

6. System Architecture §

This chapter addresses various aspects of PCI Express interconnect architecture in a platform context.

6.1 Interrupt and PME Support §

The PCI Express interrupt model supports two mechanisms:

- INTx emulation
- Message Signaled Interrupt (MSI/MSI-X)

For legacy compatibility, PCI Express provides a PCI INTx emulation mechanism to signal interrupts to the system interrupt controller (typically part of the Root Complex). This mechanism is compatible with existing PCI software, and provides the same level and type of service as the corresponding PCI interrupt signaling mechanism and is independent of system interrupt controller specifics. This legacy compatibility mechanism allows boot device support without requiring complex BIOS-level interrupt configuration/control service stacks. It virtualizes PCI physical interrupt signals by using an in-band signaling mechanism.

If an Endpoint Function supports interrupts, then this specification requires support of either MSI or MSI-X or both. PCI Compatible INTx interrupt emulation is optional. Switches are required to support forwarding the INTx interrupt emulation Messages (see § Section 2.2.8.1). The PCI Express MSI and MSI-X mechanisms are compatible with those originally defined in [PCI].

For SR-IOV Devices, PFs are permitted to implement INTx, and VFs must not implement INTx. Each PF and VF must implement its own unique interrupt capabilities.

For SIOV Devices, PFs are permitted to implement INTx, and SDIs must not implement INTx.

6.1.1 Rationale for PCI Express Interrupt Model §

PCI Express takes an evolutionary approach from PCI with respect to interrupt support.

As required for PCI/PCI-X interrupt mechanisms, each device Function is required to differentiate between INTx and MSI/MSI-X modes of operation. The device complexity required to support both schemes is no different than that for PCI/PCI-X devices. The advantages of this approach include:

- Compatibility with existing PCI Software Models
- Direct support for boot devices
- Easier End of Life (EOL) for INTx legacy mechanisms.

The existing software model is used to differentiate INTx vs. MSI/MSI-X modes of operation; thus, no special software support is required for PCI Express.

The software model does not support changing interrupt modes while the Function is in active operation. If software does this, interrupt conditions may be dropped or replicated.

6.1.2 PCI-compatible INTx Emulation §

PCI Express emulates the PCI interrupt mechanism including the Interrupt Pin and Interrupt Line registers of the PCI Configuration Space for PCI device Functions. PCI Express non-Switch devices may optionally support these registers for backwards compatibility. Switch devices are required to support them. Actual interrupt signaling uses in-band Messages rather than being signaled using physical pins.

Two types of Messages are defined, Assert_INTx and Deassert_INTx, for emulation of PCI INTx signaling, where x is A, B, C, and D for respective PCI interrupt signals. These Messages are used to provide “virtual wires” for signaling interrupts across a Link. Switches collect these virtual wires and present a combined set at the Switch’s Upstream Port. Ultimately, the virtual wires are routed to the Root Complex which maps the virtual wires to system interrupt resources. Devices must use assert/deassert Messages in pairs to emulate PCI interrupt level-triggered signaling. Actual mapping of PCI Express INTx emulation to system interrupts is implementation specific as is mapping of physical interrupt signals in conventional PCI.

The legacy INTx emulation mechanism may be deprecated in a future version of this specification.

6.1.3 INTx Emulation Software Model §

The software model for legacy INTx emulation matches that of PCI. The system BIOS reporting of chipset/platform interrupt mapping and the association of each device Function’s interrupt with PCI interrupt lines is handled in exactly the same manner as with conventional PCI systems. Legacy software reads from each device Function’s Interrupt Pin register to determine if the Function is interrupt driven. A value between 01h and 04h indicates that the Function uses an emulated interrupt pin to generate an interrupt.

Note that similarly to physical interrupt signals, the INTx emulation mechanism may potentially cause spurious interrupts that must be handled by the system software.

6.1.4 MSI and MSI-X Operation §

Message Signaled Interrupts (MSI) is an optional feature that enables a device Function to request service by writing a system-specified data value to a system-specified address (using a DWORD Memory Write transaction). System software initializes the message address and message data (from here on referred to as the “vector”) during device configuration, allocating one or more vectors to each MSI-capable Function.

Interrupt latency (the time from interrupt signaling to interrupt servicing) is system dependent. Consistent with current interrupt architectures, Message Signaled Interrupts do not provide interrupt latency time guarantees.

MSI-X defines a separate optional extension to basic MSI functionality. Compared to MSI, MSI-X supports a larger maximum number of vectors per Function, the ability for software to control aliasing when fewer vectors are allocated than requested, plus the ability for each vector to use an independent address and data value, specified by a table that resides in Memory Space. However, most of the other characteristics of MSI-X are identical to those of MSI.

For the sake of software backward compatibility, MSI and MSI-X use separate and independent Capability structures. On Functions that support both MSI and MSI-X, system software that supports only MSI can still enable and use MSI without any modification. MSI functionality is managed exclusively through the MSI Capability structure, and MSI-X functionality is managed exclusively through the MSI-X Capability structure.

A Function is permitted to implement both MSI and MSI-X, but system software is prohibited from enabling both at the same time. If system software enables both at the same time, the behavior is undefined.

All PCI Express Endpoint Functions that are capable of generating interrupts must support MSI or MSI-X or both. The MSI and MSI-X mechanisms deliver interrupts by performing Memory Write transactions. MSI and MSI-X are edge-triggered interrupt mechanisms; neither [PCI] nor this specification support level-triggered MSI/MSI-X interrupts. Certain PCI devices and their drivers rely on INTx-type level-triggered interrupt behavior (addressed by the PCI Express legacy INTx emulation mechanism). To take advantage of the MSI or MSI-X capability and edge-triggered interrupt semantics, these devices and their drivers may have to be redesigned.

MSI and MSI-X each support Per-Vector Masking (PVM). PVM is an optional ¹¹³ extension to MSI, and a standard feature with MSI-X. A Function that supports the PVM extension to MSI is backward compatible with system software that is unaware of the extension. MSI-X also supports a Function Mask bit, which when Set masks all of the vectors associated with a Function.

A Legacy Endpoint that implements MSI is required to support either the 32-bit or 64-bit Message Address version of the MSI Capability structure. A PCI Express Endpoint that implements MSI is required to support the 64-bit Message Address version of the MSI Capability structure.

The Requester of an MSI/MSI-X transaction must set the No Snoop and Relaxed Ordering attributes of the Transaction Descriptor to 0b. A Requester of an MSI/MSI-X transaction is permitted to Set the ID-Based Ordering (IDO) attribute if use of the IDO attribute is enabled.

Note that, unlike INTx emulation Messages, MSI/MSI-X transactions are not restricted to the TC0 traffic class.

IMPLEMENTATION NOTE: SYNCHRONIZATION OF DATA TRAFFIC AND MESSAGE SIGNALLED INTERRUPTS §

MSI/MSI-X transactions are permitted to use the TC that is most appropriate for the device's programming model. This is generally the same TC as is used to transfer data; for legacy I/O, TC0 should be used.

If a device uses more than one TC, it must explicitly ensure that proper synchronization is maintained between data traffic and interrupt Message(s) not using the same TC. Methods for ensuring this synchronization are implementation specific. One option is for a device to issue a zero-length Read (as described in § Section 2.2.5) using each additional TC used for data traffic prior to issuing the MSI/MSI-X transaction. Other methods are also possible. Note, however, that platform software (e.g., a device driver) is generally only capable of issuing transactions using TC0.

Within a device, different Functions are permitted to implement different sets of the MSI/MSI-X/INTx interrupt mechanisms, and system software manages each Function's interrupt mechanisms independently.

6.1.4.1 MSI Configuration §

In this section, all register and field references are in the context of the MSI Capability structure.

System software reads the Message Control register to determine the Function's MSI capabilities.

System software reads the Multiple Message Capable field (bits 3-1 of the Message Control register) to determine the number of requested vectors. MSI supports a maximum of 32 vectors per Function. System software writes to the Multiple Message Enable field (bits 6-4 of the Message Control register) to allocate either all or a subset of the requested vectors. For example, a Function can request four vectors and be allocated either four, two, or one vector. The number of

113. Exception: Within an SR-IOV Device, any PFs or VFs that implement MSI must implement MSI PVM.

vectors requested and allocated is aligned to a power of two (that is, a Function that requires three vectors must request four).

If the Per-Vector Masking Capable bit (bit 8 of the Message Control register) is Set and system software supports Per-Vector Masking, system software may mask one or more vectors by writing to the Mask Bits register.

If the 64-bit Address Capable bit (bit 7 of the Message Control register) is Set, system software initializes the MSI Capability structure's Message Address register (specifying the lower 32 bits of the message address) and the Message Upper Address register (specifying the upper 32 bits of the message address) with a system-specified message address. System software may program the Message Upper Address register to zero so that the Function uses a 32-bit address for the MSI transaction. If this bit is Clear, system software initializes the MSI Capability structure's Message Address register (specifying a 32-bit message address) with a system specified message address.

System software initializes the MSI Capability structure's Message Data register with the lower 16 bits of a system specified data value. When the Extended Message Data Capable bit is Clear, care must be taken to initialize only the Message Data register (i.e., a 2-byte value) and not modify the upper two bytes of that DWORD location.

If the Extended Message Data Capable bit is Set and system software supports 32-bit vector values, system software may initialize the MSI capability structure's Extended Message Data register with the upper 16 bits of a system specified data value, and then Set the Extended Message Data Enable bit.

6.1.4.2 MSI-X Configuration §

In this section, all register and field references are in the context of the MSI-X Capability, MSI-X Table, and MSI-X PBA structures.

System software allocates address space for the Function's standard set of Base Address registers and sets the registers accordingly. One of the Function's Base Address registers includes address space for the MSI-X Table, though the system software that allocates address space does not need to be aware of which Base Address register this is, or the fact the address space is used for the MSI-X Table. The same or another Base Address register includes address space for the MSI-X PBA, and the same point regarding system software applies.

Depending upon system software policy, system software, device driver software, or each at different times or environments may configure a Function's MSI-X Capability and table structures with suitable vectors. For example, a booting environment will likely require only a single vector, whereas a normal operating system environment for running applications may benefit from multiple vectors if the Function supports an MSI-X Table with multiple entries. For the remainder of this section, "software" refers to either system software or device driver software.

Software reads the Table Size field from the Message Control register to determine the MSI-X Table size. The field encodes the number of table entries as N-1, so software must add 1 to the value read from the field to calculate the number of table entries N. MSI-X supports a maximum table size of 2048 entries.

Software calculates the base address of the MSI-X Table by reading the 32-bit value from the Table Offset/Table BIR register, masking off the lower 3 Table BIR bits, and adding the remaining QWORD-aligned 32-bit Table offset to the address taken from the Base Address register indicated by the Table BIR. Software calculates the base address of the MSI-X PBA using the same process with the PBA Offset/PBA BIR register.

For each MSI-X Table entry that will be used, software fills in the Message Address field, Message Upper Address field, Message Data field, and Vector Control field. The Vector Control field may contain optional Steering Tag fields. Software must not modify the Address, Data, or Steering Tag fields of an entry while it is unmasked. Refer to § Section 6.1.4.5 for details.

IMPLEMENTATION NOTE: SPECIAL CONSIDERATIONS FOR QWORD ACCESSES §

Software is permitted to fill in MSI-X Table entry DWORD fields individually with DWORD writes, or software in certain cases is permitted to fill in appropriate pairs of DWORDs with a single QWORD write. Specifically, software is always permitted to fill in the Message Address and Message Upper Address fields with a single QWORD write. If a given entry is currently masked (via its Mask bit or the Function Mask bit), software is permitted to fill in the Message Data and Vector Control fields with a single QWORD write, taking advantage of the fact the Message Data field is guaranteed to become visible to hardware no later than the Vector Control field. However, if software wishes to mask a currently unmasked entry (without Setting the Function Mask bit), software must Set the entry's Mask bit using a DWORD write to the Vector Control field, since performing a QWORD write to the Message Data and Vector Control fields might result in the Message Data field being modified before the Mask bit in the Vector Control field becomes Set.

For potential use by future specifications, the Reserved bits in the Vector Control field must have their default values preserved by software. If software does not preserve their values, the result is undefined.

For each MSI-X Table entry that software chooses not to configure for generating messages, software can simply leave the entry in its default state of being masked.

Software is permitted to configure multiple MSI-X Table entries with the same vector, and this may indeed be necessary when fewer vectors are allocated than requested.

IMPLEMENTATION NOTE: HANDLING MSI-X VECTOR SHORTAGES §

For the case where fewer vectors are allocated to a Function than desired, software-controlled aliasing as enabled by MSI-X is one approach for handling the situation. For example, if a Function supports five queues, each with an associated MSI-X table entry, but only three vectors are allocated, the Function could be designed for software still to configure all five table entries, assigning one or more vectors to multiple table entries. Software could assign the three vectors {A,B,C} to the five entries as ABCCC, ABBCC, ABCBA, or other similar combinations.

Alternatively, the Function could be designed for software to configure it (using a device specific mechanism) to use only three queues and three MSI-X table entries. Software could assign the three vectors {A,B,C} to the five entries as ABC-, A-B-C, A-CB, or other similar combinations.

6.1.4.3 Enabling Operation §

To maintain backward compatibility, the MSI Enable bit in the Message Control Register for MSI and the MSI-X Enable bit in the Message Control Register for MSI-X are each Clear by default (MSI and MSI-X are both disabled). System configuration software Sets one of these bits to enable either MSI or MSI-X, but never both simultaneously. Behavior is undefined if both MSI and MSI-X are enabled simultaneously. Software disabling either mechanism during active operation may result in the Function dropping pending interrupt conditions or failing to recognize new interrupt conditions. While enabled for MSI or MSI-X operation, a Function is prohibited from using INTx interrupts (if implemented) to request service (MSI, MSI-X, and INTx are mutually exclusive).

6.1.4.4 Sending Messages §

Once MSI or MSI-X is enabled (the appropriate bit in one of the Message Control registers is Set), and one or more vectors is unmasked, the Function is permitted to send messages. To send a message, a Function does a DWORD Memory Write to the appropriate message address with the appropriate message data.

For MSI when the Extended Message Data Enable bit is Clear, the DWORD that is written is made up of the value in the MSI Message Data register in the lower two bytes and zeros in the upper two bytes. For MSI when the Extended Message Data Enable bit is Set, the DWORD that is written is made up of the value in the MSI Message Data register in the lower two bytes and the value in the MSI Extended Message Data register in the upper two bytes.

For MSI, if the Multiple Message Enable field (bits 6-4 of the Message Control Register for MSI) is non-zero, the Function is permitted to modify the low order bits of the message data to generate multiple vectors. For example, a Multiple Message Enable encoding of 010b indicates the Function is permitted to modify message data bits 1 and 0 to generate up to four unique vectors. If the Multiple Message Enable field is 000b, the Function is not permitted to modify the message data.

For MSI-X, the MSI-X Table contains at least one entry for every allocated vector, and the 32-bit Message Data field value from a selected table entry is used in the message without any modification to the low-order bits by the Function.

How a Function uses multiple vectors (when allocated) is device dependent. A Function must handle being allocated fewer vectors than requested.

6.1.4.5 Per-vector Masking and Function Masking §

Per-Vector Masking (PVM) is an optional ¹¹⁴ feature with MSI, and a standard feature in MSI-X.

Function Masking is a standard feature in MSI-X. When the MSI-X Function Mask bit is Set, all of the Function's entries must behave as being masked, regardless of the per-entry Mask bit values. Function Masking is not supported in MSI, but software can readily achieve a similar effect by Setting all MSI Mask bits using a single DWORD write.

PVM in MSI-X is controlled by a Mask bit in each MSI-X Table entry. While more accurately termed “per-entry masking”, masking an MSI-X Table entry is still referred to as “vector masking” so similar descriptions can be used for both MSI and MSI-X. However, since software is permitted to program the same vector (a unique Address/Data pair) into multiple MSI-X table entries, all such entries must be masked in order to guarantee the Function will not send a message using that Address/Data pair.

For MSI and MSI-X, while a vector is masked, the Function is prohibited from sending the associated message, and the Function must Set the associated Pending bit whenever the Function would otherwise send the message. When software unmask a vector whose associated Pending bit is Set, the Function must schedule sending the associated message, and Clear the Pending bit as soon as the message has been sent. Note that Clearing the MSI-X Function Mask bit may result in many messages needing to be sent.

If a masked vector has its Pending bit Set, and the associated underlying interrupt events are somehow satisfied (usually by software though the exact manner is Function-specific), the Function must Clear the Pending bit, to avoid sending a spurious interrupt message later when software unmask the vector. However, if a subsequent interrupt event occurs while the vector is still masked, the Function must again Set the Pending bit.

Software is permitted to mask one or more vectors indefinitely, and service their associated interrupt events strictly based on polling their Pending bits. A Function must Set and Clear its Pending bits as necessary to support this “pure polling” mode of operation.

114. Exception: Within an SR-IOV Device, any PFs or VFs that implement MSI must implement MSI PVM.

For MSI-X, a Function is permitted to cache Address and Data values from unmasked MSI-X Table entries. However, anytime software unmask a currently masked MSI-X Table entry either by Clearing its Mask bit or by Clearing the Function Mask bit, the Function must update any Address or Data values that it cached from that entry. If software changes the Address or Data value of an entry while the entry is unmasked, the result is undefined.

IMPLEMENTATION NOTE:

PER VECTOR MASKING WITH MSI/MSI-X §

Devices and drivers that use MSI or MSI-X have the challenge of coordinating exactly when new interrupt messages are generated. If hardware fails to send an interrupt message that software expects, an interrupt event might be “lost”. If hardware sends an interrupt message that software is not expecting, a “spurious” interrupt might result.

Per-Vector Masking (PVM) can be used to assist in this coordination. For example, when a software interrupt service routine begins, it can mask the vector to help avoid “spurious” interrupts. After the interrupt service routine services all the interrupt conditions that it is aware of, it can unmask the vector. If any interrupt conditions remain, hardware is required to generate a new interrupt message, guaranteeing that no interrupt events are lost.

PVM is a standard feature with MSI-X and an optional ¹¹⁵ feature for MSI. For devices that implement MSI, implementing PVM as well is strongly recommended.

6.1.4.6 Hardware/Software Synchronization §

If a Function sends messages with the same vector multiple times before being acknowledged by software, only one message is guaranteed to be serviced. If all messages must be serviced, a device driver handshake is required. In other words, once a Function sends Vector A, it cannot send Vector A again until it is explicitly enabled to do so by its device driver (provided all messages must be serviced). If some messages can be lost, a device driver handshake is not required. For Functions that support multiple vectors, a Function can send multiple unique vectors and is guaranteed that each unique message will be serviced. For example, a Function can send Vector A followed by Vector B without any device driver handshake (both Vector A and Vector B will be serviced).

¹¹⁵. Exception: Within an SR-IOV Device, any PFs or VFs that implement MSI must implement MSI PVM

IMPLEMENTATION NOTE:

SERVICING MSI AND MSI-X INTERRUPTS §

When system software allocates fewer MSI or MSI-X vectors to a Function than it requests, multiple interrupt sources within the Function, each desiring a unique vector, may be required to share a single vector. Without proper handshakes between hardware and software, hardware may send fewer messages than software expects, or hardware may send what software considers to be extraneous messages.

A rather sophisticated but resource-intensive approach is to associate a dedicated event queue with each allocated vector, with producer and consumer pointers for managing each event queue. Such event queues typically reside in host memory. The Function acts as the producer and software acts as the consumer. Multiple interrupt sources within a Function may be assigned to each event queue as necessary. Each time an interrupt source needs to signal an interrupt, the Function places an entry on the appropriate event queue (assuming there's room), updates a copy of the producer pointer (typically in host memory), and sends an interrupt message with the associated vector when necessary to notify software that the event queue needs servicing. The interrupt service routine for a given event queue processes all entries it finds on its event queue, as indicated by the producer pointer. Each event queue entry identifies the interrupt source and possibly additional information about the nature of the event. The use of event queues and producer/consumer pointers can be used to guarantee that interrupt events won't get dropped when multiple interrupt sources are forced to share a vector. There's no need for additional handshaking between sending multiple messages associated with the same event queue, to guarantee that every message gets serviced. In fact, various standard techniques for "interrupt coalescing" can be used to avoid sending a separate message for every event that occurs, particularly during heavy bursts of events.

In more modest implementations, the hardware design of a Function's MSI or MSI-X logic sends a message any time a transition to assertion would have occurred on the virtual INTx wire if MSI or MSI-X had not been enabled. For example, consider a scenario in which two interrupt events (possibly from distinct interrupt sources within a Function) occur in rapid succession. The first event causes a message to be sent. Before the interrupt service routine has had an opportunity to service the first event, the second event occurs. In this case, only one message is sent, because the first event is still active at the time the second event occurs (a virtual INTx wire signal would have had only one transition to assertion).

One handshake approach for implementations like the above is to use standard Per-Vector Masking, and allow multiple interrupt sources to be associated with each vector. A given vector's interrupt service routine sets the vector's Mask bit before it services any associated interrupting events and clears the Mask bit after it has serviced all the events it knows about. (This could be any number of events.) Any occurrence of a new event while the Mask bit is Set results in the Pending bit being Set. If one or more associated events are still pending at the time the vector's Mask bit is Cleared, the Function immediately sends another message.

A handshake approach for MSI Functions that do not implement Per-Vector Masking is for a vector's interrupt service routine to re-inspect all of the associated interrupt events after clearing what is presumed to be the last pending interrupt event. If another event is found to be active, it is serviced in the same interrupt service routine invocation, and the complete re-inspection is repeated until no pending events are found. This ensures that if an additional interrupting event occurs before a previous interrupt event is Cleared, whereby the Function does not send an additional interrupt message, that the new event is serviced as part of the current interrupt service routine invocation.

This alternative has the potential side effect of one vector's interrupt service routine processing an interrupting event that has already generated a new interrupt message. The interrupt service routine invocation resulting from the new message may find no pending interrupt events. Such occurrences are sometimes referred to as spurious interrupts, and software using this approach must be prepared to tolerate them.

An MSI or MSI-X message, by virtue of being a Posted Request, is prohibited by transaction ordering rules from passing Posted Requests sent earlier by the Function. The system must guarantee that an interrupt service routine invoked as a result of a given message will observe any updates performed by Posted Requests arriving prior to that message. Thus, the interrupt service routine of a device driver is not required to read from a device register in order to ensure data consistency with previous Posted Requests. However, if multiple MSI-X Table entries share the same vector, the interrupt service routine may need to read from some device specific register to determine which interrupt sources need servicing.

6.1.4.7 Message Transaction Reception and Ordering Requirements §

As with all Memory Write transactions, the device that includes the target of the interrupt message (the interrupt receiver) is required to complete all interrupt message transactions as a Completer without requiring other transactions to complete first as a Requester. In general, this means that the message receiver must complete the interrupt message transaction independent of when the CPU services the interrupt. For example, each time the interrupt receiver receives an interrupt message, it could Set a bit in an internal register indicating that this message had been received and then complete the transaction on the bus. The appropriate interrupt service routine would later be dispatched because this bit was Set. The message receiver would not be allowed to delay the completion of the interrupt message on the bus pending acknowledgement from the processor that the interrupt was being serviced. Such dependencies can lead to deadlock when multiple devices send interrupt messages simultaneously.

Although interrupt messages remain strictly ordered throughout the PCI Express Hierarchy, the order of receipt of the interrupt messages does not guarantee any order in which the interrupts will be serviced. Since the message receiver must complete all interrupt message transactions without regard to when the interrupt was actually serviced, the message receiver will generally not maintain any information about the order in which the interrupts were received. This is true both of interrupt messages received from different devices and multiple messages received from the same device. If a device requires one interrupt message to be serviced before another, the device must not send the second interrupt message until the first one has been serviced.

6.1.5 PME Support §

PCI Express supports power management events from native PCI Express devices as well as PME-capable PCI devices.

PME signaling is accomplished using an in-band Transaction Layer PME Message (PM_PME) as described in § Chapter 5. .

6.1.6 Native PME Software Model §

PCI Express-aware software can enable a mode where the Root Complex signals PME via an interrupt. When configured for native PME support, a Root Port receives the PME Message and sets the PME Status bit in its Root Status Register. If software has set the PME Interrupt Enable bit in the Root Control Register to 1b, the Root Port then generates an interrupt.

If the Root Port is enabled for level-triggered interrupt signaling using the INTx messages, the virtual INTx wire must be asserted whenever and as long as all of the following conditions are satisfied:

- The Interrupt Disable bit in the Command register is set to 0b.
- The PME Interrupt Enable bit in the Root Control Register is set to 1b.
- The PME Status bit in the Root Status register is set.

Note that all other interrupt sources within the same Function will assert the same virtual INTx wire when requesting service.

If the Root Port is enabled for edge-triggered interrupt signaling using MSI or MSI-X, an interrupt message must be sent every time the logical AND of the following conditions transitions from FALSE to TRUE:

- The associated vector is unmasked (not applicable if MSI does not support PVM).
- The PME Interrupt Enable bit in the Root Control Register is set to 1b.
- The PME Status bit in the Root Status Register is set.

Note that PME and Hot-Plug Event interrupts (when both are implemented) always share the same MSI or MSI-X vector, as indicated by the Interrupt Message Number field in the PCI Express Capabilities Register.

The software handler for this interrupt can determine which device sent the PME Message by reading the PME Requester ID field in the Root Status Register in a Root Port. It dismisses the interrupt by writing a 1b to the PME Status bit in the Root Status Register. Refer to § Section 7.5.3.14 for more details.

Root Complex Event Collectors provide support for the above described functionality for Root Complex Integrated Endpoints (RCIEPs).

6.1.7 Legacy PME Software Model §

Legacy software, however, will not understand this mechanism for signaling PME. In the presence of legacy system software, the system power management logic in the Root Complex receives the PME Message and informs system software through an implementation specific mechanism. The Root Complex may utilize the Requester ID in the PM_PME to inform system software which device caused the power management event.

Since it is delivered by a Message, PME has edge-triggered semantics in PCI Express, which differs from the level-triggered PME mechanism used for conventional PCI. It is the responsibility of the Root Complex to abstract this difference from system software to maintain compatibility with conventional PCI systems.

6.1.8 Operating System Power Management Notification §

In order to maintain compatibility with non-PCI Express-aware system software, system power management logic must be configured by firmware to use the legacy mechanism of signaling PME by default. PCI Express-aware system software must notify the firmware prior to enabling native, interrupt-based PME signaling. In response to this notification, system firmware must, if needed, reconfigure the Root Complex to disable legacy mechanisms of signaling PME. The details of this firmware notification are beyond the scope of this specification, but since it will be executed at system run-time, the response to this notification must not interfere with system software. Therefore, following control handoff to the operating system, firmware must not write to available system memory or any PCI Express resources (e.g., Configuration Space structures) owned by the operating system.

6.1.9 PME Routing Between PCI Express and PCI Hierarchies §

PME-capable conventional PCI and PCI-X devices assert the PME# pin to signal a power management event. The PME# signal from PCI or PCI-X devices may either be converted to a PCI Express in-band PME Message by a PCI Express-PCI Bridge or routed directly to the Root Complex.

If the PME# signal from a PCI or PCI-X device is routed directly to the Root Complex, it signals system software using the same mechanism used in present PCI systems. A Root Complex may optionally provide support for signaling PME from PCI or PCI-X devices to system software via an interrupt. In this scenario, it is recommended for the Root Complex to detect the Bus, Device and Function Number of the PCI or PCI-X device that asserted PME#, and use this information to

fill in the PME Requester ID field in the Root Port that originated the hierarchy containing the PCI or PCI-X device. If this is not possible, the Root Complex may optionally write the Requester ID of the Root Port to this field.

Since RCiEPs are not contained in any of the hierarchy domains originated by Root Ports, RCiEPs not associated with a Root Complex Event Collector signal system software of a PME using the same mechanism used in present PCI systems. A Root Complex Event Collector, if implemented, enables the PCI Express Native PME model for associated RCiEPs.

6.2 Error Signaling and Logging §

In this document, errors which must be checked and errors which may optionally be checked are identified. Each such error is associated either with the Port or with a specific device (or Function in a Multi-Function Device), and this association is given along with the description of the error. This section will discuss how errors are classified and reported.

6.2.1 Scope §

This section explains the error signaling and logging requirements for PCI Express components. This includes errors which occur on the PCI Express interface itself, those errors which occur on behalf of transactions initiated on PCI Express, and errors which occur within a component and are related to the PCI Express interface. This section does not focus on errors which occur within the component that are unrelated to a PCI Express interface. This type of error signaling is better handled through proprietary methods employing device-specific interrupts.

PCI Express defines two error reporting paradigms: baseline capability and Advanced Error Reporting Extended Capability (AER). Baseline capability is required for all PCI Express devices, and it defines the minimum error reporting requirements. AER Extended Capability defines more robust error reporting and is implemented with a specific PCI Express Capability structure (refer to § Section 7.8.4 for a definition of this optional capability). This section explicitly calls out all error handling differences between baseline and AER Extended Capability.

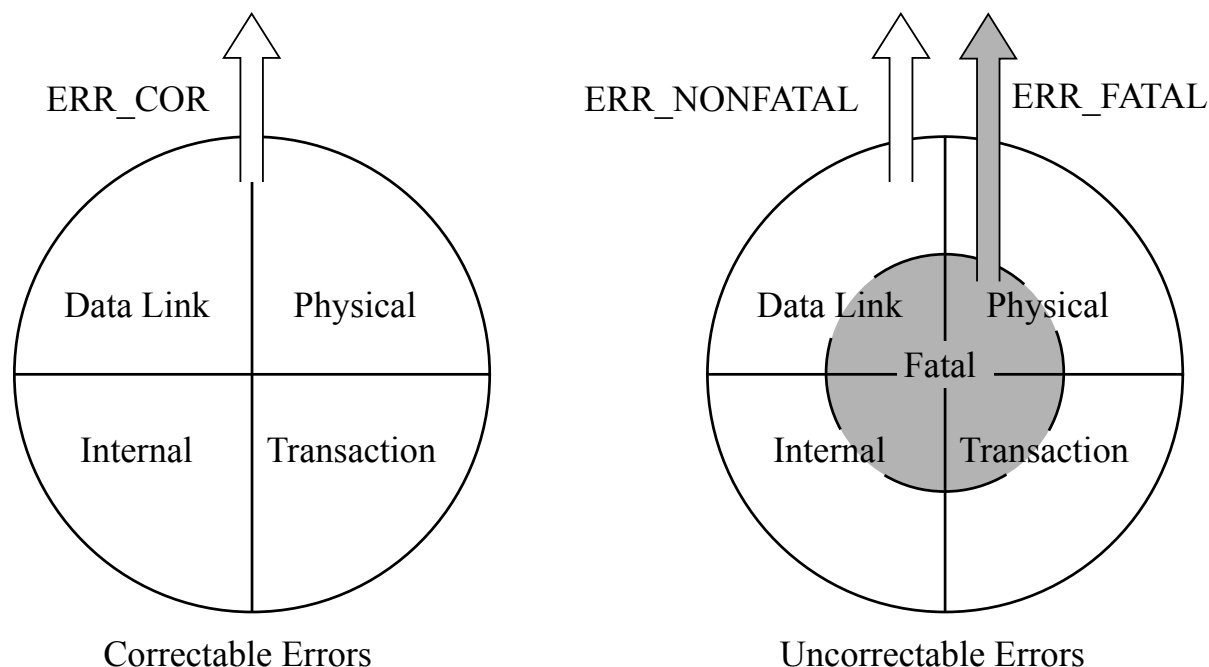
All SR-IOV Devices must support baseline Capability, with certain modifications to account for the goal of reduced cost and complexity of implementation. The AER Extended Capability is optional, but if AER is not implemented in a PF, it must not be implemented in its associated VFs. If AER is implemented in the PF, it is optional in its VFs.

All SIOV Devices must support baseline Capability, with certain modifications to account for the goal of reduced cost and complexity of implementation. The AER Extended Capability is optional.

All PCI Express devices support existing, non-PCI Express-aware, software for error handling by mapping PCI Express errors to existing PCI reporting mechanisms, in addition to the PCI Express-specific mechanisms.

6.2.2 Error Classification §

PCI Express errors can be classified as two types: Uncorrectable errors and Correctable errors. This classification separates those errors resulting in functional failure from those errors resulting in degraded performance. Uncorrectable errors can further be classified as Fatal or Non-Fatal (see § Figure 6-1).



OM13827A

Figure 6-1 Error Classification §

Classification of error severity as Fatal, Uncorrectable, and Correctable provides the platform with mechanisms for mapping the error to a suitable handling mechanism. For example, the platform might choose to respond to correctable errors with low priority, performance monitoring software. Such software could count the frequency of correctable errors and provide Link integrity information. On the other hand, a platform designer might choose to map Fatal errors to a system-wide reset. It is the decision of the platform designer to map these PCI Express severity levels onto platform level severities.

6.2.2.1 Correctable Errors §

Correctable errors include those error conditions where hardware can recover without any loss of information. Hardware corrects these errors and software intervention is not required. For example, an LCRC error in a TLP that might be corrected by Data Link Level Retry is considered a correctable error. Measuring the frequency of Link-level correctable errors may be helpful for profiling the integrity of a Link.

Correctable errors also include transaction-level cases where one agent detects an error with a TLP, but another agent is responsible for taking any recovery action if needed, such as re-attempting the operation with a separate subsequent transaction. The detecting agent can be configured to report the error as being correctable since the recovery agent may be able to correct it. If recovery action is indeed needed, the recovery agent must report the error as uncorrectable if the recovery agent decides not to attempt recovery.

The triggering of Downstream Port Containment (DPC) is not handled as an error, but it can be signaled as if it were a correctable error, since software that takes advantage of DPC can sometimes recover from the uncorrectable error that triggered DPC. See § Section 6.2.11 . An ERR_COR Message that's used for DPC signaling is intended to target system firmware, and may indicate so via the ERR_COR Subclass field.

Similarly, ERR_COR may be used by the System Firmware Intermediary (SFI) capability to signal system firmware, and must indicate so via the ERR_COR Subclass field. See § Section 6.7.4 .

IMPLEMENTATION NOTE:

ERR_COR SUBCLASS POSSIBLE USAGE §

The following is a high-level summary of how the ERR_COR Subclass (ECS) field can be used:

- DPC events: DPC Trigger Status, DL_Active
 - ECS SIG_SFW – ERR_CORs intended to signal SFW:
 - SFI events: DL_Active, SFI DRS
- ECS SIG_OS – ERR_CORs intended to signal the OS:
 - AER events: Link correctable errors, Advisory Non-Fatal Errors
 - RP PIO events: RP PIO advisory errors
- ECS Legacy – ERR_CORs from adapters that do not support ECS
 - Implementation-specific mechanisms may enable Legacy ERR_CORs to signal SFW

6.2.2.2 Uncorrectable Errors §

Uncorrectable errors are those error conditions that impact functionality of the interface. There is no mechanism defined in this specification to correct these errors. Reporting an uncorrectable error is analogous to asserting SERR# in PCI/PCI-X. For more robust error handling by the system, this specification further classifies uncorrectable errors as Fatal and Non-fatal.

6.2.2.2.1 Fatal Errors §

Fatal errors are uncorrectable error conditions which render the particular Link and related hardware unreliable. For Fatal errors, a reset of the components on the Link may be required to return to reliable operation. Platform handling of Fatal errors, and any efforts to limit the effects of these errors, is platform implementation specific.

6.2.2.2.2 Non-Fatal Errors §

Non-fatal errors are uncorrectable errors which cause a particular transaction to be unreliable but the Link is otherwise fully functional. Isolating Non-fatal from Fatal errors provides Requester/Receiver logic in a device or system management software the opportunity to recover from the error without resetting the components on the Link and disturbing other transactions in progress. Devices not associated with the transaction in error are not impacted by the error.

6.2.3 Error Signaling §

There are three complementary mechanisms which allow the agent detecting an error to alert the system or another device that an error has occurred. The first mechanism is through a Completion Status, the second method is with in-band error Messages, and the third is with Error Forwarding (also known as data poisoning).

Note that it is the responsibility of the agent detecting the error to signal the error appropriately.

§ Section 6.2.7 describes all the errors and how the hardware is required to respond when the error is detected.

6.2.3.1 Completion Status §

The Completion Status field (when status is not Successful Completion) in the Completion header indicates that the associated Request failed (see § Section 2.2.8.10). This is one method of error reporting which enables the Requester to associate an error with a specific Request. In other words, since Non-Posted Requests and UIO Requests are not considered complete until after the Completion returns, the Completion Status field gives the Requester an opportunity to “fix” the problem at some higher level protocol (outside the scope of this specification).

6.2.3.2 Error Messages §

Error Messages are sent to the Root Complex for reporting the detection of errors according to the severity of the error.

Error messages that originate from PCI Express or Legacy Endpoints are sent to corresponding Root Ports. Errors that originate from a Root Port itself are reported through the same Root Port.

If an optional Root Complex Event Collector is implemented, errors that originate from RCiEPs are sent to the corresponding Root Complex Event Collector. Errors that originate in a Root Complex Event Collector itself are reported through the same Root Complex Event Collector. The Root Complex Event Collector must declare supported RCiEPs as part of its capabilities; each RCiEP must be associated with no more than one Root Complex Event Collector.

When multiple errors of the same severity are detected, the corresponding error Messages with the same Requester ID may be merged for different errors of the same severity. At least one error Message must be sent for detected errors of each severity level. Note, however, that the detection of a given error in some cases will preclude the reporting of certain errors. Refer to § Section 6.2.3.2.3 . Also note special rules in § Section 6.2.4 regarding non-Function-specific errors in Multi-Function Devices.

Table 6-1 Error Messages §

Error Message	Description
<u>ERR_COR</u>	This Message is issued when the Function or Device detects a correctable error on the PCI Express interface. Refer to § Section 6.2.2.1 for the definition of a correctable error.
<u>ERR_NONFATAL</u>	This Message is issued when the Function or Device detects a Non-fatal, uncorrectable error on the PCI Express interface. Refer to § Section 6.2.2.2 for the definition of a Non-fatal, uncorrectable error.
<u>ERR_FATAL</u>	This Message is issued when the Function or Device detects a Fatal, uncorrectable error on the PCI Express interface. Refer to § Section 6.2.2.1 for the definition of a Fatal, uncorrectable error.

For these Messages, the Root Complex identifies the initiator of the Message by the Requester ID of the Message header. The Root Complex translates these error Messages into platform level events.

IMPLEMENTATION NOTE:

USE OF ERR_COR, ERR_NONFATAL, AND ERR_FATAL §

In [PCIe-1.0a], a given error was either correctable, non-fatal, or fatal. Assuming signaling was enabled, correctable errors were always signaled with ERR_COR, non-fatal errors were always signaled with ERR_NONFATAL, and fatal errors were always signaled with ERR_FATAL.

In subsequent specifications that support Role-Based Error Reporting, non-fatal errors are sometimes signaled with ERR_NONFATAL, sometimes signaled with ERR_COR, and sometimes not signaled at all, depending upon the role of the agent that detects the error and whether the agent implements AER (see § Section 6.2.3.2.4). On some platforms, sending ERR_NONFATAL will preclude another agent from attempting recovery or determining the ultimate disposition of the error. For cases where the detecting agent is not the appropriate agent to determine the ultimate disposition of the error, a detecting agent with AER can signal the non-fatal error with ERR_COR, which serves as an advisory notification to software. For cases where the detecting agent is the appropriate one, the agent signals the non-fatal error with ERR_NONFATAL.

For a given uncorrectable error that's normally non-fatal, if software wishes to avoid continued hierarchy operation upon the detection of that error, software can configure detecting agents that implement AER to escalate the severity of that error to fatal. A detecting agent (if enabled) will always signal a fatal error with ERR_FATAL, regardless of the agent's role.

Software should recognize that a single transaction can be signaled by multiple agents using different types of error Messages. For example, a poisoned TLP might be signaled by intermediate Receivers with ERR_COR, while the ultimate destination Receiver might signal it with ERR_NONFATAL.

6.2.3.2.1 Uncorrectable Error Severity Programming (Advanced Error Reporting) §

For device Functions implementing the Advanced Error Reporting Extended Capability, the Uncorrectable Error Severity register allows each uncorrectable error to be programmed to Fatal or Non-Fatal. Uncorrectable errors are not recoverable using defined PCI Express mechanisms. However, some platforms or devices might consider a particular error fatal to a Link or device while another platform considers that error non-fatal. The default value of the Uncorrectable Error Severity register serves as a starting point for this specification but the register can be reprogrammed if the device driver or platform software requires more robust error handling.

Baseline error handling does not support severity programming.

6.2.3.2.2 Masking Individual Errors §

§ Section 6.2.7 lists all the errors governed by this specification and describes when each of the above error Messages are issued. The transmission of these error Messages by class (correctable, non-fatal, fatal) is enabled using the Reporting Enable bits of the Device Control register (see § Section 7.5.3.4) or the SERR# Enable bit in the PCI Command register (see § Section 7.5.1.1.3).

For devices implementing the Advanced Error Reporting Extended Capability the Uncorrectable Error Mask register and Correctable Error Mask register allows each error condition to be masked independently. If Messages for a particular class of error are not enabled by the combined settings in the Device Control register and the PCI Command register, then no Messages of that class will be sent regardless of the values for the corresponding mask register.

If an individual error is masked when it is detected, its error status bit is still affected, but no error reporting Message is sent to the Root Complex, and the error is not recorded in the Header Log, TLP Prefix Log, or First Error Pointer.

6.2.3.2.3 Error Pollution §

Error pollution can occur if error conditions for a given transaction are not isolated to the most significant occurrence. For example, assume the Physical Layer detects a Receiver Error. This error is detected at the Physical Layer and an error is reported to the Root Complex. To avoid having this error propagate and cause subsequent errors at upper layers (for example, a TLP error at the Data Link Layer), making it more difficult to determine the root cause of the error, subsequent errors which occur for the same packet will not be reported by the Data Link or Transaction layers. Similarly, when the Data Link Layer detects an error, subsequent errors which occur for the same packet will not be reported by the Transaction Layer. This behavior applies only to errors that are associated with a particular packet - other errors are reported for each occurrence.

Corrected Internal Errors are errors whose effect has been masked or worked around by a component; refer to § Section 6.2.10 for details. Therefore, Corrected Internal Errors do not contribute to error pollution and should be reported when detected.

For errors detected in the Transaction layer and Uncorrectable Internal Errors, it is permitted and recommended that no more than one error be reported for a single received TLP, and that the following precedence (from highest to lowest) be used:

- Uncorrectable Internal Error
- Receiver Overflow
- Malformed TLP
- IDE Check Failed
- ECRC Check Failed
- Misrouted IDE TLP
- AtomicOp, DMWr, or TLP Translation Egress Blocked
- TLP Prefix Blocked
- ACS Violation
- MC Blocked TLP
- Unsupported Request (UR), Completer Abort (CA), or Unexpected Completion
- PCRC Check Failed
- Poisoned TLP Received or Poisoned TLP Egress Blocked

The Completion Timeout error is not in the above precedence list, since it is not detected by processing a received TLP. Errors listed under the same bullet are mutually exclusive, so their relative order does not matter.

6.2.3.2.4 Advisory Non-Fatal Error Cases §

In some cases the detector of a non-fatal error is not the most appropriate agent to determine whether the error is recoverable or not, or if it even needs any recovery action at all. For example, if software attempts to perform a configuration read from a non-existent device or Function, the resulting UR Status in the Completion will signal the error to software, and software does not need for the Completer in addition to signal the error by sending an ERR_NONFATAL Message. In fact, on some platforms, signaling the error with ERR_NONFATAL results in a System Error, which breaks normal software probing.

“Advisory Non-Fatal Error” cases are predominantly determined by the role of the detecting agent (Requester, Completer, or Receiver) and the specific error. In such cases, an agent with AER signals the non-fatal error (if enabled) by

sending an ERR_COR Message as an advisory to software, instead of sending ERR_NONFATAL. An agent without AER sends no error Message for these cases, since software receiving ERR_COR would be unable to distinguish Advisory Non-Fatal Error cases from the correctable error cases used to assess Link integrity.

Following are the specific cases of Advisory Non-Fatal Errors. Note that multiple errors from the same or different error classes (correctable, non-fatal, fatal) may be present with a single TLP. For example, an unexpected Completion might also be poisoned. Refer to § Section 6.2.3.2.3 for requirements and recommendations on reporting multiple errors. For the previous example, it is recommended that Unexpected Completion be reported, and that Poisoned TLP Received not be reported.

If software wishes for an agent with AER to handle what would normally be an Advisory Non-Fatal Error case as being more serious, software can escalate the severity of the uncorrectable error to fatal, in which case the agent (if enabled) will signal the error with ERR_FATAL.

This section covers Advisory Non-Fatal Error handling for errors managed by the PCI Express Extended Capability and AER. § Section 6.2.11.3 covers the RP PIO error handling mechanism for Root Ports that support RP Extensions for DPC. RP PIO advisory errors are similar in concept to AER Advisory Non-Fatal Errors, but apply to different error cases and are managed by different controls.

6.2.3.2.4.1 Completer Sending a Completion with UR/CA Status §

A Completer generally sends a Completion with an Unsupported Request or Completer Abort (UR/CA) Status to signal an uncorrectable error for a Non-Posted Request or UIO Request.¹¹⁶ If the severity of the UR/CA error¹¹⁷ is non-fatal, the Completer must handle this case as an Advisory Non-Fatal Error.¹¹⁸ A Completer with AER signals the non-fatal error (if enabled) by sending an ERR_COR Message. A Completer without AER sends no error Message for this case.

Even though there was an uncorrectable error for this specific transaction, the Completer must handle this case as an Advisory Non-Fatal Error, since the Requester upon receiving the Completion with UR/CA Status is responsible for reporting the error (if necessary) using a Requester-specific mechanism (see § Section 6.2.3.2.5).

6.2.3.2.4.2 Intermediate Receiver §

When a Receiver that's not serving as the ultimate PCI Express destination for a TLP detects¹¹⁹ a non-fatal error with the TLP, this "intermediate" Receiver must handle this case as an Advisory Non-Fatal Error.¹²⁰ A Receiver with AER signals the error (if enabled) by sending an ERR_COR Message. A Receiver without AER sends no error Message for this case. An exception to the intermediate Receiver case for Root Complexes (RCs) is noted below.

An example where the intermediate Receiver case occurs is a Switch that detects poison or bad ECRC in a TLP that it is routing. Even though this was an uncorrectable (but non-fatal) error at this point in the TLP's route, the intermediate Receiver handles it as an Advisory Non-Fatal Error, so that the ultimate Receiver of the TLP (i.e., the Completer for a Request TLP, or the Requester for a Completion TLP) is not precluded from handling the error more appropriately according to its error settings. For example, a given Completer that detects poison in a Memory Write Request¹²¹ might have the error masked (and thus go unsignaled), whereas a different Completer in the same hierarchy might signal that error with ERR_NONFATAL.

116. If the Completer is returning data in a Completion, and the data is bad or suspect, the Completer is permitted to signal the error using the Error Forwarding (Data Poisoning) mechanism instead of handling it as a UR or CA.

117. Certain other errors (e.g., ACS Violation) with a Non-Posted Request or UIO Request also result in the Completer sending a Completion with UR or CA Status. If the severity of the error (e.g., ACS Violation) is non-fatal, the Completer must also handle this case as an Advisory Non-Fatal Error. However, see § Section 2.7.2.1 regarding certain Requests with Poisoned data that must be handled as uncorrectable errors.

118. If the severity is fatal, the error is not an Advisory Non-Fatal Error, and must be signaled (if enabled) with ERR_FATAL.

119. If the Receiver does not implement ECRC Checking or ECRC Checking is not enabled, the Receiver will not detect an ECRC Error.

120. If the severity is fatal, the error is not an Advisory Non-Fatal Error, and must be signaled (if enabled) with ERR_FATAL.

121. See § Section 2.7.2.1 for special rules that apply for poisoned Memory Write Requests.

A Poisoned TLP Egress Blocked error is never handled as an intermediate Receiver case since it is not detected as a part of processing a received TLP.

If an RC detects a non-fatal error with a TLP it normally would forward peer-to-peer between Root Ports, but the RC does not support propagating the error related information (e.g., a TLP Digest, EP bit, or equivalent) with the forwarded transaction, the RC must signal the error (if enabled) with ERR_NONFATAL and also must not forward the transaction. An example is an RC needing to forward a poisoned TLP peer-to-peer between Root Ports, but the RC's internal fabric does not support poison indication.

6.2.3.2.4.3 Ultimate PCI Express Receiver of a Poisoned TLP or IDE TLP with PCRC Check Failed §

When a poisoned TLP is received by its ultimate PCI Express destination, if the severity is non-fatal and the Receiver legitimately chooses not to handle this case as an uncorrectable error (see below), the Receiver must handle this case as an Advisory Non-Fatal Error.¹²² When a IDE TLP is determined at its ultimate destination Port to have a PCRC Check Failed error, if the severity is non-fatal and the Receiver deals with the data payload in a manner that permits continued operation, the Receiver must handle this case as an Advisory Non-Fatal Error. A Receiver with AER signals the error (if enabled) by sending an ERR_COR Message. A Receiver without AER sends no error Message for this case.

A Receiver must not handle this case as an Advisory Non-Fatal Error if either of the following apply:

- Handling the error as a Correctable Error and continuing operation when configured correctly could lead to silent data corruption.
- Rules in § Section 2.7.2.1 require this case to be handled as an uncorrectable error.

An example is a Root Complex that receives a poisoned Memory Write TLP that targets host memory. If the Root Complex propagates the poisoned data along with its indication to host memory, it signals the error (if enabled) with an ERR_COR. If the Root Complex does not propagate the poison to host memory, it signals the error (if enabled) with ERR_NONFATAL.

Another example is a Requester that receives a poisoned Memory Read Completion TLP. If the Requester propagates the poisoned data internally or handles the error like it would for a Completion with UR/CA Status, it signals the error (if enabled) with an ERR_COR. If the Requester does not handle the poison in a manner that permits continued operation, it signals the error (if enabled) with ERR_NONFATAL.

6.2.3.2.4.4 Requester with Completion Timeout §

This section applies to Requesters other than Root Ports performing programmed I/O (PIO). See § Section 6.2.11.3 for related RP PIO functionality in Root Ports that support RP Extensions for DPC.

When the Requester of a Non-Posted Request or UIO Request times out while waiting for the associated Completion, the Requester is permitted to attempt to recover from the error by issuing a separate subsequent Request. The Requester is permitted to attempt recovery zero, one, or multiple (finite) times, but must signal the error (if enabled) with an uncorrectable error Message if no further recovery attempt will be made.

If the severity of the Completion Timeout is non-fatal, and the Requester elects to attempt recovery by issuing a new request, the Requester must first handle the current error case as an Advisory Non-Fatal Error.¹²³ A Requester with AER signals the error (if enabled) by sending an ERR_COR Message. A Requester without AER sends no error Message for this case.

122. If the severity is fatal, the error is not an Advisory Non-Fatal Error, and must be signaled (if enabled) with ERR_FATAL.

123. If the severity is fatal, the error is not an Advisory Non-Fatal Error, and must be signaled (if enabled) with ERR_FATAL. The Requester is strongly discouraged from attempting recovery since sending ERR_FATAL will often result in the entire hierarchy going down.

Note that automatic recovery by the Requester from a Completion Timeout is generally possible only if the Non-Posted Request or UIO Request has no side-effects, but may also depend upon other considerations outside the scope of this specification.

6.2.3.2.4.5 Receiver of an Unexpected Completion §

When a Receiver receives an unexpected Completion and the severity of the Unexpected Completion error is non-fatal, the Receiver must handle this case as an Advisory Non-Fatal Error.¹²⁴ A Receiver with AER signals the error (if enabled) by sending an ERR_COR Message. A Receiver without AER sends no error Message for this case.

If the unexpected Completion was a result of misrouting, the Completion Timeout mechanism at the associated Requester will trigger eventually, and the Requester may elect to attempt recovery. Interference with Requester recovery can be avoided by having the Receiver of the unexpected Completion handle the error as an Advisory Non-Fatal Error.

6.2.3.2.5 Requester Receiving a Completion with UR/CA Status §

When a Requester receives back a Completion with a UR/CA Status, generally the Completer has handled the error as an Advisory Non-Fatal Error, assuming the error severity was non-fatal at the Completer (see § Section 6.2.3.2.4.1). The Requester must determine if any error recovery action is necessary, what type of recovery action to take, and whether or not to report the error.

If the Requester needs to report the error, the Requester must do so solely through a Requester-specific mechanism. For example, many devices have an associated device driver that can report errors to software. As another important example, the Root Complex on some platforms returns all 1's to software if a Configuration Read Completion has a UR/CA Status.

§ Section 6.2.11.3 covers RP PIO controls for Root Ports that support RP Extensions for DPC. Outside of the RP PIO mechanisms, Requesters are not permitted to report the error using PCI Express logging and error Message signaling.

6.2.3.3 Error Forwarding (Data Poisoning) §

Error Forwarding, also known as data poisoning, is indicated by setting the EP bit in a TLP. Refer to § Section 2.7.2 . This is another method of error reporting in PCI Express that enables the Receiver of a TLP to associate an error with a specific Request or Completion. Unlike the Completion Status mechanism, Error Forwarding can be used with either Requests or Completions that contain data. In addition, “intermediate” Receivers along the TLP’s route, not just the Receiver at the ultimate destination, are required to detect and report (if enabled) receiving the poisoned TLP. This can help software determine if a particular Switch along the path poisoned the TLP.

6.2.3.4 Optional Error Checking §

This specification contains a number of optional error checks. Unless otherwise specified, behavior is undefined if an optional error check is not performed and the error occurs.

When an optional error check involves multiple rules, unless otherwise specified, each rule is independently optional. An implementation may check against all of the rules, none of them or any combination.

Unless otherwise specified, implementation specific criteria are used in determining whether an optional error check is performed.

124. If the severity is fatal, the error is not an Advisory Non-Fatal Error, and must be signaled (if enabled) with ERR_FATAL.

6.2.4 Error Logging §

§ Section 6.2.7 lists all the errors governed by this specification and for each error, the logging requirements are specified. Device Functions that do not support the Advanced Error Reporting Extended Capability log only the Device Status register bits indicating that an error has been detected. Note that some errors are also reported using the reporting mechanisms in the PCI-compatible (Type 00h and 01h) configuration registers. § Section 7.5.1 describes how these register bits are affected by the different types of error conditions described in this section.

For device Functions supporting the Advanced Error Reporting Extended Capability, each of the errors in § Table 6-3, § Table 6-4, and § Table 6-5 corresponds to a particular bit in the Uncorrectable Error Status register or Correctable Error Status register. These registers are used by software to determine more precisely which error and what severity occurred. For specific Transaction Layer errors and Uncorrectable Internal Errors, the associated TLP header is recorded.

In a Multi-Function Device other than an SR-IOV Device or an SIOV Device, PCI Express errors not specific to any single Function within the device must be logged in the corresponding status and logging registers of all Functions in that device.

In an SR-IOV Device or an SIOV Device, errors identified as non-Function-specific must be logged in all PFs. Such errors must also be logged in any non-IOV Functions. Non-Function-specific errors must not be logged in VFs.

The following PCI Express errors are not Function-specific:

- All Physical Layer errors
- All Data Link Layer errors
- These Transaction Layer errors:
 - ECRC Check Failed
 - Unsupported Request, when caused by no Function claiming a TLP
 - Receiver Overflow
 - Flow Control Protocol Error
 - Malformed TLP
 - Unexpected Completion, when caused by no Function claiming a Completion
 - Unexpected Completion, when caused by a Completion that cannot be forwarded by a Switch, and the Ingress Port is a Switch Upstream Port associated with a Multi-Function Device
 - Some Transaction Layer errors (e.g., Poisoned TLP Received) may be Function-specific or not, depending upon whether the associated TLP targets a single Function or all Functions in that device.
- Some Internal Errors
 - The determination of whether an Internal Error is Function-specific or not is implementation specific.

On the detection of one of these errors, a Multi-Function Device should generate at most one error reporting Message of a given severity, where the Message must report the Requester ID of a Function of the device that is enabled to report that specific type of error. If no Function is enabled to send a reporting Message, the device does not send a reporting Message. If all reporting-enabled Functions have the same severity level set for the error, only one error Message is sent. If all reporting-enabled Functions do not have the same severity level set for the error, one error Message for each severity level is sent. Software is responsible for scanning all Functions in a Multi-Function Device when it detects one of those errors.

If an SIOV Device generates the Message for an error that is attributable to an SDI, the error Message must contain the RID of the SDI. Errors attributable to a specific SDI may optionally be handled through per-SDI mechanisms that are outside the scope of this specification.

6.2.4.1 Root Complex Considerations (Advanced Error Reporting) §

6.2.4.1.1 Error Source Identification §

In addition to the above logging, a Root Port or Root Complex Event Collector that supports the Advanced Error Reporting Extended Capability is required to implement the Error Source Identification register, which records the Requester ID of the first ERR_NONFATAL/ERR_FATAL (uncorrectable errors) and applicable ¹²⁵ ERR_COR (correctable errors) Messages received by the Root Port or Root Complex Event Collector. System software written to support Advanced Error Reporting can use the Root Error Status register to determine which fields hold valid information.

If an RCiEP is associated with a Root Complex Event Collector, the RCiEP must report its errors through that Root Complex Event Collector.

For both Root Ports and Root Complex Event Collectors, in order for a received error Message or an internally generated error Message to be recorded in the Root Error Status register and the Error Source Identification register, the error Message must be “transmitted”. Refer to § Section 6.2.8.1 for information on how received Messages are forwarded and transmitted. Internally generated error Messages are enabled for transmission with the SERR# Enable bit in the Command register (ERR_NONFATAL and ERR_FATAL) or the Reporting Enable bits in the Device Control register (ERR_COR, ERR_NONFATAL, and ERR_FATAL).

6.2.4.1.2 Interrupt Generation §

The Root Error Command register allows further control of Root Complex response to Correctable, Non-Fatal, and Fatal error Messages than the basic Root Complex capability to generate system errors in response to error Messages. Bit fields enable or disable generation of interrupts for the three types of error Messages. System error generation in response to error Messages may be disabled via the PCI Express Capability structure.

If a Root Port or Root Complex Event Collector is enabled for level-triggered interrupt signaling using the INTx messages, the virtual INTx wire must be asserted whenever and as long as all of the following conditions are satisfied:

- The Interrupt Disable bit in the Command register is set to 0b.
- At least one Error Reporting Enable bit in the Root Error Command register and its associated error Messages Received bit in the Root Error Status register are both set to 1b.

Note that all other interrupt sources within the same Function will assert the same virtual INTx wire when requesting service.

If a Root Port or Root Complex Event Collector is enabled for edge-triggered interrupt signaling using MSI or MSI-X, an interrupt message must be sent every time the logical AND of the following conditions transitions from FALSE to TRUE:

- The associated vector is unmasked (not applicable if MSI does not support PVM).
- At least one Error Reporting Enable bit in the Root Error Command register and its associated error Messages Received bit in the Root Error Status register are both set to 1b.

Note that Advanced Error Reporting MSI/MSI-X interrupts always use the vector indicated by the Advanced Error Interrupt Message Number field in the Root Error Status register.

125. To see which ERR_COR Messages are not applicable for this section, see § Section 6.2.4.1.3.

6.2.4.1.3 RC ECS Handling §

If an RP or RCEC supports RC ECS Handling as indicated by the RC ECS Handling Capable bit being Set, the Function can control individually whether each of the three ECS subclasses signals SFW versus the OS.

There are three enable bits for directing ERR_CORs to be handled by SFW, one for each of the ECS subclasses (ECS SIG_SFW, ECS SIG_OS, and ECS Legacy). In each case, if the enable bit is Set, that subclass of ERR_COR Messages signals SFW via a system-specific mechanism; otherwise, that subclass of ERR_COR Messages signals the OS using architected interrupts. The specific bits are ECS SIG_SFW Handling by SFW Enable, ECS SIG_OS Handling by SFW Enable, and ECS Legacy Handling by SFW Enable.

The default is for all three enable bits to be Clear, resulting in behavior identical to RPs or RCECs that do not support RC ECS Handling.

To enable the handling of ERR_COR Messages by SFW and the OS concurrently and independently, RC ECS Handling defines different resources to be used by ERR_COR Messages being handled by SFW. These include the SFW Handling ERR_COR Received bit, the Multiple SFW Handling ERR_COR Received bit, and the SFW Handling Source ID field. These resources correspond to similar resources used for handling interrupts to the OS. See the specific resources for details on their operation.

6.2.4.2 Multiple Error Handling (Advanced Error Reporting Extended Capability) §

For the Advanced Error Reporting Extended Capability, the Uncorrectable Error Status register and Correctable Error Status register accumulate the collection of errors which correspond to that particular PCI Express interface. The bits remain set until explicitly cleared by software or reset. Since multiple bits might be set in the Uncorrectable Error Status register, the First Error Pointer (when valid) points to the oldest uncorrectable error that is recorded. The First Error Pointer is valid when the corresponding bit of the Uncorrectable Error Status register is set. The First Error Pointer is invalid when the corresponding bit of the Uncorrectable Error Status register is not set, or is an undefined bit.

The Advanced Error Reporting Extended Capability provides the ability to record headers¹²⁶ for errors that require header logging. An implementation may support the recording of multiple headers, but at a minimum must support the ability of recording at least one. The ability to record multiple headers is indicated by the state of the Multiple Header Recording Capable bit and enabled by the Multiple Header Recording Enable bit of the Advanced Error Capabilities and Control register. When multiple header recording is supported and enabled, errors are recorded in the order in which they are detected.

If no header recording resources are available when an unmasked uncorrectable error is detected, its error status bit is Set, but the error is not recorded. If an uncorrectable error is masked when it is detected, its error status bit is Set, but the error is not recorded.

When software is ready to dismiss a recorded error indicated by the First Error Pointer, software writes a 1b to the indicated error status bit to clear it, which causes hardware to free up the associated recording resources. If another instance of that error is still recorded, hardware is permitted but not required to leave that error status bit set. If any error instance is still recorded, hardware must immediately update the Header Log, TLP Prefix Log, TLP Prefix Log Present bit, First Error Pointer, and Uncorrectable Error Status register to reflect the next recorded error. If no other error is recorded, it is recommended that hardware update the First Error Pointer to indicate a status bit that it will never set, e.g., a Reserved status bit. See the Implementation Note below.

If multiple header recording is supported and enabled, and the First Error Pointer is valid, it is recommended that software not write a 1b to any status bit other than the one indicated by the First Error Pointer¹²⁷. If software writes a 1b to such non-indicated bits, hardware is permitted to clear any associated recorded errors, but is not required to do so.

126. If a Function supports TLP Prefixes, then its AER Extended Capability also records any accompanying TLP Prefix along with each recorded header. References to header recording also imply TLP Prefix recording.

If software observes that the First Error Pointer is invalid, and software wishes to clear any unmasked status bits that were set because of earlier header recording resource overflow, software should be aware of the following race condition. If any new instances of those errors happen to be recorded before software clears those status bits, one or more of the newly recorded errors might be lost.

If multiple header recording is supported and enabled, software must use special care when clearing the Multiple Header Recording Enable bit. Hardware behavior is undefined if software clears that bit while the First Error Pointer is valid. Before clearing the Multiple Header Recording Enable bit, it is recommended that software temporarily mask all uncorrectable errors, and then repetitively dismiss each error indicated by the First Error Pointer.

Since an implementation only has the ability to record a finite number of headers, it is important that software services the First Error Pointer, Header Log, and TLP Prefix Log registers in a timely manner, to limit the risk of missing this information for subsequent errors. A Header Log Overflow occurs when an error that requires header logging is detected and either the number of recorded headers supported by an implementation has been reached, or the Multiple Header Recording Enable bit is not Set and the First Error Pointer is valid.

Implementations may optionally check for this condition and report a Header Log Overflow error. This is a reported error associated with the detecting Function.

The setting of Multiple Header Recording Capable and the checking for Header Log Overflow are independently optional.

127. Status bits for masked errors are an exception. Software can safely clear them if software is certain that they have no recorded headers, as would be the case if they have remained masked since the First Error Pointer was last invalid.

IMPLEMENTATION NOTE:

FIRST ERROR POINTER REGISTER BEING VALID §

The First Error Pointer (FEP) field is defined to be valid when the corresponding bit of the Uncorrectable Error Status register is set. To avoid ambiguity with certain cases, the following is recommended:

- After an uncorrectable error has been recorded, when the associated bit in the Uncorrectable Error Status register is cleared by software writing a 1b to it, hardware should update the FEP to point to a status bit that it will never set, e.g., a Reserved status bit. (This assumes that the Function does not already have another recorded error to report, as could be the case if it supports multiple header recording.)
- The default value for the FEP should point to a status bit that hardware will never set, e.g., a Reserved status bit.

Here is an example case of ambiguity with Unsupported Request (UR) if the above recommendations are not followed:

- UR and Advisory Non-Fatal Error are unmasked while system firmware does its Configuration Space probing.
- The Function encounters a UR due to normal probing, logs it, and sets the FEP to point to UR.
- System firmware clears the UR Status bit, and hardware leaves the FEP pointing to UR.
- After the operating system has booted, it masks UR.
- Normal probing sets the UR Status bit, but the error is not recorded since UR is masked.

At this point, there's the ambiguity of the FEP pointing to a status bit that is set (thus being valid), when in fact, there is no recorded error that needs to be processed by software.

If hardware relies on this definition of the FEP being valid to determine when it's possible to record a new error, the Function can fail to record new unmasked errors, falsely determining that it has no available recording resources. Hardware implementations that rely on other internal state to determine when it's possible to record a new error might not have this problem; however, hardware implementations should still follow the above recommendations to avoid presenting this ambiguity to software.

6.2.4.2.1 Multiple Error Handling in VFs §

Header Log space for a PF is independent of that for its associated VFs, and must be implemented with dedicated storage space.

VFs that implement AER may share Header Log space among all the VFs associated with a single PF. Especially when sharing Header Log space, VFs may not have room to log a header associated with an error. In this case, the Function must update the Uncorrectable Error Status Register and Advanced Error Capabilities and Control register as required by § Section 6.2.4.2 ; however, when the Header Log Register is read, it must return all 1s to indicate an overflow condition and that no header was logged. If Flit Mode Supported in the PCI Express Capabilities Register is Set, in addition to returning all 1s, the associated Logged TLP Size field must contain 0.

The VF's header log entry shall be locked and remain valid while that VF's First Error Pointer is valid. As defined in § Section 6.2.4.2 , the First Error Pointer register is valid when the corresponding bit of the Uncorrectable Error Status

register is Set. While the header log entry is locked, additional errors shall not overwrite the locked entry for this or any other VF. When a header entry is unlocked, it shall be available to record a new error for any VF sharing the header logs.

6.2.4.3 Advisory Non-Fatal Error Logging §

§ Section 6.2.3.2.4 describes Advisory Non-Fatal Error cases, under which an agent with AER detecting an uncorrectable error of non-fatal severity signals the error (if enabled) using ERR_COR instead of ERR_NONFATAL. For the same cases, an agent without AER sends no error Message. The remaining discussion in this section is in the context of agents that do implement AER.

For Advisory Non-Fatal Error cases, since an uncorrectable error is signaled using the correctable error Message, control/status/mask bits involving both uncorrectable and correctable errors apply. § Figure 6-2 shows a flowchart of the sequence. Following are some of the unique aspects for logging Advisory Non-Fatal Errors.

First, the uncorrectable error needs to be of severity non-fatal, as determined by the associated bit in the Uncorrectable Error Severity register. If the severity is fatal, the error does not qualify as an Advisory Non-Fatal Error, and will be signaled (if enabled) with ERR_FATAL.

Next, the specific error case needs to be one of the Advisory Non-Fatal Error cases documented in § Section 6.2.3.2.4 . If not, the error does not qualify as an Advisory Non-Fatal Error, and will be signaled (if enabled) with an uncorrectable error Message.

Next, the Advisory Non-Fatal Error Status bit is Set in the Correctable Error Status register to indicate the occurrence of the advisory error, and the Advisory Non-Fatal Error Mask bit in the Correctable Error Mask register is checked, and, if set, no further processing is done.

If the Advisory Non-Fatal Error Mask bit is clear, logging proceeds by setting the “corresponding” bit in the Uncorrectable Error Status register, based upon the specific uncorrectable error that’s being reported as an advisory error. If the “corresponding” uncorrectable error bit in the Uncorrectable Error Mask register is clear and the error is one that requires header logging, then the prefix and header are recorded, subject to the availability of resources. See § Section 6.2.4.2 .

Finally, an ERR_COR Message is sent if the Correctable Error Reporting Enable bit is Set in the Device Control Register.

6.2.4.4 End-End TLP Prefix Logging - Non-Flit Mode §

For any device Function that supports both TLP Prefixes and Advanced Error Reporting the TLP Prefixes associated with the TLP in error are recorded in the TLP Prefix Log register according to the same rules as the Header Log register (such that both the TLP Prefix Log and Header Log registers always correspond to the error indicated in the First Error Pointer, when the First Error Pointer is valid).

The TLP Prefix Log Present bit (see § Section 7.8.4.7) indicates that the TLP Prefix Log register (see § Section 7.8.4.12) contains information.

Only End-End TLP Prefixes are logged by AER. Logging of Local TLP Prefixes may occur elsewhere using prefix-specific mechanisms.

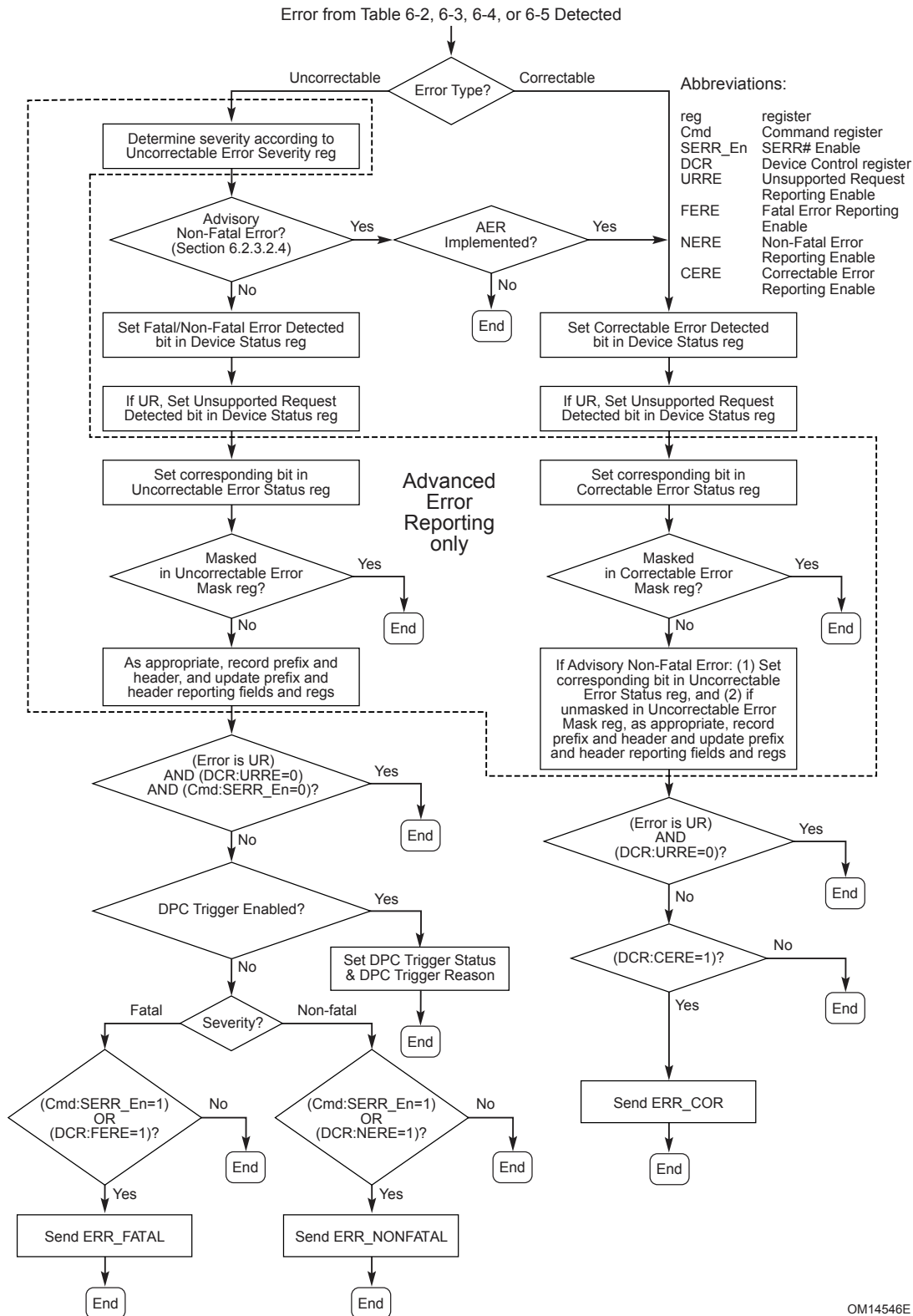
End-End TLP Prefixes are logged in the TLP Prefix Log register. The underlying TLP Header is logged in the Header Log register subject to two exceptions:

- If the Extended Fmt Field Supported bit is Set (see § Section 7.5.3.15), a Function that does not support End-End TLP Prefixes and receives a TLP containing an End-End TLP Prefix must signal Malformed TLP, and the Header Log register must contain the first four DWs of the TLP (End-End TLP Prefixes followed by as much of the TLP Header as will fit).

- A Function that receives a TLP containing more End-End TLP Prefixes than are indicated by the Function's Max End-End TLP Prefixes field must handle the TLP as an error (see § Section 2.2.10.4 for specifics) and store the first overflow End-End TLP Prefix in the 1st DW of the Header Log register with the remainder of the Header Log register being undefined.

6.2.5 Sequence of Device Error Signaling and Logging Operations §

§ Figure 6-2 shows the sequence of operations related to signaling and logging of errors detected by a device.

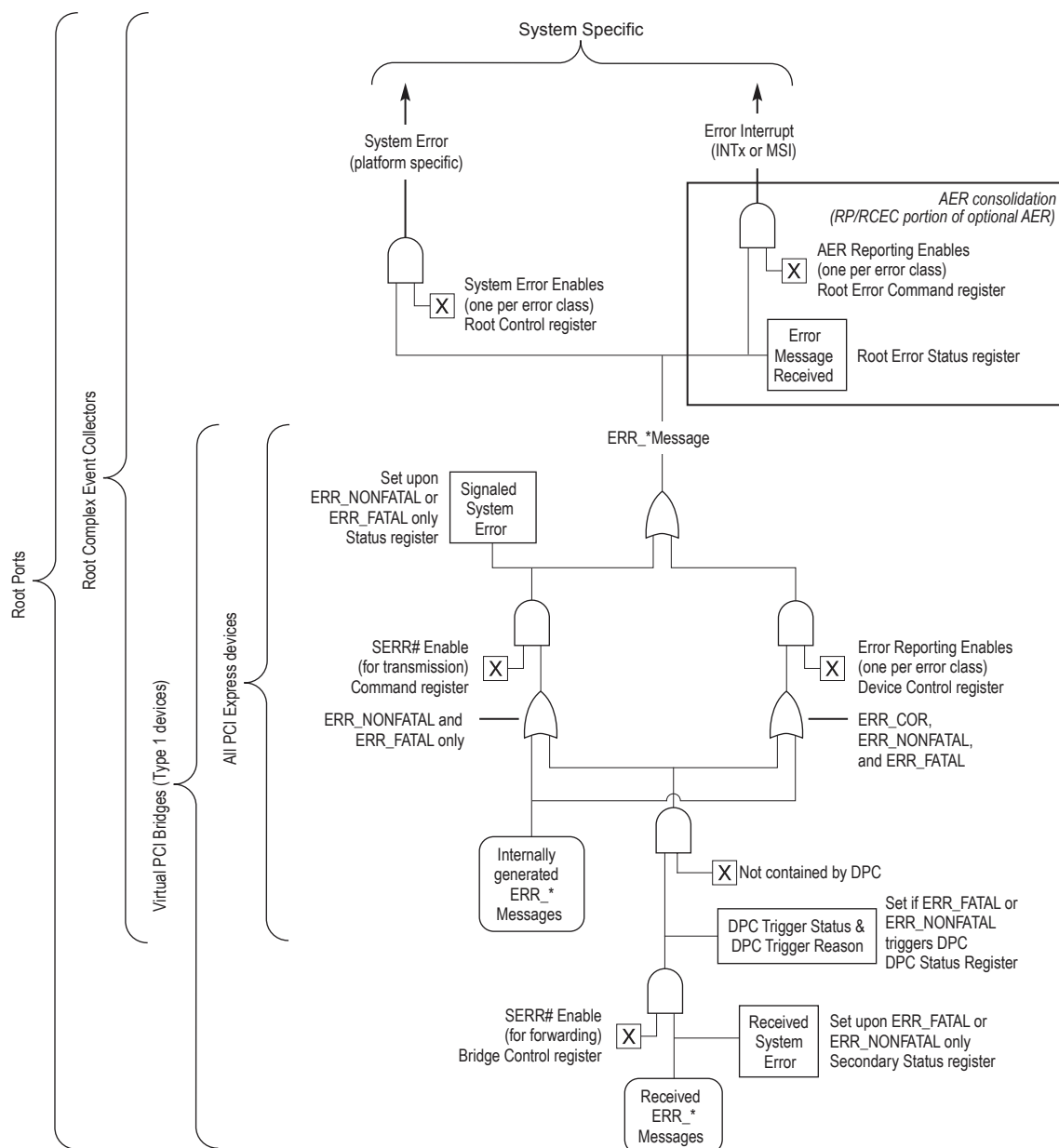


OM14546E

Figure 6-2 Flowchart Showing Sequence of Device Error Signaling and Logging Operations §

6.2.6 Error Message Controls §

Error Messages have a complex set of associated control and status bits. § Figure 6-3 provides a high-level summary in the form of a pseudo logic diagram for how error Messages are generated, logged, forwarded, and ultimately notified to the system. Not all control and status bits are shown. The logic gates shown in this diagram are intended for conveying general concepts, and not for direct implementation.



A-0479B

Figure 6-3 Pseudo Logic Diagram for Selected Error Message Control and Status Bits §

6.2.7 Error Listing and Rules §

§ Table 6-2 through § Table 6-4 list all of the PCI Express errors that are defined by this specification. Each error is listed with a short-hand name, how the error is detected in hardware, the default severity of the error, and the expected action taken by the agent which detects the error. These actions form the rules for PCI Express error reporting and logging.

The Default Severity column specifies the default severity for the error without any software reprogramming. For device Functions supporting the Advanced Error Reporting Extended Capability, the uncorrectable errors are programmable to Fatal or Non-fatal with the Error Severity register. Device Functions without Advanced Error Reporting Extended Capability use the default associations and are not reprogrammable.

The detecting agent action for Downstream Ports that implement Downstream Port Containment (DPC) and have it enabled will be different if the error triggers DPC. DPC behavior is not described in the following tables. See § Section 6.2.11 for the description of DPC behavior.

Table 6-2 General PCI Express Error List §

Error Name	Error Type (Default Severity)	Detecting Agent Action ¹²⁸	References
Corrected Internal Error	Correctable (masked by default)	<i>Component:</i> Send ERR_COR to Root Complex.	§ Section 6.2.10
Uncorrectable Internal Error	Uncorrectable (Fatal and masked by default)	<i>Component:</i> Send ERR_FATAL to Root Complex. Optionally, log the prefix/header of the first TLP associated with the error.	§ Section 6.2.10
Header Log Overflow	Correctable (masked by default)	<i>Component:</i> Send ERR_COR to Root Complex.	§ Section 6.2.4.2

Table 6-3 Physical Layer Error List §

Error Name	Error Type (Default Severity)	Detecting Agent Action ¹²⁹	References
Receiver Error	Correctable	<i>Receiver:</i> Send ERR_COR to Root Complex.	§ Section 4.2.1.1.3 § Section 4.2.1.2 § Section 4.2.5.8 § Section 4.2.7

Table 6-4 Data Link Layer Error List §

Error Name	Error Type (Default Severity)	Detecting Agent Action ¹³⁰	References
Bad TLP	Correctable	<i>Receiver:</i> Send ERR_COR to Root Complex.	§ Section 3.6.3.1
Bad DLLP		<i>Receiver:</i> Send ERR_COR to Root Complex.	§ Section 3.6.2.2 , and § Section 3.6.2.3

128. For these tables, detecting agent action is given as if all enable bits are set to “enable” and, for Advanced Error Handling, mask bits are disabled and severity bits are set to their default values. Actions must be modified according to the actual settings of these bits.

129. For these tables, detecting agent action is given as if all enable bits are set to “enable” and, for Advanced Error Handling, mask bits are disabled and severity bits are set to their default values. Actions must be modified according to the actual settings of these bits.

130. For these tables, detecting agent action is given as if all enable bits are set to “enable” and, for Advanced Error Handling, mask bits are disabled and severity bits are set to their default values. Actions must be modified according to the actual settings of these bits.

Error Name	Error Type (Default Severity)	Detecting Agent Action	References
Replay Timer Timeout	<i>Transmitter:</i> Send ERR_COR to Root Complex.	<i>Transmitter:</i> Send ERR_COR to Root Complex.	§ Section 3.6.2.1 , § Section 4.2.3.4.2.1
REPLAY_NUM Rollover		§ Section 3.6.2.1 , § Section 4.2.3.4.2.1	
Data Link Protocol Error	Uncorrectable (Fatal)	If checking, send ERR_FATAL to Root Complex.	§ Section 3.6.2.2 , § Section 3.6.2.3 , § Section 4.2.3.4.2.1
Surprise Down		If checking, send ERR_FATAL to Root Complex.	§ Section 3.2.1

Table 6-5 Transaction Layer Error List [§]

Error Name	Error Type (Default Severity)	Detecting Agent Action ¹³¹	References
Poisoned TLP Received	Uncorrectable (Non-Fatal)	<i>Receiver:</i> Send ERR_NONFATAL to Root Complex or ERR_COR for the Advisory Non-Fatal Error cases described in § Section 6.2.3.2.4.2 and § Section 6.2.3.2.4.3 . Log the prefix/header of the Poisoned TLP. ¹³²	§ Section 2.7.2.1
Poisoned TLP Egress Blocked		<i>Downstream Port Transmitter:</i> Send ERR_NONFATAL to Root Complex or ERR_COR for the Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.1 . Log the prefix/header of the poisoned TLP.	§ Section 2.7.2.1
ECRC Check Failed		<i>Receiver (if ECRC checking is supported):</i> - Send ERR_NONFATAL to Root Complex or ERR_COR for the Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.1 and § Section 6.2.3.2.4.2 . - Log the prefix/header of the TLP that	§ Section 2.7.1

131. For these tables, detecting agent action is given as if all enable bits are set to “enable” and, for Advanced Error Handling, mask bits are disabled and severity bits are set to their default values. Actions must be modified according to the actual settings of these bits.

Error Name	Error Type (Default Severity)	Detecting Agent Action	References
		encountered the ECRC error.	
Unsupported Request (UR)	Uncorrectable (Non-Fatal)	<i>Request Receiver:</i> Send <u>ERR_NONFATAL</u> to Root Complex or <u>ERR_COR</u> for the Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.1 . Log the prefix/header of the TLP that caused the error.	§ Table F-1, § Section 2.3.1 , § Section 2.3.2 , § Section 2.7.2.1 , § Section 2.9.1 , § Section 5.3.1 , § Section 6.2.3.1 , § Section 6.2.6 , § Section 6.2.8.1 , § Section 6.5.7 , § Section 7.3.1 , § Section 7.3.3 , § Section 7.5.1.1.3 , § Section 7.5.1.1.4
Completion Timeout	Uncorrectable (Non-Fatal)	<i>Requester:</i> Send <u>ERR_NONFATAL</u> to Root Complex or <u>ERR_COR</u> for the Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.4 . If the Completion Timeout Prefix/Header Log Capable bit is Set in the Advanced Error Capabilities and Control register, log the prefix/header of the Request TLP that encountered the error.	§ Section 2.8
Completer Abort		<i>Completer:</i> Send <u>ERR_NONFATAL</u> to Root Complex or <u>ERR_COR</u> for the Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.1 . Log the prefix/header of the Request that encountered the error.	§ Section 2.3.1
Unexpected Completion		<i>Receiver:</i> Send <u>ERR_COR</u> to Root Complex. This is an Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.5 . Log the prefix/header of the Completion that encountered the error.	§ Section 2.3.2
ACS Violation		<i>Receiver (if checking):</i> Send <u>ERR_NONFATAL</u> to Root Complex or <u>ERR_COR</u> for the	

Error Name	Error Type (Default Severity)	Detecting Agent Action	References
		Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.1 . Log the prefix/header of the Request TLP that encountered the error.	
MC Blocked TLP		<i>Receiver (if checking):</i> Send <u>ERR_NONFATAL</u> to Root Complex. Log the prefix/header of the Request TLP that encountered the error.	<u>§ Section 6.14.4</u>
AtomicOp Egress Blocked	Uncorrectable (Non-Fatal)	<i>Egress Port:</i> Send <u>ERR_COR</u> to Root Complex. This is an Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.1 . Log the prefix/header of the AtomicOp Request that encountered the error.	<u>§ Section 6.15.2</u>
DMWr Request Egress Blocked		<i>Egress Port:</i> Send <u>ERR_COR</u> to Root Complex. This is an Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.1 . Log the prefix/header of the DMWr Request that encountered the error.	<u>§ Section 6.32</u>
TLP Translation Egress Blocked		<i>Egress Port:</i> For error signaling, handle PR/NPR/CPL FC Types as specified in § Section 2.2.1.2 . For error logging, handle PR/NPR/CPL FC Types as specified in § Section 2.2.1.2 . Log the prefix/header of the Flit Mode TLP that was not possible to translate to send in Non-Flit Mode on the Egress Port.	<u>§ Section 2.2.1.2</u>
TLP Prefix Blocked		<i>Egress Port:</i> Send <u>ERR_NONFATAL</u> to Root Complex or <u>ERR_COR</u> for the Advisory Non-Fatal Error case described in § Section 6.2.3.2.4.1 . Log the prefix/	<u>§ Section 2.2.10.4</u>

Error Name	Error Type (Default Severity)	Detecting Agent Action	References
		header of the TLP that encountered the error.	
Receiver Overflow	Uncorrectable (Fatal)	<i>Receiver (if checking):</i> Send ERR_FATAL to Root Complex.	<u>§ Section 2.6.1.2</u>
Flow Control Protocol Error		<i>Receiver (if checking):</i> Send ERR_FATAL to Root Complex.	<u>§ Section 2.6.1</u>
Malformed TLP		<i>Receiver:</i> Send ERR_FATAL to Root Complex. Log the prefix/header of the TLP that encountered the error.	<u>§ Section 2.2.2</u> , <u>§ Section 2.2.3</u> , <u>§ Section 2.2.5</u> , <u>§ Section 2.2.7</u> , <u>§ Section 2.2.8.1</u> , <u>§ Section 2.2.8.2</u> , <u>§ Section 2.2.8.3</u> , <u>§ Section 2.2.8.4</u> , <u>§ Section 2.2.8.5</u> , <u>§ Section 2.2.8.10</u> , <u>§ Section 2.2.10</u> , <u>§ Section 2.2.10.2</u> , <u>§ Section 2.2.10.4</u> , <u>§ Section 2.3</u> , <u>§ Section 2.3.1</u> , <u>§ Section 2.3.1.1</u> , <u>§ Section 2.3.2</u> , <u>§ Section 2.5</u> , <u>§ Section 2.5.3</u> , <u>§ Section 2.6.1</u> , <u>§ Section 2.6.1.2</u> , <u>§ Section 6.2.4.4</u> , <u>§ Section 6.3.2</u>
IDE Check Failed		<i>Receiving IDE Terminus:</i> Enter Insecure All subsequent IDE TLPs over this stream treated as having MAC check failure Send ERR_FATAL to root complex. Transmit IDE Fail Message to the Partner Port Log the prefix/header of the TLP that encountered the error	<u>§ Section 6.33</u>
Misrouted IDE TLP	Uncorrectable (Non-Fatal)	<i>Ingress/Egress Port:</i> Send ERR_NONFATAL to Root Complex. Log the prefix/header of the TLP that encountered the error	<u>§ Section 6.33</u>
PCRC Check Failed		<i>Receiving IDE Terminus:</i> Send ERR_NONFATAL to Root Complex or ERR_COR for the Advisory Non-Fatal Error cases described in <u>§ Section 6.2.3.2.4.3</u> . Log the prefix/header of the TLP that encountered the error	<u>§ Section 6.33</u>

For all errors listed above, the appropriate status bit(s) must be set upon detection of the error. For Unsupported Request (UR), additional detection and reporting enable bits apply (see § Section 6.2.5).

IMPLEMENTATION NOTE: DEVICE UR REPORTING COMPATIBILITY WITH LEGACY AND 1.0A SOFTWARE §

With [PCIe-1.0a] device Functions that do not implement Role-Based Error Reporting,¹³³ the Unsupported Request Reporting Enable bit in the Device Control Register, when clear, prevents the Function from sending any error Message to signal a UR error. With Role-Based Error Reporting Functions, if the SERR# Enable bit in the Command Register is set, the Function is implicitly enabled¹³⁴ to send ERR_NONFATAL or ERR_FATAL messages to signal UR errors, even if the Unsupported Request Reporting Enable bit is clear. This raises a backward compatibility concern with software (or firmware) written for [PCIe-1.0a] devices.

With software/firmware that sets the SERR# Enable bit but leaves the Unsupported Request Reporting Enable and Correctable Error Reporting Enable bits clear, a Role-Based Error Reporting Function that encounters a UR error will send no error Message if the Request was non-posted, and will signal the error with ERR_NONFATAL if the Request was posted. The behavior with non-posted Requests supports PC-compatible Configuration Space probing, while the behavior with posted Requests restores error reporting compatibility with PCI and PCI-X, avoiding the potential in this area for silent data corruption. Thus, Role-Based Error Reporting devices are backward compatible with envisioned legacy and [PCIe-1.0a] software and firmware.

6.2.7.1 Conventional PCI Mapping §

In order to support conventional PCI driver and software compatibility, PCI Express error conditions, where appropriate, must be mapped onto the PCI Status register bits for error reporting.

In other words, when certain PCI Express errors are detected, the appropriate PCI Status register bit is Set alerting the error to legacy PCI software. While the PCI Express error results in setting the PCI Status register, clearing the PCI Status register will not result in clearing bits in the Uncorrectable Error Status register and Correctable Error Status register. Similarly, clearing bits in the Uncorrectable Error Status register and Correctable Error Status register will not result in clearing the PCI Status register.

The PCI command register has bits which control PCI error reporting. However, the PCI Command register does not affect the setting of the PCI Express error register bits.

6.2.8 Virtual PCI Bridge Error Handling §

Virtual PCI Bridge configuration headers are associated with each PCI Express Port in a Root Complex or a Switch. For these cases, PCI Express error concepts require appropriate mapping to the PCI error reporting structures.

133. As indicated by the Role-Based Error Reporting bit in the Device Capabilities register. See § Section 7.8.3 .

134. Assuming the Unsupported Request Error Mask bit is not set in the Uncorrectable Error Mask Register if the device implements AER.

6.2.8.1 Error Message Forwarding and PCI Mapping for Bridge - Rules §

In general, a TLP is either passed from one side of the Virtual PCI Bridge to the other, or is handled at the ingress side of the Bridge according to the same rules which apply to the ultimate recipient of a TLP. The following rules cover PCI Express specific error related cases. Refer to § Section 6.2.6 for a conceptual summary of Error Message Controls.

- If a Request does not address a space mapped to either the Bridge's internal space, or to the egress side of the Bridge, the Request is terminated at the ingress side as an Unsupported Request
- Poisoned TLPs are forwarded according to the same rules as non-Poisoned TLPs
 - When forwarding a Poisoned Request Downstream:
 - Set the Detected Parity Error bit in the Status register
 - Set the Master Data Parity Error bit in the Secondary Status register if the Parity Error Response Enable bit in the Bridge Control Register is set
 - When forwarding a Poisoned Completion Downstream:
 - Set the Detected Parity Error bit in the Status register
 - Set the Master Data Parity Error bit in the Status register if the Parity Error Response bit in the Command register is set
 - When forwarding a Poisoned Request Upstream:
 - Set the Detected Parity Error bit in the Secondary Status register
 - Set the Master Data Parity Error bit in the Status register if the Parity Error Response bit in the Command register is set
 - When forwarding a Poisoned Completion Upstream:
 - Set the Detected Parity Error bit in the Secondary Status register
 - Set the Master Data Parity Error bit in the Secondary Status register if the Parity Error Response Enable bit in the Bridge Control Register is set
- ERR_COR, ERR_NONFATAL, and ERR_FATAL are forwarded from the secondary interface to the primary interface, if the SERR# Enable bit in the Bridge Control Register is set. A Bridge forwarding an error Message must not set the corresponding Error Detected bit in the Device Status register. Transmission of forwarded error Messages by the primary interface is controlled by multiple bits, as shown in § Figure 6-3.
- For a Root Port, error Messages forwarded from the secondary interface to the primary interface must be enabled for “transmission” by the primary interface in order to cause a System Error via the Root Control Register or (when the Advanced Error Reporting Extended Capability is present) reporting via the Root Error Command register and logging in the Root Error Status register and Error Source Identification register.
- For a Root Complex Event Collector (technically not a Bridge), error Messages “received” from associated RCiEPs must be enabled for “transmission” in order to cause a System Error via the Root Control Register or (when the Advanced Error Reporting Extended Capability is present) reporting via the Root Error Command register and logging in the Root Error Status register and Error Source Identification register.

6.2.9 SR-IOV Baseline Error Handling §

All SR-IOV Devices must support baseline capability, with certain modifications to account for the goal of reduced cost and complexity of implementation.

The following error handling control bits are only implemented in the PF. They are RsvdP in VFs, and VFs must use the control bits in their associated PF for managing their error handling behavior.

- Command register (see § Section 7.5.1.1.3)
 - SERR# Enable
 - Parity Error Response
- Device Control register (see § Section 7.5.3.4)
 - Correctable Reporting Enable
 - Non-Fatal Reporting Enable
 - Fatal Reporting Enable
 - Unsupported Request (UR) Reporting Enable

Each VF must report its error status independently of any other Function. This is necessary to provide SI isolation for errors that are Function-specific. The following baseline error handling status bits must be implemented in each VF:

- Status register (see § Section 7.5.1.1.4)
 - Master Data Parity Error
 - Signaled Target Abort
 - Received Target Abort
 - Received Master Abort
 - Signaled System Error
 - Detected Parity Error
- Device Status register (see § Section 7.5.3.5)
 - Correctable Error Detected
 - Non-Fatal Error Detected
 - Fatal Error Detected
 - Unsupported Request Detected

Each VF must use its own Routing ID when signaling errors.

6.2.10 Internal Errors §

An Internal Error is an error associated with a PCI Express interface that occurs within a component and which may not be attributable to a packet or event on the PCI Express interface itself or on behalf of transactions initiated on PCI Express. The determination of what is considered an Internal Error is implementation specific and is outside the scope of this specification.

Internal Errors may be classified as Corrected Internal Errors or Uncorrectable Internal Errors. A Corrected Internal Error is an error that occurs within a component that has been masked or worked around by hardware without any loss of information or improper operation. An example of a possible Corrected Internal Error is an internal packet buffer memory error corrected by an Error Correcting Code (ECC). An Uncorrectable Internal Error is an error that occurs within a component that results in improper operation of the component. An example of a possible Uncorrectable Internal Error is a memory error that cannot be corrected by an ECC. The only method of recovering from an Uncorrectable Internal Error is reset or hardware replacement.

Reporting of Corrected Internal Errors and Uncorrectable Internal Errors is independently optional. If either is reported, then AER must be implemented.

Header logging is optional for Uncorrectable Internal Errors. When a header is logged, the header is that of the first TLP that was lost or corrupted by the Uncorrectable Internal Error. When header logging is not implemented or a header is not available, a header of all ones is recorded.

Internal Errors that can be associated with a specific PCI Express interface are reported by the Function(s) associated with that Port. Internal Errors detected within Switches that cannot be associated with a specific PCI Express interface are reported by the Upstream Port. Reporting of Internal Errors that cannot be associated with a specific PCI Express interface in all other multi-Port components (e.g., Root Complexes) is outside the scope of this specification.

6.2.11 Downstream Port Containment (DPC) §

Downstream Port Containment (DPC) is an optional normative feature of a Downstream Port. DPC halts PCI Express traffic below a Downstream Port after an unmasked uncorrectable error is detected at or below the Port, avoiding the potential spread of any data corruption, and supporting Containment Error Recovery (CER) if implemented by software. A Downstream Port indicates support for DPC by implementing a DPC Extended Capability structure, which contains all DPC control and status bits. See § Section 7.9.14 .

DPC is disabled by default, and cannot be triggered unless enabled by software using the DPC Trigger Enable field. When the DPC Trigger Enable field is set to 01b, DPC is enabled and is triggered when the Downstream Port detects an unmasked uncorrectable error or when the Downstream Port receives an ERR_FATAL Message. When the DPC Trigger Enable field is set to 10b, DPC is enabled and is triggered when the Downstream Port detects an unmasked uncorrectable error or when the Downstream Port receives an ERR_NONFATAL or ERR_FATAL Message. In addition to uncorrectable errors of the type managed by the PCI Express Extended Capability and Advanced Error Reporting (AER), RP PIO errors can be handled as uncorrectable errors. See § Section 6.2.11.3 . There is also a mechanism described in § Section 6.2.11.4 for software or firmware to trigger DPC.

When DPC is triggered due to receipt of an uncorrectable error Message, the Requester ID from the Message is recorded in the DPC Error Source ID Register and that Message is discarded and not forwarded Upstream. When DPC is triggered by an unmasked uncorrectable error, that error will not be signaled with an uncorrectable error Message, even if otherwise enabled. However, when DPC is triggered, DPC can signal an interrupt or send an ERR_COR Message if enabled. See § Section 6.2.11.1 and § Section 6.2.11.2 .

When DPC is triggered, the Downstream Port immediately Sets the DPC Trigger Status bit and DPC Trigger Reason field to indicate the triggering condition (unmasked uncorrectable error, ERR_NONFATAL, ERR_FATAL, RP_PIO error, or software triggered), and disables its Link by directing the LTSSM to the Disabled state. Once the LTSSM reaches the Disabled state, it remains in that state until the DPC Trigger Status bit is Cleared. To ensure that the LTSSM has time to reach the Disabled state or at least to bring the Link down under a variety of error conditions, software must leave the Downstream Port in DPC until the Data Link Layer Link Active bit in the Link Status Register reads 0b; otherwise, the result is undefined. See § Section 7.5.3.8 . See § Section 2.9.3 for other important details on Transaction Layer behavior during DPC.

After DPC has been triggered in a Root Port that supports RP Extensions for DPC, the Root Port may require some time to quiesce and clean up its internal activities, such as those associated with DMA read Requests. When the DPC Trigger Status bit is Set and the DPC RP Busy bit is Set, software must leave the Root Port in DPC until the DPC RP Busy bit reads 0b.

After software releases the Downstream Port from DPC, the Port's LTSSM must transition to the Detect state unless the Link is disabled due to other conditions (e.g., the Link Disable bit being Set), where the Link will attempt to retrain. Software can use Data Link Layer State Changed interrupts, DL_ACTIVE_ERR_COR signaling, or both, to signal when the Link reaches the DL_Active state again. See § Section 6.7.3.3 and § Section 6.2.11.5 .

IMPLEMENTATION NOTE: DATA VALUE OF ALL 1'S §

Many platforms, including those supporting RP Extensions for DPC, can return a data value of all 1's to software when an error is associated with a PCI Express Configuration, I/O, or Memory Read Request. During DPC, the Downstream Port discards Requests destined for the Link and completes them with an error (i.e., either with an Unsupported Request (UR) or Completer Abort (CA) Completion Status). By ending a series of MMIO or configuration space operations with a read to an address with a known data value not equal to all 1's, software may determine if a Completer has been removed or DPC has been triggered.

Also see the Implementation Note "Use of RP PIO Advisory Error Handling"

IMPLEMENTATION NOTE: SELECTING A RESPONSE FOR A NON-POSTED REQUEST OR A UIO REQUEST RESPONSE DURING DPC §

The DPC Completion Control bit determines how a Downstream Port responds to a Non-Posted Request (NPR) or a UIO Request received during DPC. The selection needs to take into account how the rest of the platform handles Containment Error Recovery (CER).

While specific CER policy details in a platform are outside the scope of this specification, here are some guidelines based on general considerations.

If the platform or drivers do not support CER policies, it's recommended to select UR Completions, which is the standard behavior when a device is not present.

If the CER strategy relies on software detecting containment by looking for all 1's returned by PIO reads, then a UR Completion may be the more appropriate selection, assuming the RP synthesizes an all 1's return value for PIO reads that return UR Completions. The all 1's synthesis would need to occur for PIO reads that target Configuration Space, Memory Space, and perhaps I/O Space.

If the CER strategy utilizes a mechanism that handles UR and CA Completions differently for PIO reads, then a CA Completion might be the more appropriate selection. CA Completions coming back from a PCIe device normally indicate a device programming model violation, which may need to trigger Port containment and error recovery.

IMPLEMENTATION NOTE:

SELECTING THE DPC TRIGGER CONDITION §

Non-Fatal Errors are uncorrectable errors that indicate that a particular TLP was unreliable, and in general the associated Function should not continue its normal operation. Fatal errors are uncorrectable errors that indicate that a particular Link and its related hardware are unreliable, and in general the entire hierarchy below that Link should not continue normal operation. This distinction between Non-Fatal and Fatal errors together with the Root Port error containment capabilities can sometimes be used to select the appropriate DPC trigger condition. The following assumes that there is no peer-to-peer traffic between devices.

Some RCs implement a proprietary feature that will be referred to generically as “Function Level Containment” (FLC). This is not an architected feature of PCI Express. A Root Port that implements FLC is capable of containing the traffic associated with a specific Function when a Non-Fatal Error is detected in that traffic. Switch Downstream Ports below a Root Port with FLC should be configured to trigger DPC when the Downstream Port detects an unmasked uncorrectable error itself or when the Downstream Port receives an `ERR_FATAL` Message. Under this mode, the Switch Downstream Port passes `ERR_NONFATAL` Messages it receives Upstream without triggering DPC. This enables Root Port FLC to handle Non-Fatal Errors that render a specific Function unreliable and Switch Downstream Port DPC to handle errors that render a subtree of the hierarchy domain unreliable. The Downstream Port still needs to trigger DPC for all unmasked uncorrectable errors it detects, since an `ERR_NONFATAL` it generates will have its own Requester ID, and the FLC hardware in the Root Port would not be able to determine which specific Function below the Switch Downstream Port was responsible for the Non-Fatal Error.

Switch Downstream Ports below a Root Port without FLC should be configured to trigger DPC when the Switch Downstream Port detects an unmasked uncorrectable error or when the Switch Downstream Port receives an `ERR_NONFATAL` or `ERR_FATAL` Message. This enables DPC to contain the error to the affected hierarchy below the Link and allow continued normal operation of the unaffected portion of the hierarchy domain.

IMPLEMENTATION NOTE:

SOFTWARE POLLING THE DPC RP BUSY BIT §

The DPC RP Busy bit is a means for hardware to indicate to software that the RP needs to remain in DPC containment while the RP does some internal cleanup and quiescing activities. While the details of these activities are implementation specific, the activities will typically complete within a few microseconds or less. However, under worst-case conditions such as those that might occur with certain internal errors in large systems, the busy period might extend substantially, possibly into multiple seconds. If software is unable to tolerate such lengthy delays within the current software context, software may need to rely on using timer interrupts to schedule polling under interrupt.

IMPLEMENTATION NOTE: DETERMINATION OF DPC CONTROL §

DPC may be controlled in some configurations by platform firmware and in other configurations by the operating system. DPC functionality is strongly linked with the functionality in Advanced Error Reporting. To avoid conflicts over whether platform firmware or the operating system have control of DPC, it is recommended that platform firmware and operating systems always link the control of DPC to the control of Advanced Error Reporting.

6.2.11.1 DPC Interrupts §

A DPC-capable Downstream Port must support the generation of DPC interrupts. DPC interrupts are enabled by the DPC Interrupt Enable bit in the DPC Control Register. DPC interrupts are indicated by the DPC Interrupt Status bit in the DPC Status Register.

If the Port is enabled for level-triggered interrupt signaling using INTx messages, the virtual INTx wire must be asserted whenever and as long as the following conditions are satisfied:

- The value of the Interrupt Disable bit in the Command register is 0b.
- The value of the DPC Interrupt Enable bit is 1b.
- The value of the DPC Interrupt Status bit is 1b.

Note that all other interrupt sources within the same Function will assert the same virtual INTx wire when requesting service.

If the Port is enabled for edge-triggered interrupt signaling using MSI or MSI-X, an interrupt message must be sent every time the logical AND of the following conditions transitions from FALSE to TRUE:

- The associated vector is unmasked (not applicable if MSI does not support PVM).
- The value of the DPC Interrupt Enable bit is 1b.
- The value of the DPC Interrupt Status bit is 1b.

The Port may optionally send an interrupt message if interrupt generation has been disabled, and the logical AND of the above conditions is TRUE when interrupt generation is subsequently enabled.

The interrupt message will use the vector indicated by the DPC Interrupt Message Number field in the DPC Capability register. This vector may be the same or may be different from the vectors used by other interrupt sources within this Function.

6.2.11.2 DPC ERR_COR Signaling §

A DPC-capable Downstream Port must support ERR_COR signaling, independent of whether it supports Advanced Error Reporting (AER) or not. DPC ERR_COR signaling is enabled by the DPC ERR_COR Enable bit in the DPC Control Register. DPC triggering is indicated by the DPC Trigger Status bit in the DPC Status Register. DPC ERR_COR signaling is managed independently of DPC interrupts, and it is permitted to use both mechanisms concurrently.

If the DPC ERR_COR Enable bit is Set, and the Correctable Error Reporting Enable bit in the Device Control Register or the DPC SIG_SFW Enable bit in the DPC Control Register is Set, the Port must send an ERR_COR Message each time the DPC Trigger Status bit transitions from Clear to Set. DPC ERR_COR signaling must not Set the Correctable Error Detected bit in

the Device Status Register, since this event is not handled as an error. If the Downstream Port supports ERR_COR Subclass capability, this DPC ERR_COR signaling event must set the DPC SIG_SFW Status bit in the DPC Status Register and also set the ERR_COR Subclass field in the ERR_COR Message to indicate ECS SIG_SFW.

For a given DPC trigger event, if a Port is going to send both an ERR_COR Message and an MSI/MSI-X transaction, then the Port must send the ERR_COR Message prior to sending the MSI/MSI-X transaction. There is no corresponding requirement if the INTx mechanism is being used to signal DPC interrupts, since INTx Messages won't necessarily remain ordered with respect to ERR_COR Messages when passing through routing elements.

IMPLEMENTATION NOTE:

USE OF DPC ERR_COR SIGNALING §

It is recommended that operating systems use DPC interrupts for signaling when DPC has been triggered. While DPC ERR_COR signaling indicates the same event, DPC ERR_COR signaling is primarily intended for use by system firmware, when it needs to be notified in order to do its own logging of the event or provide firmware first services.

6.2.11.3 Root Port Programmed I/O (RP PIO) Error Controls §

The RP PIO error control registers enable fine-grained control over what happens when Non-Posted Requests and UIO Requests that are tracked by the Root Port encounter certain uncorrectable or advisory errors. See § Section 2.9.3 for a description of which Non-Posted Requests are tracked. A set of control and status bits exists for receiving Completion with Unsupported Request status (UR Cpl), receiving Completion with Completer Abort status (CA Cpl), and Completion Timeout (CTO) errors. Independent sets of these error bits exist for Configuration Requests, I/O Requests, and Memory Requests. This finer granularity enables more precise error handling for this subset of uncorrectable errors (UR Cpl, CA Cpl, and CTO). As a key example, UR Cpl errors with Memory Read Requests can be configured to trigger DPC for proper containment and error handling, while UR Cpl errors with Configuration Requests can be configured to return all 1's (without triggering DPC) for normal probing and enumeration.

A UR or CA error logged in AER is the result of the Root Port operating in the role of a Completer returning a Completion for a Non-Posted Request or UIO Request. In contrast, a UR Cpl or CA Cpl error logged as an RP PIO error is the result of the Root Port operating in the role of a Requester, and receiving a Completion for a Non-Posted Request or a UIO Request. CTO errors logged in both AER and RP PIO are the result of the Root Port operating in the role of a Requester, though the RP PIO error controls support per-space granularity. Depending upon the control register settings, CTO errors can be logged in AER registers, in RP PIO registers, or both. If software unmask CTO errors in RP PIO, it is recommended that software mask CTO errors in AER to avoid unintended interactions.

The RP PIO Header Log Register, RP PIO ImpSpec Log Register, and RP PIO TLP Prefix Log Registers are referred to collectively as the RP PIO log registers. The RP PIO Header Log Register must be implemented; the RP PIO ImpSpec Log Register and RP PIO TLP Prefix Log Register are optional. The RP PIO Log Size field indicates how many DWORDs are allocated for the RP PIO log registers, and from this the allocated size for the RP PIO TLP Prefix Log Register can be calculated. See § Section 7.9.14.2. The RP PIO log registers always record information from a PIO Request, not any associated Completions.

When Flit Mode Supported is Set and the link is operating in Flit Mode, the RP PIO Header Log Register extends into additional DWs as indicated by the RP PIO Log Size field. Software must parse the Type and OHC fields to determine the size and layout of a TLP recorded in the RP PIO Header Log Register. Hardware is not required to support logging of TLP Headers larger than the largest size supported by the Port. Hardware is not required to support logging of OHC types not supported by the Port. TLP Trailers are not logged in the RP PIO Header Log Register. The required minimum size of the RP PIO Header Log Register is determined by the largest Header Base Size implemented by the Port (up to the maximum defined of 7 DW - See § Table 2-5), plus the largest number of OHC implemented by the Port (up to the maximum defined

of 7 DW). Hardware must hardwire to zero the DW of the RP PIO Header Log Register beyond those required to log the largest supported TLP Header and the overall length of the Advanced Error Reporting Extended Capability is reduced accordingly. As in Non-Flit Mode, Local TLP Prefixes are not logged.

The RP PIO Status, Mask, and Severity registers behave similarly to the Uncorrectable Error Status, Mask, and Severity registers in AER. See § Section 7.8.4.2, § Section 7.8.4.3, and § Section 7.8.4.4. When an RP PIO error is detected while it is unmasked, the associated bit in the RP PIO Status Register is Set, and the error is recorded in the RP PIO log registers (assuming that RP PIO error logging resources are available). When an RP PIO error is detected while it is masked, the associated bit is still Set in the RP PIO Status Register, but the error does not trigger DPC and the error is not recorded in the RP PIO log registers.

Each unmasked RP PIO error is handled either as uncorrectable or advisory, as determined by the value of the corresponding bit in the RP PIO Severity Register. If the associated Severity bit is Set, the error is handled as uncorrectable, triggering DPC (assuming that DPC is enabled) and signaling this event with a DPC interrupt and/or ERR_COR (if enabled). If the associated Severity bit is Clear, the error is handled as advisory (without triggering DPC) and signaled with ERR_COR (if enabled).

IMPLEMENTATION NOTE: USE OF RP PIO ADVISORY ERROR HANDLING §

Each RP PIO error can be handled either as uncorrectable or advisory. Uncorrectable error handling usually logs the error, triggers DPC, and signals the event either with a DPC interrupt, an ERR_COR, or both. Advisory error handling usually logs the error and signals the event with ERR_COR.

RP PIO advisory error handling can be used by software in certain cases to handle RP PIO errors robustly without incurring the disruption caused if DPC is triggered in the RP. If an RP PIO Exception is not enabled for a given error, an all 1's value is returned whenever the error occurs. If the error does not trigger DPC, software may be uncertain if the all 1's value returned by a given PIO read is the actual data value returned by the Completion versus indicating that an error occurred with that PIO read. If software enables advisory error handling for that error, instances of that error will be logged, enabling software to distinguish the two cases.

The use of RP PIO advisory error handling is notably beneficial if DPC is triggered in a Switch Downstream Port, and that causes one or more Completion Timeouts in the RP as a side-effect, as described in § Section 2.9.3. If the RP handles Completion Timeout errors as advisory, this avoids DPC being triggered in the RP, permitting continued operation with the other Switch Downstream Ports.

The RP PIO First Error Pointer, RP PIO Header Log, and RP PIO TLP Prefix Log behave similarly to the First Error Pointer, Header Log, and TLP Prefix Log in AER. The RP PIO First Error Pointer is defined to be valid when its value indicates a bit in the RP PIO Status Register that is Set. When the RP PIO First Error Pointer is valid, the RP PIO log registers contain the information associated with the indicated error. The RP PIO ImpSpec Log, if implemented, contains implementation specific information, e.g., the source of the Request TLP.

In contrast to AER, where the recording of CTO error information in the AER log registers is optional, RP PIO implementations must support recording RP PIO CTO error information in the RP PIO log registers.

If an error is detected with a received Completion TLP associated with an outstanding PIO Request, the set of RP PIO error control bits used to govern the error handling is determined in a similar manner. The DPC Completion Control bit determines whether UR or CA applies, and the Space (Configuration, I/O, or Memory) is that of the associated PIO Request. For example, if the DPC Completion Control bit is configured for CA, and a Root Port receives a poisoned Completion for a PIO Memory Read Request, the Mem CA Cpl bit (bit 17) is used in the RP PIO control and status registers for handling the error.

The RP PIO SysError Register provides a means to generate a System Error when an RP PIO error occurs. If an unmasked RP PIO error is detected while its associated bit in the RP PIO SysError Register is Set, a System Error is generated.

The RP PIO Exception Register provides a means to generate a synchronous processor exception¹³⁵ when an error occurs with certain tracked Non-Posted Requests and UIO Requests that are generated by a processor instruction. See § Section 2.9.3. This exception must support all such tracked Requests, and may optionally support Configuration write, I/O write, and AtomicOp Requests. If an error with an exception-supported Non-Posted Request or UIO Request is detected¹³⁶ or a Completion for it is synthesized, and its associated bit in the RP PIO Exception Register is Set, the processor instruction that generated the Request must take a synchronous exception. This still applies even if the RP PIO or AER controls specify that the error be handled as masked or advisory.

The details of a processor instruction taking a synchronous exception are processor-specific, but at a minimum, the mechanism must be able to interrupt the normal processor instruction flow either before completion of the instruction that generated the Non-Posted Request or UIO Request, or immediately following that instruction. The intent is that exception handling routines in system firmware, the operating system, or both, can examine the cause of the exception and take corrective action if necessary.

If an RP PIO error occurs with a processor-generated read or AtomicOp Request, and the RP PIO Exception Register value does not cause an exception, a value of all 1's must be returned for the instruction that generated the Request.

IMPLEMENTATION NOTE: SYNCHRONOUS EXCEPTION IMPLEMENTATION §

The exact mechanism for implementing synchronous exceptions is processor and platform specific. One possible implementation is poisoning the data returned to the processor for a read or AtomicOp Request that encounters an error. While this approach is likely to work with those Requests, it might not work with Configuration and I/O write Requests since they return no data.

Another possible implementation is marking the response transaction for processor-generated Non-Posted Requests with some other type of indication of the Request having failed, e.g., a “hard fail” response. This approach is more likely to work with all processor-generated Non-Posted Requests.

IMPLEMENTATION NOTE: RP PIO MASK BIT BEHAVIOR AND RATIONALE §

For a given RP PIO error, the associated mask bit in the RP PIO Mask Register affects its associated status bit setting, error logging, and error signaling in a manner that closely parallels the behavior of mask bits in AER.

SysError generation for a given RP PIO error is primarily controlled by the associated bit in the RP PIO SysError Register, but is also contingent upon the associated RP PIO mask bit being Clear. This behavior was chosen for consistency with AER, and also since it is poor practice to generate a SysError without logging the reason.

Exception generation for a given RP PIO error is independent of the associated RP PIO mask bit value. Usage models are envisioned where an RP PIO error needs to generate an Exception without logging an RP PIO error or triggering DPC.

Root Port error handling for tracked Non-Posted Requests and UIO Requests with errors other than receiving UR and CA Completions is governed by a combination of AER and RP PIO error controls. Examples are CTO¹³⁷, Poisoned TLP Received, and Malformed TLP. For a given error managed by AER, the associated AER Mask and Severity bits determine if

135. “Exception” is used as a generic term for a variety of mechanisms used by processors, including interrupts, traps, machine checks, instruction aborts, etc.

136. This includes any errors with the Completion TLP itself (e.g., Malformed TLP) or where the Completion Status is other than Successful Completion.

137. CTO errors have status and mask bits in both AER and RP PIO, though RP PIO has independent sets of bits for each of the 3 spaces. Other errors in AER have no equivalent errors in RP PIO.

the error must be handled as an uncorrectable error, handled as an Advisory Non-Fatal Error, or handled as a masked error.

- If the AER-managed error is to be handled as an uncorrectable error (see § Section 6.2.2.2), DPC is triggered. The RP PIO SysError and RP PIO Exception bits associated with the Request type and Completion Status apply.
- If the AER-managed error is to be handled as an Advisory Non-Fatal Error (see § Section 6.2.3.2.4), DPC is not triggered. The RP PIO SysError and RP PIO Exception bits do apply.
- If the AER-managed error is to be handled as a masked error (see § Section 6.2.3.2.2), DPC is not triggered. RP PIO SysError bit does not apply, but the RP PIO Exception bit does apply.

6.2.11.4 Software Triggering of DPC §

If the DPC Software Triggering Supported bit in the DPC Capability register is Set, then software can trigger DPC by writing a 1b to the DPC Software Trigger bit in the DPC Control Register, assuming that DPC is enabled and the Port isn't currently in DPC. This mechanism is envisioned to be useful for software and/or firmware development and testing. It also supports usage models where software or firmware examines RP PIO Exceptions or RP PIO advisory errors, and decides to trigger DPC based upon the situation.

When this mechanism triggers DPC, the DPC Trigger Reason and DPC Trigger Reason Extension fields in the DPC Status Register will indicate this as the reason.

If a Port is already in DPC when a 1b is written to the DPC Software Trigger bit, the Port remains in DPC, and the DPC Trigger Reason and DPC Trigger Reason Extension fields are not modified.

IMPLEMENTATION NOTE: AVOID DISABLE LINK AND HOT-PLUG SURPRISE USE WITH DPC §

It is recommended that software not Set the Link Disable bit in the Link Control register while DPC is enabled but not triggered. Setting the Link Disable bit will cause the Link to be directed to DL_Down, invoking some semantics similar to those in DPC, but lacking others. If DPC is enabled, the subsequent arrival of any Posted Requests will likely trigger DPC anyway. If DPC is enabled, the recommended method for software to disable the Link is to write a 1b to the optional DPC Software Trigger bit in the DPC Control Register. If the DPC Software Trigger bit is not implemented, software should disable DPC and use Link Disable instead. If the operating system is performing this action, but DPC is owned by system firmware, the operating system should coordinate disabling DPC with system firmware.

DPC is not recommended for use concurrently with the Hot-Plug Surprise mechanism, indicated by the Hot-Plug Surprise bit in the Slot Capabilities register being Set. Having this bit Set blocks the reporting of Surprise Down errors, preventing DPC from being triggered by this important error, greatly reducing the benefit of DPC. See § Section 6.7.4.5 for guidance on slots supporting both mechanisms.

6.2.11.5 DL_Active ERR_COR Signaling §

Support for this feature is indicated by the DL_Active ERR_COR Signaling Supported bit in the DPC Capability register. The feature is enabled by the DL_ACTIVE ERR_COR Enable bit in the DPC Control Register. The DL_ACTIVE state is indicated by the Data Link Layer Link Active bit in the Link Status Register. DL_ACTIVE ERR_COR signaling is managed independently of Data Link Layer State Changed interrupts, and it is permitted to use both mechanisms concurrently.

If the DL_ACTIVE ERR_COR Enable bit is Set, and the Correctable Error Reporting Enable bit in the Device Control register or the DPC SIG_SFW Enable bit in the DPC Control Register is Set, the Port must send an ERR_COR Message each time the Link transitions into the DL_Active state. DL_ACTIVE ERR_COR signaling must not Set the Correctable Error Detected bit in the Device Status register, since this event is not handled as an error. If the Downstream Port supports ERR_COR Subclass capability, this DPC ERR_COR signaling event must set the DPC SIG_SFW Status bit in the DPC Status register and also set the ERR_COR Subclass field in the ERR_COR Message to indicate ECS SIG_SFW. In contrast to Data Link Layer State Changed interrupts, DL_ACTIVE ERR_COR signaling only indicates the Link enters the DL_Active state, not when the Link exits the DL_Active state.

For a given DL_ACTIVE event, if a Port is going to send both an ERR_COR Message and an MSI/MSI-X transaction, then the Port must send the ERR_COR Message prior to sending the MSI/MSI-X transaction. There is no corresponding requirement if the INTx mechanism is being used to signal DL_ACTIVE interrupts, since INTx Messages won't necessarily remain ordered with respect to ERR_COR Messages when passing through routing elements.

IMPLEMENTATION NOTE:

USE OF DL_ACTIVE ERR_COR SIGNALING §

It is recommended that operating systems use Data Link Layer State Changed interrupts for signaling when DL_ACTIVE changes state. While DL_ACTIVE ERR_COR signaling indicates a subset of the same events, DL_ACTIVE ERR_COR signaling is primarily intended for use by system firmware, when it needs to be notified in order to do Downstream Port configuration or provide firmware first services.

6.3 Virtual Channel Support §

6.3.1 Introduction and Scope §

The Virtual Channel mechanism provides a foundation for supporting differentiated services within the PCI Express fabric. It enables deployment of independent physical resources that together with traffic labeling are required for optimized handling of differentiated traffic. Traffic labeling is supported using Traffic Class TLP-level labels. The policy for traffic differentiation is determined by the TC/VC mapping and by the VC-based, Port-based, and Function-based arbitration mechanisms supported by certain VC capabilities. The TC/VC mapping depends on the platform application requirements. These requirements drive the choice of the arbitration algorithms and configurability/programmability of arbiters allows detailed tuning of the traffic servicing policy.

The definition of the Virtual Channel and associated Traffic Class mechanisms is covered in § Chapter 2. . The VC configuration/programming models are defined in § Section 7.9.1 , § Section 7.9.2 , and § Section 7.9.29 .

This section covers VC mechanisms from the system perspective. It addresses the next level of details on:

- Supported TC/VC configurations
- VC-based arbitration - algorithms and rules
- Traffic ordering considerations
- Isochronous support as a specific usage model
- SVC and VC/MFVC capability coexistence

6.3.2 TC/VC Mapping and Example Usage §

A Virtual Channel is established when one or more TCs are associated with a physical resource designated by a VC ID. Every Traffic Class that is supported on a given path within the fabric must be mapped to one of the enabled Virtual Channels. Every Port must support the default TC0/VC0 pair - this is “hardwired”. Any additional TC mapping or additional VC resource enablement is optional and is controlled by system software using the programming model described in Sections 7.9.1 and 7.9.2.

The number of VC resources provisioned within a component or enabled within a given fabric may vary due to implementation and usage model requirements, due to Hot-Plug of disparate components with varying resource capabilities, or due to system software restricting what resources may be enabled on a given path within the fabric.

Some examples to illustrate:

- A set of components (Root Complex, Endpoints, Switches) may only support the mandatory VC0 resource that must have TC0 mapped to VC0. System software may, based on application usage requirements, map one or all non-zero TCs to VC0 as well on any or all paths within the fabric.
- A set of components may support two VC resources, e.g., VC0 and VC1. System software must map TC0/VC0 and in addition, may map one or all non-zero TC labels to either VC0 or VC1. As above, these mappings may be enabled on any or all paths within the fabric. Refer to the examples below for additional information.
- A Switch may be implemented with eight Ports - seven x1 Links with two VC resources and one x16 Link with one VC resource. System software may enable both VC resources on the x1 Links and assign one or more additional TCs to either VC thus allowing the Switch to differentiate traffic flowing between any Ports. The x16 Link must also be configured to map any non-TC0 traffic to VC0 if such traffic is to flow on this Link. Note: multi-Port components (Switches and Root Complex) are required to support independent TC/VC mapping per Port.

In any of the above examples, system software has the ability to map one, all, or a subset of the TCs to a given VC. Should system software wish to restrict the number of traffic classes that may flow through a given Link, it may configure only a subset of the TCs to the enabled VC resources. Any TLP indicating a TC that has not been mapped to an enabled VC resource must be treated as a Malformed TLP. This is referred to as TC Filtering. Flow Control credits for this TLP will be lost, and an uncorrectable error will be generated, so software intervention will usually be required to restore proper operation after a TC Filtering event occurs.

A graphical example of TC filtering is illustrated in § Figure 6-4, where TCs (2:6) are not mapped to the Link that connects Endpoint A and the Switch. This means that the TLPs with TCs (2:6) are not allowed between the Switch and Endpoint A.

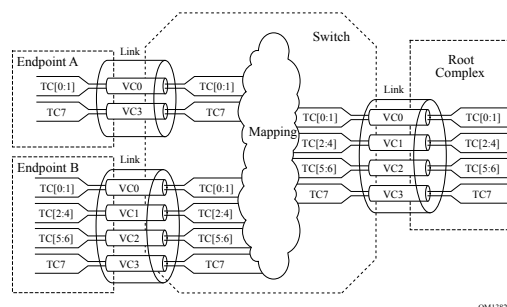


Figure 6-4 TC Filtering Example §

§ Figure 6-5 shows an example of TC to VC mapping. A simple Switch with one Downstream Port and one Upstream Port connects an Endpoint to a Root Complex. At the Upstream Port, two VCs (VC0 and VC1) are enabled with the following

mapping: TC(0-6)/VC0, TC7/VC1. At the Downstream Port, only VC0 is enabled and all TCs are mapped to VC0. In this example while TC7 is mapped to VC0 at the Downstream Port, it is re-mapped to VC1 at the Upstream Port. Although the Endpoint only supports VC0, when it labels transactions with different TCs, transactions associated with TC7 from/to the Endpoint can take advantage of the second Virtual Channel enabled between the Switch and the Root Complex.

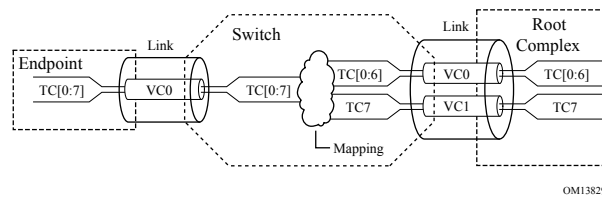


Figure 6-5 TC to VC Mapping Example §

IMPLEMENTATION NOTE: MULTIPLE TCS OVER A SINGLE VC §

A single VC implementation may benefit from using multiple TCs. TCs provide ordering domains that may be used to differentiate traffic within the Endpoint or the Root Complex independent of the number of VCs supported.

In a simple configuration, where only VC0 is supported, traffic differentiation may not be accomplished in an optimum manner since the different TCs cannot be physically segregated. However, the benefits of carrying multiple TCs can still be exploited particularly in the small and “shallow” topologies where Endpoints are connected directly to Root Complex rather than through cascaded Switches. In these topologies traffic that is targeting Root Complex only needs to traverse a single Link, and an optimized scheduling of packets on both sides (Endpoint and Root Complex) based on TCs may accomplish significant improvement over the case when a single TC is used. Still, the inability to route differentiated traffic through separate resources with fully independent flow control and independent ordering exposes all of the traffic to the potential head-of-line blocking conditions. Optimizing Endpoint internal architecture to minimize the exposure to the blocking conditions can reduce those risks.

6.3.3 VC Arbitration §

Arbitration is one of the key aspects of the Virtual Channel mechanism and is defined in a manner that fully enables configurability to the specific application. In general, the definition of the VC-based arbitration mechanism is driven by the following objectives:

- To prevent false transaction timeouts and to guarantee data flow forward progress
- To provide differentiated services between data flows within the fabric
- To provide guaranteed bandwidth with deterministic (and reasonably small) end-to-end latency between components

Links are bidirectional, i.e., each Port can be an Ingress or an Egress Port depending on the direction of traffic flow. This is illustrated by the example of a 3-Port Switch in § Figure 6-6, where the paths for traffic flowing between Switch Ports are highlighted with different types of lines. In the following sections, VC Arbitration is defined using a Switch arbitration model since the Switch represents a functional superset from the arbitration perspective.

In addition, one-directional data flow is used in the description.

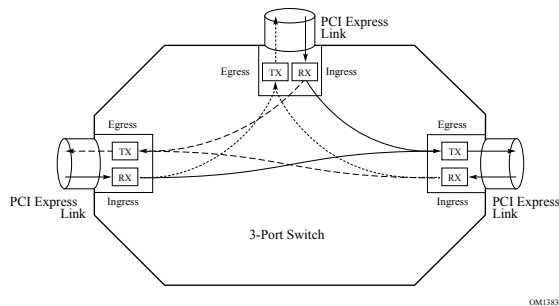


Figure 6-6 An Example of Traffic Flow Illustrating Ingress and Egress §

6.3.3.1 Traffic Flow and Switch Arbitration Model §

The following set of figures (§ Figure 6-7 and § Figure 6-8) illustrates traffic flow through the Switch and summarizes the key aspects of the arbitration.

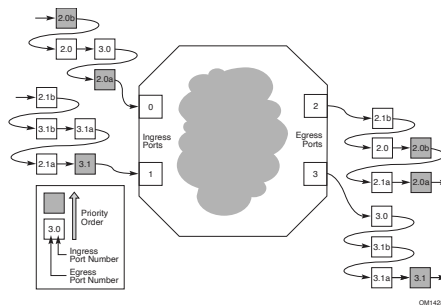


Figure 6-7 An Example of Differentiated Traffic Flow Through a Switch §

At each Ingress Port an incoming traffic stream is represented in § Figure 6-7 by small boxes. These boxes represent packets that are carried within different VCs that are distinguished using different levels of gray. Each of the boxes that represents a packet belonging to different VC includes designation of Ingress and Egress Ports to indicate where the packet is coming from and where it is going to. For example, designation “3.0” means that this packet is arriving at Port #0 (Ingress) and is destined to Port #3 (Egress). Within the Switch, packets are routed and serviced based on Switch internal arbitration mechanisms.

Switch arbitration model defines a required arbitration infrastructure and functionality within a Switch. This functionality is needed to support a set of arbitration policies that control traffic contention for an Egress Port from multiple Ingress Ports.

§ Figure 6-8 shows a conceptual model of a Switch highlighting resources and associated functionality in ingress to egress direction. Note that each Port in the Switch can have the role of an Ingress or Egress Port. Therefore, this figure only shows one particular scenario where the 4-Port Switch in this example has ingress traffic on Port #0 and Port #1, that targets Port #2 as an Egress Port. A different example may show different flow of traffic implying different roles for Ports on the Switch. The PCI Express architecture enables peer-to-peer communication through the Switch and, therefore, possible scenarios using the same example may include multiple separate and simultaneous ingress to egress flows (e.g., Port 0 to Port 2 and Port 1 to Port 3).

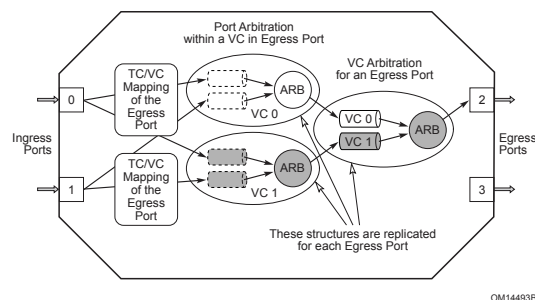


Figure 6-8 Switch Arbitration Structure §

The following two steps conceptually describe routing of traffic received by the Switch on Port 0 and Port 1 and destined to Port 2. First, the target Egress Port is determined based on address/routing information in the TLP header. Secondly, the target VC of the Egress Port is determined based on the TC/VC map of the Egress Port. Transactions that target the same VC in the Egress Port but are from different Ingress Ports must be arbitrated before they can be forwarded to the corresponding resource in the Egress Port. This arbitration is referred to as the Port Arbitration.

Once the traffic reaches the destination VC resource in the Egress Port, it is subject to arbitration for the shared Link. From the Egress Port point of view this arbitration can be conceptually defined as a simple form of multiplexing where the multiplexing control is based on arbitration policies that are either fixed or configurable/programmable. This stage of arbitration between different VCs at an Egress Port is called the VC Arbitration of the Egress Port.

Independent of VC arbitration policy, a management/control logic associated with each VC must observe transaction ordering and flow control rules before it can make pending traffic visible to the arbitration mechanism.

IMPLEMENTATION NOTE: VC CONTROL LOGIC AT THE EGRESS PORT §

VC control logic at every Egress Port includes:

- VC Flow Control logic
- VC Ordering Control logic

Flow control credits are exchanged between two Ports connected to the same Link. Availability of flow control credits is one of the qualifiers that VC control logic must use to decide when a VC is allowed to compete for the shared Link resource (i.e., Data Link Layer transmit/retry buffer). If a candidate packet cannot be submitted due to the lack of an adequate number of flow control credits, VC control logic must mask the presence of pending packet to prevent blockage of traffic from other VCs. Note that since each VC includes buffering resources for Posted Requests, Non-Posted Requests, and Completion packets, the VC control logic must also take into account availability of flow control credits for the particular candidate packet. In addition, VC control logic must observe ordering rules (see § Section 2.4 for more details) for Posted/Non-Posted/Completion transactions to prevent deadlocks and violation of producer/consumer ordering model.

6.3.3.2 VC Arbitration - Arbitration Between VCs §

This specification defines a default VC prioritization via the VC Identification (VC ID) assignment, i.e., the VC IDs are arranged in ascending order of relative priority in the Virtual Channel Capability structure or Multi-Function Virtual

Channel Capability structure. The example in § Figure 6-9 illustrates a Port that supports eight VCs with VC0 treated as the lowest priority and VC7 as the highest priority.

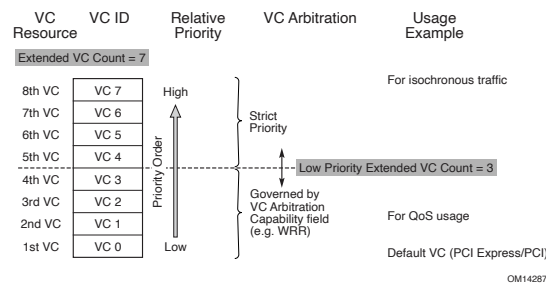


Figure 6-9 VC ID and Priority Order - An Example §

The availability of default prioritization does not restrict the type of algorithms that may be implemented to support VC arbitration - either implementation specific or one of the architecture-defined methods:

- Strict Priority - Based on inherent prioritization, i.e., VC0 = lowest, VC7 = highest
- Round Robin (RR) - Simplest form of arbitration where all VCs have equal priority
- Weighted RR - Programmable weight factor determines the level of service

If strict priority arbitration is supported by the hardware for a subset of the VC resources, software can configure the VCs into two priority groups - a lower and an upper group. The upper group is treated as a strict priority arbitration group while the lower group is arbitrated to only when there are no packets to process in the upper group. § Figure 6-9 illustrates an example configuration that supports eight VCs separated into two groups - the lower group consisting of VC0-VC3 and the upper group consisting of VC4-VC7. The arbitration within the lower group can be configured to one of the supported arbitration methods. The Low Priority Extended VC Count field in the Port VC Capability Register 1 indicates the size of this group. The arbitration methods are listed in the VC Arbitration Capability field in the Port VC Capability Register 2. Refer to § Section 7.9.1 and § Section 7.9.2 for details. When the Low Priority Extended VC Count field is set to zero, all VCs are governed by the strict-priority VC arbitration; when the field is equal to the Extended VC Count, all VCs are governed by the VC arbitration indicated by the VC Arbitration Capability field.

6.3.3.2.1 Strict Priority Arbitration Model §

Strict priority arbitration enables minimal latency for high-priority transactions. However, there is potential danger of bandwidth starvation should it not be applied correctly. Using strict priority requires all high-priority traffic to be regulated in terms of maximum peak bandwidth and Link usage duration. Regulation must be applied either at the transaction injection point or within subsequent Egress Ports where data flows contend for a common Link. System software must configure traffic such that lower priority transactions will be serviced at a sufficient rate to avoid transaction timeouts.

6.3.3.2.2 Round Robin Arbitration Model §

Round Robin arbitration is used to provide, at the transaction level, equal¹³⁸ opportunities to all traffic. Note that this scheme is used where different unordered streams need to be serviced with the same priority.

In the case where differentiation is required, a Weighted Round Robin scheme can be used. The WRR scheme is commonly used in the case where bandwidth regulation is not enforced by the sources of traffic and therefore it is not

138. Note that this does not imply equivalence and fairness in the terms of bandwidth usage.

possible to use the priority scheme without risking starvation of lower priority traffic. The key is that this scheme provides fairness during traffic contention by allowing at least one arbitration win per arbitration loop. Assigned weights regulate both minimum allowed bandwidth and maximum burstiness for each VC during the contention. This means that it bounds the arbitration latency for traffic from different VCs. Note that latencies are also dependent on the maximum packet sizes allowed for traffic that is mapped onto those VCs.

One of the key usage models of the WRR scheme is support for QoS policy where different QoS levels can be provided using different weights.

Although weights can be fixed (by hardware implementation) for certain applications, to provide more generic support for different applications, components that support the WRR scheme are recommended to implement programmable WRR. Programming of WRR is controlled using the software interface defined in Sections 7.9.1 and 7.9.2.

6.3.3.3 Port Arbitration - Arbitration Within VC §

For Switches, Port Arbitration refers to the arbitration at an Egress Port between traffic coming from other Ingress Ports that is mapped to the same VC. For Root Ports, Port Arbitration refers to the arbitration at a Root Egress Port between peer-to-peer traffic coming from other Root Ingress Ports that is mapped to the same VC. For RCRBs, Port Arbitration refers to the arbitration at the RCRB (e.g., for host memory) between traffic coming from Root Ports that is mapped to the same VC. An inherent prioritization scheme for arbitration among VCs in this context is not applicable since it would imply strict arbitration priority for different Ports. Traffic from different Ports can be arbitrated using the following supported schemes:

- Hardware-fixed arbitration scheme, e.g., Round Robin
- Programmable WRR arbitration scheme
- Programmable Time-based WRR arbitration scheme

Hardware-fixed RR or RR-like scheme is the simplest to implement since it does not require any programmability. It makes all Ports equal priority, which is acceptable for applications where no software-managed differentiation or per-Port-based bandwidth budgeting is required.

Programmable WRR allows flexibility since it can operate as flat RR or if differentiation is required, different weights can be applied to traffic coming from different Ports in the similar manner as described in § Section 6.3.3.2 . This scheme is used where different allocation of bandwidth needs to be provided for different Ports.

A Time-based WRR is used for applications where not only different allocation of bandwidth is required but also a tight control of usage of that bandwidth. This scheme allows control of the amount of traffic that can be injected from different Ports within a certain fixed period of time. This is required for certain applications such as isochronous services, where traffic needs to meet a strict deadline requirement. § Section 6.3.4 provides basic rules to support isochronous applications. For more details on time-based arbitration and on the isochronous service as a usage model for this arbitration scheme refer to Appendix A.

6.3.3.4 Multi-Function Devices and Function Arbitration §

The multi-Function arbitration model defines an optional arbitration infrastructure and functionality within a Multi-Function Device. This functionality is needed to support a set of arbitration policies that control traffic contention for the device's Upstream Egress Port from its multiple Functions.

§ Figure 6-10 shows a conceptual model of a Multi-Function Device highlighting resources and associated functionality. Note that each Function optionally contains a VC Extended Capability structure, which if present manages TC/VC mapping, optional Port Arbitration, and optional VC Arbitration, all within the Function. The MFVC Extended Capability structure manages TC/VC mapping, optional Function Arbitration, and optional VC Arbitration for the device's Upstream Egress Port. Together these resources enable enhanced QoS management for Upstream requests. However, unlike a

complete Switch with devices on its Downstream Ports, the Multi-Function Device model does not support full QoS management for peer-to-peer requests between Functions or for Downstream requests.

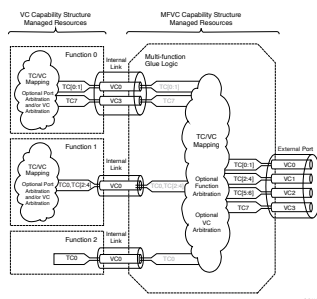


Figure 6-10 Multi-Function Arbitration Model §

QoS for an Upstream request originating at a Function is managed as follows. First, a Function-specific mechanism applies a TC to the request. For example, a device driver might configure a Function to tag all its requests with TC7.

Next, if the Function contains a VC Extended Capability structure, it specifies the TC/VC mapping to one of the Function's VC resources (perhaps the Function's single VC resource). In addition, the VC Extended Capability structure supports the enablement and configuration of the Function's VC resources.

If the Function is a Switch and the target VC resource supports Port Arbitration, this mechanism governs how the Switch's multiple Downstream Ingress Ports arbitrate for that VC resource. If the Port Arbitration mechanism supports time-based WRR, this also governs the injection rate of requests from each Downstream Ingress Port.

If the Function supports VC arbitration, this mechanism manages how the Function's multiple VC resources arbitrate for the conceptual internal link to the MFVC resources.

Once a request packet conceptually arrives at MFVC resources, address/routing information in the TLP header determines whether the request goes Upstream or peer-to-peer to another Function. For the case of peer-to-peer, QoS management is left to unarchitected device-specific mechanisms. For the case of Upstream, TC/VC mapping in the MFVC Extended Capability structure determines which VC resource the request will target. The MFVC Extended Capability structure also supports enablement and configuration of the VC resources in the multi-Function glue logic. If the target VC resource supports Function Arbitration, this mechanism governs how the multiple Functions arbitrate for this VC resource. If the Function Arbitration mechanism supports time-based WRR, this governs the injection rate of requests for each Function into this VC resource.

Finally, if the MFVC Extended Capability structure supports VC Arbitration, this mechanism governs how the MFVC's multiple VCs compete for the device's Upstream Egress Port. Independent of VC arbitration policy, management/control logic associated with each VC must observe transaction ordering and flow control rules before it can make pending traffic visible to the arbitration mechanism.

IMPLEMENTATION NOTE:

MULTI-FUNCTION ARBITRATION ERROR BEHAVIOR §

§ Table 6-6 shows the expected error behavior associated with the example topology shown in § Figure 6-10.

Table 6-6 Multi-Function Arbitration Error Model Example §

Source	TC	Destination			
		Function 0	Function 1	Function 2	External Port
Function 0	0	n/a	OK	OK	OK
	1		MF @ F1	MF @ F2	OK
	2 - 6		MF @ F0	MF @ F0	MF @ F0
	7		MF @ F1	MF @ F2	OK
Function 1	0	OK	n/a	OK	OK
	1	MF @ F1		MF @ F1	MF @ F1
	2 - 4	MF @ F0		MF @ F2	OK
	5 - 7	MF @ F1		MF @ F1	MF @ F1
Function 2	0	OK	OK	n/a	OK
	1 - 7	MF @ F2	MF @ F2		MF @ F2
External Port	0	OK	OK	OK	n/a
	1	OK	MF @ F1	MF @ F2	
	2 - 4	MF @ F0	OK		
	5 - 6		MF @ F1		
	7				
Legend:					
OK	Success				
MF @ F0	Malformed TLP, reported at Function 0				
MF @ F1	Malformed TLP, reported at Function 1				
MF @ F2	Malformed TLP, reported at Function 2				
n/a	Not Applicable (Function/Port sending to itself)				

IMPLEMENTATION NOTE: MULTI-FUNCTION DEVICES WITHOUT THE MFVC EXTENDED CAPABILITY STRUCTURE §

If a Multi-Function Device lacks an MFVC Extended Capability structure, the arbitration of data flows from different Functions of a Multi-Function Device is beyond the scope of this specification.

However, as stated in this specification, if a Multi-Function Device supports TCs other than TC0 and does not implement an MFVC Extended Capability structure, it is required to implement a single VC Extended Capability structure in Function 0 to provide architected TC/VC mappings for the Link.

6.3.4 Isochronous Support §

Servicing isochronous data transfer requires a system to provide not only guaranteed data bandwidth but also deterministic service latency. The isochronous support mechanisms are defined to ensure that isochronous traffic receives its allocated bandwidth over a relevant period of time while also preventing starvation of the other traffic in the system. Isochronous support mechanisms apply to communication between Endpoint and Root Complex as well as to peer-to-peer communication.

Isochronous service is realized through proper use of mechanisms such as TC transaction labeling, VC data-transfer protocol, and TC-to-VC mapping. End-to-end isochronous service requires software to set up proper configuration along the path between the Requester and the Completer. This section describes the rules for software configuration and the rules hardware components must follow to provide end-to-end isochronous services. More information and background material regarding isochronous applications and isochronous service design guidelines can be found in Appendix A.

6.3.4.1 Rules for Software Configuration §

System software must obey the following rules to configure PCI Express fabric for isochronous traffic:

- Software must designate one or more TCs for isochronous transactions.
- Software must ensure that the Attribute fields of all isochronous requests targeting the same Completer are fixed and identical.
- Software must configure all VC resources used to support isochronous traffic to be serviced (arbitrated) at the requisite bandwidth and latency to meet the application objectives. This may be accomplished using strict priority, WRR, or hardware-fixed arbitration.
- Software should not intermix isochronous traffic with non-isochronous traffic on a given VC.
- Software must observe the Maximum Time Slots capability reported by the Port or RCRB.
- Software must not assign all Link capacity to isochronous traffic. This is required to ensure the requisite forward progress of other non-isochronous transactions to avoid false transaction timeouts.
- Software must limit the Max_Payload_Size for each path that supports isochronous to meet the isochronous latency. For example, all traffic flowing on a path from an isochronous capable device to the Root Complex should be limited to packets that do not exceed the Max_Payload_Size required to meet the isochronous latency requirements.

- Software must set Max_Read_Request_Size of an isochronous-configured device with a value that does not exceed the Max_Payload_Size set for the device.

6.3.4.2 Rules for Requesters §

A Requester requiring isochronous services must obey the following rules:

- The value in the Length field of read requests must never exceed Max_Payload_Size.
- If isochronous traffic targets the Root Complex and the RCRB indicates it cannot meet the isochronous bandwidth and latency requirements without requiring all transactions to set the No Snoop attribute bit, indicated by setting the Reject Snoop Transactions bit, then this bit must be set within the TLP header else the transaction will be rejected.

6.3.4.3 Rules for Completers §

A Completer providing isochronous services must obey the following rules:

- A Completer should not apply flow control induced backpressure to uniformly injected isochronous requests under normal operating conditions.
- A Completer must report its isochronous bandwidth capability in the Maximum Time Slots field in the VC Resource Capability register. Note that a Completer must account for partial writes.
- A Completer must observe the maximum isochronous transaction latency.
- A Root Complex as a Completer must implement at least one RCRB and support time-based Port Arbitration for the associated VCs. Note that time-based Port Arbitration only applies to request transactions.

6.3.4.4 Rules for Switches and Root Complexes §

A Switch providing isochronous services must obey the following rules. The same rules apply to Root Complexes that support isochronous data flows peer-to-peer between Root Ports, abbreviated in this section as “P2P-RC”.

- An isochronous-configured Switch or P2P-RC Port should not apply flow control induced backpressure to uniformly injected isochronous requests under normal operating conditions.
- An isochronous-configured Switch or P2P-RC Port must observe the maximum isochronous transaction latency.
- A Switch or P2P-RC component must support time-based Port Arbitration for each Port that supports one or more VCs capable of supporting isochronous traffic. Note that time-based Port Arbitration applies to request transactions but not to completion transactions.

6.3.4.5 Rules for Multi-Function Devices §

A Multi-Function Device that includes an MFVC Extended Capability structure providing isochronous services must obey the following rules:

- MFVC glue logic configured for isochronous operation should not apply backpressure to uniformly injected isochronous requests from its Functions under normal operating conditions.

- The MFVC Extended Capability structure must support time-based Function Arbitration for each VC capable of supporting isochronous traffic. Note that time-based Function Arbitration applies only to Upstream request transactions; it does not apply to any Downstream or peer-to-peer request transactions, nor to any completion transactions.

A Multi-Function Device that lacks an MFVC Extended Capability structure has no architected mechanism to provide isochronous services for its multiple Functions concurrently.

6.3.5 SVC and VC/MFVC Capability Coexistence §

The Virtual Channel Extended Capability (VC capability) and Multi-Function Virtual Channel Extended Capability (MFVC capability) are referred to as VC/MFVC capabilities. A Port is permitted to implement a Streamlined Virtual Channel Extended Capability (SVC capability) as well as one or more VC/MFVC capabilities. During Link training, VC0 in each VC/MFVC capability will automatically initialize, and VC0 in the SVC capability will remain disabled. This ensures backward compatibility with software that is unaware of SVC.

SVC capability is incompatible with VC/MFVC capabilities, and hardware mechanisms ensure that for a given Port, SVC capability is never enabled at the same time as VC/MFVC capabilities.

SVC-aware software chooses when to use SVC capability instead of VC/MFVC capabilities on a Port that implements both. Before configuring VC/MFVC capabilities, software Clears the Use VC/MFVC bit in the SVC Port Status register. Doing so immediately clears the VC Enable bit for each VC resource in every VC/MFVC capability (VC Resource Control Register or MFVC VC Resource Control Register). This also Sets the SVC VC Enable bit for VC0 in its SVC Resource Control Register. The Use VC/MFVC bit will remain Clear until the next Conventional Reset. This presents a simplified and consistent operational state, reducing hardware and software complexity.

If a Port implements an SVC capability and no VC/MFVC capabilities, the Use VC/MFVC bit in the SVC Port Status register must be hardwired to 0b. During Link training, VC0 in the SVC capability automatically initializes. See the Use VC/MFVC bit description for further required semantics.

6.4 Device Synchronization §

System software requires a “stop” mechanism for ensuring that there are no outstanding transactions for a particular device in a system. For example, without such a mechanism renumbering Bus Numbers during system operation may cause the Requester ID (which includes the Bus Number) for a given device to change while Requests or Completions for that device are still in flight, and may thus be rendered invalid due to the change in the Requester ID. It is also desirable to be able to ensure that there are no outstanding transactions during a Hot-Plug orderly removal.

The details of stop mechanism implementation depend on the device hardware, device driver software, and system software. However, the fundamental requirements which must be supported to allow system software management of the fabric include the abilities to:

- Block the device from generating new Requests
- Block the generation of Requests issued to the device
- Determine that all Requests being serviced by the device have been completed
- Determine that all Non-Posted Requests and UIO Requests initiated by the device have completed
- Determine that all posted Requests initiated by the device have reached their destination

The ability of the driver and/or system software to block new Requests from the device is supported by the Bus Master Enable, SERR# Enable, and Interrupt Disable bits in the Command register (§ Section 7.5.1.1.3) of each device Function, and other such control bits.

Requests issued to the device are generally under the direct control of the driver, so system software can block these Requests by directing the driver to stop generating them (the details of this communication are system software specific). Similarly, Requests serviced by the device are normally under the device driver's control, so determining the completion of such requests is usually trivial.

The Transactions Pending bit provides a consistent way for software to determine if the Function is expecting one or more Completions (see § Section 7.5.3.5).

Determining that posted Requests have reached their destination is handled by generating a transaction to “flush” any outstanding Requests. Writes to system memory using TC0 will be flushed by host reads of the device, and so require no explicit flush protocol. Writes using TCs other than TC0 require some type of flush synchronization mechanism. The mechanism itself is implementation specific to the device and its driver software. However, in all cases the device hardware and software implementers should thoroughly understand the ordering rules described in § Section 2.4. This is especially true if the Relaxed Ordering or ID-Based Ordering attributes are set for any Requests initiated by the device.

IMPLEMENTATION NOTE: FLUSH MECHANISMS §

In a simple case such as that of an Endpoint communicating only with host memory through TC0, “flush” can be implemented simply by reading from the Endpoint. If the Endpoint issues writes to main memory using TCs other than TC0, “flush” can be implemented with a memory read on the corresponding TCs directed to main memory. The memory read needs to be performed on all TCs that the Endpoint is using.

If a memory read is used to “flush” outstanding transactions, but no actual read is required, it may be desirable to use the zero-length read semantic described in § Section 2.2.5.

Peer-to-peer interaction between devices requires an explicit synchronization protocol between the involved devices, even if all communication is through TC0. For a given system, the model for managing peer-to-peer interaction must be established. System software, and device hardware and software must then conform to this model. The requirements for blocking Request generation and determining completion of Requests match the requirements for non-peer interaction, however the determination that Posted Requests have reached peer destination device(s) requires an explicit synchronization mechanism. The mechanism itself is implementation specific to the device, its driver software, and the model used for the establishment and disestablishment of peer communications.

6.5 Locked Transactions §

6.5.1 Introduction §

Locked Transaction support is required to prevent deadlock in systems that use legacy software which causes the accesses to I/O devices. Note that some CPUs may generate locked accesses as a result of executing instructions that implicitly trigger lock. Some legacy software misuses these transactions and generates locked sequences even when exclusive access is not required. Since locked accesses to I/O devices introduce potential deadlocks apart from those mentioned above, as well as serious performance degradation, PCI Express Endpoints are prohibited from supporting locked accesses, and new software must not use instructions which will cause locked accesses to I/O devices. Legacy Endpoints support locked accesses only for compatibility with existing software.

Only the Root Complex is allowed to initiate Locked Requests on PCI Express. Locked Requests initiated by Endpoints and Bridges are not supported. This is consistent with limitations for locked transaction use outlined in [PCI] (§ Appendix F. - Exclusive Accesses).

This section specifies the rules associated with supporting locked accesses from the Host CPU to Legacy Endpoints, including the propagation of those transactions through Switches and PCI Express/PCI Bridges.

6.5.2 Initiation and Propagation of Locked Transactions - Rules §

Locked transaction sequences are generated by the Host CPU(s) as one or more reads followed by a number of writes to the same location(s). When a lock is established, all other traffic is blocked from using the path between the Root Complex and the locked Legacy Endpoint or Bridge.

- A locked transaction sequence or attempted locked transaction sequence is initiated on PCI Express using the “lock”-type Read Request/Completion (MRdLk/CplDLk) and terminated with the Unlock Message
 - Locked Requests which are completed with a status other than Successful Completion do not establish lock (explained in detail in the following sections)
 - Regardless of the status of any of the Completions associated with a locked sequence, all locked sequences and attempted locked sequences must be terminated by the transmission of an Unlock Message.
 - MRdLk, CplDLk, and Unlock semantics are allowed only for the default Traffic Class (TC0)
 - Only one locked transaction sequence attempt may be in progress at a given time within a single hierarchy domain
- The Unlock Message is sent from the Root Complex down the locked transaction path to the Completer, and may be broadcast from the Root Complex to all Endpoints and Bridges
 - Any device which is not involved in the locked sequence must ignore this Message
- Any violation of the rules for initiation and propagation of locked transactions can result in undefined device and/or system behavior
 - The initiation and propagation of a locked transaction sequence through PCI Express is performed as follows:
- A locked transaction sequence is started with a MRdLk Request
 - Any successive reads for the locked transaction sequence must also use MRdLk Requests
 - The Completions for any MRdLk Request use the CplDLk Completion type for successful Requests, and the CplLk Completion type for unsuccessful Requests
- If any read associated with a locked sequence is completed unsuccessfully, the Requester must assume that the atomicity of the lock is no longer assured, and that the path between the Requester and Completer is no longer locked
- All writes for the locked sequence use MWr Requests
- The Unlock Message is used to indicate the end of a locked sequence
 - A Switch propagates Unlock Messages to the locked Egress Port
- Upon receiving an Unlock Message, a Legacy Endpoint or Bridge must unlock itself if it is in a locked state
 - If not locked, or if the Receiver is a PCI Express Endpoint or Bridge which does not support lock, the Unlock Message is ignored and discarded

6.5.3 Switches and Lock - Rules §

Switches must distinguish transactions associated with locked sequences from other transactions to prevent other transactions from interfering with the lock and potentially causing deadlock. The following rules cover how this is done. Note that locked accesses are limited to TC0, which is always mapped to VC0.

- When a Switch propagates a MRdLk Request from the Ingress Port (closest to the Root Complex) to the Egress Port, it must block all Requests which map to the default Virtual Channel (VC0) from being propagated to the Egress Port
 - If a subsequent MRdLk Request is Received at this Ingress Port addressing a different Egress Port, the behavior of the Switch is undefined

Note: This sort of split-lock access is not supported by PCI Express and software must not cause such a locked access. System deadlock may result from such accesses.
- When the CplDLk for the first MRdLk Request is returned, if the Completion indicates a Successful Completion status, the Switch must block all Requests from all other Ports from being propagated to either of the Ports involved in the locked access, except for Requests which map to non-VC0 on the Egress Port
- The two Ports involved in the locked sequence must remain blocked as described above until the Switch receives the Unlock Message (at the Ingress Port for the initial MRdLk Request)
 - The Unlock Message must be forwarded to the locked Egress Port
 - The Unlock Message may be broadcast to all other Ports
 - The Ingress Port is unblocked once the Unlock Message arrives, and the Egress Port(s) which were blocked are unblocked following the Transmission of the Unlock Message out of the Egress Ports
 - Ports which were not involved in the locked access are unaffected by the Unlock Message

6.5.4 PCI Express/PCI Bridges and Lock - Rules §

The requirements for PCI Express/PCI Bridges are similar to those for Switches, except that, because PCI Express/PCI Bridges use only the default Virtual Channel and Traffic Class, all other traffic is blocked during the locked access. The requirements on the PCI bus side of the PCI Express/PCI Bridge match the requirements for a PCI/PCI Bridge (see [\[PCI-to-PCI-Bridge\]](#) and [\[PCIe-to-PCI-PCI-X-Bridge\]](#)).

6.5.5 Root Complex and Lock - Rules §

A Root Complex is permitted to support locked transactions as a Requester. If locked transactions are supported, a Root Complex must follow the sequence described in [§ Section 6.5.2](#) to perform a locked access. The mechanisms used by the Root Complex to interface PCI Express to the Host CPU(s) are outside the scope of this document.

6.5.6 Legacy Endpoints §

Legacy Endpoints are permitted to support locked accesses, although their use is discouraged. If locked accesses are supported, Legacy Endpoints must handle them as follows:

- The Legacy Endpoint becomes locked when it Transmits the first Completion for the first Read Request of the locked access with a Successful Completion status
 - If the completion status is not Successful Completion, the Legacy Endpoint does not become locked
 - Once locked, the Legacy Endpoint must remain locked until it receives the Unlock Message
- While locked, a Legacy Endpoint must not issue any Requests using TCs which map to the default Virtual Channel (VC0)

Note that this requirement applies to all possible sources of Requests within the Endpoint, in the case where there is more than one possible source of Requests.

 - Requests may be issued using TCs which map to VCs other than the default Virtual Channel

6.5.7 PCI Express Endpoints §

PCI Express Endpoints do not support lock. A PCI Express Endpoint must treat a MRdLk Request as an Unsupported Request (see § Chapter 2.).

6.6 PCI Express Reset - Rules §

This section specifies the PCI Express Reset mechanisms. This section covers the relationship between the architectural mechanisms defined in this document and the reset mechanisms defined in this document. Any relationship between the PCI Express Conventional Reset and component or platform reset is component or platform specific (except as explicitly noted).

6.6.1 Conventional Reset §

Conventional Reset includes all reset mechanisms other than Function Level Reset. There are two categories of Conventional Resets: Fundamental Reset and resets that are not Fundamental Reset. This section applies to all types of Conventional Reset.

In all form factors and system hardware configurations, there must, at some level, be a hardware mechanism for setting or returning all Port states to the initial conditions specified in this document - this mechanism is called “Fundamental Reset”. This mechanism can take the form of an auxiliary signal provided by the system to a component or adapter card, in which case the signal must be called PERST#, and must conform to the rules specified in § Section 4.2.5.9.1 . When PERST# is provided to a component or adapter, this signal must be used by the component or adapter as Fundamental Reset. When PERST# is not provided to a component or adapter, Fundamental Reset is generated autonomously by the component or adapter, and the details of how this is done are outside the scope of this document. If a Fundamental Reset is generated autonomously by the component or adapter, and if power is supplied by the platform to the component/adapter, the component/adapter must generate a Fundamental Reset to itself if the supplied power goes outside of the limits specified for the form factor or system.

- There are three distinct types of Conventional Reset: Cold, Warm, and Hot:
 - A Fundamental Reset must occur following the application of power to the component. This is called a Cold Reset.
 - In some cases, it may be possible for the Fundamental Reset mechanism to be triggered by hardware without the removal and re-application of power to the component. This is called a Warm Reset.
 - This document does not specify a means for generating a Warm or Cold Reset.
 - There is an in-band mechanism for propagating Conventional Reset across a Link. This is called a Hot Reset and is described in § Section 4.2.5.9.2 .

There is an in-band mechanism for software to force a Link into Electrical Idle, “disabling” the Link. The Disabled LTSSM state is described in § Section 4.2.6.10 , the Link Disable control bit is described in § Section 7.5.3.7 , and the Downstream Port Containment mechanism is described in § Section 6.2.11 . Disabling a Link causes Downstream components to undergo a hot reset.

See § Section 2.9 for additional requirements regarding the effects of the Data Link Layer reporting DL_Down status, and how those effects relate to hot reset.

- On exit from any type of Conventional Reset (cold, warm, or hot), all Port registers and state machines must be set to their initialization values as specified in this document, except for sticky registers (see § Section 7.4).

- Note that, from a device point of view, any type of Conventional Reset (cold, warm, hot, or DL_Down) has the same effect at the Transaction Layer and above as would RST# assertion and deassertion in conventional PCI.
- On exit from a Fundamental Reset, the Physical Layer will attempt to bring up the Link (see § Section 4.2.6). Once both components on a Link have entered the initial Link Training state, they will proceed through Link initialization for the Physical Layer and then through Flow Control initialization for VC0, making the Data Link and Transaction Layers ready to use the Link.
 - Following Flow Control initialization for VC0, it is possible for TLPs and DLLPs to be transferred across the Link.

Following exit from a Conventional Reset, some devices may require additional time before they are able to respond to Requests they receive. Particularly for Configuration Requests it is necessary that components and devices behave in a deterministic way, which the following rules address.

The first set of rules addresses requirements for components and devices:

- A component that supports Link speeds greater than 5.0 GT/s must enter the LTSSM Detect state within 100 ms of the end of Fundamental Reset (Link Training is described in § Section 4.2.5). A component that supports only Link speeds 5.0 GT/s or less must do this within 20 ms. All components are strongly encouraged to minimize this time period. This also applies to Retimers, see § Section 4.3.4 .
 - Note: In some systems, it is possible that the components on a Link may exit Fundamental Reset at different times. Each component must observe the requirement to enter the initial active Link Training state within the applicable time period based on the end of Fundamental Reset from its own point of view.
- On the completion of Link Training (entering the DL_Active state, see § Section 3.2), a component must be able to receive and process TLPs and DLLPs.
- Following exit from a Conventional Reset of a device, within 1.0 s the device must be able to receive a Configuration Request and return a Successful Completion if the Request is valid. This period is independent of how quickly Link training completes. If Readiness Notifications mechanisms are used (see § Section 6.22 .), this period may be shorter, or, with appropriate system support, longer.

The second set of rules addresses requirements placed on the system:

- To allow components to perform internal initialization, system software must wait a specified minimum period following exit from a Conventional Reset of one or more devices before it is permitted to issue Configuration Requests to those devices, unless Readiness Notifications mechanisms are used (see § Section 6.22). System software is also exempted from these minimum waiting period requirements if it knows of the specific device's requirements via means outside the scope of this specification.
- System software is permitted to immediately issue Configuration Requests to a device/Function if Immediate Readiness is indicated by the device/Function (see § Section 3.3 and § Section 7.9.16 for two ways Immediate Readiness support is permitted to be indicated)
- System software is permitted to immediately issue Configuration Requests to a device/Function below a Downstream Port if the DRS Message Received bit is Set.
- Devices that support Flit Mode are required to implement DRS
- Because DRS *MUST*@FLIT be supported, system software can use the Downstream Component Presence and Flit Mode Status fields in a Downstream Port to determine that DRS is supported by the attached Device. For cases where system software cannot determine that DRS is supported by the attached device, or by the Downstream Port above the attached device:
 - With a Downstream Port that does not support Link speeds greater than 5.0 GT/s, software must wait a minimum of 100 ms following exit from a Conventional Reset before sending a Configuration Request to the device immediately below that Port.

- With a Downstream Port that supports Link speeds greater than 5.0 GT/s, software must wait a minimum of 100 ms after Link training completes before sending a Configuration Request to the device immediately below that Port. Software can determine when Link training completes by polling the Data Link Layer Link Active bit or by setting up an associated interrupt (see § Section 6.7.3.3). It is strongly recommended for software to use this mechanism whenever the Downstream Port supports it.
- For a Device that implements the Readiness Time Reporting Extended Capability, and has reported a Reset Time shorter than 100 ms, software is permitted to send a Configuration Request to the Device after waiting the reported Reset Time from Conventional Reset.
- A system must guarantee that all components intended to be software visible at boot time are ready to receive Configuration Requests within the applicable minimum period based on the end of Conventional Reset at the Root Complex - how this is done is beyond the scope of this specification.
- It is strongly recommended that software use 100 ms wait periods only if software enables Configuration RRS Software Visibility. Otherwise, Completion timeouts, platform timeouts, or lengthy processor instruction stalls may result. See the Request Retry Status for Configuration Requests Implementation Note in § Section 2.3.1 .
- Unless Readiness Notifications mechanisms are used, the Root Complex and/or system software must allow at least 1.0 s following exit from a Conventional Reset of a device, before determining that the device is broken if it fails to return a Successful Completion status for a valid Configuration Request. This period is independent of how quickly Link training completes.
Note: This delay is analogous to the T_{rhfa} parameter specified for PCI/PCI-X, and is intended to allow an adequate amount of time for devices which require self initialization.
- When attempting a Configuration access to devices on a PCI or PCI-X bus segment behind a PCI Express/PCI(-X) Bridge, the timing parameter T_{rhfa} must be respected.

For this second set of rules, if system software does not have direct visibility into the state of Fundamental Reset (e.g., Hot-Plug; see § Section 6.7), software must base these timing parameters on an event known to occur after the end of Fundamental Reset.

When a Link is in normal operation, the following rules apply:

- If, for whatever reason, a normally operating Link goes down, the Transaction and Data Link Layers will enter the DL_Inactive state (see § Section 2.9 and § Section 3.2.1).
- For any Root or Switch Downstream Port, setting the Secondary Bus Reset bit of the Bridge Control Register associated with the Port must cause a hot reset to be sent (see § Section 4.2.5.9.2).
- For a Switch, the following must cause a hot reset to be sent on all Downstream Ports:
 - Setting the Secondary Bus Reset bit of the Bridge Control Register associated with the Upstream Port
 - The Data Link Layer of the Upstream Port reporting DL_Down status. In Switches that support Link speeds greater than 5.0 GT/s, the Upstream Port must direct the LTSSM of each Downstream Port to the Hot Reset state, but not hold the LTSSMs in that state. This permits each Downstream Port to begin Link training immediately after its hot reset completes. This behavior is recommended for all Switches.
 - Receiving a hot reset on the Upstream Port

Certain aspects of Fundamental Reset are specified in this document and others are specific to a platform, form factor and/or implementation. Specific platforms, form factors or application spaces may require the additional specification of the timing and/or sequencing relationships between the components of the system for Fundamental Reset. For example, it might be required that all PCI Express components within a chassis observe the assertion and deassertion of Fundamental Reset at the same time (to within some tolerance). In a multi-chassis environment, it might be necessary to specify that the chassis containing the Root Complex be the last to exit Fundamental Reset.

In all cases where power and PERST# are supplied, the following parameters must be defined:

- T_{pvperl} - PERST# must remain asserted at least this long after power becomes valid
- T_{perst} - When asserted, PERST# must remain asserted at least this long
- T_{fail} - When power becomes invalid, PERST# must be asserted within this time
- $T_{perstslew}$ - The slew rate of PERST# transition to deasserted through its logic input switching range. $T_{perstslew}$ is specified as a minimum of 50 mV/ns unless the form factor specification states otherwise.

Additional parameters may be specified.

In all cases where a reference clock is supplied, the following parameter must be defined:

- $T_{perst-clk}$ - PERST# must remain asserted at least this long after any supplied reference clock is stable

Additional parameters may be specified.

6.6.2 Function Level Reset (FLR) §

The FLR mechanism enables software to quiesce and reset Endpoint hardware with Function-level granularity. Three example usage models illustrate the benefits of this feature:

- In some systems, it is possible that the software entity that controls a Function will cease to operate normally. To prevent data corruption, it is necessary to stop all PCI Express and external I/O (not PCI Express) operations being performed by the Function. Other defined reset operations do not guarantee that external I/O operations will be stopped.
- In a partitioned environment where hardware is migrated from one partition to another, it is necessary to ensure that no residual “knowledge” of the prior partition be retained by hardware, for example, a user’s secret information entrusted to the first partition but not to the second. Further, due to the wide range of Functions, it is necessary that this be done in a Function-independent way.
- When system software is taking down the software stack for a Function and then rebuilding that stack, it is sometimes necessary to return the state to an uninitialized state before rebuilding the Function’s software stack.

Implementation of FLR is optional (not required), but is strongly recommended.

FLR applies on a per Function basis. Only the targeted Function is affected by the FLR operation. The Link state must not be affected by an FLR.

FLR modifies the Function state described by this specification as follows:

- Function registers and Function-specific state machines must be set to their initialization values as specified in this document, except that in the following cases, the values are not affected by FLR:
 - sticky-type registers (ROS, RWS, RW1CS)
 - registers defined as type HwInit
 - these other fields or registers:
 - Captured Slot Power Limit Value in the Device Capabilities Register
 - Captured Slot Power Limit Scale in the Device Capabilities Register
 - Max_Payload_Size in the Device Control Register

- Active State Power Management (ASPM) Control in the Link Control Register
- Read Completion Boundary (RCB) in the Link Control Register
- Common Clock Configuration in the Link Control Register
- Extended Synch in the Link Control Register
- Enable Clock Power Management in the Link Control Register
- Hardware Autonomous Width Disable in Link Control Register
- Hardware Autonomous Speed Disable in the Link Control 2 Register
- Link Equalization Request 8.0 GT/s in the Link Status 2 Register
-
- Enable Lower SKP OS Generation Vector in the Link Control 3 register
- Lane Equalization Control Register in the Secondary PCI Express Extended Capability structure
- LTR Mechanism Enable bit in the Device Control 2 Register if the LTR Enable Preserved Across FLR bit is Set
-
- All registers in the Virtual Channel Extended Capability structure
- All registers in the Multi-Function Virtual Channel Extended Capability structure
- All registers in the Streamlined Virtual Channel Extended Capability structure
- All registers in the Data Link Feature Extended Capability structure
- All registers in the Physical Layer 16.0 GT/s Extended Capability structure
- All registers in the Physical Layer 32.0 GT/s Extended Capability structure
- All registers in the Physical Layer 64.0 GT/s Extended Capability structure
- All registers in the Lane Margining at the Receiver Extended Capability structure
- All registers in the Flit Logging Extended Capability structure
- All registers in the Flit Error Injection Extended Capability structure
- All registers in the Flit Performance Measurement Extended Capability
- All registers in the NOP Flit Extended Capability
- CMA-SPDM session state
- The following registers *MUST@FLIT* also not reset to their initialization values:
 - ARI Control Register in the ARI Extended Capability Structure
 - All registers in the L1 PM Substates Extended Capability structure
 - All registers in the Latency Tolerance Reporting Extended Capability structure
 - All registers in the Precision Time Measurement Extended Capability structure
- It is strongly recommended that the following registers are also not reset to their initialization values:
 - All registers in the NPEM Extended Capability structure

Future revisions of this specification may change this recommendation to a requirement.

Note that the controls that enable the Function to initiate requests on PCI Express are cleared, including Bus Master Enable, MSI Enable, and the like, effectively causing the Function to become quiescent on the Link.

Note that Port state machines associated with Link functionality including those in the Physical and Data Link Layers are not reset by FLR, and VC0 remains initialized following an FLR.

- Any outstanding INTx interrupt asserted by the Function must be deasserted by sending the corresponding Deassert_INTx Message prior to starting the FLR.

Note that when the FLR is initiated to a Function of a Multi-Function Device, if another Function continues to assert a matching INTx, no Deassert_INTx Message will be transmitted.

After an FLR has been initiated by writing a 1b to the Initiate Function Level Reset bit, the Function must complete the FLR within 100 ms. If software initiates an FLR when the Transactions Pending bit is 1b, then software must not initialize the Function until allowing adequate time for any associated Completions to arrive, or to achieve reasonable certainty that any remaining Completions will never arrive. For this purpose, it is recommended that software allow as much time as provided by the pre-FLR value for Completion Timeout on the device. If Completion Timeouts were disabled on the Function when FLR was issued, then the delay is system dependent but must be no less than 100 ms. If Function Readiness Status (FRS - see § Section 6.22.2) is implemented, then system software is permitted to issue Configuration Requests to the Function immediately following receipt of an FRS Message indicating Configuration-Ready, however, this does not necessarily indicate that outstanding Requests initiated by the Function have completed.

Note that upon receipt of an FLR, a device Function may either clear all transaction status including Transactions Pending or set the Completion Timeout to its default value so that all pending transactions will time out during FLR execution. Regardless, the Transactions Pending bit must be clear upon completion of the FLR.

Since FLR modifies Function state not described by this specification (in addition to state that is described by this specification), it is necessary to specify the behavior of FLR using a set of criteria that, when applied to the Function, show that the Function has satisfied the requirements of FLR. The following criteria must be applied using Function-specific knowledge to evaluate the Function's behavior in response to an FLR:

- The Function must not give the appearance of an initialized adapter with an active host on any external interfaces controlled by that Function. The steps needed to terminate activity on external interfaces are outside of the scope of this specification.
 - For example, a network adapter must not respond to queries that would require adapter initialization by the host system or interaction with an active host system, but is permitted to perform actions that it is designed to perform without requiring host initialization or interaction. If the network adapter includes multiple Functions that operate on the same external network interface, this rule affects only those aspects associated with the particular Function reset by FLR.
- The Function must not retain within itself software readable state that potentially includes secret information associated with any preceding use of the Function. Main host memory assigned to the Function must not be modified by the Function.
 - For example, a Function with internal memory readable directly or indirectly by host software must clear or randomize that memory.
- The Function must return to a state such that normal configuration of the Function's PCI Express interface will cause it to be usable by drivers normally associated with the Function

When an FLR is initiated, the targeted Function must behave as follows:

- The Function must return the Completion for the configuration write that initiated the FLR operation and then initiate the FLR.
- While an FLR is in progress:
 - If a Request arrives, the Request is permitted to be silently discarded (following update of flow control credits) without logging or signaling it as an error.

- If a Completion arrives, the Completion is permitted to be handled as an Unexpected Completion or to be silently discarded (following update of flow control credits) without logging or signaling it as an error.
- While a Function is required to complete the FLR operation within the time limit described above, the subsequent Function-specific initialization sequence may require additional time. If additional time is required, the Function must return a Request Retry Status (RRS) Completion Status when a Configuration Request is received after the time limit above. After the Function responds to a Configuration Request with a Completion status other than RRS, it is not permitted to return RRS in response to a Configuration Request until it is reset again.

IMPLEMENTATION NOTE:

AVOIDING DATA CORRUPTION FROM STALE COMPLETIONS §

An FLR causes a Function to lose track of any outstanding Non-Posted Requests and UIO Requests. Any corresponding Completions that later arrive are referred to as being "stale". If software issues an FLR while there are outstanding Requests, and then re-enables the Function for operation without waiting for potential stale Completions, any stale Completions that arrive afterwards may cause data corruption by being mistaken by the Function as belonging to Requests issued since the FLR.

Software can avoid data corruption from stale Completions in a variety of ways. Here's a possible algorithm:

1. Software that's performing the FLR synchronizes with other software that might potentially access the Function directly, and ensures such accesses do not occur during this algorithm.
2. Software clears the entire Command register, disabling the Function from issuing any new Requests.
3. Software polls the Transactions Pending bit in the Device Status register either until it is clear or until it has been long enough that software is reasonably certain that Completions associated with any remaining outstanding Transactions will never arrive. On many platforms, the Transactions Pending bit will usually clear within a few milliseconds, so software might choose to poll during this initial period using a tight software loop. On rare cases when the Transactions Pending bit does not clear by this time, software will need to poll for a much longer platform-specific period (potentially seconds), so software might choose to conduct this polling using a timer-based interrupt polling mechanism.
4. Software initiates the FLR.
5. Software waits 100 ms.
6. Software reconfigures the Function and enables it for normal operation.

6.7 PCI Express Native Hot-Plug §

The PCI Express architecture is designed to natively support both hot-add and hot-removal ("hot-plug") of cables, add-in cards, and modules. PCI Express native hot-plug provides a "toolbox" of mechanisms that allow different user/operator models to be supported using a self-consistent infrastructure. These mechanisms may be used to implement orderly addition/removal that relies on coordination with the operating system (e.g., traditional PCI hot-plug), as well as async removal, which proceeds without lock-step synchronization with the operating system. This section defines the set of hot-plug mechanisms and specifies how the elements of hot-plug, such as indicators and push buttons, must behave if implemented in a system.

6.7.1 Elements of Hot-Plug §

§ Table 6-7 lists the physical elements comprehended in this specification for support of hot-plug models. A form factor specification must define how these elements are used in that form factor. For a given form factor specification, it is possible that only some of the available hot-plug elements are required, or even that none of these elements are required. In all cases, the form factor specification must define all assumptions and limitations placed on the system or the user by the choice of elements included. Silicon component implementations that are intended to be used only with selected form factors are permitted to support only those elements that are required by the associated form factor(s).

Table 6-7 Elements of Hot-Plug §

Element	Purpose
Indicators	Show the power and attention state of the slot
Manually-operated Retention Latch (MRL)	Holds adapter in place
MRL Sensor	Allows the Port and system software to detect the MRL being opened
Electromechanical Interlock	Prevents removal of adapter from slot
Attention Button	Allows user to request hot-plug operations
Software User Interface	Allows user to request hot-plug operations
Slot Numbering	Provides visual identification of slots
Power Controller	Software-controlled electronic component or components that control power to a slot or adapter and monitor that power for fault conditions
Out-of-band Presence Detect	Method of determining physical presence of an adapter in a slot that does not rely on the Physical Layer

6.7.1.1 Indicators §

Two indicators are defined: the Power Indicator and the Attention Indicator. Each indicator is in one of three states: on, off, or blinking. Hot-plug system software has exclusive control of the indicator states by writing the command registers associated with the indicator (with one exception noted below). The indicator requirements must be included in all form factor specifications. For a given form factor, the indicators may be required or optional or not applicable at all.

The hot-plug capable Port controls blink frequency, duty cycle, and phase of the indicators. Blinking indicators must operate at a frequency of between 1 and 2 Hz, with a 50% (+/- 5%) duty cycle. Blinking indicators are not required to be synchronous or in-phase between Ports.

Indicators may be physically located on the chassis or on the adapter (see the associated form factor specification for Indicator location requirements). Regardless of the physical location, logical control of the indicators is by the Downstream Port of the Upstream component on the Link.

The Downstream Port must not change the state of an indicator unless commanded to do so by software, except for platforms capable of detecting stuck-on power faults (relevant only when a power controller is implemented). In the case of a stuck-on power fault, the platform is permitted to override the Downstream Port and force the Power Indicator to be on (as an indication that the adapter should not be removed). The handling by system software of stuck-on faults is optional and not described in this specification. Therefore, the platform vendor must ensure that this feature, if implemented, is addressed via other software, platform documentation, or by other means.

6.7.1.1.1 Attention Indicator §

The Attention Indicator, which must be yellow or amber in color, indicates that an operational problem exists or that the hot-plug slot is being identified so that a human operator can locate it easily.

Table 6-8 Attention Indicator States §

Indicator Appearance	Meaning
Off	Normal - Normal operation
On	Attention - Operational problem at this slot
Blinking	Locate - Slot is being identified at the user's request

Attention Indicator Off

The Attention Indicator in the Off state indicates that neither the adapter (if one is present) nor the hot-plug slot requires attention.

Attention Indicator On

The Attention Indicator in the On state indicates that an operational problem exists at the adapter or slot.

An operational problem is a condition that prevents continued operation of an adapter. The operating system or other system software determines whether a specific condition prevents continued operation of an adapter and whether lighting the Attention Indicator is appropriate. Examples of operational problems include problems related to external cabling, adapter, software drivers, and power faults. In general, the Attention Indicator in the On state indicates that an operation was attempted and failed or that an unexpected event occurred.

The Attention Indicator is not used to report problems detected while validating the request for a hot-plug operation. Validation is a term applied to any check that system software performs to assure that the requested operation is viable, permitted, and will not cause problems. Examples of validation failures include denial of permission to perform a hot-plug operation, insufficient power budget, and other conditions that may be detected before a hot-plug request is accepted.

Attention Indicator Blinking

A blinking Attention Indicator indicates that system software is identifying this slot for a human operator to find. This behavior is controlled by a user (for example, from a software user interface or management tool).

6.7.1.1.2 Power Indicator §

The Power Indicator, which must be green in color, indicates the power state of the slot. § Table 6-9 lists the Power Indicator states.

Table 6-9 Power Indicator States §

Indicator Appearance	Meaning
Off	Power Off - Insertion or removal of the adapter is permitted.
On	Power On - Insertion or removal of the adapter is not permitted.
Blinking	Power Transition - Hot-plug operation is in progress and insertion or removal of the adapter is not permitted.

Power Indicator Off

The Power Indicator in the Off state indicates that insertion or removal of the adapter is permitted. Main power to the slot is off if required by the form factor. Note that, depending on the form factor, other power/signals may remain on, even when main power is off and the Power Indicator is off. In an example using the [CEM] form factor, if the platform provides Vaux to hot-plug slots and the MRL is closed, any signals switched by the MRL are connected to the slot even when the Power Indicator is off. Signals switched by the MRL are disconnected when the MRL is opened. System software must cause a slot's Power Indicator to be turned off when the slot is not powered and/or it is permissible to insert or remove an adapter. Refer to the appropriate form factor specification for details.

Power Indicator On

The Power Indicator in the On state indicates that the hot-plug operation is complete and that main power to the slot is On and that insertion or removal of the adapter is not permitted.

Power Indicator Blinking

A blinking Power Indicator indicates that the slot is powering up or powering down and that insertion or removal of the adapter is not permitted.

The blinking Power Indicator also provides visual feedback to the operator when the Attention Button is pressed or when hot-plug operation is initiated through the hot-plug software interface.

6.7.1.2 Manually-operated Retention Latch (MRL) §

An MRL is a manually-operated retention mechanism that holds an adapter in the slot and prevents the user from removing the device. The MRL rigidly holds the adapter in the slot so that cables may be attached without the risk of creating intermittent contact. MRLs that hold down two or more adapters simultaneously are permitted in platforms that do not provide MRL Sensors.

6.7.1.3 MRL Sensor §

The MRL Sensor is a switch, optical device, or other type of sensor that reports the position of a slot's MRL to the Downstream Port. The MRL Sensor reports closed when the MRL is fully closed and open at all other times (that is, if the MRL fully open or in an intermediate position).

If a power controller is implemented for the slot, the slot main power must be automatically removed from the slot when the MRL Sensor indicates that the MRL is open. If signals such as Vaux and SMBus are switched by the MRL, then these signals must be automatically removed from the slot when the MRL Sensor indicates that the MRL is open and must be restored to the slot when the MRL Sensor indicates that MRL has closed again. Refer to the appropriate form factor specification to identify the signals, if any, switched by the MRL.

Note that the Hot-Plug Controller does not autonomously change the state of either the Power Indicator or the Attention Indicator based on MRL sensor changes.

IMPLEMENTATION NOTE:

MRL SENSOR HANDLING §

In the absence of an MRL sensor, for some form factors, out-of-band presence detect may be used to handle the switched signals. In this case, when out-of-band presence detect indicates the absence of an adapter in a slot, the switched signals will be automatically removed from the slot.

If an MRL Sensor is implemented without a corresponding MRL Sensor input on the Hot-Plug Controller, it is recommended that the MRL Sensor be routed to power fault input of the Hot-Plug Controller. This allows an active adapter to be powered off when the MRL is opened.

6.7.1.4 Electromechanical Interlock §

An electromechanical interlock is a mechanism for physically locking the adapter or MRL in place until system software releases it. The state of the electromechanical interlock is set by software and must not change except in response to a subsequent software command. In particular, the state of the electromechanical interlock must be maintained even when power to the hot-plug slot is removed.

The current state of the electromechanical interlock must be reflected at all times in the Electromechanical Interlock Status bit in the Slot Status register, which must be updated within 200 ms of any commanded change. Software must wait at least 1 second after issuing a command to toggle the state of the Electromechanical Interlock before another command to toggle the state can be issued. Systems may optionally expand control of interlocks to provide physical security of the adapter.

6.7.1.5 Attention Button §

The Attention Button is a momentary-contact push button switch, located adjacent to each hot-plug slot or on the adapter that is pressed by the user to initiate a hot-plug operation at that slot. Regardless of the physical location of the button, the signal is processed and indicated to software by hot-plug hardware associated with the Downstream Port corresponding to the slot.

The Attention Button must allow the user to initiate both hot add and hot remove operations regardless of the physical location of the button.

If present, the Power Indicator provides visual feedback to the human operator (if the system software accepts the request initiated by the Attention Button) by blinking. Once the Power Indicator begins blinking, a 5-second abort interval exists during which a second depression of the Attention Button cancels the operation.

If an operation initiated by an Attention Button fails for any reason, it is recommended that system software present an error message explaining the failure via a software user interface or add the error message to a system log.

6.7.1.6 Software User Interface §

System software provides a user interface that allows hot insertions and hot removals to be initiated and that allows occupied slots to be monitored. A detailed discussion of hot-plug user interfaces is operating system specific and is therefore beyond the scope of this document.

On systems with multiple hot-plug slots, the system software must allow the user to initiate operations at each slot independent of the states of all other slots. Therefore, the user is permitted to initiate a hot-plug operation on one slot

using either the software user interface or the Attention Button while a hot-plug operation on another slot is in process, regardless of which interface was used to start the first operation.

6.7.1.7 Slot Numbering §

A Physical Slot Identifier (as defined in [PCI-Hot-Plug-1.1], § Section 1.5) consists of an optional chassis number and the physical slot number of the slot. The physical slot number is a chassis unique identifier for a slot. System software determines the physical slot number from registers in the Port. Chassis number 0 is reserved for the main chassis. The chassis number for other chassis must be a non-zero value obtained from a PCI-to-PCI Bridge's Chassis Number register (see [PCI-to-PCI-Bridge], Section 13.4).

Regardless of the form factor associated with each slot, each physical slot number must be unique within a chassis.

IMPLEMENTATION NOTE:

DETERMINATION OF SLOT NUMBER INFORMATION §

The Slot Numbering Capability, as defined in the PCI-to-PCI Bridge Specification, is being considered for deprecation in future versions of this specification. System software should use other means to identify physical slot number or chassis ID. Examples include ACPI _DSM for PCI Express Slot Number method (see [Firmware]), ACPI _SUN method (see [UEFI]), and the SMBIOS Type 9 structure (see [SMBIOS]).

6.7.1.8 Power Controller §

The power controller is an element composed of one or more discrete components that acts under control of software to set the power state of the hot-plug slot as appropriate for the specific form factor. The power controller must also monitor the slot for power fault conditions (as defined in the associated form factor specification) that occur on the slot's main power rails and, if supported, auxiliary power rail.

If a power controller is not present, the power state of the hot-plug slot must be set automatically by the hot-plug controller in response to changes in the presence of an adapter in the slot.

The power controller monitors main and auxiliary power faults independently. If a power controller detects a main power fault on the hot-plug slot, it must automatically set its internal main power fault latch and remove main power from the hot-plug slot (without affecting auxiliary power). Similarly, if a power controller detects an auxiliary power fault on the hot-plug slot, it must automatically set its internal auxiliary power fault latch and remove auxiliary power from the hot-plug slot (without affecting main power). Power must remain off to the slot as long as the power fault condition remains latched, regardless of any writes by software to turn on power to the hot-plug slot. The main power fault latch is cleared when software turns off power to the hot-plug slot. The mechanism by which the auxiliary power fault latch is cleared is form factor specific but generally requires auxiliary power to be removed from the hot-plug slot. For example, one form factor may remove auxiliary power when the MRL for the slot is opened while another may require the adapter to be physically removed from the slot. Refer to the associated form factor specifications for specific requirements.

Since the Power Controller Control bit in the Slot Control register reflects the last value written and not the actual state of the power controller, this means there may be an inconsistency between the value of the Power Controller Control bit and the state of the power to the slot in a power fault condition. To determine whether slot is off due to a power fault, software must use the power fault software notification to detect power faults. To determine that a requested power-up operation has otherwise failed, software must use the hot-plug slot power-up time out mechanism described in § Section 6.7.3.3 .

Software must not assume that writing to the Slot Control register to change the power state of a hot-plug slot causes an immediate power state transition. After turning power on, software must wait for a Data Link Layer State Changed event, as described in § Section 6.7.3.3 . After turning power off, software must wait for at least 1 second before taking any

action that relies on power having been removed from the hot-plug slot. For example, software is not permitted to turn off the power indicator (if present) or attempt to turn on the power controller before completing the 1 second wait period.

6.7.2 Registers Grouped by Hot-Plug Element Association §

The registers described in this section are grouped by hot-plug element to convey all registers associated with implementing each element. Register fields associated with each Downstream Port implementing a hot-plug capable slot are located in the Device Capabilities, Slot Capabilities, Slot Control, Slot Status, and Slot Capabilities 2 registers in the PCI Express Capability structure (see § Section 7.5.3). Registers reporting the presence of hot-plug elements associated with the device Function on an adapter are located in the Device Capabilities register (also in the PCI Express Capability structure).

6.7.2.1 Attention Button Registers §

Attention Button Present (Slot Capabilities Register and Device Capabilities Register) - This bit indicates if an Attention Button is electrically controlled by the chassis (Slot Capabilities Register) or by the adapter (Device Capabilities Register).

Attention Button Pressed (Slot Status Register) - This bit is Set when an Attention Button electrically controlled by the chassis is pressed.

Attention Button Pressed Enable (Slot Control Register) - When Set, this bit enables software notification on an Attention Button Pressed event (see § Section 6.7.3.4).

6.7.2.2 Attention Indicator Registers §

Attention Indicator Present (Slot Capabilities Register and Device Capabilities Register) - This bit indicates if an Attention Indicator is electrically controlled by the chassis (Slot Capabilities Register) or by the adapter (Device Capabilities Register).

Attention Indicator Control (Slot Control Register) - When written, sets an Attention Indicator electrically controlled by the chassis to the written state.

6.7.2.3 Power Indicator Registers §

Power Indicator Present (Slot Capabilities Register and Device Capabilities Register) - This bit indicates if a Power Indicator is electrically controlled by the chassis (Slot Capabilities Register) or by the adapter (Device Capabilities Register).

Power Indicator Control (Slot Control Register) - When written, sets a Power Indicator electrically controlled by the chassis to the written state.

6.7.2.4 Power Controller Registers §

Power Controller Present (Slot Capabilities Register) - This bit indicates if a Power Controller is implemented.

Power Controller Control (Slot Control Register) - Turns the Power Controller on or off according to the value written.

Power Fault Detected (Slot Status Register) - This bit is Set when a power fault is detected at the slot or the adapter.

Power Fault Detected Enable (Slot Control Register) - When Set, this bit enables software notification on a power fault event (see § Section 6.7.3.4).

6.7.2.5 Presence Detect Registers §

In-Band PD Disable Supported (Slot Capabilities 2 Register) - This bit indicates if the slot supports the disabling of in-band presence detect, which allows the out-of-band presence detect state to be reported independently of the in-band presence detect state.

In-Band PD Disable (Slot Control Register) - When Set, this bit disables the in-band presence detect mechanism from affecting the Presence Detect State bit, allowing that bit to be dedicated to reporting out-of-band presence detect.

Presence Detect State (Slot Status Register) - This bit indicates the presence of an adapter in the slot.

Presence Detect Changed (Slot Status Register) - This bit is Set when a presence detect state change is detected.

Presence Detect Changed Enable (Slot Control Register) - When Set, this bit enables software notification on a presence detect changed event (see § Section 6.7.3.4).

6.7.2.6 MRL Sensor Registers §

MRL Sensor Present (Slot Capabilities Register) - This bit indicates if an MRL Sensor is implemented.

MRL Sensor Changed (Slot Status Register) - This bit is Set when the value of the MRL Sensor state changes.

MRL Sensor Changed Enable (Slot Control Register) - When Set, this bit enables software notification on a MRL Sensor changed event (see § Section 6.7.3.4).

MRL Sensor State (Slot Status Register) - This register reports the status of the MRL Sensor if one is implemented.

6.7.2.7 Electromechanical Interlock Registers §

Electromechanical Interlock Present (Slot Capabilities Register) - This bit indicates if an Electromechanical Interlock is implemented.

Electromechanical Interlock Status (Slot Status Register) - This bit reflects the current state of the Electromechanical Interlock.

Electromechanical Interlock Control (Slot Control Register) - This bit when set to 1b toggles the state of the Electromechanical Interlock.

6.7.2.8 Command Completed Registers §

No Command Completed Support (Slot Capabilities Register) - This bit when set to 1b indicates that this slot does not generate software notification when an issued command is completed by the Hot-Plug Controller.

Command Completed (Slot Status Register) - This bit is Set when the Hot-Plug Controller completes an issued command and is ready to accept the next command.

Command Completed Interrupt Enable (Slot Control Register) - When Set, this bit enables software notification (see § Section 6.7.3.4) when a command is completed by the hot-plug control logic.

6.7.2.9 Port Capabilities and Slot Information Registers §

Slot Implemented (PCI Express Capabilities Register) - When Set, this bit indicates that the Link associated with this Downstream Port is connected to a slot.

Physical Slot Number (Slot Capabilities Register) - This hardware initialized field indicates the physical slot number attached to the Port.

Hot-Plug Capable (Slot Capabilities Register) - When Set, this bit indicates this slot is capable of supporting hot-plug.

Hot-Plug Surprise (Slot Capabilities Register) - When Set, this bit indicates that the Hot-Plug Surprise mechanism for handling async removal is enabled for this slot. See § Section 6.7.6 .

6.7.2.10 Hot-Plug Interrupt Control Register §

Hot-Plug Interrupt Enable (Slot Control Register) - When Set, this bit enables generation of the hot-plug interrupt on enabled hot-plug events.

6.7.3 PCI Express Hot-Plug Events §

A Downstream Port with hot-plug capabilities supports the following hot-plug events:

- Slot Events:
 - Attention Button Pressed
 - Power Fault Detected
 - MRL Sensor Changed
 - Presence Detect Changed
- Command Completed Events
- Data Link Layer State Changed Events

Each of these events has a status field, which indicates that an event has occurred but has not yet been processed by software, and an enable field, which indicates whether the event is enabled for software notification. Some events also have a capability field, which indicates whether the event type is supported on the Port. The grouping of these fields by event type is listed in § Section 6.7.2 , and each individual field is described in § Section 7.5.3 .

6.7.3.1 Slot Events §

A Downstream Port with hot-plug capabilities monitors the slot it controls for the slot events listed above. When one of these slot events is detected, the Port indicates that the event has occurred by setting the status field associated with the event. At that point, the event is pending until software clears the status field.

Once a slot event is pending on a particular slot, all subsequent events of that type are ignored on that slot until the event is cleared. The Port must continue to monitor the slot for all other slot event types and report them as they occur.

If enabled through the associated enable field, slot events must generate a software notification. If the event is not supported on the Port as indicated by the associated capability field, software must not enable software notification for the event. The mechanism by which this notification is reported to software is described in § Section 6.7.3.4 .

6.7.3.2 Command Completed Events §

Since changing the state of some hot-plug elements may not happen instantaneously, PCI Express supports hot-plug commands and command completed events. All hot-plug capable Ports are required to support hot-plug commands and, if the capability is reported, command completed events.

Software issues a command to a hot-plug capable Downstream Port by issuing a write transaction that targets any portion of the Port's Slot Control register. A single write to the Slot Control register is considered to be a single command, even if the write affects more than one field in the Slot Control register. In response to this transaction, the Port must carry out the requested actions and then set the associated status field for the command completed event. The Port must process the command normally even if the status field is already set when the command is issued. If a single command results in more than one action being initiated, the order in which the actions are executed is unspecified. All actions associated with a single command execution must not take longer than 1 second.

If command completed events are not supported as indicated by a value of 1b in the No Command Completed Support field of the Slot Capabilities register, a hot-plug capable Port must process a write transaction that targets any portion of the Port's Slot Control register without any dependency on previous Slot Control writes. Software is permitted to issue multiple Slot Control writes in sequence without any delay between the writes.

If command completed events are supported, then software must wait for a command to complete before issuing the next command. However, if the status field is not set after the 1 second limit on command execution, software is permitted to repeat the command or to issue the next command. If software issues a write before the Port has completed processing of the previous command and before the 1 second time limit has expired, the Port is permitted to either accept or discard the write. Such a write is considered a programming error, and could result in a discrepancy between the Slot Control register and the hot plug element state. To recover from such a programming error and return the controller to a consistent state, software must issue a write to the Slot Control register which conforms to the command completion rules.

If enabled through the associated enable field, the completion of a command must generate a software notification. The exception to this rule is a command that occurs as a result of a write to the Slot Control register that disables software notification of command completed events. Such a command must be processed as described above, but must not generate a software notification.

6.7.3.3 Data Link Layer State Changed Events §

The Data Link Layer State Changed event provides an indication that the state of the Data Link Layer Link Active bit in the Link Status Register has changed. Support for Data Link Layer State Changed events and software notification of these events are required for hot-plug capable Downstream Ports. If this event is supported, the Port sets the status field associated with the event when the value in the Data Link Layer Link Active bit changes.

This event allows software to indirectly determine when power has been applied to a newly hot-plugged adapter. Software must wait for 100 ms after the Data Link Layer Link Active bit reads 1b before initiating a configuration access to the hot added device (see § Section 6.6). Software must allow 1 second after the Data Link Layer Link Active bit reads 1b before it is permitted to determine that a hot plugged device which fails to return a Successful Completion for a Valid Configuration Request is a broken device (see § Section 6.6).

The Data Link Layer State Changed event must occur within 1 second of the event that initiates the hot-insertion. If a power controller is supported, the time out interval is measured from when software initiated a write to the Slot Control register to turn on the power. If the Power Disable mechanism is supported, the time out interval is measured from when that mechanism is deasserted (power is restored). If neither mechanism is supported, the time out interval is measured from presence detect slot event. Software is allowed to time out on a hot add operation if the Data Link Layer State Changed event does not occur within 1 second. The action taken by software after such a timeout is implementation specific.

6.7.3.4 Software Notification of Hot-Plug Events §

A hot-plug capable Downstream Port must support generation of an interrupt on a hot-plug event. As described in § Section 6.7.3.1 and § Section 6.7.3.2, each hot-plug event has both an enable bit for interrupt generation and a status bit that indicates when an event has occurred but has not yet been processed by software. There is also a Hot-Plug Interrupt Enable bit in the Slot Control register that serves as a master enable/disable bit for all hot-plug events.

If the Port is enabled for level-triggered interrupt signaling using the INTx messages, the virtual INTx wire must be asserted whenever and as long as the following conditions are satisfied:

- The Interrupt Disable bit in the Command register is set to 0b.
- The Hot-Plug Interrupt Enable bit in the Slot Control register is set to 1b.
- At least one hot-plug event status bit in the Slot Status register and its associated enable bit in the Slot Control register are both set to 1b.

Note that all other interrupt sources within the same Function will assert the same virtual INTx wire when requesting service.

If the Port is enabled for edge-triggered interrupt signaling using MSI or MSI-X, an interrupt message must be sent every time the logical AND of the following conditions transitions from FALSE to TRUE:

- The associated vector is unmasked (not applicable if MSI does not support PVM).
- The Hot-Plug Interrupt Enable bit in the Slot Control register is set to 1b.
- At least one hot-plug event status bit in the Slot Status register and its associated enable bit in the Slot Control register are both set to 1b.

Note that PME and Hot-Plug Event interrupts (when both are implemented) always share the same MSI or MSI-X vector, as indicated by the Interrupt Message Number field in the PCI Express Capabilities Register.

The Port may optionally send an MSI when there are hot-plug events that occur while interrupt generation is disabled, and interrupt generation is subsequently enabled.

If wake generation is required by the associated form factor specification, a hot-plug capable Downstream Port must support generation of a wakeup event (using the PME mechanism) on hot-plug events that occur when the system is in a sleep state or the Port is in device state D1, D2, or D3_{Hot}.

Software enables a hot-plug event to generate a wakeup event by enabling software notification of the event as described in § Section 6.7.3.1. Note that in order for software to disable interrupt generation while keeping wakeup generation enabled, the Hot-Plug Interrupt Enable bit must be cleared. For form factors that support wake generation, a wakeup event must be generated if all three of the following conditions occur:

- The status register for an enabled event transitions from Clear to Set
- The Port is in device state D1, D2, or D3_{Hot}, and
- The PME_En bit in the Port's Power Management Control/Status register is Set

Note that the Hot-Plug Controller generates the wakeup on behalf of the hot-plugged device, and it is not necessary for that device to have auxiliary (or main) power.

6.7.4 System Firmware Intermediary (SFI) Support §

The System Firmware Intermediary (SFI) Capability is an optional normative feature of a Downstream Port. Some SFI functionality is focused on hot-pluggable slots, as indicated by the Hot-Plug Capable bit in the Slot Capabilities register being Set, while some SFI functionality is useful outside that context. If a Downstream Port supports an SFI Extended Capability structure, the following bits must be Set:

- Data Link Layer Link Active Reporting Capable bit in the Link Capabilities register
- DRS Supported bit in the Link Capabilities 2 register
- ERR_COR Subclass Capable bit in the Device Capabilities register

As originally introduced, SFI is focused on system firmware running on the host CPU(s) as the intermediary. Enhanced SFI (eSFI) adds important new functionality for improved support of the BMC as the intermediary, as well as increased protection against malicious components or adapters. Enhanced SFI also makes SFI CAM support optional instead of mandatory for Root Ports, easing the implementation of eSFI on Root Ports intended for platforms that have no need to support system firmware running on the host CPU(s) as the intermediary.

6.7.4.1 SFI ERR_COR Event Signaling §

The SFI Extended Capability has no support for generating INTx or MSI/MSI-X interrupts, since the capability is intended for use by system firmware.

A Downstream Port with SFI must support ERR_COR signaling, regardless of whether it supports Advanced Error Reporting (AER) or not. SFI ERR_COR event signaling is enabled independently by the SFI OOB PD Changed Enable, SFI DLL State Changed Enable, and SFI DRS Signaling Enable bits in the SFI Control Register. These events are indicated by the SFI OOB PD Changed, SFI DLL State Changed, and SFI DRS Received bits in the SFI Status Register.

If the Correctable Error Reporting Enable bit in the Device Control Register is Set, the Port must send an ERR_COR Message each time one of the enabled conditions becomes satisfied. SFI ERR_COR event signaling must not Set the Correctable Error Detected bit in the Device Status Register, since this event is not handled as an error.

IMPLEMENTATION NOTE: ERR_COR SIGNALING FOR DPC DL_ACTIVE VS. SFI DLL STATE CHANGED §

DPC implements ERR_COR signaling for DL_Active, whereas SFI implements ERR_COR signaling for SFI DLL State Changed, which are related but non-identical conditions. The DL_Active condition occurs when the Data Link Layer Link Active bit in the Link Status register changes from 0b to 1b, and this bit can be masked by the SFI DLL State Mask bit in the SFI Control register. The SFI DLL State Changed condition occurs when the SFI DLL State bit in the SFI Status Register changes its value either by becoming Set or becoming Clear, and this condition is always based on the actual Data Link Layer state.

6.7.4.2 SFI Downstream Port Filtering (DPF) §

Downstream Port Filtering (DPF) is a mechanism where a Downstream Port can filter, or block, specified TLPs passing through the Downstream Port. A filtered Request TLP is handled as an Unsupported Request and a filtered Completion TLP is silently discarded (following update of flow control credits for upstream flowing TLPs).

DPF supports up to three modes of filtering. The first two modes filter Downstream Request TLPs that are passing through the Downstream Port other than Configuration Request TLPs generated by that Port's SFI CAM mechanism. The first mode filters all such Request TLPs; the second mode filters only Downstream Configuration Request TLPs not generated by the Port's SFI CAM mechanism. Other TLPs must not be filtered or blocked by these two modes.

Enhanced SFI also supports a third mode of filtering that allows Upstream and Downstream MCTP VDMs to be passed through the Downstream Port. In addition, this third mode does not filter Configuration Request TLPs generated by PCIe-MI accesses to the Port's SFI CAM mechanism or the Completion TLPs for those Configuration Request TLPs. All other TLPs are filtered.

One key use case for DPF is guaranteeing that asynchronous system software activities like bus scans do not unintentionally send Configuration Requests to devices that are not yet ready following a Conventional Reset, since such accesses result in undefined hardware behavior. See § Section 6.6.1 .

Another key use case for DPF is supporting firmware first functionality, enabling system firmware running on the host CPU(s), when notified of an async hot add, to configure the newly added device before making the device visible to the operating system. For this use case, the SFI CAM mechanism enables the Downstream Port itself to generate Configuration Request TLPs targeting Downstream Components, and those TLPs and their corresponding Completion TLPs are not filtered or blocked by the DPF mechanism. See § Section 6.7.4.3 , § Section 7.9.20.5 , and § Section 7.9.20.6 .

The use case for the third DPF mode (block all TLPs except MCTP VDMs and PCIe-MI initiated SFI CAM accesses) is for enabling BMC management of components or adapters using [MCTP-VDM] and the SFI CAM without permitting other PCIe traffic, which malicious components or adapters might use as attack vectors against the system. Key use case examples include authenticating a component or adapter with Component Measurement and Authentication (CMA-SPDM) and configuring a component or adapter with PCIe-MI.

6.7.4.3 SFI CAM §

The SFI Configuration Access Method (CAM) provides a means for SFI-aware system firmware to have the Downstream Port proxy (pass through) Configuration Requests targeting Components below the Downstream Port when DPF is enabled. If supported, the SFI CAM is always enabled.

The following rules apply to SFI CAM support (see the SFI CAM Unsupported bit and IMPLEMENTATION NOTE: SFI CAM SUPPORT CONSIDERATIONS).

- For Root Ports that support eSFI, SFI CAM support is optional when accessed using Configuration Read Requests or Configuration Writes Requests.
- For Root Ports that do not support eSFI, SFI CAM support is mandatory when accessed using Configuration Read Requests or Configuration Writes Requests.
- For Switch Downstream Ports that support SFI or eSFI, SFI CAM support is mandatory when accessed using Configuration Read Requests or Configuration Writes Requests.
- For all Downstream Ports that support SFI or eSFI, SFI CAM support is optional when accessed using PCIe-MI.

If the SFI RO In-Band bit is Set, Configuration Read Requests and Configuration Write Requests to the SFI CAM are impacted. See § Section 6.7.4.8 for more detail.

To use the SFI CAM in cases where the SFI CAM is supported and writeable, software first writes to the SFI CAM Address Register, specifying the target Configuration address. Software then reads or writes the SFI CAM Data Register to cause a proxied Configuration Request to be generated and transmitted to the Downstream Component.

The following rules apply when the SFI CAM is accessed using Configuration Request TLPs:

- All TLP fields used for the proxied Configuration Request are identical to those in the Configuration Request that targeted the SFI CAM Data Register, with the following exceptions:

- The target Bus Number, Device Number, and Function Number come from the SFI CAM Address Register.
- The Extended Register Number and Register Number come from the SFI CAM Address Register.
- The LCRC is regenerated.
- If present, the ECRC is regenerated.
- If a Completion TLP flowing Upstream is permitted to pass through the Downstream Port (e.g., the Completion is not blocked due to the SFI Downstream Port Filtering field being set to a value of 11b), the Completion TLP must be passed through the Downstream Port unmodified.
- If there is a detected error associated with the proxied Configuration Request, this is a reported error associated with the Downstream Port implementing the SFI CAM (see § Section 6.2).

The following rules apply when the SFI CAM is accessed using PCIe-MI:

- All TLP fields used for the proxied Configuration Request are populated by the Port as necessary to satisfy the corresponding PCIe-MI access.
- If there is a detected error associated with the proxied Configuration Request, this is a reported error in the PCIe-MI Completion.

The following rules apply when the SFI CAM is accessed using Configuration Request TLPs or PCIe-MI:

- The SFI CAM must not apply the Completion Timeout mechanism to the Request.
- The intermediary must ensure that between the time it writes to the SFI CAM Address Register and its subsequent read or write of the SFI CAM Data Register completes, no other threads modify the SFI CAM Address Register; otherwise, the result is undefined.

IMPLEMENTATION NOTE:

USE OF ASSIGNED BUS NUMBERS WITH THE SFI CAM §

When a Downstream Port has DPF enabled, the SFI CAM, if supported, can be used by SFI-aware system firmware to configure and access the sub-hierarchy below the Port without other software being able to do so. While the Bus Number configuration below the Port is generally not visible to other software, Bus Numbers configured for use below the Port should be limited to those already assigned to the Port since TLPs coming Upstream through the Port may contain IDs with the configured Bus Numbers. If any errors are detected and logged with those TLPs, the Bus Numbers can become visible to other software, creating confusion if they overlap with Bus Numbers used elsewhere in the system.

IMPLEMENTATION NOTE:

SFI CAM SUPPORT CONSIDERATIONS §

SFI as originally architected prior to eSFI support being architected requires implementing the SFI CAM and was intended for use by system firmware running on the host CPU(s) to configure Functions hidden from the view of the operating system running on the host CPU(s) using SFI Downstream Port Filtering. If eSFI is supported, SFI CAM support is optional under certain conditions. See the SFI CAM Unsupported bit. By permitting the SFI CAM to be optional, this eases the implementation of eSFI. However, there are other considerations besides this key benefit.

Enterprise and cloud systems use BMCs extensively and may require support for a BMC to access the SFI CAM (e.g., using PCIe-MI) to generate Configuration Read Requests and Configuration Write Requests to Functions below the Port, allowing a BMC to configure such Functions, which may be necessary if the Functions below the Port do not support a way for the BMC to access the Functions' Configuration Space directly (e.g., using PCIe-MI).

Other systems, e.g., those used for client, mobile, embedded, or automotive, may find it more suitable or even necessary to use the operating system running on the host CPU(s) as the intermediary. This specification does not architect such use of SFI, but such use is not forbidden. In fact, the PCI Firmware Specification architects an _OSC mechanism for system firmware/operating system negotiation of SFI ownership, though it recommends that general-purpose operating systems never request ownership, and general-purpose platforms never grant ownership to an operating system if requested.

Additionally, any system software running on the host CPU(s) developed for SFI as originally architected may assume that the SFI CAM is implemented and supported, and such software may break if an unimplemented SFI CAM might be mistaken as being fully functional. If an eSFI-capable Port implementation does not support the SFI CAM, the SFI CAM registers are required to behave “as if” the Link below the Downstream Port is in DL_Down. See the SFI CAM Data register. This mandatory semantic helps reduce the risk of non-eSFI-aware software breaking when the SFI CAM isn't supported.

Since system firmware running on the host CPU(s) is usually customized for the system's Root Complex(es), there is less concern for non-eSFI-aware system firmware running on the host CPU(s) possibly needing to be modified to accommodate eSFI-capable Root Ports that do not support the SFI CAM.

See the SFI Hidden In-Band bit and the SFI RO In-Band bit for other approaches to avoid implementation complexities due to SFI CAM.

IMPLEMENTATION NOTE:

SERIALIZED USE OF THE SFI CAM ADDRESS AND DATA REGISTERS §

As described above, system firmware must ensure that between the time it writes to the SFI CAM Address Register and its subsequent read or write of the SFI CAM Data Register completes, no other threads modify the SFI CAM Address Register. For example, a semaphore or other synchronization mechanism can be used to ensure this serialization.

For platforms where a processor store instruction to Configuration Space is effectively posted, software must still ensure that the resulting Configuration Write completes before another software thread modifies the SFI CAM Data Register. On such platforms, the mechanism for determining when a Configuration Write completes is platform specific.

Given appropriate serialization, the SFI CAM works correctly with Configuration Requests that result in RRS Completions, even when the Root Complex automatically re-issues the Configuration Request as a new Request. The re-issued Configuration Request will again be sent to the SFI CAM Data Register, and the associated Downstream Port will again generate a Configuration Request targeting the Downstream Component. As long as the SFI CAM Address Register isn't modified by other software until the Configuration Request completes, the sequence can repeat indefinitely until a non-RRS Completion is returned or a Completion Timeout occurs.

When Configuration RRS Software Visibility is enabled, the SFI CAM still works correctly with Configuration Requests that result in RRS Completions. Any Completions with a RRS Completion Status flow back to the original Requester, which handles them as required by Configuration RRS Software Visibility semantics. See § Section 2.3.2.

6.7.4.4 SFI Interactions with Readiness Notifications §

The SFI Extended Capability is able to mask the reporting of received Device Readiness Status (DRS) Messages as well as emulate them being received. This functionality is useful when SFI's Downstream Port Filtering (DPF) mechanism is being used to block operating system visibility of a device or sub-hierarchy below the Downstream Port.

Rules:

- When the SFI DRS Mask bit is Set, the DRS Message Received bit in the Link Status 2 Register value must be 0b.
- The SFI DRS Received bit must always indicate the actual state of the DRS Message Received condition.
- When the SFI DRS Mask bit is Clear and a 1b is written to the SFI DRS Trigger bit, the Downstream Port must behave as if a DRS Message was received.

IMPLEMENTATION NOTE:

SFI TRANSPARENT OPTIMIZATIONS FOR DEVICE READINESS §

Certain devices may need more time to become Configuration-Ready following a hot-add operation than permitted. See § Section 6.6.1 .

If system firmware is aware of such devices, it can use the SFI DPF mechanism to block operating system visibility of a newly added device, wait the necessary amount of time for the device to become Configuration-Ready, and then expose the device to the operating system.

To avoid the operating system from unnecessarily waiting additional time for the newly exposed device to become Configuration-Ready, system firmware can use the SFI DRS Trigger bit to have the Downstream Port emulate the reception of a DRS Message. An operating system that supports DRS can then immediately discover and configure the newly exposed device.

The newly exposed device doesn't necessarily need to be DRS capable itself. Since an Upstream Port is expressly permitted to send DRS Messages even when its DRS Supported bit is Clear, the Downstream Port above it can legitimately emulate receiving a DRS Message from it even if it is incapable of sending DRS Messages.

It should also be noted that in cases where system firmware is aware of a device becoming Configuration-Ready early, system firmware can expose this to the operating system using the SFI DRS Trigger mechanism.

Although SFI is not intended to be used by operating system software, it is recommended that operating systems used in platforms supporting SFI implement support for DRS, so that the system as a whole can have the benefits of this optimized Device Readiness timing.

IMPLEMENTATION NOTE:

SFI DPF AND FUNCTION READINESS STATUS (FRS) MESSAGES §

Downstream Port Filtering (DPF) does not affect the generation or propagation of FRS Messages. No FRS Messages are generated by a device when it becomes ready as part of an async hot-add operation. However, if system firmware performs operations on a device that result in FRS events, the resulting FRS Messages may be visible to the operating system. See § Section 2.2.8.6.3 and § Section 6.22.2 .

6.7.4.5 SFI Suppression of Hot-Plug Surprise Functionality §

If a slot supports Hot-Plug Surprise (HPS) functionality as indicated by the Hot-Plug Surprise bit in the Slot Capabilities Register being Set, the SFI HPS Suppress bit in the SFI Control Register can be used to force the Hot-Plug Surprise bit to be Clear, and disable the associated Hot-Plug Surprise functionality.

HPS suppression is useful when a Downstream Port / slot combination supports both HPS and Downstream Port Containment (DPC). DPC is not recommended for concurrent use with HPS, so if a slot has HPS capability enabled, DPC should not be enabled. If software wishes to use DPC, software should first Set the SFI HPS Suppress bit in order to disable HPS functionality, allowing DPC to function properly.

IMPLEMENTATION NOTE: SOFTWARE NEGOTIATION OF HOT-PLUG SURPRISE FUNCTIONALITY §

Assuming that system firmware owns the SFI Extended Capability structure, it is recommended that for backward compatibility with older operating systems, Hot-Plug Surprise functionality be enabled by default on slots supporting async removal. Then, if the slot also supports DPC and the operating system wishes to use it instead, the operating system will request that HPS be suppressed by system firmware, and system firmware will determine whether to Set or Clear the SFI HPS Suppress bit.

6.7.4.6 SFI Quarantine Mode §

The SFI Quarantine Mode field and the SFI Quarantine bit provide an intermediary (e.g., a BMC) with the ability to configure an eSFI-capable Port to be quarantined (i.e., Functions below the Port are hidden from system software running on the host CPU(s) and blocked from injecting traffic into the platform over the Link) under a variety of conditions such as Conventional Reset of the Port, hot removal of an adapter connected to the Port, or the LTSSM of the Link below the Port reporting DL_Down status. Quarantining of a Port is accomplished by directing the LTSSM of the Link below the Port to the Disabled state and for hot-plug use cases, masking the presence detect state by keeping the Presence Detect State bit in the Slot Status Register Cleared. Once quarantined, a Port remains quarantined until the SFI quarantine mode is disabled (i.e., the SFI Quarantine bit is Cleared).

A Port transitions to SFI quarantine when the SFI Quarantine bit is Set which occurs in the following conditions.

- The SFI Quarantine Mode field is set to 01b and the actual presence detect state is deasserted (i.e., the SFI PD State bit or the SFI OOB PD State bit transitions from Set to Clear).
 - See IMPLEMENTATION NOTE: SFI QUARANTINE UPON PRESENCE DETECT STATE DEASSERTION and IMPLEMENTATION NOTE: SECURITY AND FUNCTIONAL RISKS WHEN USING IN-BAND PRESENCE DETECT for information on the use case for this mode.
- The SFI Quarantine Mode field is set to 10b and a) the actual presence detect state is deasserted (i.e., the SFI PD State bit or the SFI OOB PD State bit transitions from Set to Clear) or b) the Port's Data Link Layer reports DL_Down status.
 - See IMPLEMENTATION NOTE: SFI QUARANTINE UPON PRESENCE DETECT STATE DEASSERTION or DL_DOWN for information on the use case for this mode.
- The SFI Quarantine Mode field transitions to a value of 11b (e.g., due to a Conventional Reset of Port if the default value of the SFI Quarantine Mode field is 11b or due to the intermediary modifying the SFI Quarantine Mode field to a value of 11b).
 - See IMPLEMENTATION NOTE: SFI QUARANTINE AS THE DEFAULT UPON CONVENTIONAL RESET for information on the use case for this mode.

If the SFI Quarantine bit is Set, the following rules apply.

- The Port's LTSSM is immediately directed to the Disabled state until the SFI Quarantine bit is Cleared.
- If the Port's actual presence detect state is deasserted (i.e., because the SFI PD State bit or the SFI OOB PD State bit transitioned from Set to Clear), the Port masks (i.e., Clears) the Presence Detect State bit until the SFI Quarantine bit is Cleared.

Once the Port is taken out of SFI quarantine (i.e., the SFI Quarantine bit is Cleared), the following rules apply.

- The Port's LTSSM must transition to the Detect State where the Link will attempt to retrain, unless otherwise specified (e.g., the Link is disabled due to another condition such as the Link Disable bit being Set).
 - The intermediary can use the SFI DLL State bit or the SFI DLL State Changed bit to monitor when the Link reaches the DL_Active state again.
 - Once the Link reaches the DL_Active state (i.e., the SFI DLL State bit is Set), software is permitted to modify the value of the SFI Quarantine Mode field or the SFI Quarantine bit.
 - After re-enabling the Link, software must honor the timing requirements defined in § Section 6.6.1 with respect to the first Configuration Read following a Conventional Reset.
- The value of the Presence Detect State bit must indicate the actual state, unless otherwise specified (e.g., the Presence Detect State bit is masked by the SFI PD State Mask bit).
 - If the value of the Presence Detect State bit changes when the SFI Quarantine bit value changes, this must cause a Presence Detect Changed event (see § Section 6.7.3).

IMPLEMENTATION NOTE:

SFI QUARANTINE UPON PRESENCE DETECT STATE DEASSERTION §

One of the possible use cases for this mode, enabled if the SFI Quarantine Mode field is Set to 01b, is to permit BMCs acting as the intermediary to authenticate (e.g., using CMA-SPDM) and configure (e.g., using PCIe-MI) an adapter that is hot inserted into a slot after a prior removal of an adapter from that slot.

Example:

- Upon system boot, the BMC enables this mode on hot-plug capable Ports of slots with an adapter installed.
- The system is booted into the operating system running on the host CPU(s).
- If the adapter is removed, the Presence Detect State bit will be Cleared due to the removal, the Port will immediately direct the LTSSM to the Disabled state, and the Port will begin masking (i.e., Clearing) the Presence Detect State bit.
- The Port will continue to direct the LTSSM to the Disabled state and will continue to mask the Presence Detect State bit until the BMC Clears the SFI Quarantine bit, ensuring that any subsequently inserted adapter is not exposed to the operating system running on the host CPU(s).
- The user inserts another adapter into the slot.
- The Link remains in the Disabled state, ensuring the adapter is not able to inject traffic into the platform over the Link.
- Since the Link remains in the Disabled state and the Presence Detect State bit remains masked, no Data Link Layer State Changed or Presence Detect Changed event is generated to notify the operating system running on the host CPU(s) of the newly inserted adapter.
- Any Configuration Read Requests attempting to pass through the quarantined Port would result in a UR Completion Status due to the Link being in the Disabled state, which hides the Adapter from any bus scans initiated by the operating system running on the host CPU(s).
- While the LTSSM is in the Disabled state and the adapter is hidden from the view of the operating system running on the host CPU(s), the BMC may authenticate/configure the adapter using a sideband interface (e.g., MCTP over SMBus/I2C or MCTP over USB).
- Once the BMC has authenticated/configured the adapter, the BMC Clears the SFI Quarantine bit which enables the Link and unmask Presence Detect State in order to expose the adapter to the operating system running on the host CPU(s).

IMPLEMENTATION NOTE: SECURITY AND FUNCTIONAL RISKS WHEN USING IN-BAND PRESENCE DETECT §

If in-band presence detect is enabled, the presence detect state deassertion due to an adapter removal while the LTSSM is in certain states (e.g., the L1 or Disabled states) is not detected because in-band presence detect state is required to be 1b in those states, regardless of whether the adapter is present or not.

In these cases, if the Port is configured to direct the Link to the Disabled state upon presence detect state deassertion (i.e., the SFI Quarantine Mode field is Set to 01b), the LTSSM will not be directed to the Disabled state due to adapter removal. The BMC may be able to detect the removal and configure the Port to hide a subsequently inserted adapter from the operating system by Setting the SFI DLL State Mask bit and the SFI PD State Mask bit; however, if the subsequent insertion of another adapter occurs before the BMC has an opportunity to set these bits, the newly inserted adapter will be inadvertently exposed to the operating system running on the host CPU(s) prior to the BMC having the opportunity to authenticate/configure the newly inserted adapter. See IMPLEMENTATION NOTE: IN-BAND PRESENCE DETECT MECHANISM DEPRECATED FOR ASYNC HOT-PLUG for related functional issues.

To avoid this race condition for slots where the BMC is required to authenticate/configure a newly inserted adapter prior to exposing the newly inserted adapter to the operating system running on the host CPU(s), it is strongly recommended that the slot implement out-of-band presence detect, the Downstream Port implement the In-Band PD Disable bit, and in-band presence detect be disabled by Setting the In-Band PD Disable bit.

IMPLEMENTATION NOTE:

SFI QUARANTINE UPON PRESENCE DETECT STATE DEASSERTION OR DL_DOWN §

This mode, enabled if the SFI Quarantine Mode field is Set to 10b, works similar to the mode that directs the LTSSM to the Disabled state upon presence detect state deassertion (see IMPLEMENTATION NOTE: IN-BAND PRESENCE DETECT MECHANISM DEPRECATED FOR ASYNC HOT-PLUG). But in addition to directing the LTSSM to the Disabled state upon presence detect state deassertion, this mode also directs the LTSSM to the Disabled state upon the Port's Data Link Layer reporting DL_Down status.

This mode is intended to handle cases where in-band presence detect is not disabled (e.g., because the slot does not support out-of-band presence detect). In this case, presence detection does not detect the removal of a device when the LTSSM is in certain states (e.g., the L1 or Disabled states) since in-band presence detect state is required to be 1b in those states. Directing the LTSSM to the Disabled state upon the Port's Data Link Layer reporting DL_Down status ensures that an adapter, inserted into a slot after the removal of another adapter in that slot, is not exposed to the operating system running on the host CPU(s) until the BMC has had an opportunity to authenticate/configure the newly inserted adapter.

This mode is intended to handle cases where in-band presence detect is not disabled (e.g., because the slot does not support out-of-band presence detect). In this case, presence detection does not detect the removal of a device when the LTSSM is in certain states (e.g., the L1 or Disabled states) since in-band presence detect state is required to be 1b in those states. Directing the LTSSM to the Disabled state upon the Port's Data Link Layer reporting DL_Down status ensures that an adapter, inserted into a slot after the removal of another adapter in that slot, is not exposed to the operating system running on the host CPU(s) until the BMC has had an opportunity to authenticate/configure the newly inserted adapter.

Note that if this mode is enabled and the operating system running on the host CPU(s) does cause a DL_Down (e.g., due to setting the Secondary Bus Reset bit or setting the Link Disable bit), after re-enabling the Link (e.g., by clearing the Secondary Bus Reset bit or clearing the Link Disable bit), the operating system waits the amount of time defined in Section 6.6.1 with respect to the first Configuration Read following a Conventional Reset. If the BMC has not re-exposed the component within this amount of time, the operating system may time out. The mechanism the BMC uses to detect the DL_Down in a timely manner is outside the scope of this specification.

IMPLEMENTATION NOTE:

SFI QUARANTINE AS THE DEFAULT UPON CONVENTIONAL RESET §

The SFI Quarantine Mode field is HwInit allowing a Port to set the default value upon Conventional Reset to 11b. In this mode, the Port's LTSSM is directed to the Disabled state upon Conventional Reset of the Port (e.g., due to platform power on or platform reset). This mode enables a “security quarantine” of the Downstream component, preventing the component from being exposed to system software running on the host CPU(s) or being able to inject traffic into the platform over the Link until the BMC has authenticated the component and enabled the Link.

6.7.4.7 SFI Extended Capability In-Band Hiding §

If the SFI Hidden In-Band bit is Set, the SFI Extended Capability structure and all fields within the SFI Extended Capability structure must not be readable or writable when accessed using Configuration Read Requests or Configuration Write Requests (e.g., initiated by in-band management system software).

If the SFI Hidden In-Band bit is Clear, accesses to the SFI Extended Capability follows all rules for an extended capability structure.

The value of the SFI Hidden In-Band bit must have no effect on accesses to the SFI Extended Capability using PCIe-MI.

IMPLEMENTATION NOTE: USE OF SFI HIDDEN IN-BAND §

This “hidden in-band” feature is intended for use when this SFI Extended Capability is configured by a BMC using PCIe-MI or other out-of-band management mechanisms. Setting the SFI Hidden In-Band bit hides the SFI Extended Capability when accessed by Configuration Read Requests or Configuration Write Requests (e.g., initiated by in-band management system software).

Hiding the SFI Extended Capability from system software running on the host CPU(s) prevents inadvertent or intentional security exploits from system software running on the host CPU(s) (e.g., by preventing system software running on the host CPU(s) from being able to modify the SFI Capability Structure to unblock a component the BMC has not authenticated or that has failed authentication). But hiding the SFI Extended Capability also prevents system software running on the host CPU(s) from being able to read the SFI Extended Capability for information that may be useful (e.g., the SFI OOB PD Supported bit).

If system software running on the host CPU(s) requires the ability to read the SFI Extended Capability but must be prevented from modifying the SFI Extended Capability Structure, the functionality enabled by Setting the SFI RO In-Band bit should be used instead of hiding the SFI Extended Capability.

The SFI Hidden In-Band bit is HwInit allowing a Port to set the default value upon Conventional Reset to 1b such that the SFI Extended Capability is always hidden when accessed by Configuration Read Requests or Configuration Write Requests. This option may be used to simplify implementations since it does not require any functionality related to SFI Extended Capability when access using Configuration Read Requests or Configuration Write Requests to be developed or validated.

The SFI OOB PD Supported bit may be useful to the operating system running on the host CPU(s) for configuring hot-plug related functionality and determining when an adapter is present, but that bit is not visible if the SFI Hidden In-Band bit is Set. In this case, the SCap2 OOB PD Supported bit (if implemented) can be used by software instead.

For systems that support firmware first DPC, the SFI HPS Suppress bit supports the use case defined by the _DSM for Downstream Port Containment and Hot-Plug Surprise Control in the PCI Firmware Specification. If the SFI Extended Capability is hidden on such systems when accessed by Configuration Read Requests or Configuration Write Requests, mechanisms outside the scope of this specification may be required to support this use case.

6.7.4.8 SFI Extended Capability In-Band Read-Only §

If the SFI RO In-Band [hotlink] bit is Set, Configuration Read Requests (e.g., initiated by in-band management [hotlink] system software) to the SFI Extended Capability [hotlink] are processed with normal semantics, except Configuration

Read Requests to the SFI CAM Data [hotlink] register which must return a value of 0h without generating a Configuration Read Request on the Link below the Port. If the SFI RO In-Band [hotlink] bit is Set and a Configuration Write Request accesses the SFI CAM Data [hotlink] register, the value of the SFI CAM Data [hotlink] register must not be altered and a Configuration Write Request must not be generated on the Link below the Port.

If the SFI RO In-Band [hotlink] bit is Clear, Configuration Read Requests and Configuration Write Requests to the SFI Extended Capability must be processed with normal semantics, unless otherwise specified (e.g., the SFI CAM Unsupported bit is Set).

The value of the SFI RO In-Band [hotlink] bit must have no effect on accesses to the SFI Extended Capability [hotlink] using PCIe-MI [hotlink].

IMPLEMENTATION NOTE: USE OF SFI RO IN-BAND §

This “read-only in-band” feature is intended for use when the SFI Extended Capability is configured by a BMC using PCIe-MI [hotlink] or other out-of-band management [hotlink] mechanisms. Setting the SFI RO In-Band [hotlink] bit blocks system software running on the host CPU(s) from writing to the SFI Extended Capability [hotlink]. Blocking such Configuration Write Requests prevents inadvertent or intentional security exploits from system software running on the host CPU(s) (e.g., by preventing the ability to modify the SFI Capability Structure to unblock access to an unauthenticated component) while still allowing Configuration Read Requests so that system software running on the host CPU(s) is able to read the SFI Extended Capability [hotlink] for information that may be useful (e.g., the SFI OOB PD Supported bit).

The SFI RO In-Band [hotlink] bit is HwInit [hotlink] allowing a Port to set the default value upon Conventional Reset to 1b such that the SFI Extended Capability [hotlink] is always read-only when accessed by Configuration Read Requests or Configuration Write Requests. This option may be useful to avoid race cases where system software running on the host CPUs may be able to write to the SFI Extended Capability after a Conventional Reset and before the BMC has an opportunity to set the SFI RO In-Band [hotlink] bit.

For systems that support firmware first DPC, the SFI HPS Suppress [hotlink] bit supports the use case defined by the _DSM for Downstream Port Containment and Hot-Plug Surprise Control in the PCI Firmware Specification. If the SFI Extended Capability [hotlink] is read-only on such systems when accessed by Configuration Read Requests or Configuration Write Requests, mechanisms outside the scope of this specification may be required to support this use case.

6.7.5 Firmware Support for Hot-Plug §

Some systems that include hot-plug capable Root Ports and Switches that are released before ACPI-compliant operating systems with native hot-plug support are available, can use ACPI firmware for propagating hot-plug events. Firmware control of the hot-plug registers must be disabled if an operating system with native support is used. Platforms that provide ACPI firmware to propagate hot-plug events must also provide a mechanism to transfer control to the operating system. The details of this method are described in the *PCI Firmware Specification*.

6.7.6 Async Removal §

Async removal refers to the removal of an adapter or disabling of a Downstream Port Link due to error containment without prior warning to the operating system. This is in contrast to orderly removal, where removal operations are performed in a lock-step manner with the operating system through a well defined sequence of user actions and system management facilities. For example, the user presses the Attention Button to request permission from the operating

system to remove the adapter, but the user doesn't actually remove the adapter from the slot until the operating system has quiesced activity to the adapter and granted permission for removal.

Since async removal proceeds before the rest of the PCI Express hierarchy or operating system necessarily becomes aware of the event, special consideration is required beyond that needed for standard PCI hot-plug. This section outlines PCI Express events that may occur as a side effect of async removal and mechanisms for handling async removal.

Since async removal may be unexpected to both the Physical and Data Link Layers of the Downstream Port associated with the slot, Correctable Errors may be reported as a side effect of the event (i.e., Receiver Error, Bad TLP, and Bad DLLP). If these errors are reported, software should handle them as an expected part of this event.

Requesters may experience Completion Timeouts associated with Requests that were accepted, but will never be completed by removed Completers. Any resulting Completion Timeout errors in this context should be handled as an expected part of this event.

Async removal may result in a transition from DL_Active to DL_Down in the Downstream Port. This transition may result in a Surprise Down error. In addition, Requesters in the PCI Express hierarchy domain may not become immediately aware of this transition and continue to issue Requests to removed Completers that must be handled by the Downstream Port associated with the slot.

Either Downstream Port Containment (DPC) or the Hot-Plug Surprise (HPS) mechanism may be used to support async removal as part of an overall async hot-plug architecture. See § Appendix I. for the associated reference model.

IMPLEMENTATION NOTE: HOT-PLUG SURPRISE MECHANISM DEPRECATED FOR ASYNC HOT-PLUG §

The Hot-Plug Surprise (HPS) mechanism, as indicated by the Hot-Plug Surprise bit in the Slot Capabilities Register being Set, is deprecated for use with async hot-plug. DPC is the recommended mechanism for supporting async hot-plug. See § Section 6.7.4.4 for guidance on slots supporting both mechanisms.

With async removal, using HPS has serious downsides. Uncorrectable errors other than those that inherently bring down the Link need to be configured either to crash the system, be handled asynchronously by software, or be ignored. These include uncorrectable errors associated with Posted Memory Writes, TLPs with poisoned data, and Completion Timeouts. Uncorrectable errors ignored or handled asynchronously by software may make it impossible for the driver to determine which high-level operations complete successfully versus those that do not.

DPC provides a robust mechanism for supporting async removal. The TLP stream cleanly stops upon an uncorrectable error that triggers DPC. Operating System / driver stacks that support Containment Error Recovery (CER) can fully and transparently recover from many transient PCIe uncorrectable errors. DPC can support async removal and CER concurrently

6.8 Power Budgeting Mechanism §

With the addition of a hot-plug capability for adapters, the need arises for the system to be capable of properly allocating power to any new devices added to the system. This capability is a separate and distinct function from power management and a basic level of support is required to ensure proper operation of the system. The power budgeting concept puts in place the building blocks that allow devices to interact with systems to achieve these goals. There are many ways in which the system can implement the actual power budgeting capabilities, and as such, they are beyond the scope of this specification.

Implementation of the Power Budgeting Extended Capability is optional for devices that are implemented either in a form factor that does not require hot-plug support, or that are integrated on the system board. Form factor specifications may require support for power budgeting. The devices and/or adapters are required to remain under the configuration power limit specified in the corresponding electromechanical specification until they have been configured and enabled by the system. The system should guarantee that power has been properly budgeted prior to enabling an adapter.

When enabled, Extended Power Budgeting provides power consumption information on a per-connector basis. This allows a system to manage power consumption more accurately.

§ Table 6-10 shows the deployment mechanisms for Power Budgeting.

Table 6-10 Power Budgeting Deployments §

Device Type	Power Budgeting Extended Capability	Description	Notes
Single Function, Multi-Function Device, or Multi-Device add-in card	Not Present	No Power Budgeting information is available	Some form factors may not permit this combination
Single-Function Device	Present	Power Budgeting represents Device power	
Multi-Function Device	Present in exactly one Function		
Multi-Function Device	Present in more than one Function	Power Budgeting represents per-Function power	Sum of all Functions represents power consumed by the Device. Functions that do not implement Power Budgeting have negligible power consumption. Power that is not associated with a specific Function is included in an implementation specific manner.
Multi-Device add-in card	Present in one or more Functions of exactly one Device	Power Budgeting represents add-in card power	Sum of all Functions represents power consumed by the add-in card. Functions and Devices that do not implement Power Budgeting have negligible power consumption. Power that is not associated with a specific Function is included in an implementation specific manner.
Multi-Device add-in card	Present in more than one Device	Power Budgeting represents per-Device power	Sum of all Devices represents power consumed by the add-in card. Devices that do not implement Power Budgeting have negligible power consumption. Power that is not associated with a specific Device is included in an implementation specific manner.

6.8.1 System Power Budgeting Process Recommendations §

It is recommended that system firmware provide the power budget management agent the following information:

- Total system power budget (power supply information).
- Total power allocated by system firmware (system board devices).
- Total number of slots and the types of slots.

System firmware is responsible for allocating power for all devices on the system board that do not have power budgeting capabilities. The firmware may or may not include devices that are connected to the standard power rails. When the firmware allocates the power for a device that implements the Power Budgeting Extended Capability it must set the System Allocated bit to 1b in the Power Budget Capability register to indicate that it has been properly allocated. The power budget manager is responsible for allocating all PCI Express devices including system board devices that have the Power Budgeting Extended Capability and have the System Allocated bit Clear. The power budget manager is responsible for determining if hot-plugged devices can be budgeted and enabled in the system.

There are alternate methods which may provide the same functionality, and it is not required that the power budgeting process be implemented in this manner.

6.8.2 Device Power Considerations §

When the Power Budgeting Extended Capability is present in more than one Function of a Device (or the only Function of a Single-Function Device), power is reported on a per-Function basis (see § Section 7.8.1). When Power Budgeting is present in exactly one Function of a Multi-Function Device, power is reported on a per-Device basis. When Power Budgeting is present in more than one Function, but not in all Functions, power is reported on a per-Function basis and the missing Functions are treated as consuming negligible power.

When Power Budgeting is present in exactly one Function, this represents per-Device Power. When Power Budgeting is present in exactly one Device of a Multi-Device add-in card, power is reported on a per-add-in card basis. The following rules apply:

- When all Functions in the Device are in the same PM State, power budgeting for that PM State applies.
 - If no power budgeting is reported for a given PM State, the next higher PM State that is reported applies.
- When Functions are in different PM States, power budgeting is at an implementation specific value between the power budgeting reported for highest and lowest power PM State (e.g., if no Functions are in D0, one Function is in D1 and another Function is in D3, power budgeting is somewhere between the values reported for D1 and D3, inclusive).
 - If no power budgeting is reported for the highest PM State, power budgeting for the next higher PM State that is reported applies.
 - If no power budgeting is reported for the lowest PM State, power budgeting for the next lower PM State that is reported applies.

6.8.3 Power Limit Mechanisms §

An add-in card must not consume more power than it is granted by the system. There are six mechanisms defined to grant power to an add-in card:

1. **Initial Power** – this is power granted to all similarly situated add-in cards by the form factor. This value is form factor specific and typically is based on factors like add-in card size and external power configuration.
2. **Set_Slot_Power_Limit power** – this is power granted to an add-in card when it receives a Set_Slot_Power_Limit message with a power value greater than the Initial Power grant for the add-in card.
3. **Power Limit PM Sub State power** – this is power granted to an add-in card through the Power Limit mechanism (see below). This mechanism overrides power grants from either the Initial Power or Set_Slot_Power_Limit mechanisms. Unlike Set_Slot_Power_Limit, this mechanism is permitted to grant a power level that is lower than the Initial Power grant.
4. **Out of Band Power Limit PM Sub State power** – this is power granted to an add-in card through the Out of Band Power Limit mechanism (see below). This mechanism overrides power grants from either the Initial Power or Set_Slot_Power_Limit mechanisms. Unlike Set_Slot_Power_Limit, this mechanism is permitted to grant a power level that is lower than the Initial Power grant.
5. **Firmware based additional Aux Power** – this is Aux power granted when the driver uses the Request D3_{Cold} Aux Power Limit_DSM call as defined in [Firmware]. This mechanism overrides Aux power grants using any of the above mechanisms.
6. **Form factor specific or Device Class (specific e.g., NVMe) mechanisms** – this is power granted to an add-in card using mechanisms defined by other specifications and are outside the scope of this specification. Interactions between these mechanisms and the mechanisms listed above are form factor or Device Class specific.

The Power Limit mechanism is optional. Support is indicated by the Power Limit Supported bit in the lowest-numbered Function that contains a Power Budgeting Extended Capability. The Power Limit fields in that Function control power for the entire add-in card.

The Power Budgeting Data Register (see § Section 7.8.1.3) contains a PM Sub State field. This permits a Device to report power consumption for up to 8 PM Sub States. This PM Sub State is implementation specific and is unrelated to other similarly named concepts in PCIe (e.g., PM State, L1 PM Substates).

Each PM Sub State represents additional optional power consumption levels. Each PM Sub State is mutually exclusive. At any specific point in time, a Device operates in exactly one PM Sub State. Each defined power consumption level includes a full complement of Data entries (e.g., Aux Power, Thermal, Max, Min for all appropriate connectors and power rails).

Device behavior is undefined if software attempts to transition the Device to a PM Sub State for which there are no Data entries.

When both the Power Limit Enable and the Out of Band Power Limit Enable bits are Clear, for each power rail and connector combination, the Device must operate in the PM Sub State determined by an implementation specific mechanism.

When both the Power Limit Enable and Out of Band Power Limit Enable bits are Set, individually for each power rail and connector combination, the Device must operate in the PM Sub State indicated by the smaller power consumption indicated by Power Limit PM Sub State or Out of Band Power Limit PM Sub State fields.

The Power Limit Enable and Power Limit PM Sub State fields are configured using Configuration Write transactions while all Functions of the Device are in D0_{uninitialized}. Behavior is undefined if these fields change after any Function exits D0_{uninitialized}.

The Out of Band Power Limit Enable and Out of Band Power Limit PM Sub State values are configured using implementation specific mechanisms. The mechanism used for out of band configuration is outside the scope of this specification.

6.9 Slot Power Limit Control §

PCI Express provides a mechanism for software controlled limiting of the Maximum Power per slot that an adapter (associated with that slot) can consume. If supported, the Emergency Power Reduction State, over-rides the mechanisms listed here (see § Section 6.24). The key elements of this mechanism are:

- Slot Power Limit Value and Scale fields of the Slot Capabilities register implemented in the Downstream Ports of a Root Complex or a Switch
- Captured Slot Power Limit Value and Scale fields of the Device Capabilities register implemented in Endpoint, Switch, or PCI Express-PCI Bridge Functions present in an Upstream Port
- Set_Slot_Power_Limit Message that conveys the content of the Slot Power Limit Value and Scale fields of the Slot Capabilities register of the Downstream Port (of a Root Complex or a Switch) to the corresponding Captured Slot Power Limit Value and Scale fields of the Device Capabilities register in the Upstream Port of the component connected to the same Link

Power limits on the platform are typically controlled by the software (for example, platform firmware) that comprehends the specifics of the platform such as:

- Partitioning of the platform, including slots for I/O expansion using adapters
- Power delivery capabilities
- Thermal capabilities

This software is responsible for correctly programming the Slot Power Limit Value and Scale fields of the Slot Capabilities registers of the Downstream Ports connected to slots. After the value has been written into the register within the Downstream Port, it is conveyed to the adapter using the Set_Slot_Power_Limit Message (see § Section 2.2.8.5). The recipient of the Message must use the value in the Message data payload to limit usage of the power for the entire adapter, unless the adapter will never exceed the lowest value specified in the corresponding form factor specification. It is required that device driver software associated with the adapter be able (by reading the values of the Captured Slot Power Limit Value and Scale fields of the Device Capabilities register) to configure hardware of the adapter to guarantee that the adapter will not exceed the imposed limit. In the case where the platform imposes a limit that is below the minimum needed for adequate operation, the device driver will be able to communicate this discrepancy to higher level configuration software. Configuration software is required to set the Slot Power Limit to one of the maximum values specified for the corresponding form factor based on the capability of the platform.

The following rules cover the Slot Power Limit control mechanism:

For Adapters:

- Until and unless a Set_Slot_Power_Limit Message is received indicating a Slot Power Limit value greater than the lowest value specified in the form factor specification for the adapter's form factor, the adapter must not consume more than the lowest value specified.
- An adapter must never consume more power than what was specified in the most recently received Set_Slot_Power_Limit Message or the minimum value specified in the corresponding form factor specification, whichever is higher.
- Components with Endpoint, Switch, or PCI Express-PCI Bridge Functions that are targeted for integration on an adapter where total consumed power is below the lowest limit defined for the targeted form factor are permitted to ignore Set_Slot_Power_Limit Messages, and to return a value of 0 in the Captured Slot Power Limit Value and Scale fields of the Device Capabilities register
 - Such components still must be able to receive the Set_Slot_Power_Limit Message without error but simply discard the Message value

For Root Complex and Switches which source slots:

- Configuration software must not program a Set_Slot_Power_Limit value that indicates a limit that is lower than the lowest value specified in the form factor specification for the slot's form factor.

IMPLEMENTATION NOTE:

EXAMPLE ADAPTER BEHAVIOR BASED ON THE SLOT POWER LIMIT CONTROL CAPABILITY §

The following power limit scenarios are examples of how an adapter must behave based on the Slot Power Limit control capability. The form factor limits are representations, and should not be taken as actual requirements.

Note: Form factor #1 has a Maximum Power requirement of 40 W and 25 W; form factor #2 has a Maximum Power requirement of 15 W.

Scenario 1: An Adapter Consuming 12 W

- If the adapter is plugged into a form factor #1 40 W slot, the Slot Power Limit control mechanism is followed, and the adapter operates normally.
- If the adapter is plugged into a form factor #1 25 W slot, the Slot Power Limit control mechanism is followed, and the adapter operates normally.
- If the adapter is plugged into a form factor #2 15 W slot, the Slot Power Limit control mechanism is followed, and the adapter operates normally.

In all cases, since the adapter operates normally within all the form factors, it can ignore any of the slot power limit Messages.

Scenario 2: An Adapter Consuming 18 W

- If the adapter is plugged into a form factor #1 40 W slot, the Slot Power Limit control mechanism is followed, and the adapter operates normally.
- If the adapter is plugged into a form factor #1 25 W slot, the Slot Power Limit control mechanism is followed, and the adapter operates normally.
- If the adapter is plugged into a form factor #2 15 W slot, the Slot Power Limit control mechanism is followed, and the adapter must scale down to 15 W or disable operation. An adapter that does not scale within any of the power limits for a given form factor will always be disabled in that form factor and should not be used.

In this case, if the adapter is only to be used in form factor #1, it can ignore any of the slot power limit Messages. To be useful in form factor #2, the adapter should be capable of scaling to the power limit of form factor #2.

Scenario 3: An Adapter Consuming 30 W

- If the adapter is plugged into a form factor #1 40 W slot, the Slot Power Limit control mechanism is followed, and the device operates normally.
- If the adapter is plugged into a form factor #1 25 W slot, the Slot Power Limit control mechanism is followed, and the device must scale down to 25 W or disable operation.
- If the adapter is plugged into a form factor #2 15 W slot, the Slot Power Limit control mechanism is followed, and the adapter must scale down to 15 W or disable operation. An adapter that does not scale within any of the power limits for a given form factor will always be disabled in that form factor and should not be used.

In this case, since the adapter consumes power above the lowest power limit for a slot, the adapter must be capable of scaling or disabling to prevent system failures. Operation of adapters at power levels that exceed the capabilities of the slots in which they are plugged must be avoided.

IMPLEMENTATION NOTE: SLOT POWER LIMIT CONTROL REGISTERS §

Typically Slot Power Limit register fields within Downstream Ports of a Root Complex or a Switch will be programmed by platform-specific software. Some implementations may use a hardware method for initializing the values of these registers and, therefore, do not require software support.

Components with Endpoint, Switch, or PCI Express-PCI Bridge Functions that are targeted for integration on the adapter where total consumed power is below the lowest limit defined for that form factor are allowed to ignore Set_Slot_Power_Limit Messages. Note that components that take this implementation approach may not be compatible with potential future defined form factors. Such form factors may impose lower power limits that are below the minimum required by a new adapter based on the existing component.

IMPLEMENTATION NOTE: AUTO SLOT POWER LIMIT DISABLE §

In some environments host software may wish to directly manage the transmission of a Set_Slot_Power_Limit message by performing a Configuration Write to the Slot Capabilities register rather than have the transmission automatically occur when the Link transitions from a non-DL_Up to a DL_Up status. This allows host software to limit power supply surge current by staggering the transition of Endpoints to a higher power state following a Link Down or when multiple Endpoints are simultaneously hot-added due to cable or adapter insertion.

6.10 Root Complex Topology Discovery §

A Root Complex may present one of the following topologies to configuration software:

- A single opaque Root Complex such that software has no visibility with respect to internal operation of the Root Complex. All Root Ports are independent of each other from a software perspective; no mechanism exists to manage any arbitration among the various Root Ports for any differentiated services.
- A single Root Complex Component such that software has visibility and control with respect to internal operation of the Root Complex Component. As shown in § Figure 6-11, software views the Root Ports as Ingress Ports for the component. The Root Complex internal Port for traffic aggregation to a system Egress Port or an internal sink unit (such as memory) is represented by an RCRB structure. Controls for differentiated services are provided through a Virtual Channel Capability structure located in the RCRB.

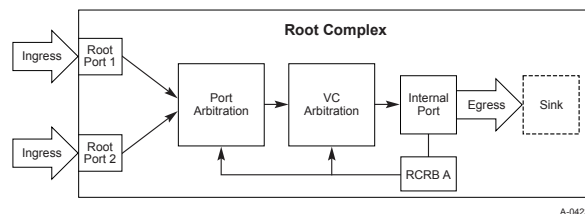


Figure 6-11 Root Complex Represented as a Single Component §

- Multiple Root Complex Components such that software not only has visibility and control with respect to internal operation of a given Root Complex Component but also has the ability to discover and control arbitration between different Root Complex Components. As shown in § Figure 6-12, software views the Root Ports as Ingress Ports for a given component. An RCRB structure controls egress from the component to other Root Complex Components (RCRB C) or to an internal sink unit such as memory (RCRB A). In addition, an RCRB structure (RCRB B) may also be present in a given component to control traffic from other Root Complex Components. Controls for differentiated services are provided through Virtual Channel Capability structures located appropriately in the RCRBs respectively.

More complex topologies are possible as well.

A Root Complex topology can be represented as a collection of logical Root Complex Components such that each logical component has:

- One or more Ingress Ports.
- An Egress Port.
- Optional associated Virtual Channel capabilities located either in the Configuration Space (for Root Ports) or in an RCRB (for internal Ingress/Egress Ports) if the Root Complex supports Virtual Channels.
- Optional devices/Functions integrated in the Root Complex.

In order for software to correctly program arbitration and other control parameters for PCI Express differentiated services, software must be able to discover a Root Complex's internal topology. Root Complex topology discovery is accomplished by means of the Root Complex Link Declaration Capability as described in § Section 7.9.8.

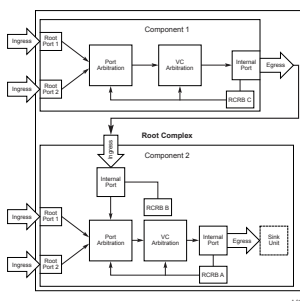


Figure 6-12 Root Complex Represented as Multiple Components §

6.11 Link Speed Management §

This section describes how Link speed management is coordinated between the LTSSM (§ Section 4.2.7) and the software Link observation and control mechanisms (see § Section 7.5.3.6 , § Section 7.5.3.7 , § Section 7.5.3.8 , § Section 7.5.3.18 , § Section 7.5.3.19 , and § Section 7.5.3.20).

The Target Link Speed field in the Link Control 2 register in the Downstream Port sets the upper bound for the Link speed. Except as described below, the Upstream component must attempt to maintain the Link at the Target Link Speed, or at the highest speed supported by both components on the Link (as reported by the values in the training sets - see § Section 4.2.5.1), whichever is lower.

Any Upstream Port or Downstream Port with the Hardware Autonomous Speed Disable bit in the Link Control 2 register clear is permitted to autonomously change the Link speed using implementation specific criteria.

If the reliability of the Link is unacceptably low, then either component is permitted to lower the Link speed by removing the unreliable Link speed from the list of supported speeds advertised in the training sets the component transmits. The criteria for determination of acceptable Link reliability are implementation specific, and are not dependent on the setting of the Hardware Autonomous Speed Disable bit.

During any given speed negotiation it is possible that one or both components will advertise a subset of all speeds supported, as a means to cap the post-negotiation Link speed. It is permitted for a component to change its set of advertised supported speeds without requesting a Link speed change by driving the Link through Recovery without setting the speed change bit.

When a component's attempt to negotiate to a particular Link speed fails, that component is not permitted to attempt negotiation to that Link speed, or to any higher Link speed, until 200 ms has passed from the return to L0 following the failed attempt, or until the other component on the Link advertises support for the higher Link speed through its transmitted training sets (with or without a request to change the Link speed), whichever comes first.

Software is permitted to restrict the maximum speed of Link operation and set the preferred Link speed by setting the value in the Target Link Speed field in the Upstream component. After modifying the value in the Target Link Speed field, software must trigger Link retraining by writing 1b to the Retrain Link bit. Software is notified of any Link speed changes (as well as any Link width changes) through the Link Bandwidth Notification Mechanism.

Software is permitted to cause a Link to transition to the Polling.Compliance LTSSM state at a particular speed by writing the Link Control 2 register in both components with the same value in the Target Link Speed field and Setting the Enter Compliance bit, and then initiating a Hot Reset on the Link (through the Downstream Port).

Note that this will take the Link to a DL_Down state and therefore cannot be done transparently to other software that is using the Link. The Downstream Port will return to Polling.Active when the Enter Compliance bit is cleared.

6.12 Access Control Services (ACS) §

ACS defines a set of control points within a PCI Express topology to determine whether a TLP is to be routed normally, blocked, or redirected. ACS is applicable to RCs, Switches, and Multi-Function Devices.¹³⁹ For ACS requirements, Single-Function Devices that are SR-IOV capable or SIOV capable must be handled as if they were Multi-Function Devices, since they essentially behave as Multi-Function Devices after their Virtual Functions (VFs) or Scalable Device Interfaces (SDIs) are enabled.

Implementation of ACS in RCiEPs is permitted but not required. It is explicitly permitted that, within a single Root Complex, some RCiEPs implement ACS and some do not. It is strongly recommended that Root Complex

139. Applicable Functions within Multi-Function Devices specifically include PCI Express Endpoints, Switch Upstream Ports, Legacy PCI Express Endpoints, and Root Complex Integrated Endpoints.

implementations ensure that all accesses originating from RCiEPs (PFs, VFs, and SDIs) without ACS support are first subjected to processing by the Translation Agent (TA) in the Root Complex before further decoding and processing. The details of such Root Complex handling are outside the scope of this specification.

ACS provides the following types of access control:

- [ACS Source Validation](#)
- [ACS Translation Blocking](#)
- [ACS P2P Request Redirect](#)
- [ACS P2P Completion Redirect](#)
- [ACS Upstream Forwarding](#)
- [ACS P2P Egress Control](#)
- [ACS Direct Translated P2P](#)
- [ACS I/O Request Blocking](#)
- [ACS DSP Memory Target Access](#)
- [ACS USP Memory Target Access](#)
- [ACS Unclaimed Request Redirect](#)

The specific requirements for each of these are discussed in the following section.

ACS hardware functionality is disabled by default, and is enabled only by ACS-aware software. With the exception of ACS Source Validation, ACS access controls are not applicable to Multicast TLPs (see [§ Section 6.14](#)), and have no effect on them.

6.12.1 ACS Component Capability Requirements [§](#)

ACS functionality is reported and managed via ACS Extended Capability structures. PCI Express components are permitted to implement ACS Extended Capability structures in some, none, or all of their applicable Functions. The extent of what is implemented is communicated through capability bits in each ACS Extended Capability structure. A given Function with an ACS Extended Capability structure may be required or forbidden to implement certain capabilities, depending upon the specific type of the Function and whether it is part of a [Multi-Function Device](#).

ACS is never applicable to a PCI Express to PCI Bridge Function or a Root Complex Event Collector Function, and such Functions must never implement an [ACS Extended Capability structure](#).

6.12.1.1 ACS Downstream Ports [§](#)

This section applies to Root Ports and Switch Downstream Ports that implement an [ACS Extended Capability structure](#). This section applies to Downstream Port Functions both for [Single-Function Devices](#) and [Multi-Function Devices](#).

- [ACS Source Validation](#): must be implemented.

When enabled, the Downstream Port tests the Bus Number from the Requester ID of each Upstream Request received by the Port to determine if it is associated with the Secondary side of the virtual bridge associated with the Downstream Port, by either or both of:

- Determining that the Requester ID falls within the Bus Number “aperture” of the Port - the inclusive range specified by the [Secondary Bus Number Register](#) and the [Subordinate Bus Number Register](#).

- If FPB is implemented and enabled, determining that the Requester ID is associated with the bridge's Secondary Side by the application of the FPB Routing ID mechanism.

If the Bus Number from the Requester ID of the Request is not within this aperture, this is a reported error (ACS Violation) associated with the Receiving Port (see § Section 6.12.5 .)

ACS mechanisms are not architected to check Segment Numbers in FM TLPs. Segment Number checking that applies independently of ACS is specified under Segment Rules in § Section 2.2.1.2 .

Completions are never affected by ACS Source Validation.

IMPLEMENTATION NOTE: UPSTREAM MESSAGES AND ACS SOURCE VALIDATION §

Functions are permitted to transmit Upstream Messages before they have been assigned a Bus Number. Such messages will have a Requester ID with a Bus Number of 00h. If the Downstream Port has ACS Source Validation enabled, these Messages (see § Table F-1, § Section 2.2.8.2 , and § Section 6.22.1) will likely be detected as an ACS Violation error.

- ACS Translation Blocking: must be implemented.

When enabled, the Downstream Port checks the Address Type (AT) field of each Upstream Memory Request and, in Flit Mode only, of each Upstream Address Routed Message received by the Port. If the AT field is not 00b (Untranslated), this is a reported error (ACS Violation) associated with the Receiving Port (see § Section 6.12.5). This error must take precedence over ACS Upstream Forwarding and any applicable ACS P2P control mechanisms.

Completions are never affected by ACS Translation Blocking.

- ACS P2P Request Redirect: must be implemented by Root Ports that support peer-to-peer traffic with other Root Ports;¹⁴⁰ must be implemented by Switch Downstream Ports.

ACS P2P Request Redirect is subject to interaction with the ACS P2P Egress Control and ACS Direct Translated P2P mechanisms (if implemented). Refer to § Section 6.12.3 for more information.

When ACS P2P Request Redirect is enabled in a Switch Downstream Port, peer-to-peer Requests must be redirected Upstream towards the RC.

When ACS P2P Request Redirect is enabled in a Root Port, peer-to-peer Requests must be sent to Redirected Request Validation logic within the RC that determines whether the Request is “reflected” back Downstream towards its original target, or blocked as an ACS Violation error. The algorithms and specific controls for making this determination are not architected by this specification.

Downstream Ports never redirect Requests that are traveling Downstream.

Completions are never affected by ACS P2P Request Redirect.

- ACS P2P Completion Redirect: must be implemented by Root Ports that implement ACS P2P Request Redirect; must be implemented by Switch Downstream Ports.
The intent of ACS P2P Completion Redirect is to avoid ordering rule violations between Completions and Requests when Requests are redirected. Refer to § Section 6.12.6 for more information.

ACS P2P Completion Redirect does not interact with ACS controls that govern Requests.

140. Root Port indication of ACS P2P Request Redirect or ACS P2P Completion Redirect support does not imply any particular level of peer-to-peer support by the Root Complex, or that peer-to-peer traffic is supported at all.

When ACS P2P Completion Redirect is enabled in a Switch Downstream Port, peer-to-peer Completions¹⁴¹ that do not have the Relaxed Ordering Attribute bit set (1b) must be redirected Upstream towards the RC. Otherwise, peer-to-peer Completions must not be redirected.

When ACS P2P Completion Redirect is enabled in a Root Port, peer-to-peer Completions that do not have the Relaxed Ordering bit set must be handled such that they do not pass Requests that are sent to Redirected Request Validation logic within the RC. Such Completions must eventually be sent Downstream towards their original peer-to-peer targets, without incurring additional ACS access control checks.

Downstream Ports never redirect Completions that are traveling Downstream.

Requests are never affected by ACS P2P Completion Redirect.

- ACS Upstream Forwarding: must be implemented by Root Ports if the RC supports Redirected Request Validation; must be implemented by Switch Downstream Ports.
When ACS Upstream Forwarding is enabled in a Switch Downstream Port, and its Ingress Port receives an Upstream Request or Completion TLP targeting the Port's own Egress Port, the Port must instead forward the TLP Upstream towards the RC.

When ACS Upstream Forwarding is enabled in a Root Port, and its Ingress Port receives an Upstream Request or Completion TLP that targets the Port's own Egress Port, the Port must handle the TLP as follows. For a Request, the Root Port must handle it the same as a Request that the Port "redirects" with the ACS P2P Request Redirect mechanism. For a Completion, the Root Port must handle it the same as a Completion that the Port "redirects" with the ACS P2P Completion Redirect mechanism.

When ACS Upstream Forwarding is not enabled on a Downstream Port, and its Ingress Port receives an Upstream Request or Completion TLP that targets the Port's own Egress Port, the handling of the TLP is undefined.

- ACS P2P Egress Control: implementation is optional.
ACS P2P Egress Control is subject to interaction with the ACS P2P Request Redirect and ACS Direct Translated P2P mechanisms (if implemented). Refer to § Section 6.12.3 for more information.

A Switch that supports ACS P2P Egress Control can be selectively configured to block peer-to-peer Requests between its Downstream Ports. Software can configure the Switch to allow none or only a subset of its Downstream Ports to send peer-to-peer Requests to other Downstream Ports. This is configured on a per Downstream Port basis.

An RC that supports ACS P2P Egress Control can be selectively configured to block peer-to-peer Requests between its Root Ports. Software can configure the RC to allow none or only a subset of the Hierarchy Domains to send peer-to-peer Requests to other Hierarchy Domains. This is configured on a per Root Port basis.

With ACS P2P Egress Control in Downstream Ports, controls in the Ingress Port ("sending" Port) determine if the peer-to-peer Request is blocked, and if so, the Ingress Port handles the ACS Violation error per § Section 6.12.5.

Completions are never affected by ACS P2P Egress Control.

- ACS Direct Translated P2P: must be implemented by Root Ports that support Address Translation Services (ATS) and also support peer-to-peer traffic with other Root Ports;¹⁴² must be implemented by Switch Downstream Ports.

When ACS Direct Translated P2P is enabled in a Downstream Port, peer-to-peer Memory Requests and, in Flit Mode only, peer-to-peer Address Routed Messages whose Address Type (AT) field indicates a Translated address must be routed to the peer Egress Port without redirection, regardless of ACS P2P Request Redirect

141. This includes Read Completions, AtomicOp Completions, and other Completions with or without Data.

142. Root Port indication of ACS Direct Translated P2P support does not imply any particular level of peer-to-peer support by the Root Complex, or that peer-to-peer traffic is supported at all.

and ACS P2P Egress Control settings. All other peer-to-peer Requests must still be subject to ACS P2P Request Redirect and ACS P2P Egress Control settings.

Completions are never affected by ACS Direct Translated P2P.

- ACS I/O Request Blocking: must be implemented by Root Ports and Switch Downstream Ports that support ACS Enhanced Capability.

When enabled, the Port must handle an Upstream I/O Request received by the Port's Ingress as an ACS Violation.

- ACS DSP Memory Target Access: must be implemented by Root Ports and Switch Downstream Ports that support ACS Enhanced Capability and that have applicable Memory BAR Space to protect.

ACS DSP Memory Target Access determines how an Upstream Request received by the Downstream Port's Ingress and targeting any Memory BAR Space¹⁴³ associated with an applicable Downstream Port is handled. The Request can be blocked, redirected, or allowed to proceed directly to its target. In a Switch, all Downstream Ports are applicable, including the one on which the Request was received. In a Root Complex, the set of applicable Root Ports is implementation specific, but always includes the one on which the Request was received.

- ACS USP Memory Target Access: must be implemented by Switch Downstream Ports that support ACS Enhanced Capability and that have applicable Memory BAR Space in the Switch Upstream Port to protect; is not applicable to Root Ports.

ACS USP Memory Target Access determines how an Upstream Request received by the Switch Downstream Port's Ingress and targeting any Memory BAR Space¹⁴⁴ associated with the Switch's Upstream Port is handled. The Request can be blocked, redirected, or allowed to proceed directly to its target.

If any Functions other than the Switch Upstream Port are associated with the Upstream Port, this field has no effect on accesses to their Memory BAR Space¹⁴⁵. Such access is controlled by the ACS Extended Capability (if present) in the Switch Upstream Port.

- ACS Unclaimed Request Redirect: must be implemented by Switch Downstream Ports that support ACS Enhanced Capability; is not applicable to Root Ports.

When enabled, incoming Requests received by the Switch Downstream Port's Ingress and targeting Memory Space within the memory window of a Switch Upstream Port that is not within a memory window or Memory BAR Target of any Downstream Port within the Switch are redirected Upstream out of the Switch.

When not enabled, such Requests are handled by the Switch Downstream Port as an Unsupported Request (UR).

6.12.1.2 ACS Functions in SR-IOV, SIOV, and Multi-Function Devices §

This section applies to Multi-Function Device ACS Functions, with the exception of Downstream Port Functions, which are covered in the preceding section. For ACS requirements, Single-Function Devices that are SR-IOV capable or SIOV capable must be handled as if they were Multi-Function Devices.

- ACS Source Validation: must not be implemented.
- ACS Translation Blocking: must not be implemented.

143. This also includes any Memory Space allocated by an Expansion ROM Base Address register (BAR). This also includes any Memory Space allocated by EA entries with a BEI value of 0, 1, 7, or 8. See § Section 7.8.5.3.

144. This also includes any Memory Space allocated by an Expansion ROM Base Address register (BAR). This also includes any Memory Space allocated by EA entries with a BEI value of 0, 1, 7, or 8. See § Section 7.8.5.3.

145. This also includes any Memory Space allocated by an Expansion ROM Base Address register (BAR). This also includes any Memory Space allocated by EA entries with a BEI value of 0, 1, 7, or 8. See § Section 7.8.5.3.

- ACS P2P Request Redirect: must be implemented by Functions that support peer-to-peer traffic with other Functions. This includes SR-IOV Virtual Functions (VFs). SDIs that support peer-to-peer traffic with other Functions or SDIs in an SIOV Device must implement this in their associated PF. SIOV Devices must ensure that peer-to-peer traffic initiated by SDIs follow the policy set in their associated PF.

ACS P2P Request Redirect is subject to interaction with the ACS P2P Egress Control and ACS Direct Translated P2P mechanisms (if implemented). Refer to § Section 6.12.3 for more information.

When ACS P2P Request Redirect is enabled in a Multi-Function Device that is not an RCiEP, peer-to-peer Requests (between Functions of the device) must be redirected Upstream towards the RC.

It is permitted but not required to implement ACS P2P Request Redirect in an RCiEP. When ACS P2P Request Redirect is enabled in an RCiEP, peer-to-peer Requests, defined as all Requests that do not target system memory, must be sent to implementation specific logic within the Root Complex that determines whether the Request is directed towards its original target, or blocked as an ACS Violation error. The algorithms and specific controls for making this determination are not architected by this specification.

Completions are never affected by ACS P2P Request Redirect.

- ACS P2P Completion Redirect: must be implemented by Functions that implement ACS P2P Request Redirect. The intent of ACS P2P Completion Redirect is to avoid ordering rule violations between Completions and Requests when Requests are redirected. Refer to § Section 6.12.6 for more information.

ACS P2P Completion Redirect does not interact with ACS controls that govern Requests.

When ACS P2P Completion Redirect is enabled in a Multi-Function Device that is not an RCiEP, peer-to-peer Completions that do not have the Relaxed Ordering bit set must be redirected Upstream towards the RC. Otherwise, peer-to-peer Completions must not be redirected.

Requests are never affected by ACS P2P Completion Redirect.

- ACS Upstream Forwarding: must not be implemented.
- ACS P2P Egress Control: implementation is optional; is based on Function Numbers or Function Group Numbers; controls peer-to-peer Requests between the different Functions within a Multi-Function Device. SDIs within an SIOV Device do not have ACS P2P Egress Control of their own and follow the policy set in their associated PF.

ACS P2P Egress Control is subject to interaction with the ACS P2P Request Redirect and ACS Direct Translated P2P mechanisms (if implemented). Refer to § Section 6.12.3 for more information.

Each Function within a Multi-Function Device that supports ACS P2P Egress Control can be selectively enabled to block peer-to-peer communication with other Functions or Function Groups¹⁴⁶ within the device. This is configured on a per Function basis.

With ACS P2P Egress Control, controls in the 'sending' Function determine if the Request is blocked, and if so, the 'sending' Function handles the ACS Violation error per § Section 6.12.5. For an SDI within an SIOV Device, controls in the associated PF determine if the Request is blocked, and if so, the associated PF handles the ACS Violation error per § Section 6.12.5.

When ACS Function Groups are enabled in an ARI Device (ACS Function Groups Enable is Set), ACS P2P Egress Controls are enforced on a per Function Group basis instead of a per Function basis. See § Section 6.13.

Completions are never affected by ACS P2P Egress Control.

- ACS Direct Translated P2P: must be implemented if the Function supports Address Translation Services (ATS) and also peer-to-peer traffic with other Functions. SDIs that support ATS and peer-to-peer traffic with other

146. ACS Function Groups capability is optional for ARI Devices that implement ACS P2P Egress Controls.

Functions or SDIs in an SIOV Device must implement this in their associated PF. SIOV Devices must ensure that peer-to-peer traffic initiated by SDIs follow the policy set in their associated PF.

When ACS Direct Translated P2P is enabled in a Multi-Function Device, peer-to-peer Memory Requests whose Address Type (AT) field indicates a Translated address must be routed without redirection to the peer Function, regardless of ACS P2P Request Redirect and ACS P2P Egress Control settings. All other peer-to-peer Requests must still be subject to ACS P2P Request Redirect and ACS P2P Egress Control settings.

Completions are never affected by ACS Direct Translated P2P.

6.12.1.3 Functions in Single-Function Devices §

This section applies to Single-Function Device Functions, with the exception of Downstream Port Functions and SR-IOV capable or SIOV capable Functions, which are covered in a preceding section. For ACS requirements, Single-Function Devices that are SR-IOV capable or SIOV capable must be handled as if they were Multi-Function Devices.

No ACS capabilities are applicable, and the Function must not implement an ACS Extended Capability structure.

6.12.2 Interoperability §

The following rules govern interoperability between ACS and non-ACS components:

- When ACS P2P Request Redirect and ACS P2P Completion Redirect are not being used, ACS and non-ACS components may be intermixed within a topology and will interoperate fully. ACS can be enabled in a subset of the ACS components without impacting interoperability.
- When ACS P2P Request Redirect, ACS P2P Completion Redirect, or both are being used, certain components in the PCI Express hierarchy must support ACS Upstream Forwarding (of Upstream redirected Requests). Specifically:
The associated Root Port ¹⁴⁷ must support ACS Upstream Forwarding. Otherwise, how the Root Port handles Upstream redirected Request or Completion TLPs is undefined. The RC must also implement Redirected Request Validation.

Between each ACS component where P2P TLP redirection is enabled and its associated Root Port, any intermediate Switches must support ACS Upstream Forwarding. Otherwise, how such Switches handle Upstream redirected TLPs is undefined.

6.12.3 ACS Peer-to-Peer Control Interactions §

With each peer-to-peer Request, multiple ACS control mechanisms may interact to determine whether the Request is routed directly towards its peer-to-peer target, blocked immediately as an ACS Violation, or redirected Upstream towards the RC for access validation. Peer-to-peer Completion redirection is determined exclusively by the ACS P2P Completion Redirect mechanism.

If ACS Direct Translated P2P is enabled in a Port/Function, peer-to-peer Memory Requests whose Address Type (AT) field indicates a Translated address must be routed to the peer Port/Function without redirection, regardless of ACS P2P Request Redirect and ACS P2P Egress Control settings. Otherwise such Requests, and unconditionally all other peer-to-peer Requests, must be subject to ACS P2P Request Redirect and ACS P2P Egress Control settings. Specifically, the applicable Egress Control Vector bit, along with the ACS P2P Egress Control Enable bit (E) and the ACS P2P Request Redirect Enable bit (R), determine how the Request is handled. It must be noted that atomicity of accesses cannot be

147. Not applicable for ACS Redirect between Functions of a multi-Function Root Complex Integrated Endpoint.

guaranteed if ACS peer-to-peer Request Redirect targets a legacy device location that can be the target of a locked access. Refer to § Section 7.7.11 for descriptions of these control bits. § Table 6-11 specifies the interactions.

Table 6-11 ACS P2P Request Redirect and ACS P2P Egress Control Interactions §

Control Bit E (b)	Control Bit R (b)	Egress Control Vector Bit for the Associated Egress Switch Port, Root Port, Function, or <u>Function Group</u>	Required Handling for Peer-to-Peer Requests
0	0	X - Don't care	Route directly to peer-to-peer target
0	1	X - Don't Care	Redirect Upstream
1	0	1	Handle as an ACS Violation
1	0	0	Route directly to peer-to-peer target
1	1	1	Redirect Upstream
1	1	0	Route directly to peer-to-peer target

IMPLEMENTATION NOTE:

ACCESS CONTROL SERVICES IN SYSTEMS THAT SUPPORT DIRECT ASSIGNMENT OF FUNCTIONS §

General-purpose VIs typically have separate address spaces for each SI and for the VI itself. If such a VI also supports direct assignment of a Function to an SI, Untranslated Memory Request transactions issued by directly assigned Functions are under the complete control of software operating within the associated SI and typically reference the address space associated with that SI. In contrast, Memory Request transactions issued by the Host (MMIO requests) and by Functions that are not directly assigned are under the control of the VI and typically reference one or more system address spaces (e.g., the PCIe physical address space, the address space associated with the VI, or the address space associated with some designated SI). General-purpose VIs are not expected to establish a dependency between these various address spaces. Consequently, these address spaces may freely overlap which could lead to unintended routing of TLPs by Switches. For example, Upstream Memory Request TLPs originated by a directly assigned Function and intended for main memory could instead be routed to a Downstream Port if the address in the Request falls within the MMIO address region associated with that Downstream Port. Such unintended routing poses a threat to SI and/or VI stability and integrity.

To guard against this concern, vendors are strongly recommended to implement ACS in platforms that support general purpose VIs with direct assignment of Functions. Such support should include:

- In Switches or Root Complexes located below the TA, the level of ACS support should follow the guidelines established in this document for Downstream Switch Ports that implement an ACS Extended Capability structure.

Note: Components located above the TA only see Translated Memory Requests, consequently this concern does not apply to those components.

- In SR-IOV Devices that are capable of peer-to-peer transactions, ACS support is required.
- In Multi-Function Devices that are capable of peer-to-peer transactions, vendors are strongly recommended to implement ACS with ACS P2P Egress Control.

Additionally, platform vendors should test for the presence of ACS and enable it in Root Complexes and Switches on the path from the TA to a Function prior to directly assigning that Function. If the Function is peer-to-peer capable, ACS should be enabled in the Function as well.

6.12.4 ACS Enhanced Capability §

ACS Enhanced Capability is an additional set of ACS control mechanisms to improve the level of isolation and protection provided by ACS. ACS Enhanced Capability defines the following additional access control mechanisms:

- ACS I/O Request Blocking
- ACS DSP Memory Target Access
- ACS USP Memory Target Access
- ACS Unclaimed Request Redirect

Through these mechanisms, ACS Enhanced Capability provides protection and consistent handling of Requests directed toward regions not covered by the original ACS mechanisms.

IMPLEMENTATION NOTE:

ACS REDIRECT AND GUEST PHYSICAL ADDRESSES (GPAS) §

ACS redirect mechanisms were originally architected to enable fine-grained access control for P2P Memory Requests, by redirecting selected Requests Upstream to the RC, where validation logic determines whether to allow or deny access. However, ACS redirect mechanisms can also ensure that Functions under the direct control of VMs have their DMA Requests routed correctly to the Translation Agent in the host, which then translates their guest physical addresses (GPAs) into host physical addresses (HPAs).

GPA ranges used for Memory Space vs. DMA are not guaranteed to coincide with HPA ranges, which the PCIe fabric uses for Memory Request routing and access control. If any GPAs used for DMA fall within the HPA ranges used for Memory Space, legitimate or malicious packet misrouting can result.

ACS redirect mechanisms can ensure that Upstream Memory Requests with GPAs intended for DMA never get routed to HPA Memory ranges. ACS P2P Request Redirect handles this for (1) peer accesses between Functions within a Multi-Function Device and (2) peer accesses between Downstream Ports within a Switch or RC. ACS P2P Egress Control with redirect handles this in a more fine-grained manner for the same two cases.

Redirect mechanisms introduced with ACS Enhanced Capability handle this for additional cases. ACS DSP Memory Target Access with redirect handles this for Downstream Port Memory Resource ranges. ACS USP Memory Target Access with redirect handles this for Switch Upstream Port Memory Resource ranges. In Switches, ACS Unclaimed Request Redirect handles this for any areas within Upstream Port Memory apertures that are not handled by the other ACS redirect mechanisms.

Together these ACS redirect mechanisms can ensure that Upstream Memory Requests with GPAs intended for DMA are always routed or redirected to the Translation Agent in the host, and those with GPAs intended for P2P are still routed as originally architected.

6.12.5 ACS Violation Error Handling §

ACS Violations may occur due to either hardware or software defects/failures. To assist in fault isolation and root cause analysis, it is recommended that AER be implemented in ACS components. AER prefix/header logging and the Prefix Log/Header Log registers may be used to determine the prefix/header of the offending Request. The ACS Violation Status, Mask, and Severity bits provide positive identification of the error and increased control over error logging and signaling.

When an ACS Violation is detected, the detecting ACS component operates as the Completer¹⁴⁸ and must do the following:

- For Non-Posted and UIO Requests, the Completer must generate a Completion with a Completer Abort (CA) Completion Status (see § Section 2.2.9.1).
- This Completer must log and signal the ACS Violation as indicated in § Figure 6-2. Note the following:
 - Even though the Completer uses a CA Completion Status when it sends a Completion, the Completer must log an ACS Violation error instead of a Completer Abort error.
 - If the severity of the ACS Violation is non-fatal and the Completer sends a Completion with CA Completion Status, this case must be handled as an Advisory Non-Fatal Error as described in § Section 6.2.3.2.4.1.

148. In all cases but one, the ACS component that detects the ACS Violation also operates as the Completer. The exception case is when Root Complex Redirected Request Validation logic disallows a redirected Request. If the redirected Request came through a Root Port, that Root Port must operate as the Completer. If the redirected Request came from a Root Complex Integrated Endpoint, the associated Root Complex Event Collector must operate as the Completer.

- This Completer¹⁴⁹ must set the Signaled Target Abort bit in either its Status register or Secondary Status register as appropriate.

6.12.6 ACS Redirection Impacts on Ordering Rules §

When ACS P2P Request Redirect is enabled, some or all peer-to-peer Requests are redirected, which can cause ordering rule violations in some cases. This section explores those cases, plus a similar case that occurs with RCs that implement “Request Retargeting” as an alternative mechanism for enforcing peer-to-peer access control.

6.12.6.1 Completions Passing Posted Requests §

When a peer-to-peer Posted Request is redirected, a subsequent peer-to-peer non-RO¹⁵⁰ Completion that is routed directly can effectively pass the redirected Posted Request, violating the ordering rule that non-RO Completions must not pass Posted Requests. Refer to § Section 2.4.1 for more information.

ACS P2P Completion Redirect can be used to avoid violating this ordering rule. When ACS P2P Completion Redirect is enabled, all peer-to-peer non-RO Completions will be redirected, thus taking the same path as redirected peer-to-peer Posted Requests. Enabling ACS P2P Completion Redirect when some or all peer-to-peer Requests are routed directly will not cause any ordering rule violations, since it is permitted for a given Completion to be passed by any TLP other than another Completion with the same Transaction ID.

As an alternative mechanism to ACS P2P Request Redirect for enforcing peer-to-peer access control, some RCs implement “Request Retargeting”, where the RC supports special address ranges for “peer-to-peer” traffic, and the RC will retarget validated Upstream Requests to peer devices. Upon receiving an Upstream Request targeting a special address range, the RC validates the Request, translates the address to target the appropriate peer device, and sends the Request back Downstream. With retargeted Requests that are Non-posted, if the RC does not modify the Requester ID, the resulting Completions will travel “directly” peer-to-peer back to the original Requester, creating the possibility of non-RO Completions effectively passing retargeted Posted Requests, violating the same ordering rule as when ACS P2P Request Redirect is being used. ACS P2P Completion Redirect can be used to avoid violating this ordering rule here as well.

If ACS P2P Request Redirect and RC P2P Request Retargeting are not being used, there is no envisioned benefit to enabling ACS P2P Completion Redirect, and it is recommended not to do so because of potential performance impacts.

149. Similarly, if the Request was a Non-Posted or UIO Request, when the Requester receives the resulting Completion with CA Completion Status, the Requester must set the Received Target Abort bit in either its Status register or Secondary Status register as appropriate. Note that for the case of a Multi-Function Device incurring an ACS Violation error with a peer-to-peer Request between its Functions, the same Function might serve both as Requester and Completer.

150. In this section, “non-RO” is an abbreviation characterizing TLPs whose Relaxed Ordering Attribute field is not set.

IMPLEMENTATION NOTE: PERFORMANCE IMPACTS WITH ACS P2P COMPLETION REDIRECT §

While the use of ACS P2P Completion Redirect can avoid ordering violations with Completions passing Posted Requests, it also may impact performance. Specifically, all redirected Completions will have to travel up to the RC from the point of redirection and back, introducing extra latency and possibly increasing Link and RC congestion.

Since peer-to-peer Completions with the Relaxed Ordering bit set are never redirected (thus avoiding performance impacts), it is strongly recommended that Requesters be implemented to maximize the proper use of Relaxed Ordering, and that software enable Requesters to utilize Relaxed Ordering by setting the Enable Relaxed Ordering bit in the Device Control Register.

If software enables ACS P2P Request Redirect, RC P2P Request Retargeting, or both, and software is certain that proper operation is not compromised by peer-to-peer non-RO Completions passing peer-to-peer¹⁵¹ Posted Requests, it is recommended that software leave ACS P2P Completion Redirect disabled as a way to avoid its performance impacts.

6.12.6.2 Requests Passing Posted Requests §

When some peer-to-peer Requests are redirected but other peer-to-peer Requests are routed directly, the possibility exists of violating the ordering rules where Non-Posted Requests or non-RO Posted Requests must not pass Posted Requests. Refer to § Section 2.4.1 for more information.

These ordering rule violation possibilities exist only when ACS P2P Request Redirect and ACS Direct Translated P2P are both enabled. Software should not enable both these mechanisms unless it is certain either that such ordering rule violations cannot occur, or that proper operation will not be compromised if such ordering rule violations do occur.

151. These include true peer-to-peer Requests that are redirected by the ACS P2P Request Redirect mechanism, as well as “logically peer-to-peer” Requests routed to the Root Complex that the Root Complex then retargets to the peer device.

IMPLEMENTATION NOTE: ENSURING PROPER OPERATION WITH ACS DIRECT TRANSLATED P2P §

The intent of ACS Direct Translated P2P is to optimize performance in environments where Address Translation Services (ATS) are being used with peer-to-peer communication whose access control is enforced by the RC. Permitting peer-to-peer Requests with Translated addresses to be routed directly avoids possible performance impacts associated with redirection, which introduces extra latency and may increase Link and RC congestion.

For the usage model where peer-to-peer Requests with Translated addresses are permitted, but those with Untranslated addresses are to be blocked as ACS Violations, it is recommended that software enable ACS Direct Translated P2P and ACS P2P Request Redirect, and configure the Redirected Request Validation logic in the RC to block the redirected Requests with Untranslated addresses. This configuration has no ordering rule violations associated with Requests passing Posted Requests.

For the usage model where some Requesters use Translated addresses exclusively with peer-to-peer Requests and some Requesters use Untranslated addresses exclusively with peer-to-peer Requests, and the two classes of Requesters do not communicate peer-to-peer with each other, proper operation is unlikely to be compromised by redirected peer-to-peer Requests (with Untranslated addresses) being passed by direct peer-to-peer Requests (with Translated addresses). It is recommended that software not enable ACS Direct Translated P2P unless software is certain that proper operation is not compromised by the resulting ordering rule violations.

For the usage model where a single Requester uses both Translated and Untranslated addresses with peer-to-peer Requests, again it is recommended that software not enable ACS Direct Translated P2P unless software is certain that proper operation is not compromised by the resulting ordering rule violations. This requires a detailed analysis of the peer-to-peer communications models being used, and is beyond the scope of this specification.

6.13 Alternative Routing-ID Interpretation (ARI) §

Routing IDs, Requester IDs, and Completer IDs are 16-bit identifiers traditionally composed of three fields: an 8-bit Bus Number, a 5-bit Device Number, and a 3-bit Function Number. With ARI, the 16-bit field is interpreted as two fields instead of three: an 8-bit Bus Number and an 8-bit Function Number - the Device Number field is eliminated. This new interpretation enables an ARI Device to support up to 256 Functions [0..255] instead of 8 Functions [0..7].

ARI is controlled by a new set of optional capability and control register bits. These provide:

- Software the ability to detect whether a component supports ARI.
- Software the ability to configure an ARI Downstream Port so the logic that determines when to turn a Type 1 Configuration Request into a Type 0 Configuration Request no longer enforces a restriction on the traditional Device Number field being 0.
- Software the ability to configure an ARI Device to assign each Function to a Function Group. Controls based on Function Groups may be preferable when finer granularity controls based on individual Functions are not required.
 - If Multi-Function VC arbitration is supported and enabled, arbitration can optionally be based on Function Groups instead of individual Functions.

- If ACS P2P Egress Controls are supported and enabled, access control can optionally be based on Function Groups instead of individual Functions.

The following illustrates an example flow for enabling these capabilities and provides additional details on their usage:

1. Software enumerates the PCI Express hierarchy and determines whether ARI is supported.
 - a. For an ARI Downstream Port, the capability is communicated through the Device Capabilities 2 register.
 - b. For an ARI Device, the capability is communicated through the ARI Extended Capability structure.
2. Software enables ARI functionality in each component.
 - a. In an ARI Downstream Port immediately above an ARI Device, software sets the ARI Forwarding Enable bit in the Device Control 2 register. Setting this bit ensures the logic that determines when to turn a Type 1 Configuration Request into a Type 0 Configuration Request no longer enforces a restriction on the traditional Device Number field being 0.
 - b. In an ARI Device, Extended Functions must respond if addressed with a Type 0 Configuration Request. It is necessary for ARI-aware software to enable ARI Forwarding in the Downstream Port immediately above the ARI Device, in order for ARI-aware software to discover and configure the Extended Functions.
 - c. If an ARI Device implements a Multi-Function VC Extended Capability structure with Function arbitration, and also implements MFVC Function Groups, ARI-aware software categorizes Functions into Function Groups.
 - i. Each Function is assigned to a Function Group represented by a **Function Group Number**.
 - ii. A maximum of 8 Function Groups can be configured.
 - iii. Within the Multi-Function VC Arbitration Table, a Function Group Number is used in place of a Function Number in each arbitration slot.
 1. Arbitration occurs on a Function Group basis instead of an individual Function basis.
 2. All other aspects of Multi-Function VC arbitration remain unchanged. See § Section 7.9.2.10 for additional details.
 - iv. Function arbitration within each Function Group is implementation specific.
 - d. If an ARI Device supports ACS P2P Egress Control, access control can be optionally implemented on a Function Group basis.
 - e. To improve the enumeration performance and create a more deterministic solution, software can enumerate Functions through a linked list of Function Numbers. The next linked list element is communicated through each Function's ARI Capability Register.
 - i. Function 0 acts as the head of a linked list of Function Numbers. Software detects a non-Zero Next Function Number field within the ARI Capability Register as the next Function within the linked list. Software issues a configuration probe using the Bus Number captured by the Device and the Function Number derived from the ARI Capability Register to locate the next associated Function's configuration space.
 - ii. Function Numbers may be sparse and non-sequential in their consumption by an ARI Device.

With an ARI Device, the Phantom Functions Supported field within each Function's Device Capabilities register (see § Section 7.5.3.3 , § Table 7-20) must be set to 00b to indicate that Phantom Functions are not supported. The Extended Tag Field Enable bit, the 10-Bit Tag Requester Enable bit, and the 14-Bit Tag Requester Enable bit can still be used to enable each Function to support higher numbers of outstanding non-UIO Requests. See § Section 2.2.6.2 .

§ Figure 6-13 shows an example system topology with two ARI Devices, one below a Root Port and one below a Switch. For access to Extended Functions in ARI Device X, Root Port A must support ARI Forwarding and have it enabled by software. For access to Extended Functions in ARI Device Y, Switch Downstream Port D must support ARI Forwarding and have it enabled by software. With this configuration, it is recommended that software not enable ARI Forwarding in Root Port B or Switch Downstream Port C.

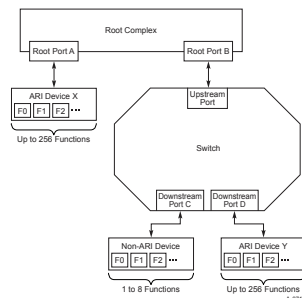


Figure 6-13 Example System Topology with ARI Devices §

IMPLEMENTATION NOTE:

ARI FORWARDING ENABLE BEING SET INAPPROPRIATELY §

It is strongly recommended that software in general Set the ARI Forwarding Enable bit in a Downstream Port only if software is certain that the device immediately below the Downstream Port is an ARI Device. If the bit is Set when a non-ARI Device is present, the non-ARI Device can respond to Configuration Space accesses under what it interprets as being different Device Numbers, and its Functions can be aliased under multiple Device Numbers, generally leading to undesired behavior.

Following a hot-plug event below a Downstream Port, it is strongly recommended that software Clear the ARI Forwarding Enable bit in the Downstream Port until software determines that a newly added component is in fact an ARI Device.

IMPLEMENTATION NOTE:

ARI FORWARDING ENABLE SETTING AT FIRMWARE/ OPERATING SYSTEM CONTROL HANDOFF §

It is strongly recommended that firmware not have the ARI Forwarding Enable bit Set in a Downstream Port upon control handoff to an operating system unless firmware knows that the operating system is ARI-aware. With this bit Set, a non-ARI-aware operating system might be able to discover and enumerate Extended Functions in an ARI Device below the Downstream Port, but such an operating system would generally not be able to manage Extended Functions successfully, since it would interpret there being multiple Devices below the Downstream Port instead of a single ARI Device. As one example of many envisioned problems, the interrupt binding for INTx virtual wires would not be consistent with what the non-ARI-aware operating system would expect.

6.14 Multicast Operations §

The Multicast Capability structure defines a Multicast address range, the segmentation of that range into a number, N , of equal sized Multicast Windows, and the association of each Multicast Window with a Multicast Group, MCG. Each Function that supports Multicast within a component implements a Multicast Capability structure that provides routing directions and permission checking for each MCG for TLPs passing through or to the Function. The Multicast Group is a field of up to 6 bits in width embedded in the address beginning at the MC_Index_Position, as defined in § Section 7.9.11.4 .

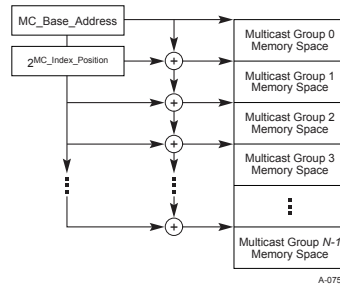


Figure 6-14 Segmentation of the Multicast Address Range §

6.14.1 Multicast TLP Processing §

A Multicast Hit occurs if all of the following are true:

- MC_Enable is Set
- TLP is a non-UIO Memory Write Request or an Address Routed Message, both of which are non-UIO Posted Requests
- $\text{Address}_{\text{TLP}} \geq \text{MC_Base_Address}$
- $\text{Address}_{\text{TLP}} < (\text{MC_Base_Address} + (2^{\text{MC_Index_Position}} * (\text{MC_Num_Group} + 1)))$

In this step, each Switch Ingress Port and other components use values of MC_Enable, MC_Base_Address, MC_Index_Position, and MC_Num_Group from any one of their Functions. Software is required to configure all Functions of a Switch and all Functions of a Multi-Function Upstream Port to have the same values in each of these fields and results are indeterminate if this is not the case.

If the address in a Non-Posted Memory Request hits in a Multicast Window, no Multicast Hit occurs and the TLP is processed normally per the base specification - i.e., as a unicast.

If a Multicast Hit occurs, the only ACS access control that can still apply is ACS Source Validation. In particular, neither ACS P2P Request Redirect nor ACS P2P Egress Control affects operations during a Multicast Hit. See § Section 6.12 .

If a Multicast Hit occurs, normal address routing rules do not apply. Instead, the TLP is processed as follows:

The Multicast Group is extracted from the address in the TLP using any Function's values for MC_Base_Address and MC_Index_Position. Specifically:

$$\text{MCG} = ((\text{Address}_{\text{TLP}} - \text{MC_Base_Address}) \gg \text{MC_Index_Position}) \& 3\text{Fh}$$

In this process, the component may use any Function's values for MC_Base_Address and MC_Index_Position. Which Function's values are used is device-specific.

Components next check the MC_Block_All and the MC_Block_Untranslated bits corresponding to the extracted MCG. Switches and Root Ports check Multicast TLPs in their Ingress Ports using the MC_Block_All and MC_Block_Untranslated registers associated with the Ingress Port. Endpoint Functions check Multicast TLPs they are preparing to send, using their MC_Block_All and MC_Block_Untranslated registers. If the MC_Block_All bit corresponding to the extracted MCG is set, the TLP is handled as an MC Blocked TLP. If the MC_Block_Untranslated bit corresponding to the extracted MCG is set and the TLP contains an Untranslated Address, the TLP, is also handled as an MC Blocked TLP.

IMPLEMENTATION NOTE:

MC_BLOCK_UNTRANSLATED AND PIO WRITES §

Programmed I/O (PIO) Writes to Memory Space generally have Untranslated addresses since there is no architected mechanism for software to control the Address Type (AT) field for PIO Requests. Thus, if it's necessary for a given Switch to Multicast any PIO Writes, software should ensure that the appropriate MC_Block_Untranslated bits in the Upstream Port of that Switch are Clear. Otherwise, the Switch Upstream Port may block PIO Writes that legitimately target Multicast Windows. Since it may be necessary for software to clear MC_Block_Untranslated bits in a Switch Upstream Port for the sake of PIO Writes, the following are strongly recommended for a Root Complex capable of Address translation:

- All Integrated Endpoints each implement a Multicast Capability structure to provide access control for sending Untranslated Multicast TLPs.
- All peer-to-peer capable Root Ports each implement a Multicast Capability structure to provide access control for Untranslated Multicast TLPs that are forwarded peer-to-peer.

For similar reasons, with Multicast-capable Switch components where the Upstream Port is a Function in a Multi-Function Device, it is strongly recommended that any Endpoints in that Multi-Function Device each implement a Multicast Capability structure.

IMPLEMENTATION NOTE:

MULTICAST WINDOW SIZE §

Each ultimate Receiver of a Multicast TLP may have a different Multicast Window size requirement. At one extreme, a Multicast Window may be required to cover a range of memory implemented within the device. At the other, it may only need to cover a particular offset at which a FIFO register is located. The MC_Window_Size_Requested field within the Multicast Capability register is used by an Endpoint to advertise the size of Multicast Window that it requires.

Unless available address space is limited, resource allocation software may be able to treat each request as a minimum and set the Multicast Window size via MC_Index_Position to accommodate the largest request. In some cases, a request for a larger window size can be satisfied by configuring a smaller window size and assigning the same membership to multiple contiguous MCGs.

IMPLEMENTATION NOTE:

MULTICAST, ATS, AND REDIRECTION §

The ACS P2P Request Redirect and ACS Direct Translated P2P mechanisms provide a means where P2P Requests with Untranslated Addresses can be redirected to the Root Complex (RC) for access control checking, whereas P2P Requests with Translated Addresses can be routed “directly” to their P2P targets for improved performance. See § Section 6.12 . No corresponding redirection mechanism exists for Multicast TLPs.

To achieve similar functionality, an RC might be configured to provide one or more target Memory Space ranges that are not in the Multicast address range, but the RC maps to “protected” Multicast Windows. Multicast TLP senders either with or without ATS capability then target these RC Memory Space ranges in order to access the protected Multicast Windows indirectly. When either type of sender targets these ranges with Memory Writes, each TLP that satisfies the access control checks will be reflected back down by the RC with a Translated Address targeting a protected Multicast Window.¹⁵² ATS-capable senders can request and cache Translated Addresses using the RC Memory Space range, and then later use those Translated Addresses for Memory Writes that target protected Multicast Windows directly and can be Multicast without a taking a trip through the RC.

For hardware enforcement that only Translated Addresses can be used to target the protected Multicast Windows directly, software Sets appropriate MCG bits in the `MC_Block_Untranslated` register in all applicable Functions throughout the platform. Each MCG whose bit is Set will cause its associated Multicast Window to be protected from direct access using Untranslated Addresses.

If the TLP is not blocked in a Switch or Root Complex it is forwarded out all of the Ports, except its Ingress Port, whose `MC_Receive` bit corresponding to the extracted MCG is set. In an Endpoint, it is consumed by all Functions whose `MC_Receive` bit corresponding to the extracted MCG is set. If no Ports forward the TLP or no Functions consume it, the TLP is silently dropped.

To prevent loops, it is prohibited for a Root Port or a Switch Port to forward a TLP back out its Ingress Port, even if so specified by the `MC_Receive` register associated with the Port. An exception is the case described in the preceding Implementation Note, where an RC reflects a unicast TLP that came in on an Ingress Root Port to a Multicast Window. In that case, when specified by the `MC_Receive` register associated with that Ingress Root Port, the RC is required to send the reflected TLP out the same Root Port that it originally came in.

A Multicast Hit suspends normal address routing, including default Upstream routing in Switches. When a Multicast Hit occurs, the TLP will be forwarded out only those Egress Ports whose `MC_Receive` bit associated with the MCG extracted from the address in the TLP is set. If the address in the TLP does not decode to any Downstream Port using normal address decode, the TLP will be copied to the Upstream Port only if so specified by the Upstream Port’s `MC_Receive` register.

6.14.2 Multicast Ordering §

No new ordering rules are defined for processing Multicast TLPs. All Multicast TLPs are Posted Requests and follow Posted Request ordering rules. Multicast TLPs are ordered per normal ordering rules relative to other TLPs in a component’s ingress stream through the point of replication. Once copied into an egress stream, a Multicast TLP follows the same ordering as other Posted Requests in the stream.

152. If the original sender belongs to the MCG associated with this Window, the original sender will also receive a copy of the reflected TLP.

6.14.3 Multicast Capability Structure Field Updates §

Some fields of the Multicast Capability structure may be changed at any time. Others cannot be changed with predictable results unless the MC_Enable bit is Clear in every Function of the component. The latter group includes MC_Base_Address and MC_Index_Position.

Fields which software may change at any time include MC_Enable, MC_Num_Group, MC_Receive, MC_Block_All, and MC_Block_Untranslated. Updates to these fields must themselves be ordered. Consider, for example, TLPs A and B arriving in that order at the same Ingress Port and in the same TC. If A uses value X for one of these fields, then B must use the same value or a newer value.

For Multi-Function Upstream Switch Ports Multicast TLPs received by one Switch or transmitted by one Endpoint Function are presented to the other parallel Endpoint Functions and the Downstream Switch Ports of the other parallel Switches (Functions are considered to be parallel if they are in the same Device). A single Multicast TLP is forwarded Upstream when any of the Upstream Switch Functions has the appropriate MC_Receive bit Set.

6.14.4 MC Blocked TLP Processing §

When a TLP is blocked by the MC_Block_All or the MC_Block_Untranslated mechanisms, the TLP is dropped. The Function blocking the TLP serves as the Completer. The Completer must log and signal this MC Blocked TLP error as indicated in § Figure 6-2. In addition, the Completer must set the Signaled Target Abort bit in either its Status register or Secondary Status register as appropriate. To assist in fault isolation and root cause analysis, it is strongly recommended that AER be implemented in Functions with Multicast capability.

In Root Complexes and Switches, if the error occurs with a TLP received by an Ingress Port, the error is reported by that Ingress Port. If the error occurs in an Endpoint Function preparing to send the TLP, the error is reported by that Endpoint Function.

6.14.5 MC_Overlay Mechanism §

The MC_Overlay mechanism is provided to allow a single BAR in an Endpoint that doesn't contain a Multicast Capability structure to be used for both Multicast and unicast TLP reception. Software can configure the MC_Overlay mechanism to affect this by setting the MC_Overlay_BAR in a Downstream Port so that the Multicast address range, or a portion of it, is remapped (overlaid) onto the Memory Space range accepted by the Endpoint's BAR. At the Upstream Port of a Switch, the mechanism can be used to overlay a portion of the Multicast address range onto a Memory Space range associated with host memory.

A Downstream Port's MC_Overlay mechanism applies to TLPs exiting that Port. An Upstream Port's MC_Overlay mechanism applies to TLPs exiting the Switch heading Upstream. A Port's MC_Overlay mechanism does not apply to TLPs received by the Port, to TLPs targeting memory space within the Port, or to TLPs routed Peer-to-Peer between Functions in a Multi-Function Upstream Port.

When enabled, the overlay operation specifies that bits in the address in the Multicast TLP, whose bit numbers are equal to or higher than the MC_Overlay_Size field, be replaced by the corresponding bits in the MC_Overlay_BAR. In other words:

```

If (MC_Overlay_Size < 6)
  Then Egress_TLP_Addr = Ingress_TLP_Addr;
  Else Egress_TLP_Addr = { MC_Overlay_BAR[63:MC_Overlay_Size],
                          Ingress_TLP_Addr[MC_Overlay_Size-1:0] };

```

Equation 6-1 MC_Overlay Transform rules §

If the TLP with modified address contains the optional ECRC, the unmodified ECRC will almost certainly indicate an error. The action to be taken if a TLP containing an ECRC is Multicast copied to an Egress Port that has MC_Overlay enabled depends upon whether or not optional support for ECRC regeneration is implemented. All of the contingent actions are outlined in § Table 6-12. If MC_Overlay is not enabled, the TLP is forwarded unmodified. If MC_Overlay is enabled and the TLP has no ECRC, the modified TLP, with its address replaced as specified in the previous paragraph is forwarded. If the TLP has an ECRC but ECRC regeneration is not supported, then the modified TLP is forwarded with its ECRC dropped and the TD bit in the header cleared to indicate no ECRC attached. If the TLP has an ECRC and ECRC regeneration is supported, then an ECRC check is performed before the TLP is forwarded. If the ECRC check passes, the TLP is forwarded with regenerated ECRC. If the ECRC check fails, the TLP is forwarded with inverted regenerated ECRC.

Table 6-12 ECRC Rules for MC_Overlay §

MC_Overlay Enabled	TLP has ECRC	ECRC Regeneration Supported	Action if ECRC Check Passes	Action if ECRC Check Fails
No	x	x	Forward TLP unmodified	
Yes	No	x	Forward modified TLP	
Yes	Yes	No	Forward modified TLP with ECRC dropped and TD bit clear	
Yes	Yes	Yes	Forward modified TLP with regenerated ECRC	Forward modified TLP with inverted regenerated ECRC

IMPLEMENTATION NOTE:

MC_OVERLAY AND ECRC REGENERATION §

Switch and Root Complex Ports have the option to support ECRC regeneration. If ECRC regeneration is supported, then it is strongly recommended to do so robustly by minimizing the time between checking the ECRC of the original TLP and replacing it with an ECRC computed on the modified TLP. The TLP is unprotected during this time, leaving a data integrity hole if the pre-check and regeneration aren't accomplished in the same pipeline stage.

Stripping the ECRC from Multicast TLPs passing through a Port that has MC_Overlay enabled but doesn't support ECRC regeneration allows the receiving Endpoint to enable ECRC checking. In such a case, the Endpoint will enjoy the benefits of ECRC on non-Multicast TLPs without detecting ECRC on Multicast TLPs modified by the MC_Overlay mechanism.

When Multicast ECRC regeneration is supported, and an ECRC error is detected prior to TLP modification, then inverting the regenerated ECRC ensures that the ECRC error isn't masked by the regeneration process.

IMPLEMENTATION NOTE: MULTICAST TO ENDPOINTS THAT DON'T HAVE MULTICAST CAPABILITY §

An Endpoint Function that doesn't contain a Multicast Capability structure cannot distinguish Multicast TLPs from unicast TLPs. It is possible for a system designer to take advantage of this fact to employ such Endpoints as Multicast targets. The primary requirement for doing so is that the base and limit registers of the virtual PCI to PCI Bridge in the Switch Port above the device be configured to overlap at least part of the Multicast address range or that the MC_Overlay mechanism be employed. Extending this reasoning, it is even possible that a single Multicast target Function could be located on the PCI/PCI-X side of a PCI Express to PCI/PCI-X Bridge.

If an Endpoint without a Multicast Capability structure is being used as a Multicast target and the MC_Overlay mechanism isn't used, then it may be necessary to read from the Endpoint's Memory Space using the same addresses used for Multicast TLPs. Therefore, Memory Reads that hit in a Multicast Window aren't necessarily errors. Memory Reads that hit in a Multicast Window and that don't also hit in the aperture of an RCiEP or the Downstream Port of a Switch will be routed Upstream, per standard address routing rules, and be handled as a UR there.

IMPLEMENTATION NOTE: MULTICAST IN A ROOT COMPLEX §

A Root Complex with multiple Root Ports that supports Multicast may implement as many Multicast Capability structures as its implementation requires. If it implements more than one, software should ensure that certain fields, as specified in § Section 6.14.3 , are configured identically. To support Multicast to RCiEPs, the implementation needs to expose all TLPs identified as Multicast via the MC_Base_Address register to all potential Multicast target Endpoints integrated within it. Each such Integrated Endpoint then uses the MC_Receive register in its Multicast Capability structure to determine if it should receive the TLP.

IMPLEMENTATION NOTE: MULTICAST AND MULTI-FUNCTION DEVICES §

All Port Functions and Endpoint Functions that are potential Multicast targets need to implement a Multicast Capability structure so that each has its own MC_Receive vector. Within a single component, software should configure the MC_Enable, MC_Base_Address, MC_Index_Position, and MC_Num_Group fields of these Capability structures identically. That being the case, it is sufficient to implement address decoding logic on only one instance of the Multicast BAR in the component.

IMPLEMENTATION NOTE: CONGESTION AVOIDANCE §

The use of Multicast increases the output link utilization of Switches to a degree proportional to both the size of the Multicast groups used and the fraction of Multicast traffic to total traffic. This results in an increased risk of congestion and congestion spreading when Multicast is used.

To mitigate this risk, components that are intended to serve as Multicast targets should be designed to consume Multicast TLPs at wire speed. Components that are intended to serve as Multicast sources should consider adding a rate limiting mechanism.

In many applications, the application's Multicast data flow will have an inherent rate limit and can be accommodated without causing congestion. Others will require an explicit mechanism to limit the injection rate, selection of a Switch with buffers adequate to hold the requisite bursts of Multicast traffic without asserting flow control, or selection of Multicast target components capable of sinking the Multicast traffic at the required rate. It is the responsibility of the system designer to choose the appropriate mechanisms and components to serve the application.

IMPLEMENTATION NOTE: THE HOST AS A MULTICAST RECIPIENT §

For general-purpose systems, it is anticipated that the Multicast address range will usually not be configured to overlap with Memory Space that's directly mapped to host memory. If host memory is to be included as a Multicast recipient, the Root Complex may need to have some sort of I/O Memory Management Unit (IOMMU) that is capable of remapping portions of Multicast Windows to host memory, perhaps with page-level granularity. Alternatively, the MC_Overlay mechanism in the Upstream Port of a Switch can be used to overlay a portion of the Multicast address range onto host memory.

For embedded systems that lack an IOMMU, it may be feasible to configure Multicast Windows overlapping with Memory Space that's directly mapped to host memory, thus avoiding the need for an IOMMU. Specific details of this approach are beyond the scope of this specification.

6.15 Atomic Operations (AtomicOps) §

An Atomic Operation (AtomicOp) is a single PCI Express transaction that targets a location in Memory Space, reads the location's value, potentially writes a new value back to the location, and returns the original value. This "read-modify-write" sequence to the location is performed atomically. AtomicOps include the following:

- FetchAdd (Fetch and Add): Request contains a single operand, the "add" value
 - Read the value of the target location.
 - Add the "add" value to it using two's complement arithmetic ignoring any carry or overflow.
 - Write the sum back to the target location.
 - Return the original value of the target location.
- Swap (Unconditional Swap): Request contains a single operand, the "swap" value

- Read the value of the target location.
- Write the “swap” value back to the target location.
- Return the original value of the target location.
- CAS (Compare and Swap): Request contains two operands, a “compare” value and a “swap” value
 - Read the value of the target location.
 - Compare that value to the “compare” value.
 - If equal, write the “swap” value back to the target location.
 - Return the original value of the target location.

A given AtomicOp transaction has an associated operand size, and the same size is used for the target location accesses and the returned value. FetchAdd and Swap support operand sizes of 32 and 64 bits. CAS supports operand sizes of 32, 64, and 128 bits.

AtomicOp capabilities are optional normative. Endpoints and Root Ports are permitted to implement AtomicOp Requester capabilities. PCI Express Functions with Memory Space BARs as well as all Root Ports are permitted to implement AtomicOp Completer capabilities. Routing elements (Switches, as well as Root Complexes supporting peer-to-peer access between Root Ports) require AtomicOp routing capability in order to route AtomicOp Requests. AtomicOps are architected for device-to-host, device-to-device, and host-to-device transactions. In each case, the Requester, Completer, and all intermediate routing elements must support the associated AtomicOp capabilities.

AtomicOp capabilities are not supported on PCI Express to PCI/PCI-X Bridges. If need be, Locked Transactions can be used for devices below such Bridges. AtomicOps and Locked Transactions can operate concurrently on the same hierarchy.

Software discovers specific AtomicOp Completer capabilities via three bits in the Device Capabilities 2 register (see § Section 7.5.3.15). For increased interoperability, Root Ports are required to implement certain AtomicOp Completer capabilities in sets if at all (see § Section 6.15.3.1). Software discovers AtomicOp routing capability via the AtomicOp Routing Supported bit in the Device Capabilities 2 register. Software discovery of AtomicOp Requester capabilities is outside the scope of this specification, but software must set the AtomicOp Requester Enable bit in a Function’s Device Control 2 register before the Function can initiate AtomicOp Requests (see § Section 7.5.3.16).

With routing elements, software can Set an AtomicOp Egress Blocking bit (see § Section 7.5.3.16) on a Port-by-Port basis to avoid AtomicOp Requests being forwarded to components that shouldn’t receive them, and might, if operating in NFM, handle each as a Malformed TLP, which by default is a Fatal Error. Each blocked Request is handled as an AtomicOp Egress Blocked error, which by default is an Advisory Non-Fatal Error. For Root Ports, to ensure that AtomicOps are not transmitted on a given Link, software should Clear AtomicOp Requester Enable and Set AtomicOp Egress Blocking.

The AtomicOp Requester Enable bit, when Clear, prevents transmission of AtomicOps initiated by an entity. When AtomicOps are initiated by a programmable entity (e.g., CPU or accelerator Function), this specification does not architect a mechanism for reporting errors when the bit is Clear and software attempts to initiate an AtomicOp. For Root Ports, when AtomicOp Requester Enable is Clear and AtomicOp Egress Blocking is Set, it is outside the scope of this specification which error mechanism (or both) will be triggered if software attempts to initiate an AtomicOp.

AtomicOps are Memory Transactions, so existing standard mechanisms for managing Memory Space access (e.g., Bus Master Enable, Memory Space Enable, and Base Address registers) apply.

6.15.1 AtomicOp Use Models and Benefits §

AtomicOps enable advanced synchronization mechanisms that are particularly useful when there are multiple producers and/or multiple consumers that need to be synchronized in a non-blocking fashion. For example, multiple producers can safely enqueue to a common queue without any explicit locking.

AtomicOps also enable lock-free statistics counters, for example where a device can atomically increment a counter, and host software can atomically read and clear the counter.

Direct support for the three chosen AtomicOps over PCI Express enables easier migration of existing high-performance SMP applications to systems that use PCI Express as the interconnect to tightly-coupled accelerators, co-processors, or GP-GPUs. For example, a ported application that uses PCI Express-attached accelerators may be able to use the same synchronization algorithms and data structures as the earlier SMP application.

An AtomicOp to a given target generally incurs latency comparable to a Memory Read to the same target. Within a single hierarchy, multiple AtomicOps can be “in flight” concurrently. AtomicOps generally create negligible disruption to other PCI Express traffic.

Compared to Locked Transactions, AtomicOps provide lower latency, higher scalability, advanced synchronization algorithms, and dramatically less impact to other PCI Express traffic.

6.15.2 AtomicOp Transaction Protocol Summary §

Detailed protocol rules and requirements for AtomicOps are distributed throughout the rest of this specification, but here is a brief summary plus some unique requirements.

- AtomicOps are Non-Posted Memory Transactions, supporting 32- and 64-bit address formats.
- FetchAdd, Swap, and CAS each use a distinct type code.
- The Completer infers the operand size from the Length field value and type code in the AtomicOp Request.
- The endian format used by AtomicOp Completers to read and write data at the target location is implementation specific, and permitted to be whatever the Completer determines to be appropriate for the target memory (e.g., little-endian, big-endian, etc.). See § Section 2.2.2 .
- If an AtomicOp Requester supports Address Translation Services (ATS), the Requester is permitted to use a Translated address in an AtomicOp Request only if the Translated address has appropriate access permissions. Specifically, the Read (R) and Write (W) fields must both be Set, and the Untranslated access only (U) field must be Clear. See § Section 2.2.4.1 .
- If a component supporting Access Control Services (ACS) supports AtomicOp routing or AtomicOp Requester capability, it handles AtomicOp Requests and Completions the same as with other Memory Requests and Completions with respect to ACS functionality.
- The No Snoop attribute is applicable and permitted to be Set with AtomicOp Requests, but atomicity must be guaranteed regardless of the No Snoop attribute value.
- The Relaxed Ordering attribute is applicable and permitted to be Set with AtomicOp Requests, where it affects the ordering of both the Requests and their associated Completions.
- Ordering requirements for AtomicOp Requests are similar to those for Non-Posted Write Requests. Thus, if a Requester wants to ensure that an AtomicOp Request is observed by the Completer before a subsequent Posted or Non-Posted Request, the Requester must wait for the AtomicOp Completion before issuing the subsequent Request.
- Ordering requirements for AtomicOp Completions are similar to those for Read Completions.
- Unless there's a higher precedence error, an AtomicOp-aware Completer must handle a Poisoned AtomicOp Request as a Poisoned TLP Received error, and must also return a Completion with a Completion Status of Unsupported Request (UR). See § Section 2.7.2.1 . The value of the target location must remain unchanged.
- If the Completer of an AtomicOp Request encounters an uncorrectable error accessing the target location or carrying out the Atomic operation, the Completer must handle it as a Completer Abort (CA). The subsequent state of the target location is implementation specific.

- AtomicOp-aware Completers are required to handle any properly formed AtomicOp Requests with types or operand sizes they don't support as an Unsupported Request (UR). If the Length field in an AtomicOp Request contains an unarchitected value, the Request must be handled by an AtomicOp-aware Completer as a Malformed TLP. See § Section 2.2.7 .
- If any Function in a Multi-Function Device supports AtomicOp Completer or AtomicOp routing capability, all Functions with Memory Space BARs in that device must decode properly formed AtomicOp Requests and handle any they don't support as an Unsupported Request (UR). Note that in such devices, Functions lacking AtomicOp Completer capability are forbidden to handle properly formed AtomicOp Requests as Malformed TLPs.
- If an RC has any Root Ports that support AtomicOp routing capability, all RCiEPs in the RC reachable by forwarded AtomicOp Requests must decode properly formed AtomicOp Requests and handle any they don't support as an Unsupported Request (UR).
- With an AtomicOp Request having a supported type and operand size, the AtomicOp-aware Completer is required either to carry out the Request or handle it as Completer Abort (CA) for any location in its target Memory Space. Completers are permitted to support AtomicOp Requests on a subset of their target Memory Space as needed by their programming model (see § Section 2.3.1). Memory Space structures defined or inherited by PCI Express (e.g., the MSI-X Table structure) are not required to be supported as AtomicOp targets unless explicitly stated in the description of the structure.
- For a Switch or an RC, when AtomicOp Egress Blocking is enabled in an Egress Port, and an AtomicOp Request targets going out that Egress Port, the Egress Port must handle the Request as an AtomicOp Egress Blocked error¹⁵³ (see § Figure 6-2) and must also return a Completion with a Completion Status of UR. If the severity of the AtomicOp Egress Blocked error is non-fatal, this case must be handled as an Advisory Non-Fatal Error as described in § Section 6.2.3.2.4.1 .

6.15.3 Root Complex Support for AtomicOps §

RCs have unique requirements and considerations with respect to AtomicOp capabilities.

6.15.3.1 Root Ports with AtomicOp Completer Capabilities §

AtomicOp Completer capability for a Root Port indicates that the Root Port supports receiving at its Ingress Port AtomicOp Requests that target host memory or Memory Space allocated by a Root Port BAR. This is independent of any RCiEPs that have AtomicOp Completer capabilities.

If a Root Port implements any AtomicOp Completer capability for host memory access, it must implement all 32-bit and 64-bit AtomicOp Completer capabilities. Implementing 128-bit CAS Completer capability is optional.

If an RC has one or more Root Ports that implement AtomicOp Completer capability, the RC must ensure that host memory accesses to a target location on behalf of a given AtomicOp Request are performed atomically with respect to each host processor or device access to that target location range.

If a host processor supports atomic operations via its instruction set architecture, the RC must also ensure that host memory accesses on behalf of a given AtomicOp Request preserve the atomicity of any host processor atomic operations.

153. Though an AtomicOp Egress Blocked error is handled by returning a Completion with UR Status, the error is not otherwise handled as an Unsupported Request. For example, it does not set the Unsupported Request Detected bit in the Device Status register.

6.15.3.2 Root Ports with AtomicOp Routing Capability §

As with other PCI Express Transactions, the support for peer-to-peer routing of AtomicOp Requests and Completions between Root Ports is optional and implementation dependent. If an RC supports AtomicOp routing capability between two or more Root Ports, it must indicate that capability in each associated Root Port via the AtomicOp Routing Supported bit in the Device Capabilities 2 register.

An RC is not required to support AtomicOp routing between all pairs of Root Ports that have the AtomicOp Routing Supported bit Set. An AtomicOp Request that would require routing between unsupported pairs of Root Ports must be handled as an Unsupported Request (UR), and reported by the “sending” Port.

The AtomicOp Routing Supported bit must be Set for any Root Port that supports forwarding of AtomicOp Requests initiated by host software or RCiEPs. The AtomicOp Routing Supported bit must be Set for any Root Ports that support forwarding of AtomicOp Requests received on their Ingress Port to RCiEPs.

6.15.3.3 RCs with AtomicOp Requester Capabilities §

An RC is permitted to implement the capability for either host software or RCiEPs to initiate AtomicOp Requests.

Software discovery of AtomicOp Requester capabilities is outside the scope of this specification.

If an RC supports software-initiated AtomicOp Requester capabilities, the specific mechanisms for how software running on a host processor causes the RC to generate AtomicOp Requests is outside the scope of this specification.

IMPLEMENTATION NOTE: GENERATING ATOMICOP REQUESTS VIA HOST PROCESSOR SOFTWARE §

If a host processor instruction set architecture (ISA) supports atomic operation instructions that directly correspond to one or more PCI Express AtomicOps, an RC might process the associated internal atomic transaction that targets PCI Express Memory Space much like it processes the internal read transaction resulting from a processor load instruction. However, instead of “exporting” the internal read transaction as a PCI Express Memory Read Request, the RC would export the internal atomic transaction as a PCI Express AtomicOp Request. Even if an RC uses the “export” approach for some AtomicOp types and operand sizes, it would not need to use this approach for all.

For AtomicOp types and operand sizes where the RC does not use the “export” approach, the RC might use an RC register-based mechanism similar to one where some PCI host bridges use CONFIG_ADDRESS and CONFIG_DATA registers to generate Configuration Requests. Refer to the [PCI] for details.

The “export” approach may permit a large number of concurrent AtomicOp Requests without becoming RC register limited. It may also be easier to support AtomicOp Request generation from user space software using this approach.

The RC register-based mechanism offers the advantage of working for all AtomicOp types and operand sizes even if the host processor ISA doesn’t support the corresponding atomic instructions. It might also support a polling mode for waiting on AtomicOp Completions as opposed to stalling the processor while waiting for a Completion.

6.15.4 Switch Support for AtomicOps §

If a Switch supports AtomicOp routing capability for any of its Ports, it must do so for all of them.

6.16 Dynamic Power Allocation (DPA) Capability §

A common approach to managing power consumption is through a negotiation between the device driver, operating system, and executing applications. Adding Dynamic Power Allocation for such devices is anticipated to be done as an extension of that negotiation, through software mechanisms that are outside of the scope of this specification. Some devices do not have a device specific driver to manage power efficiently. The DPA Capability provides a mechanism to allocate power dynamically for these types of devices. DPA is optional normative functionality applicable to Endpoint Functions that can benefit from the dynamic allocation of power and do not have an alternative mechanism. If supported, the Emergency Power Reduction State, over-rides the mechanisms listed here (see § Section 6.24).

The DPA Capability enables software to actively manage and optimize Function power usage when in the D0 state. DPA is not applicable to power states D1-D3 therefore the DPA Capability is independently managed from the PCI-PM Capability.

DPA defines a set of power substates, each of which with an associated power allocation. Up to 32 substates [0..31] can be defined per Function. Substate 0, the default substate, indicates the maximum power the Function is ever capable of consuming.

Substates must be contiguously numbered from 0 to Substate_Max, as defined in § Section 7.9.12.2 . Each successive substate has a power allocation lower than or equal to that of the prior substate. For example, a Function with four substates could be defined as follows:

1. Substate 0 (the default) defines a power allocation of 25 Watts.
2. Substate 1 defines a power allocation of 20 Watts.
3. Substate 2 defines a power allocation of 20 Watts.
4. Substate 3 defines a power allocation of 10 Watts.

When the Function is initialized, it will operate within the power allocation associated with substate 0. Software is not required to progress through intermediate substates. Over time, software may dynamically configure the Function to operate at any of the substates in any sequence it chooses. Software is permitted to configure the Function to operate at any of the substates before the Function completes a previously initiated substate transition.

On the completion of the substate transition(s) the Function must compare its substate with the configured substate. If the Function substate does not match the configured substate, then the Function must begin transition to the configured substate. It is permitted for the Function to dynamically alter substate transitions on Configuration Requests instructing the Function to operate in a new substate.

In the prior example, software can configure the Function to transition to substate 4, followed by substate 1, followed by substate 3, and so forth. As a result, the Function must be able to transition between any substates when software configures the associated control field.

The Substate Control Enabled bit provides a mechanism that allows the DPA Capability to be used in conjunction with the software negotiation mechanism mentioned above. When Set, power allocation is controlled by the DPA Capability. When Clear, the DPA Capability is disabled, and the Function is not permitted to directly initiate substate transitions based on configuration of the Substate Control register field. At an appropriate point in time, software participating in the software negotiation mechanism mentioned above clears the bit, effectively taking over control of power allocation for the Function.

It is required that the Function respond to Configuration Space accesses while in any substate.

At any instant, the Function must never draw more power than it indicates through its Substate Status. When the Function is configured to transition from a higher power substate to a lower power substate, the Function's Substate Status must indicate the higher power substate during the transition, and must indicate the lower power substate after completing the transition. When the Function is configured to transition from a lower power substate to a higher power substate, the Function's Substate Status must indicate the higher power substate during the transition, as well as after completing the transition.

Due to the variety of applications and the wide range of maximum power required for a given Function, the transition time required between any substates is implementation specific. To enable software to construct power management policies (outside the scope of this specification), the Function defines two Transition Latency Values. Each of the Function substates associates its maximum Transition Latency with one of the Transition Latency Values, where the maximum Transition Latency is the time it takes for the Function to enter the configured substate from any other substate. A Function is permitted to complete the substate transition faster than the maximum Transition Latency for the substate.

6.16.1 DPA Capability with Multi-Function Devices §

Except as stated below, it is permitted for some or all Functions of a Multi-Function Device to implement a DPA Capability. The power allocation for the Multi-Function Device is the sum of power allocations set by the DPA Capability for each Function. It is permitted for the DPA Capability of a Function to include the power allocation for the Function itself as well as account for power allocation for other Functions that do not implement a DPA Capability. The association between multiple Functions for DPA is implementation specific and beyond the scope of this specification.

Power allocation for VFs is managed using their associated PF's DPA Capability, if implemented. VFs must not implement the Dynamic Power Allocation Capability.

6.17 TLP Processing Hints (TPH) §

TLP Processing Hints is an optional feature that provides hints in Request TLP headers to facilitate optimized processing of Requests that target Memory Space. These Processing Hints enable the system hardware (e.g., the Root Complex and/or Endpoints) to optimize platform resources such as system and memory interconnect on a per TLP basis. The TPH mechanism defines Processing Hints that provide information about the communication models between Endpoints and the Root Complex. Steering Tags are system-specific values used to identify a processing resource that a Requester explicitly targets. System software discovers and identifies TPH capabilities to determine the Steering Tag allocation for each Function that supports TPH.

6.17.1 Processing Hints §

The Requester provides hints to the Root Complex or other targets about the intended use of data and data structures by the host and/or device. The hints are provided by the Requester, which has knowledge of upcoming Request patterns, and which the Completer would not be able to deduce autonomously (with good accuracy). Cases of interest to distinguish with such hints include:

DWHR: Device writes then host reads soon

HWDR: Device reads data that the host is believed to have recently written

D*D*: Device writes/reads, then device reads/writes soon

Includes DWDW, DWDR, DRDW, DRDR

Bi-Directional: Data structure that is shared and has equal read/write access by host and device.

The usage models are mapped to the Processing Hint encodings as described in § Table 6-13.

Table 6-13 Processing Hint Mapping §

PH[1:0] (b)	Processing Hint	Usage Model
00	Bi-directional data structure	Bi-Directional shared data structure
01	Requester	D*D*
10	Target	DWHR HWDR
11	Target with Priority	Same as target but with temporal reuse priority

6.17.2 Steering Tags §

Functions that intend to target a TLP towards a specific processing resource such as a host processor or system cache hierarchy require topological information of the target cache (e.g., which host cache). Steering Tags are system-specific values that provide information about the host or cache structure in the system cache hierarchy. These values are used to associate processing elements within the platform with the processing of Requests.

Software programmable Steering Tag values to be used are stored in an ST Table that is permitted to be located either in the TPH Requester Extended Capability structure (see § Section 7.9.13) or combined with the MSI-X Table (see § Section 7.7), but not in both locations for a given Function. When the ST Table is combined with the MSI-X Table, the 2 most significant bytes of the Vector Control register of each MSI-X Table entry are used to contain the Steering Tag value.

The choice of ST Table location is implementation specific and is discoverable by software. A Function that implements MSI-X is permitted to locate the ST Table in either location (see § Section 7.9.13.2). A Function that implements both MSI and MSI-X is permitted to combine the ST Table with the MSI-X Table and use it, even when MSI-X is disabled (i.e., when MSI is enabled). Each ST Table entry is 2 bytes. The size of the ST Table is indicated in the TPH Requester Extended Capability structure.

For some usage models the Steering Tags are not required or not provided, and in such cases a Function is permitted to use a value of all zeros in the ST field to indicate no ST preference. The association of each Request with an ST Table entry is device specific and outside the scope of this specification.

6.17.3 ST Modes of Operation §

The ST Table Location field in the TPH Requester Extended Capability structure indicates where (if at all) the ST Table is implemented by the Function. If an ST Table is implemented, software can program it with the system-specific Steering Tag values.

Table 6-14 ST Modes of Operation §

ST Mode Select [2:0] (b)	ST Mode Name	Description
000	No ST Mode	The Function must use a value of all zeros for all Steering Tags.

ST Mode Select [2:0] (b)	ST Mode Name	Description
001	Interrupt Vector Mode	Each Steering Tag is selected by an MSI/MSI-X interrupt vector number. The Function is required to use the Steering Tag value from an ST Table entry that can be indexed by a valid MSI/MSI-X interrupt vector number.
010	Device Specific Mode	It is recommended for the Function to use a Steering Tag value from an ST Table entry, but it is not required.
All other encodings	Reserved	Reserved for future use.

In the No ST Mode of operation, the Function must use a value of all zeros for each Steering Tag, enabling the use of Processing Hints without software-provided Steering Tags.

In the Interrupt Vector Mode of operation, Steering Tags are selected from the ST Table using MSI/MSI-X interrupt vector numbers. For Functions that have MSI enabled, the Function is required to select tags within the range specified by the Multiple Message Enable field in the MSI Capability structure. For Functions that have MSI-X enabled, the Function is required to select tags within the range of the MSI-X Table size. If the ST Table Size is smaller than the enabled range of interrupt vector numbers, the Function is permitted to either not use TPH for certain transactions, to use TPH with a Steering Tag of 0 or to use TPH with an implementation defined mechanism used to select a Steering Tag value from the ST Table. If the ST Table Size is larger than the enabled range of interrupt vector numbers, ST Table Entries corresponding to out of range interrupt vector numbers are ignored by the Function.

In the Device Specific Mode of operation, the assignment of the Steering Tags to Requests is device specific. The number of Steering Tags used by the Function is permitted to be different than the number of interrupt vectors allocated for the Function, irrespective of the ST Table location, and Steering Tag values used in Requests are not required to come from the architected ST Table.

A Function that is capable of generating TPH Requests is required to support the No ST Mode of operation. Support for other ST Modes of operation is optional. Only one ST Mode of operation can be selected at a time by programming ST Mode Select.

IMPLEMENTATION NOTE: ST TABLE PROGRAMMING §

To ensure that deterministic Steering Tag values are used in Requests, it is recommended that software either quiesce the Function or disable the TPH Requester capability during the process of performing ST Table updates. Failure to do so may result in non-deterministic values of ST values being used during ST Table updates.

6.17.4 TPH Capability §

TPH capabilities are optional normative. Each Function capable of generating Request TLPs with TPH is required to implement a TPH Requester Extended Capability structure. Functions that support processing of TLPs with TPH as Completers are required to indicate TPH Completer capability via the Device Capabilities 2 register. TPH is architected to be applied for transactions that target Memory Space, and is applicable for transaction flows between device-to-host, device-to-device and host-to-device. In each case for TPH to be supported, the Requester, Completer, and all intermediate routing elements must support the associated TPH capabilities.

Software discovers the Requester capabilities via the TPH Requester Extended Capability structure and Completer capabilities via the Device Capabilities 2 Register (see § Section 7.5.3.15). Software must program the TPH Requester Enable field in the TPH Requester Extended Capability structure to enable the Function to initiate Requests with TPH.

TPH only provides additional information to enable optimized processing of Requests that target Memory Space, so existing mechanisms and rules for managing Memory Space access (e.g., Bus Master Enable, Memory Space Enable, and Base Address registers) are not altered.

6.18 Latency Tolerance Reporting (LTR) Mechanism §

The Latency Tolerance Reporting (LTR) mechanism enables Endpoints to report their service latency requirements for Memory Reads and Writes to the Root Complex, so that power management policies for central platform resources (such as main memory, RC internal interconnects, and snoop resources) can be implemented to consider Endpoint service requirements. The LTR Mechanism does not directly affect Link power management or Switch internal power management, although it is possible that indirect effects will occur.

The implications of “latency tolerance” will vary significantly between different device types and implementations. When implementing this mechanism, it will generally be desirable to consider if service latencies impact functionality or only performance, if performance impacts are linear, and how much it is possible for the device to use buffering and/or other techniques to compensate for latency sensitivities.

The Root Complex is not required to honor the requested service latencies, but is strongly encouraged to provide a worst case service latency that does not exceed the latencies indicated by the LTR mechanism.

LTR support is discovered and enabled through reporting and control registers described in § Chapter 7. . Software must not enable LTR in an Endpoint unless the Root Complex and all intermediate Switches indicate support for LTR. Note that it is not required that all Endpoints support LTR to permit enabling LTR in those Endpoints that do support it. When enabling the LTR mechanism in a hierarchy, devices closest to the Root Port must be enabled first.

If an LTR Message is received at a Downstream Port that does not support LTR or if LTR is not enabled, the Message must be treated as an Unsupported Request.

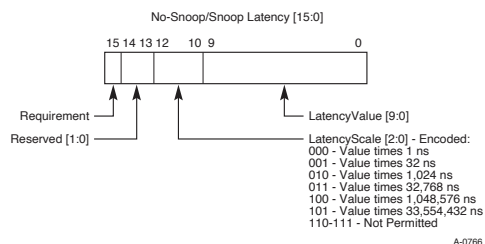


Figure 6-15 Latency Fields Format for LTR Messages §

No-Snoop Latency and Snoop Latency: As shown in § Figure 6-15, these fields include a Requirement bit that indicates if the device has a latency requirement for the given type of Request. If the Requirement bit is Set, the LatencyValue and LatencyScale fields describe the latency requirement. If the Requirement bit is Clear, there is no latency requirement and the LatencyValue and LatencyScale fields are ignored. With any LTR Message transmission, it is permitted for a device to indicate that a requirement is being reported for only no-snoop Requests, for only snoop Requests, or for both types of Requests. It is also permitted for a device to indicate that it has no requirement for either type of traffic, which it does by clearing the Requirement bit in both fields.

Each field also includes value and scale fields that encode the reported latency. Values are multiplied by the indicated scale to yield an absolute time value, expressible in a range from 1 ns to $2^{25} \times (2^{10} - 1) = 34,326,183,936$ ns.

Setting the value and scale fields to all 0's indicates that the device will be impacted by any delay and that the best possible service is requested.

If a device doesn't implement or has no service requirements for a particular type of traffic, then it must have the Requirement bit clear for the associated latency field.

When directed to a non-D0 state by a Write to the PMCSR register, if a device's most recently transmitted LTR message (since the last DL_Down to DL_Up transition) reported one or both latency fields with any Requirement bit set, then it must send a new LTR Message with both of the Requirement bits clear prior to transitioning to the non-D0 state. If subsequently directed back to D0 by a Write to the PMCSR register, it is recommended that the device send a new LTR Message to re-establish its latency requirements after the transition back to D0.

When the LTR Mechanism Enable bit is cleared, if a device's most recently sent LTR Message (since the last DL_Down to DL_Up transition) reported latency tolerance values with any Requirement bit set, then one of the following applies:

- If the bit was cleared due to a Configuration Write to the Device Control 2 Register, the device must send a new LTR Message with all the Requirement bits clear.
- If the bit was cleared due to an FLR, the device *MUST@FLIT* send a new LTR Message with all the Requirement bits clear.

When a Downstream Port goes to DL_Down status, any previous latencies recorded for that Port must be treated as invalid.

An LTR Message from a device reflects the tolerable latency from the perspective of the device, for which the platform must consider the service latency itself, plus the delay added by the use of Clock Power Management (CLKREQ#), if applicable. The service latency itself is defined as follows:

- When a device issues a Non-Posted Request, service latency of that Request is the delay from transmission of the last symbol of the Request TLP to the receipt of the first symbol of the first Completion TLP for that Request¹⁵⁴.
- When a device issues one or more Posted Requests such that it cannot issue another Posted Request due to Flow Control backpressure, service latency of the blocked Request is the delay from the transmission of the last symbol of the previous Posted Request to the receipt of the first symbol of the DLLP returning the credit(s) that allows transmission of the blocked Request¹¹⁸.

If Clock Power Management is used, then the platform implementation-dependent period between when a device asserts CLKREQ# and the device receives a valid clock signal constitutes an additional component of the platform service latency that must be comprehended by the platform when setting platform power management policy.

It is recommended that Endpoints transmit an LTR Message Upstream shortly after LTR is enabled, and subsequently when the Endpoint's service requirements change.

It is strongly recommended that Endpoints send no more than two LTR Messages within any 500 μ s time period, except where required to by the specification. Downstream Ports must not generate an error if more than two LTR Messages are received within a 500 μ s time period, and must properly handle all LTR messages regardless of the time interval between them.

Multi-Function Devices (MFDs) associated with an Upstream Port must transmit a "conglomerated" LTR Message Upstream according to the following rules:

- The acceptable latency values for the Message sent Upstream by the MFD must reflect the lowest values associated with any Function.

154. For this definition, all of the symbols of a DLLP or TLP are included. For 8b/10b, the first and last symbols are framing symbols (SDP, STP or END, see § Section 4.2.1). For 128b/130b, the first symbol of a packet is the first symbol of a framing token (SDP or STP) and the last symbol of a packet is the last symbol of a CRC or LCRC (see § Section 4.2.2).

- It is permitted that the snoop and no-snoop latencies reported in the conglomerated Message are associated with different Functions.
- If none of the Functions report a requirement for a certain type of traffic (snoop/no-snoop), the Message sent by the MFD must not set the Requirement bit corresponding to that type of traffic.
- The MFD must transmit a new LTR Message Upstream when any Function of the MFD changes the values it has reported internally in such a way as to change the conglomerated value earlier reported by the MFD.

Switches must collect the Messages from Downstream Ports that have the LTR mechanism enabled and transmit a “conglomerated” Message Upstream according to the following rules:

- If a Switch supports the LTR feature, it must support the feature on its Upstream Port and all Downstream Ports.
- A Switch Upstream Port is permitted to transmit LTR Messages only when its LTR Mechanism Enable bit is Set or shortly after software clears its LTR Mechanism Enable bit as described earlier in this section.
- The acceptable latency values for the Message sent Upstream by the Switch must be calculated as follows:
 - If none of the Downstream Ports receive an LTR Message containing a requirement for a certain type of traffic (snoop/no-snoop), then any LTR Message sent by the Switch must not set the Requirement bit corresponding to that type of traffic.
 - Define LTRdnport[N] as the value reported in the LTR Message received at Downstream Port N, with these adjustments if applicable:
 - LTRdnport[N] is effectively infinite if the Requirement bit is clear or if a Not Permitted LatencyScale value is used
 - LTRdnport[N] must be 0 if the Requirement bit is 1 and the LatencyValue field is all 0's regardless of the LatencyScale value
 - Define LTRdnportMin as the minimum value of LTRdnport[N] across all Downstream Ports
 - Define Lswitch as all latency induced by the Switch
 - If Lswitch dynamically changes based on the Switch's operational mode, the Switch must not allow Lswitch to exceed 20% of LTRdnportMin, unless Lswitch for the Switch's lowest latency mode is greater, in which case the lowest latency state must be used
 - Calculate the value to transmit upstream, LTRconglomerated, as $\text{LTRdnportMin} - \text{Lswitch}$, unless this value is less than 0 in which case LTRconglomerated is 0
 - If LTRconglomerated is 0, both the LatencyValue and LatencyScale fields must be all 0's in the conglomerated LTR message
- A new LTR message must be transmitted Upstream if the conglomerated latencies are changed as a result of DL_Down invalidating the previous latencies recorded for that Port.
- If a Switch Downstream Port has the LTR Mechanism Enable bit cleared, the Latency Tolerance values recorded for that Port must be treated as invalid, and the latencies to be transmitted Upstream updated and a new conglomerated Message transmitted Upstream if the conglomerated latencies are changed as a result.
- A Switch must transmit an LTR Message Upstream when any Downstream Port Function changes the latencies it has reported in such a way as to change the conglomerated latency reported by the Switch.
- A Switch must not transmit LTR Messages Upstream unless triggered to do so by one of the events described above.

The RC is permitted to delay processing of device Request TLPs provided it satisfies the device's service requirements.

When the latency requirement is updated during a series of Requests, it is required that the updated latency figure be comprehended by the RC prior to servicing a subsequent Request. In all cases the updated latency value must take effect

within a time period equal to or less than the previously reported latency requirement. It is permitted for the RC to comprehend the updated latency figure earlier than this limit.

IMPLEMENTATION NOTE: OPTIMAL USE OF LTR §

It is recommended that Endpoints transmit an updated LTR Message each time the Endpoint's service requirements change. If the latency tolerance is being reduced, it is recommended to transmit the updated LTR Message ahead of the first anticipated Request with the new requirement, allowing the amount of time indicated in the previously issued LTR Message. If the tolerance is being increased, then the update should immediately follow the final Request with the preceding latency tolerance value.

Typically, the Link will be in ASPM L1, and, if Clock Power Management (Clock PM) is supported, CLKREQ# will be deasserted, at the time an Endpoint reaches an internal trigger that causes the Endpoint to initiate Requests to the RC. The following text shows an example of how LTR is applied in such a case. Key time points are illustrated in § Figure 6-16.

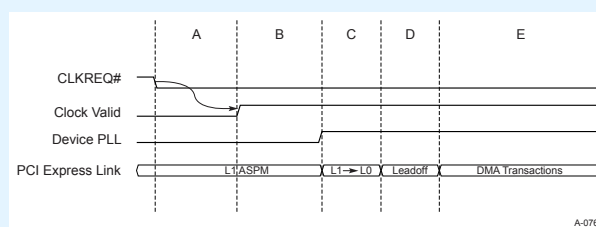


Figure 6-16 CLKREQ# and Clock Power Management §

Time A is a platform implementation-dependent period between when a device asserts CLKREQ# and the device receives a valid clock signal. This value will not exceed the latency in effect.

Time B is the device implementation-dependent period between when a device has a valid clock and it can initiate the retraining sequence to transition from L1 ASPM to L0.

Time C is the period during which the transition from L1 ASPM to L0 takes place

Time D for a Read transaction is the time between the transmission of the END symbol in the Request TLP to the receipt of the STP symbol in the Completion TLP for that Request. Time D for a Write transaction is the time between the transmission of the END symbol of the TLP that exhausts the FC credits to the receipt of the SDP symbol in the DLLP returning more credits for that Request type. This value will not exceed the latency in effect.

Time E is the period where the data path from the Endpoint to system memory is open, and data transactions are not subject to the leadoff latency.

The LTR latency semantic reflects the tolerable latency seen by the device as measured by one or both of the following:

Case 1: the device may or may not support Clock PM, but has not deasserted its CLKREQ# signal - The latency observed by the device is represented in § Figure 6-16 as the sum of times C and D.

Case 2: the device supports Clock PM and has deasserted CLKREQ#- The latency observed by the device is represented as the sum of times A, C, and D.

To effectively use the LTR mechanism in conjunction with Clock PM, the device will know or be able to measure times B and C, so that it knows when to assert CLKREQ#. The actual values of Time A, Time C, and Time D may vary dynamically, and it is the responsibility of the platform to ensure the sum will not exceed the latency.

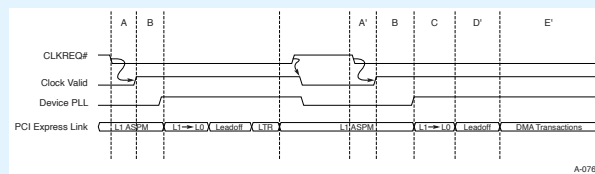


Figure 6-17 Use of LTR and Clock Power Management §

In a very simple model, an Endpoint may choose to implement LTR as shown in § Figure 6-17. When an Endpoint determines that it is idle, it sends an LTR Message with the software configured maximum latency or the maximum latency the Endpoint can support.

When the Endpoint determines that it has a need to maintain sustained data transfers with the Root Complex, the Endpoint sends a new LTR Message with a shorter latency (at Time E). This LTR Message is sent prior to the next data flush by a time equal to the maximum latency sent before (the time between Time E and Time D'). In between Time E and Time A', the Endpoint can return to a low power state, while the platform transitions to a state where it can provide the shorter latency when the device next needs to transmit data.

Note that the RC may delay processing of device Request TLPs, provided it satisfies the device's service requirements. If, for example, an Endpoint connected to Root Port 1 reports a latency tolerance of 100 μ s, and an Endpoint on Root Port 2 report a value of 30 μ s, the RC might implement a policy of stalling an initial Request following an idle period from Root Port 1 for 70 μ s before servicing the Request with a 30 μ s latency, thus providing a perceived service latency to the first Endpoint of 100 μ s. This RC behavior provides the RC the ability to batch together Requests for more efficient servicing.

It is possible that, after it is determined that the RC can service snoop and no-snoop Requests from all Endpoints within the maximum snoop and maximum no-snoop time intervals, this information may be communicated to Endpoints by updating the Max Snoop LatencyValue, Max Snoop LatencyScale and Max No-Snoop LatencyValue, Max No-Snoop LatencyScale fields. The intention of this communication would be to prevent Endpoints from sending needless LTR updates.

When an Endpoint's LTR value for snoop Requests changes to become larger (looser) than the value indicated in the Max Snoop LatencyValue/Scale fields, it is recommended that the Endpoint send an LTR message with the snoop LTR value indicated in the Max Snoop LatencyValue/Scale fields. Likewise, when an Endpoint's LTR value for no-snoop Requests changes to become larger (looser) than the value indicated in the Max No-Snoop LatencyValue/Scale fields, it is recommended that the Endpoint send an LTR message with the no-snoop LTR value indicated in the Max No-Snoop LatencyValue/Scale fields.

It is recommended that Endpoints buffer Requests as much as possible, and then use the full Link bandwidth in bursts that are as long as the Endpoint can practically support, as this will generally lead to the best overall platform power efficiency.

Note that LTR may be enabled in environments where not all Endpoints support LTR, and in such environments, Endpoints that do not support LTR may experience suboptimal service.

6.19 Optimized Buffer Flush/Fill (OBFF) Mechanism §

The Optimized Buffer Flush/Fill (OBFF) Mechanism enables a Root Complex to report to Endpoints (throughout a hierarchy) time windows when the incremental platform power cost for Endpoint bus mastering and/or interrupt activity is relatively low. Typically this will correspond to time that the host CPU(s), memory, and other central resources

associated with the Root Complex are active to service some other activity, for example the operating system timer tick. The nature and determination of such windows is platform/implementation specific.

An OBFF indication is a hint - Functions are still permitted to initiate bus mastering and/or interrupt traffic whenever enabled to do so, although this will not be optimal for platform power and should be avoided as much as possible.

OBFF is indicated using either the WAKE# signal or a message (see § Section 2.2.8.9). The message is to be used exclusively on interconnects where the WAKE# signal is not available. WAKE# signaling of OBFF or CPU Active must only be initiated by a Root Port when the system is in an operational state, which in an ACPI compliant system corresponds to the S0 state. Functions that are in a non-D0 state must not respond to OBFF or CPU Active signaling.

OBFF messages use Message Routing 100b, "Local - Terminate at Receiver" (see § Table 2-20), and are only permitted to be transmitted in the Downstream direction. There are multiple OBFF events distinguished. When using the OBFF Message, the OBFF Code field (see § Figure 2-65 and § Figure 2-66) is used to distinguish between different OBFF cases:

1111b “CPU Active”

System fully active for all Device actions including bus mastering and interrupts

0001b “OBFF”

System memory path available for Device memory read/write bus master activities

0000b “Idle”

System in an idle, low power state

Others

All other codes are Reserved.

These codes correspond to various assertion patterns of WAKE# when using WAKE# signaling, as shown in § Figure 6-18. There is one negative-going transition when signaling OBFF and two negative going transitions each time CPU Active is signaled. The electrical parameters required when using WAKE# are defined in the WAKE# Signaling section of [CEM-2.0] (or later).

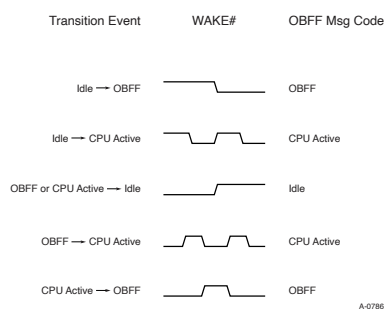


Figure 6-18 Codes and Equivalent WAKE# Patterns §

When an OBFF Message is received that indicates a Reserved code, the Receiver, if OBFF is enabled, must treat the indication as a “CPU Active” indication.

An OBFF Message received at a Port that does not implement OBFF or when OBFF is not enabled must be handled as an Unsupported Request (UR). This is a reported error associated with the receiving Port (see § Section 6.2). If a Port has OBFF enabled using WAKE# signaling, and that Port receives an OBFF Message, the behavior is undefined.

OBFF indications reflect central resource power management state transitions, and are signaled using WAKE# when this is supported by the platform topology, or using a Message when WAKE# is not available. OBFF support is discovered and enabled through reporting and control registers described in § Chapter 7. . Software must not enable OBFF in an Endpoint unless the platform supports delivering OBFF indications to that Endpoint.

When the platform indicates the start of a CPU Active or OBFF window, it is recommended that the platform not return to the Idle state in less than 10 μ s. It is permitted to indicate a return to Idle in advance of actually entering platform idle, but it is strongly recommended that this only be done to prevent late Endpoint activity from causing an immediate exit from the idle state, and that the advance time be as short as possible.

It is recommended that Endpoints not assume CPU Active or OBFF windows will remain open for any particular length of time.

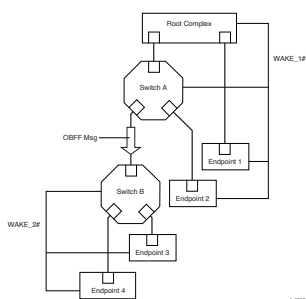


Figure 6-19 Example Platform Topology Showing a Link Where OBFF is Carried by Messages §

§ Figure 6-19 shows an example system where it is necessary for a Switch (A) to translate OBFF indications received using WAKE# into OBFF Messages, which in this case are received by another Switch (B) and translated back to using WAKE# signaling. A HwInit configuration mechanism (set by hardware or firmware) is used to identify cases such as shown in this example (where the link between Switch A and Switch B requires the use of OBFF Messages), and system firmware/software must configure OBFF accordingly.

When a Switch is configured to use OBFF Message signaling at its Upstream Port and WAKE# at one or more Downstream Ports, or vice-versa, when enabled for OBFF, the Switch is required to convert all OBFF indications received at the Upstream Port into the appropriate form at the Downstream Port(s).

When using WAKE#, the enable for any specific Root Port enables the global use of WAKE# unless there are multiple WAKE# signals, in which case only the associated WAKE# signals are affected. When using Message signaling for OBFF, the enable for a particular Root Port enables transmission of OBFF messages from that Root Port only. To ensure OBFF is fully enabled in a platform, all Root Ports indicating OBFF support must be enabled for OBFF. It is permitted for system firmware/software to selectively enable OBFF, but such enabling is beyond the scope of this specification.

To minimize power consumption, system firmware/software is strongly recommended to enable Message signaling of OBFF only when WAKE# signaling is not available for a given link.

OBFF signaling using WAKE# must only be reported as supported by all components connected to a Switch if it is a shared WAKE# signal. In these topologies it is permitted for software to enable OBFF for components connected to the Switch even if the Switch itself does not support OBFF.

It is permitted, although not encouraged, to indicate the same OBFF event more than once in succession.

When a Switch is propagating OBFF indications Downstream, it is strongly encouraged to propagate all OBFF indications. However, especially when using Messages, it may be necessary for the Switch to discard or collapse OBFF indications. It is permitted to discard and replace an earlier indication of a given type when an indication of the same or a different type is received.

Downstream Ports can be configured to transmit OBFF Messages in two ways, which are referred to as Variation A and Variation B. For Variation A, the Port must transmit the OBFF Message if the Link is in the L0 state, but discard the Message when the Link is in the Tx_L0s or L1 state. This variation is preferred when the Downstream Port leads to Devices that are expected to have communication requirements that are not time-critical, and where Devices are expected to signal a non-urgent need for attention by returning the Link state to L0. For Variation B, the Port must

transmit the OBFF Message if the Link is in the L0 state, or, if the Link is in the Tx_L0s or L1 state, it must direct the Link to the L0 state and then transmit the OBFF Message. This variation is preferred when the Downstream Port leads to devices that can benefit from timely notification of the platform state.

When initially configured for OBFF operation, the initial assumed indication must be the CPU Active state, regardless of the logical value of the WAKE# signal, until the first transition is observed.

When enabling Ports for OBFF, it is recommended that all Upstream Ports be enabled before Downstream Ports, and Root Ports must be enabled after all other Ports have been enabled. For hot pluggable Ports this sequence will not generally be possible, and it is permissible to enable OBFF using WAKE# to an unconnected hot pluggable Downstream Port. It is recommended that unconnected hot pluggable Downstream Ports not be enabled for OBFF message transmission.

IMPLEMENTATION NOTE: OBFF CONSIDERATIONS FOR ENDPOINTS §

It is possible that during normal circumstances, events could legally occur that could cause an Endpoint to misinterpret transitions from an Idle window to a CPU Active window or OBFF window. For example, a non-OBFF Endpoint could assert WAKE# as a wakeup mechanism, masking the system's transitions of the signal. This could cause the Endpoint to behave in a manner that would be less than optimal for power or performance reasons, but should not be unrecoverable for the Endpoint or the host system.

In order to allow an Endpoint to maintain the most accurate possible view of the host state, it is recommended that the Endpoint place its internal state tracking logic in the CPU Active state when it receives a request that it determines to be host-initiated, and at any point where the Endpoint has a pending interrupt serviced by host software.

6.20 PASID §

In Non-Flit Mode, the PASID TLP Prefix is an End-End TLP Prefix as defined in § Section 2.2.1 . Layout of the PASID TLP Prefix is shown in § Figure 6-20 and § Table 6-15.

In Flit Mode, the PASID, when present, is included in OHC-A1 or OHC-A4.

When a PASID is present, the PASID value, in conjunction with the Requester ID, identifies the Process Address Space ID associated with the Request. Each Function has a distinct set of PASID values. PASID values used by one Function are unrelated to PASID values used by any other Function.

PASID is permitted only for specific types of TLPs:

- Requests that include an Address with Address Type (AT) of Untranslated or Translated (see § Section 2.2.4.1).
- Address Translation Requests (i.e., MRd with AT=01b), ATS Invalidate Request Messages, Page Request Messages, Address routed messages in Flit Mode, and PRG Response Messages (see § Section 10.1.3).

PASID is not permitted on any other type of TLP.

6.20.1 Managing PASID Usage §

Usage of PASID is permitted only when specifically enabled.

For Endpoint Functions (including Root Complex Integrated Devices), the following rules apply:

- A Function is not permitted to send and receive TLPs with a PASID unless PASID Enable is Set (see § Section 7.8.9.3):
- A Function is not permitted to generate Requests using Translated Addresses and a PASID unless both PASID Enable and Translated Requests with PASID Enable are Set.
- A Function must have a mechanism for associating use of a PASID with a particular Function context. This mechanism is outside the scope of this specification.
- A Function must have a mechanism to request that it gracefully stop using a specific PASID. This mechanism is device specific but must satisfy the following rules:
 - A Function may support a limited number of simultaneous PASID stop requests. Software should defer issuing new stop requests until older stop requests have completed.
 - A stop request in one Function must not affect operation of any other Function.
 - A stop request must not affect operation of any other PASID within the Function.
 - A stop request must not affect operation of transactions that are not associated with a PASID.
 - When the stop request mechanism indicates completion, the Function has:
 - Stopped queuing new Requests for this PASID.
 - Completed all Non-Posted Requests and UIO Requests associated with this PASID.
 - Flushed to the host all Posted Requests addressing host memory in all TCs that were used by the PASID. The mechanism used for this is device specific (for example: a non-relaxed Posted Write to host memory or a processor read of the Function can flush TC0; a zero length read to host memory can flush non-zero TCs).
 - Optionally flushed all Peer-to-Peer Posted Requests to their destination(s). The mechanism used for this is device specific.
 - Complied with additional rules described in Address Translation Services (§ Chapter 10.) if Address Translations or Page Requests were issued on the behalf of this PASID.

For Root Complexes, the following rules apply:

- A Root Complex must have an implementation specific mechanism for indicating support for PASID.
- A Root Complex that supports PASID must have an implementation specific mechanism for enabling them. By default usage of PASID is disabled.
- A Root Complex that supports PASID may optionally have an implementation specific mechanism for enabling them at a finer granularity than the entire Root Complex (e.g., distinct enables for a specific Root Port, Requester ID, Bus Number, Requester ID, or Requester ID/PASID combination).

6.20.2 PASID Information Layout §

In Flit Mode, the PASID information is contained in OHC-A1 or OHC-A4. In Non-Flit Mode, the PASID information is contained in a PASID TLP Prefix.

6.20.2.1 PASID TLP Prefix - Non-Flit Mode §

A TLP may contain at most one PASID TLP Prefix.

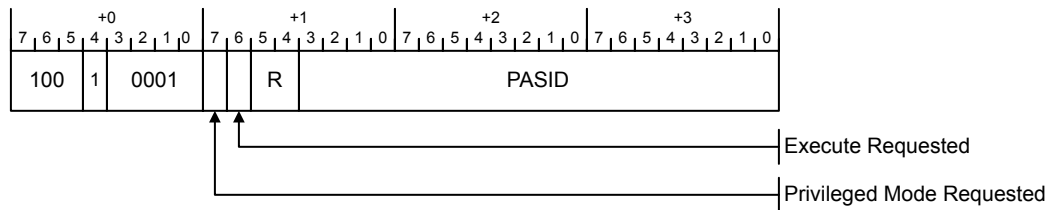


Figure 6-20 PASID TLP Prefix §

Table 6-15 PASID TLP Prefix §

Bits	Description
Byte 0 bits 7:5	100b - indicating TLP Prefix
Byte 0 bit 4	1b - indicating End-End TLP Prefix
Byte 0 bits 3:0	0001b - indicating PASID TLP Prefix
Byte 1 bit 7	Privileged Mode Requested - If Set indicates a Privileged Mode entity in the Endpoint is issuing the Request, If Clear, indicates a Non-Privileged Mode entity in the Endpoint is issuing the Request. Usage of this bit is specified in § Section 6.20.2.4 .
Byte 1 bit 6	Execute Requested - If Set indicates the Endpoint is requesting Execute Permission. If Clear, indicates the Endpoint is not requesting Execute Permission. Usage of this bit is specified in § Section 6.20.2.3 .
Byte 1 bit 3: Byte 3 bit 0	Process Address Space ID (PASID) - This field contains the PASID value associated with the TLP. Usage of this field is defined in § Section 6.20.2.2 .

6.20.2.2 PASID field (Flit Mode and Non-Flit Mode) §

The PASID field identifies the user process associated with a Request.

The PASID field is 20 bits wide. Endpoints and Root Complexes need not support the entire range of the field. For Endpoints, the Max PASID Width field indicates the supported range of PASID values (§ Section 7.8.9.2). For Root Complexes, an implementation specific mechanism is used to provide this information.

Endpoints are not permitted to send TLPs with a PASID unless the PASID Enable bit (§ Section 7.8.9.3) is Set. Endpoints that support PASID must signal Unsupported Request (UR) when they receive a TLP with a PASID and the PASID Enable bit is Clear.

Root Complexes may optionally support TLPs with a PASID. The mechanism used to detect whether a Root Complex supports PASID is implementation specific.

For Endpoints, the following rules apply:

- The Endpoint is not permitted to send TLPs with a PASID value greater than or equal to $2^{\text{Max PASID Width}}$.
- The Endpoint is optionally permitted to signal an error when it receives a Request with a PASID value greater than or equal to $2^{\text{Max PASID Width}}$. This is an Unsupported Request error associated with the Receiving Port (see § Section 6.2).

For Root Complexes, the following rules apply:

- A Root Complex is not permitted to send a TLP with a PASID value greater than it supports.
- A Root Complex is optionally permitted to signal an error when it receives a Request with a PASID value greater than it supports. This is an Unsupported Request error associated with the Receiving Port (see § Section 6.2).

For Completers, the following rules apply:

- For Untranslated Memory Requests, the PASID value and the Untranslated Address are both used in determining the Translated Address used in satisfying the Request. For address translation related TLPs, usage of this field is defined in Address Translation Services (§ Chapter 10.).

IMPLEMENTATION NOTE: PASID WIDTH HOMOGENEITY §

The PASID value is unique per Function and thus the original intent was that the width of the PASID value supported by that Function could be based on the needs of that Function. However, current system software typically does not follow that model and instead uses the same PASID value in all Functions that access a specific address space. To enable this, system software will typically ensure a common system PASID width for Root Complex and persistent translation agents. Such system software will typically disable ATS on any hot plugged Endpoint Functions or translation agents reporting PASID width support which is less than that of the common system PASID width.

The Root Complex, Endpoints, and translation agents, are often implemented independently of system software, therefore it is strongly recommended that hardware implement the maximum width of 20 bits to ensure interoperability with system software.

Endpoints may, in an implementation specific way, be able to map the 20 bit system PASID to an internal representation carrying a smaller width. If this is done, it is critical that the Endpoint do so without impacting system software, which has no mechanism to differentiate such implementation from those that implement the full 20 bit width natively.

6.20.2.3 Execute Requested §

If the Execute Requested bit is Set, the Endpoint is requesting permission for the Endpoint to Execute instructions in the memory range associated with this request. The meaning of Execute permission is outside the scope of this specification.

Endpoints are not permitted to send TLPs with the Execute Requested bit Set unless the Execute Permission Supported bit (§ Section 7.8.9.2) and the Execute Permission Enable bit (§ Section 7.8.9.3) are both Set.

For Root Complexes, the following rules apply:

- Support for Execute Requested by the Root Complex is optional. The mechanism used to determine whether a Root Complex supports Execute Requested is implementation specific.
 - It is strongly recommended that Root Complexes not support Execute Requested.
- A Root Complex that supports the Execute Requested bit must have an implementation specific mechanism to enable it to use the bit. This mechanism must default to disabled.
- A Root Complex that supports the Execute Requested bit is permitted to have an implementation specific mechanism to enable use of the bit at a finer granularity (e.g., for a specific Root Port, for a specific Bus Number, for a specific Requester ID, or for a specific Requester ID/PASID combination), and its default value is implementation specific.

For Completers, the following rules apply:

- Completers have a concept of an effective value of the bit. For a given Request, if the Execute Requested bit is supported and its usage is enabled for the Request, the effective value of the bit is the value in the Request; otherwise the effective value of the bit is 0b.
- For Untranslated Memory Read Requests, Completers use the effective value of the bit as part of the protection check. If this protection check fails, Completers treat the Request as if the memory was not mapped.
- For Memory Requests, other than an Untranslated Memory Read Request, the bit is Reserved.
For address translation related TLPs, usage of this bit is defined in Address Translation Services (§ Chapter 10.).

6.20.2.4 Privileged Mode Requested §

If Privileged Mode Requested is Set, the Endpoint is issuing a Request that targets memory associated with Privileged Mode. If Privileged Mode Requested is Clear, the Endpoint is issuing a Request that targets memory associated with Non-Privileged Mode.

The meaning of Privileged Mode and Non-Privileged Mode and what it means for an Endpoint to be operating in Privileged or Non-Privileged Mode depends on the protection model of the system and is outside the scope of this specification.

Endpoints are not permitted to send a TLP with the Privileged Mode Requested bit Set unless both the Privileged Mode Supported bit (§ Section 7.8.9.2) and the Privileged Mode Enable bit (§ Section 7.8.9.3) are Set.

For Root Complexes, the following rules apply:

- Support for the Privileged Mode Requested bit by the Root Complex is optional. The mechanism used to determine whether a Root Complex supports the Privileged Mode Requested bit is implementation specific.
- A Root Complex that supports the Privileged Mode Requested bit should have an implementation specific mechanism to enable it to use the bit.
- A Root Complex that supports the Privileged Mode Requested bit may have an implementation specific mechanism to enable use of the bit at a finer granularity (e.g., for a specific Root Port, for a specific Bus Number, for a specific Requester ID, or for a specific Requester ID/PASID combination).

For Completers, the following rules apply:

- Completers have the concept of an effective value of the bit. For a given Request, if the Privileged Mode Requested bit is supported and its usage is enabled for the Request, the effective value of the bit is the value in the Request; otherwise the effective value of the bit is the 0b.
- For Untranslated Memory Requests, Completers use the effective value of the bit as part of its protection check. If this protection check fails, Completers treat the Request as if the memory was not mapped.

- For address translation related TLPs, usage of this bit is defined in Address Translation Services (§ Chapter 10).

6.21 Precision Time Measurement (PTM) Mechanism §

6.21.1 Introduction §

Precision Time Measurement (PTM) enables precise coordination of events across multiple components with independent local time clocks. Ordinarily, such precise coordination would be difficult given that individual time clocks have differing notions of the value and rate of change of time. To work around this limitation, PTM enables components to calculate the relationship between their local times and a shared PTM Master Time: an independent time domain associated with a PTM Root.

Enhanced Precision Time Measurement (ePTM) places additional requirements on PTM Devices. Support for ePTM is indicated by the ePTM Capable bit.

PTM defines the following:

- PTM Requester - A Function capable of using PTM as a consumer associated with an Endpoint or an Upstream Port.
- PTM Responder - A Function capable of using PTM to supply PTM Master Time associated with a Port or an RCRB.
- Time Source - A local clock associated with a PTM Responder.
- PTM Root - The source of PTM Master Time for a PTM Hierarchy. A PTM Root must also be a Time Source and is typically also a PTM Responder.

Each PTM Root supplies a single PTM Master Time to all of the PTM Hierarchy: a set of PTM Requesters associated with a single PTM Root.

§ Figure 6-21 illustrates some example system topologies using PTM. These are only illustrative examples, and are not intended to imply any limits or requirements.

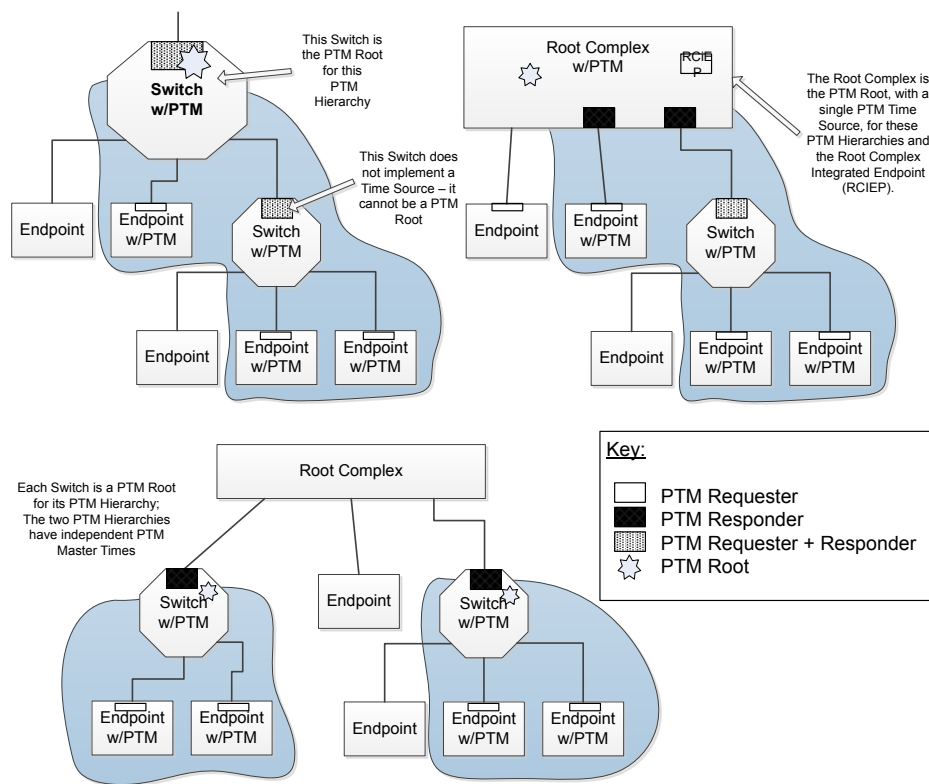


Figure 6-21 Example System Topologies using PTM §

IMPLEMENTATION NOTE:

PTM AND RETIMERS §

PCIe Retimers can impact PTM accuracy by introducing asymmetric link delays. Retimers designed to maintain symmetric link delays will enable the best PTM accuracy. The larger and more variable the asymmetry, the greater the impact to PTM. Consult the manufacturer's documentation to determine the suitability of a Retimer implementation for use with PTM.

6.21.2 PTM Link Protocol §

When using PTM between two components on a Link, the Upstream Port, which acts on behalf of the PTM Requester, sends PTM Requests to the Downstream Port on the same Link, which acts on behalf of the PTM Responder.

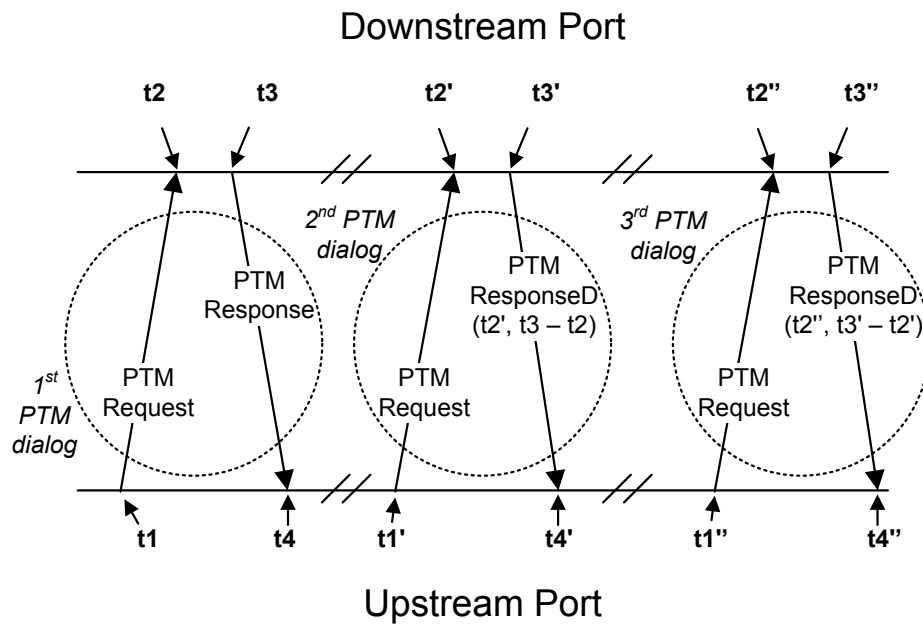


Figure 6-22 Precision Time Measurement Link Protocol §

§ Figure 6-22 illustrates the PTM link protocol. The points t1, t2, t3, and t4 in the above diagram represent timestamps captured locally by each Port as they transmit and receive PTM Messages. The component associated with each Port stores these timestamps from the 1st PTM dialog in internal registers for use in the 2nd PTM dialog, and so on for subsequent PTM dialogs.

The Upstream Port, on behalf of the PTM Requester, initiates the PTM dialog by transmitting a PTM Request message.

The Downstream Port, on behalf of the PTM Responder, has knowledge of or access (directly or indirectly) to the PTM Master Time.

During each dialog, the Downstream Port populates the PTM ResponseD message based on timestamps stored during previous PTM dialogs, as defined in § Section 6.21.3.2 .

Once each component has historical timestamps from the preceding dialog, the component associated with the Upstream Port can combine its timestamps with those passed in the PTM ResponseD message to calculate the PTM Master Time using the following formula:

$$\text{PTM Master Time at } t1' = t2' - \frac{\left((t4 - t1) - (t3 - t2) \right)}{2}$$

Equation 6-2 PTM Master Time §

The values t1, t2, t3, t4, and t2' indicate the timestamps captured during the PTM dialog as illustrated in § Figure 6-22.

PTM capable components would typically record the results of these timestamp calculations, and may make them available to software via implementation specific means. Herein, this document refers to this resultant timing information as the component's "PTM context".

For a Switch implementing PTM, the time synchronization mechanism(s) within the Switch itself are implementation specific.

IMPLEMENTATION NOTE:

PTM THEORY AND OPERATION §

The timestamps captured during the PTM dialogs enable the calculation of the timing relationship between the PTM Requester and PTM Responder. The value $(t_3 - t_2)$ measures the time consumed by the PTM Responder for a given PTM dialog. The time $(t_4 - t_1)$ is the time from request to response. Therefore $((t_4 - t_1) - (t_3 - t_2))$ effectively gives the round trip message transit time between the two components, and that quantity divided by 2 approximates the Link delay - the time difference between t_1 and t_2 . It is assumed that the Link transit times from PTM Requester to PTM Responder and back again are symmetric, which is typically a good assumption (see also the Implementation Note on PTM Timestamp Capture Mechanisms).

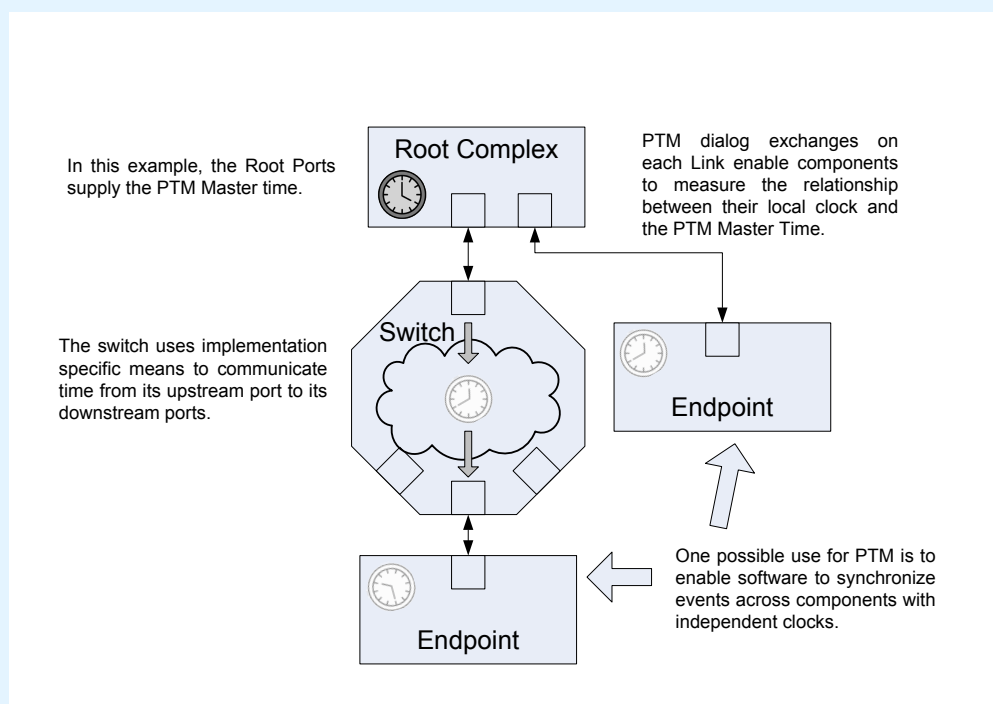


Figure 6-23 Precision Time Measurement Example §

§ Figure 6-23 illustrates a simple device hierarchy employing PTM. Each Upstream Port initiates PTM dialogs to establish the relationship between its local time and the PTM Master Time provided by the Root Port.

In this example, the Switch initiates PTM dialogs on its Upstream Port to obtain the PTM Master Time for use in fulfilling PTM Request Message received at its Downstream Ports. This Switch employs implementation specific means to communicate the PTM Master Time from its Upstream Port to its Downstream Ports.

PTM capable components can make their PTM context available for inspection by software, enabling software to translate timing information between local times and PTM Master Time. In turn, this capability enables software to coordinate events across multiple components with very fine precision.

Similarly, it is strongly recommended that platforms implementing PTM also make the PTM Master Time available to software.

6.21.3 Configuration and Operational Requirements §

Software must not have the PTM Enable bit Set in the PTM Control register on a Function associated with an Upstream Port unless the associated Downstream Port on the Link already has the PTM Enable bit Set in its associated PTM Control register.

PTM support by a Function is indicated by the presence of a PTM Extended Capability structure. It is not required that all Endpoints in a hierarchy support PTM, and it is not required for software to enable PTM in all Endpoints that do support it.

If a PTM Message is received by a Port that does not support PTM, or by a Downstream Port when the PTM Enable bit is clear, the Message must be treated as an Unsupported Request. This is a reported error associated with the Receiving Port (see § Section 6.2). A properly formed PTM Response received by an Upstream Port that supports PTM, but for which the PTM Enable bit is clear, must be silently discarded.

As observed through PTM, the PTM Master Time must satisfy the following behavioral requirements:

- Time values must be monotonic, and strictly increasing.
- The perceived granularity must be no greater than the value reported in the Local Clock Granularity field of the PTM Capability Register.
- The perceived time must start no later than when the PTM Root processes its first PTM Request Message.

Referring to § Figure 6-22, the following rules define timestamp capture:

- A PTM Requester must update its stored t1 timestamp when transmitting a PTM Request Message, even if that transmission is a replay.
- A PTM Responder must update its stored t2 timestamp when receiving a PTM Request Message, even if received TLP is a duplicate.
- A PTM Responder must update its stored t3 timestamp when transmitting a PTM Response or ResponseD Message, even if that transmission is a replay.
- A PTM Requester must update its stored t4 timestamp when receiving a PTM Response Message, even if received TLP is a duplicate.

In NFM, Timestamps must be based on the STP Symbol or Token that frames the TLP, as if observing the first bit of that Symbol or Token at the Port's pins.

In FM, a single Flit is permitted to include zero or one PTM Messages, and Timestamps must be based on the Flit_Marker (see § Section 4.2.3.4.2) that has the PTM Message contained in this Flit bit Set, as if observing the Flit_Marker Indicator bit at the Port's pins. Typically this will require an implementation specific adjustment to compensate for the inability to directly measure the time at the actual pins, as the time will commonly be measured at some internal point in the Rx or Tx path. The accuracy and consistency of this measurement are not bounded by this specification, but it is strongly recommended that the highest practical level of accuracy and consistency be achieved.

As illustrated in § Figure 2-68, the bytes within the Propagation Delay[31:0] field are such that:

- Data Byte 0 contains Propagation Delay [31:24] (most significant byte)
- Data Byte 1 contains Propagation Delay [23:16]
- Data Byte 2 contains Propagation Delay [15:8]
- Data Byte 3 contains Propagation Delay [7:0] (least significant byte)

All implementations compliant to this document are required to follow the above interpretation (Propagation Delay interpretation A). Due to ambiguity in earlier versions of this document, some implementations made this interpretation (Propagation Delay interpretation B):

- Data Byte 0 contains Propagation Delay [7:0] (least significant byte)
- Data Byte 1 contains Propagation Delay [15:8]
- Data Byte 2 contains Propagation Delay [23:16]
- Data Byte 3 contains Propagation Delay [31:24] (most significant byte)

To allow implementations using interpretation A to interoperate with implementations using interpretation B the PTM Propagation Delay Adaptation Capability can be used. For an Upstream Port, this capability applies to the interpretation of received PTM ResponseD Messages, for a Downstream Port, this capability applies to the interpretation of transmitted PTM ResponseD Messages. Support for the PTM Propagation Delay Adaptation Capability is indicated by Setting the PTM Propagation Delay Adaptation Capable bit in the PTM Capability Register. When supported, the Port must use interpretation A, unless the PTM Propagation Delay Adaptation Interpretation B bit in the Link Control Register is Set, in which case the Port changes to interpretation B. For a Switch, if the PTM Propagation Delay Adaptation Capable bit is Set, then all Ports of the Switch must support the PTM Propagation Delay Adaptation Capability, and each Switch Port must be controlled independently by the value of the PTM Propagation Delay Adaptation Interpretation B bit in the Link Control Register for the Port.

It is strongly recommended that software not enable PTM on a link until it has first assured, either by means of the PTM Propagation Delay Adaptation Capability or in an implementation specific manner, that both Ports on the Link are able to compatibly interpret the Propagation Delay value.

6.21.3.1 PTM Requester Role §

- Support for the PTM Requester role is indicated by setting the PTM Requester Capable bit in the PTM Capability Register.
- PTM Requesters are permitted to request PTM Master Time only when PTM is enabled. The mechanism for directing a PTM Requester to issue such a request is implementation specific.
 - Upstream Ports obtain PTM Master Time via PTM dialogs as described in § Section 2.2.8.10 .
 - The mechanism by which RCiEPs request PTM Master Time is implementation specific.
- Once having issued a PTM Request Message, the Upstream Port must not issue another PTM Request Message prior to the receipt of a PTM Response Message, PTM ResponseD Message, Reset, or the passage of 100 μ s without a corresponding PTM Message from the Downstream Port.
- Upon receiving a PTM Response, the Upstream Port must wait at least 1 μ s before issuing another PTM Request Message.
- For Multi-Function Devices (MFDs) containing multiple PTM Requesters, the Upstream Port associated with that MFD must issue a single PTM dialog during each PTM context refresh. PTM Requesters within the MFD maintain their individual PTM contexts using this one, Device-wide PTM dialog. The mechanism for refreshing multiple PTM contexts from one PTM dialog is implementation specific.
- An Upstream Port *MUST@FLIT* invalidate its internal PTM context when any of the following occur. If ePTM is supported, then an Upstream Port must invalidate its internal PTM context when any of the following occur:
 - A PTM Request is replayed.
 - A duplicate PTM ResponseD TLP is received.

- The relationship between PTM Master Time and the Upstream Port's local time changes, as determined by implementation specific criteria. For example, this may occur as a result of a transition to a non-D0 state or due to accumulated PPM drift.

These events are grouped under the label “Local Time Invalidation Event” in § Figure 6-24.

- If ePTM is supported, an Upstream Port, upon replaying a PTM TLP, must invalidate its PTM context until two successive PTM dialogs have been completed successfully and without replays.

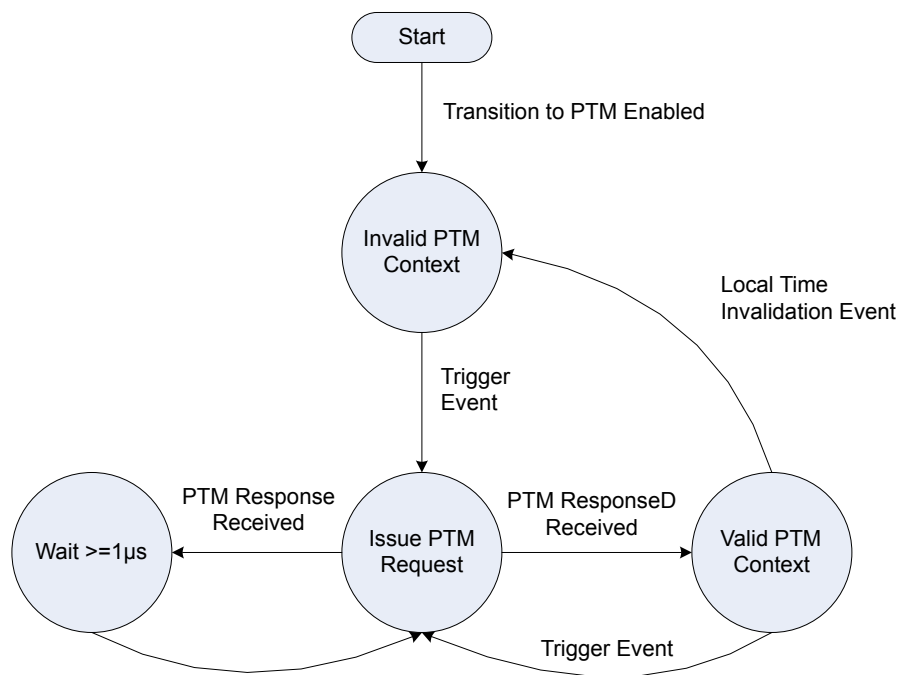


Figure 6-24 PTM Requester Operation §

IMPLEMENTATION NOTE:

PTM INVALIDATION ON THE RECEPTION OF DUPLICATE TLPs §

Duplicate TLPs are detected and discarded in the Data Link Layer, whereas PTM messages are identified in the Transaction Layer. In some implementations it may be difficult or excessively complicated to distinguish a duplicate PTM TLP from other duplicate TLPs.

Because Upstream Ports are permitted to invalidate their internal PTM context for implementation specific criteria, a PTM Requester is allowed to invalidate its internal PTM context upon the reception of any duplicate TLP in addition to any duplicate PTM TLP. Similarly, if ePTM is supported, then a PTM Responder is allowed to invalidate its historical timestamps (t2 - t3) upon the reception of any duplicate TLP.

6.21.3.2 PTM Responder Role §

- Support for the PTM Responder role is indicated by setting the PTM Responder Capable bit in the PTM Capability register.
- Switches and Root Complexes are permitted to implement the PTM Responder Role.
 - A PTM capable Switch, when enabled for PTM by setting the PTM Enable bit in the PTM Control register associated with the Switch Upstream Port, must respond to all PTM Request Messages received at any of its Downstream Ports.
 - The mechanism by which Root Complexes communicate PTM Master Time to RCIEPs is implementation specific.
- PTM Responders must populate PTM ResponseD Messages as follows (refer to § Figure 6-22 and the accompanying implementation note):
 - The PTM Master Time field is a 64-bit value containing the value of PTM Master Time at the receipt of the PTM Request Message for the current PTM Dialog. In § Figure 6-22, for the 2nd PTM dialog, this is the PTM Master Time at time t2'.
 - The Propagation Delay field is a 32-bit value containing the interval between the receipt of the PTM Request Message and the transmission of the PTM Response Message for the previous PTM dialog. In § Figure 6-22, for the 2nd PTM dialog, this is the time interval between t2 and t3 captured during the 1st PTM dialog.
 - The unit of measurement for both fields is one ns.
 - A PTM Responder with multiple Downstream Ports must populate all PTM ResponseD Messages with values from a single PTM Root across all its Downstream Ports.
- Switch Downstream Ports and Root Ports acting as PTM Responders must respond to each PTM Request Message received at their Downstream Ports with either PTM Response or PTM ResponseD according to the following rules:
 - A PTM Responder must not send a PTM Response or PTM ResponseD Message without first receiving a PTM Request Message.
 - Upon receipt of a PTM Request Message, a PTM Responder must attempt to issue a PTM Response or PTM ResponseD Message within 10 μ s.
 - A PTM Responder must issue PTM Response when the Downstream Port does not have valid historical timestamps (t3 - t2) with which to fulfill a PTM Request Message.
 - If ePTM is supported, a PTM Responder must invalidate its historical timestamps (t3 - t2) immediately upon replaying any PTM Response or PTM ResponseD.
 - If ePTM is supported a PTM Responder must, and if ePTM is not supported a PTM Responder is recommended to, invalidate its historical timestamps (t3 - t2) after receiving any duplicate PTM Request.
- A PTM Responder must issue PTM ResponseD when it has stored copies of the values required to populate the PTM ResponseD Message: historical timestamps (t3 - t2) and the PTM Master Time at the receipt of the most recent PTM Request Message (time t2').
- A PTM Responder is permitted to issue PTM Response when it has stored copies of the historical timestamps (t3 - t2) but must request the PTM Master Time from elsewhere. In this case, it is permitted to issue PTM Response messages in response to PTM Request Messages while it retrieves the PTM Master Time if that retrieval is expected to take more than 10 μ s.

- The perceived granularity of the historical timestamps and PTM Master Time values transmitted by a PTM Responder must not exceed that reported in the Local Clock Granularity field of the PTM Capability Register.

6.21.3.3 PTM Time Source Role - Rules Specific to Switches §

In addition to the requirements listed above for the PTM Requester and PTM Responder Roles, Switches must follow these requirements:

- When the Upstream Port is associated with a Multi-Function Device, only a single Function associated with that Upstream Port is permitted to implement the PTM Extended Capability structure. For Switches, all PTM functionality associated with the Switch must be controlled through that structure. It is not required that the Function implementing the PTM Extended Capability structure be the Switch Upstream Port Function.
- The PTM Extended Capability structure for a Switch must indicate support for both the PTM Requester and PTM Responder roles.
- The PTM Extended Capability in the Upstream Port controls all Switches in that Upstream Port.
- A Switch is permitted to act as a PTM Root, or to issue PTM Requests on its Upstream Port to obtain the PTM Master Time for use in fulfilling PTM Requests received at its Downstream Ports. In the latter case the Switch must account for any internal delays within the Switch.
- A Switch is permitted to maintain a local PTM context for use in fulfilling PTM Requests received on its Downstream Ports.
- A Switch which is not acting as a PTM Root must invalidate its local context no more than 10 ms from the last PTM dialog on its Upstream Port. The Switch must then refresh its local PTM context prior to issuing further PTM ResponseD Messages on its Downstream Ports. This requirement for periodic refreshes is optional if it is guaranteed by implementation specific means that the Switch's local clock is phase locked with PTM Master Time.
- Any Switch implementing a local clock for the purpose of maintaining a local PTM context must report the granularity of this clock as defined in the PTM Capabilities structure (§ Section 7.9.15).

IMPLEMENTATION NOTE:

PTM TIMESTAMP CAPTURE MECHANISMS §

PTM uses services from both the Data Link and Transaction Layers. Accuracy requires that time measurements be taken as close to the Physical Layer as possible. Conversely, the messaging protocol itself properly belongs to the Transaction Layer. The PTM message protocol applies to a single Link, where the Upstream Port is the requester and the Downstream Port is the responder.

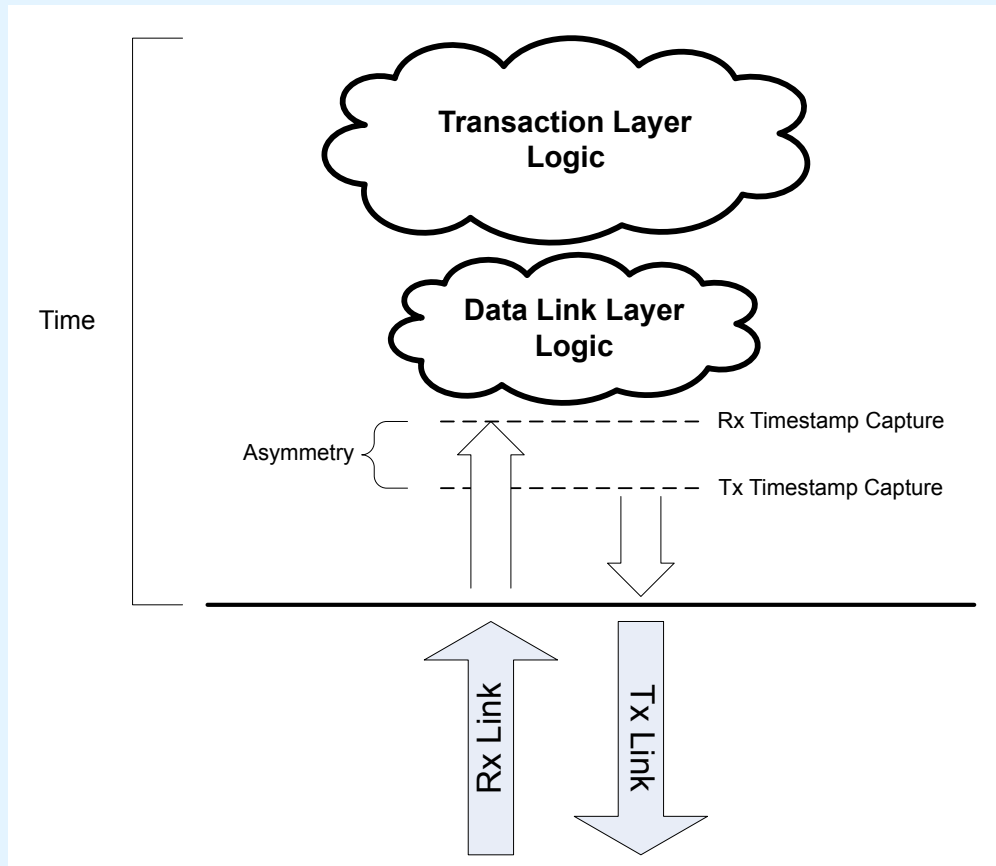


Figure 6-25 PTM Timestamp Capture Example §

§ Figure 6-25 illustrates how to select suitable timestamp capture points. For some implementations, the logic within the Transaction Layer and Data Link Layers is non-deterministic. Implementation details and current conditions have considerable impact on exactly when a particular packet may encounter any particular processing step. This makes it effectively impossible to capture any timestamp that accurately records the time of a particular physical event if timestamps are captured in the higher layers.

6.22 Readiness Notifications (RN) §

Readiness Notifications (RN) is intended to reduce the time software needs to wait before issuing Configuration Requests to a Device or Function following DRS Events or FRS Events. RN includes both the Device Readiness Status (DRS) and

Function Readiness Status (FRS) mechanisms. These mechanisms provide a direct indication of Configuration-Readiness (see Terms and Acronyms entry for Configuration-Ready). When used, DRS and FRS allow an improved behaviour over the Configuration RRS mechanism, and eliminate its associated periodic polling time of up to 1 second following a reset. With appropriate system support, the DRS mechanism can be used to support Devices that require more than 1 second to become Configuration-Ready (see HARDWARE/SOFTWARE RECOMMENDATIONS FOR OPTIMIZING CONFIGURATION READINESS below).

It is permitted that system software/firmware provide mechanisms that supersede the FRS and/or DRS mechanisms, however such software/firmware mechanisms are outside the scope of this specification.

IMPLEMENTATION NOTE: HARDWARE/SOFTWARE RECOMMENDATIONS FOR OPTIMIZING CONFIGURATION READINESS §

It is strongly recommended that implementers of System Software avoid unnecessary delays wherever possible. It is strongly recommended that hardware be designed to eliminate or minimize required delays, and utilize mechanisms provided in this and related specifications to communicate any required delays. Hardware implementers should document hardware behavior sufficiently to enable System Software to implement optimal behaviors.

Even before a Link is in L0, it is possible for System Software to determine if the DRS mechanism can be used to determine when a device becomes Configuration Ready. System Software may do this as follows:

1. If the DRS Supported bit in the Downstream Port above the device is Clear, stop this procedure and follow the procedure in § Section 2.3.1.
2. Check the Downstream Component Presence field in the Downstream Port.
 - If Downstream Component Presence equals Link Up – Component Present and DRS Received, the device is already Configuration Ready.
 - If System Software is informed, through implementation specific mechanisms, that the device supports DRS, continue with Step 3. System Software is strongly recommended to take advantage of such knowledge when available.
 - If Downstream Component Presence equals Link Down – Flit Mode Negotiation Completed or Link Up Component Present:
 - If Flit Mode Status is Set, assume the device also supports DRS since DRS is mandatory if a component supports Flit Mode. Continue with Step 3.
 - Otherwise, continue with Step 7.
 - If Otherwise, if the Link is Up, continue with Step 7.

When System Software determines that DRS is supported by both the Downstream Port and the device below it, the following DRS-Only procedure is strongly recommended:

3. The timeout based procedure defined in § Section 2.3.1 is not used. That procedure supports devices that can take a maximum of 1 second to become Configuration Ready. This DRS-Only procedure supports devices that take longer than 1 second to become Configuration Ready.
4. Either DRS Message Received or Downstream Component Presence are used to determine when the device is or becomes Configuration Ready.
5. The Downstream Component Presence field is used to avoid polling when a component is not present.
6. A non-blocking polling capability is implemented so that unrelated operations are not stalled. Software may configure DRS Signaling Control to generate an interrupt or FRS Message to indicate when polling should occur. It may be desirable to implement a timeout mechanism to terminate polling and indicate an error condition if the timeout expires. The timeout should be determined by system use case requirements. If none applies and a component is known to be present, a value of 10 seconds is recommended.

When System Software cannot determine that DRS is supported by both the Downstream Port and the device below it, then this hybrid approach for determining Configuration Ready is recommended:

7. The timeout based procedure defined in § [Section 2.3.1](#) is run in parallel with the DRS-Only procedure in [Step 4](#) through [Step 6](#).
8. The receipt of a DRS Message indicates the Device is Configuration Ready, and System Software proceeds without further delay to configure the device.
9. If no DRS Message is received within an appropriate period determined by system use case requirements, then System Software may issue a Configuration Request to the Device per the procedure described in § [Section 2.3.1](#).

6.22.1 Device Readiness Status (DRS) §

DRS *MUST@FLIT* be implemented, and the DRS Supported bit in the Link Capabilities 2 register *MUST@FLIT* be Set in all Downstream Ports and in Function 0 of all Upstream Ports. DRS is optional for Ports that do not support Flit Mode.

DRS must be used to indicate when a Device is Configuration-Ready following any of the following Device-level occurrences, which are subsequently referred to as “DRS Events”:

- Exit from [Cold Reset](#)
- Exit from [Warm Reset](#), [Hot Reset](#), [Loopback](#), or [Disabled](#)
- Exit from [L2/L3 Ready](#)
- Any other scenario where the Port transitions from [DL_Down](#) to [DL_Up](#) status.

The DRS Message protocol requirements include the following:

- There is no enable or disable mechanism for DRS.
- It is expressly permitted for Upstream Ports to send DRS Messages even when the [DRS Supported](#) bit is Clear.
- A DRS Message must be transmitted by a DRS-capable Upstream Port following every [DL_Down](#) to [DL_Up](#) transition when all non-VF Functions on the Logical Bus associated with that Upstream Port become ready.
 - A [Type 0 Function](#) is ready when it is [Configuration-Ready](#).
 - A [Type 1 Function](#) that is a Switch Upstream Port is ready when it is [Configuration-Ready](#) and all Functions on its secondary bus are [Configuration-Ready](#).
 - A [Type 1 Function](#) that is not a Switch Upstream Port is ready when the Function itself is [Configuration-Ready](#).
- After a Device transmits a DRS Message, non-VF Functions indicated as Configuration-Ready by that DRS Message must not return Completions with RRS in response to Configuration Requests unless a subsequent DRS Event occurs.

Additional requirements relating to Switches implementing DRS include:

- Must support DRS functionality in all Ports
- Implementation at each Downstream Port of the [DRS Signaling Control](#) field.
- For any physically-integrated Device that appears beneath a Switch Downstream Port, the DRS sent by the Switch does not indicate Configuration Readiness for that Device
 - For such a Device, implementation and use of DRS is independent of the Switch

Additional requirements for Root Ports and Switch Downstream Ports include:

Implementation of the DRS Message Received bit, which indicates receipt of a DRS Message

IMPLEMENTATION NOTE: DRS MESSAGES AND ACS SOURCE VALIDATION §

Functions are permitted to transmit DRS Messages before they have been assigned a Bus Number. Such messages will have a Requester ID with a Bus Number of 00h. If the Downstream Port has ACS Source Validation enabled, these Messages (see § Section 6.12.1.1) will likely be detected as an ACS Violation error.

6.22.2 Function Readiness Status (FRS) §

When implemented, FRS must be used to indicate a specific Function as being Configuration-Ready following any of the following Function-level occurrences, which are subsequently referred to as “FRS Events”:

- Function Level Reset (FLR)
- Completion of D3_{Hot} to D0 transition
- Setting or Clearing of VF Enable in a PF (SR-IOV)

The FRS Message protocol requirements include the following:

- The Requester ID of the FRS Message must indicate the Function that has changed readiness status (see § Section 2.2.8.6.3)
- The FRS Reason field in the FRS Message must indicate why that Function changed readiness status
- After a Function transmits an FRS Message, the indicated Function(s) must not return Completions with RRS in response to a Configuration Request unless a subsequent DRS Event or FRS Event occurs

Additional requirements for Switches implementing FRS include:

- Must support FRS functionality in the Upstream Port and all Downstream Ports
- The ability to transmit FRS Messages Upstream when required by the FRS protocol

Additional requirements for Physical Functions (PFs) include:

- The ability to transmit FRS Message Upstream when the VF Enable or VF Disable process completes

Additional requirements for Root Ports and Root Complex Event Collectors implementing FRS include:

- Must implement the FRS Queuing Extended Capability (see § Section 7.8.10)

6.22.3 FRS Queuing §

Root Ports and Root Complex Event Collectors that support FRS must implement the FRS Queuing Extended Capability (see § Section 7.8.10).

For a Root Port, the FRS Message Queue contains FRS Messages received by the Root Port or generated by the Root Port.

For a Root Complex Event Collector, the FRS Message Queue contains FRS Messages generated by RCiEPs associated with the Root Complex Event Collector (see § Section 7.9.10) or generated by the Root Complex Event Collector.

The FRS Message Queue must satisfy the following requirements:

- The FRS Message Queue must be empty following Reset.
- For a Root Port, the FRS Message Queue must be emptied when the Link goes to DL_Down.
- FRS Messages must be queued in the order received.
- If the FRS Message Queue is not full at the time an FRS Message is received or is internally generated, that FRS Message must be entered in the queue and the FRS Message Received bit must set to 1b.
- If the FRS Message Queue is full at the time an FRS Message is received or is internally generated, that FRS Message must be discarded and the FRS Message Overflow bit must be set to 1b. The pre-existing FRS Message Queue must be preserved.
- The oldest FRS Message must be visible in the FRS Message Queue register (see § Section 7.8.10.4).
- Writing the FRS Message Queue register must remove the oldest element from the queue.
- When either FRS Message Received or FRS Message Overflow transitions from 0b to 1b, an interrupt must be generated if enabled.

6.23 Enhanced Allocation §

The Enhanced Allocation (EA) Capability is an optional Capability that allows the allocation of I/O, Memory and Bus Number resources in ways not possible with the BAR and Base/Limit mechanisms in the Type 0 and Type 1 Configuration Headers.

It is only permitted to apply EA to certain functions, based on the hierarchical structure of the functions as seen in PCI configuration space, and based on certain aspects of how functions exist within a platform environment (see § Figure 6-26).

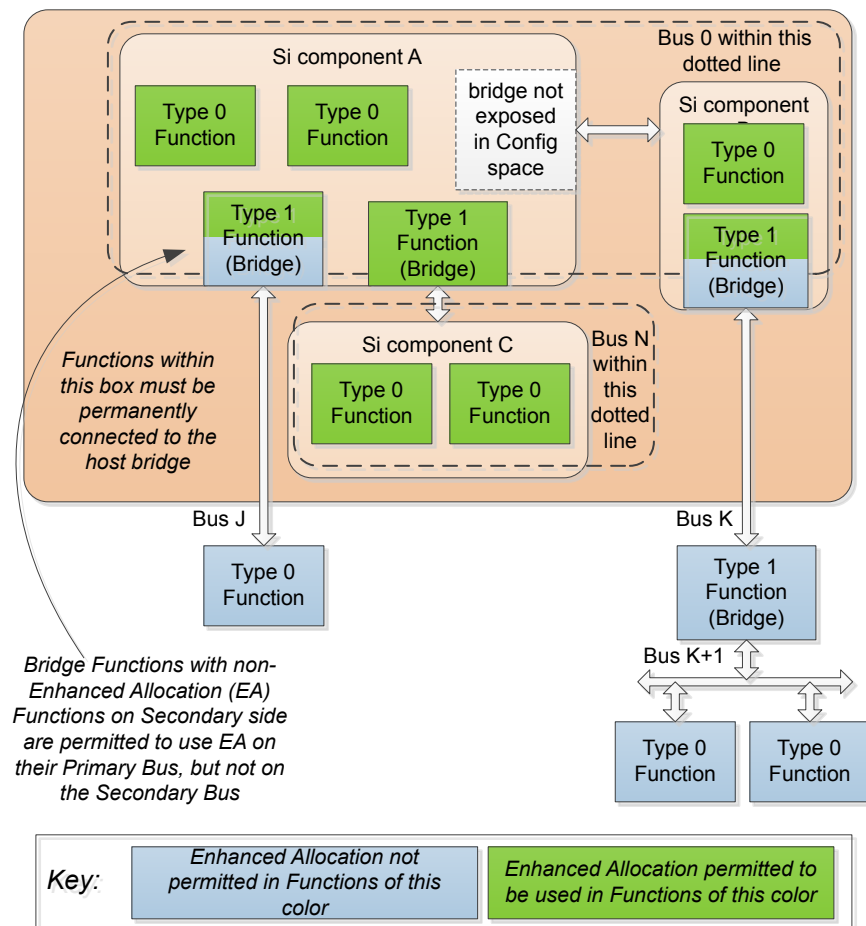


Figure 6-26 Example Illustrating Application of Enhanced Allocation §

Only functions that are permanently connected to the host bridge are permitted to use EA. A bridge function (i.e., any function with a Type 1 Configuration Header), is permitted to use EA for both its Primary Side and Secondary Side if and only if the function(s) behind the bridge are also permanently connected (below one or more bridges) to the host bridge, as shown for “Si component C” in § Figure 6-26.

A bridge function is permitted to use EA only for its Primary Side if the function(s) behind the bridge are not permanently connected to the bridge, as with the bridges above Bus J and Bus K in § Figure 6-26, and in this case the non-EA resource allocation mechanisms in the Type 1 Header for Bus numbers, MMIO ranges and I/O ranges are used for the Secondary side of the bridge. System software must ensure that the allocated Bus numbers are within the range indicated in the Fixed Secondary Bus Number and Fixed Subordinate Bus Number registers of the EA capability. System software must ensure that the allocated MMIO and I/O ranges are within ranges indicated with the corresponding Properties in the EA capability for resources to be allocated behind the bridge. For Bus numbers, MMIO and I/O ranges behind the bridge, hardware is permitted to indicate overlapping ranges in multiple bridge functions, however, in such cases, system software must ensure that the ranges actually assigned are non-overlapping.

Functions that rely exclusively on EA for I/O and Memory address allocation must hardwire all bits of all BARs in the PCI Header to 0. Such Functions must be clearly documented as relying on EA for correct operation, and platform integrators must ensure that only EA-aware firmware/software are used with such Functions.

When a Function allocates resources using EA and indicates that a resource range is associated with an equivalent BAR number, the Function must not request resources through the equivalent BAR, which must be indicated by hardwiring all bits of the equivalent BAR to 0.

For a bridge function that is permitted to implement EA based on the rules above, it is permitted, but not required, for the bridge function to use EA mechanisms to indicate resource ranges that are located behind the bridge Function. In the example shown in § Figure 6-26, the bridge above Bus N is permitted to use EA mechanisms to indicate the resources used by the two functions in “Si component C”, or that bridge is permitted to not indicate the resources used by the two functions in “Si component C”. System firmware/software must comprehend that such bridge functions are not required to indicate inclusively all resources behind the bridge, and as a result system firmware/software must make a complete search of all functions behind the bridge to comprehend the resources used by those functions.

A Function with an Expansion ROM is permitted to use the existing mechanism or the EA mechanism, but is not permitted to support both. If a Function uses the EA mechanism (EA entry with BEI of 8), the Expansion ROM Base Address and Expansion ROM Enable fields must be hardwired to 0 (see § Section 7.5.1.2.4). The Enable bit of the EA entry is equivalent to the Expansion ROM Enable bit. If a Function uses Expansion ROM Base Address Register mechanism, no EA entry with a BEI of 8 is permitted. In both cases, Expansion ROM Validation, if supported, uses the Expansion ROM Validation Status and Expansion ROM Validation Details fields (see § Section 7.5.1.2.4).

The requirements for enabling and/or disabling the decode of I/O and/or Memory ranges are unchanged by EA, including but not limited to the Memory Space and I/O Space enable bits in the Command register.

Any resource allocated using EA must not overlap with any other resource allocated using EA, except as permitted above for identifying permitted address ranges for resources behind a bridge.

6.24 Emergency Power Reduction State §

Emergency Power Reduction State is an optional mechanism to request that Functions quickly reduce their power consumption. Emergency Power Reduction is a fail-safe mechanism intended to be used to prevent system damage and is not intended to provide normal dynamic power management.

If a Function implements Emergency Power Reduction State, it must also implement the Power Budgeting extended capability and must report Power Budgeting values for this state (see § Section 7.8.1). Devices that are integrated on the system board are not required to implement the Power Budgeting extended capability, but if they do so, they must meet the preceding requirement.

Functions enter and exit this state either autonomously or based on external requests. External requests may be either following a signaling protocol defined in an applicable form factor specification, or by a vendor-specific method. § Table 6-16 defines how the Emergency Power Reduction Supported and Emergency Power Reduction Initialization Required fields determine the mechanisms that are allowed to trigger entry and exit from this state (see § Section 7.5.3.15).

Table 6-16 Emergency Power Reduction Supported Values §

Emergency Power Reduction Supported	Emergency Power Reduction Initialization Required	Entry/Exit Permitted by		
		Form Factor Mechanism	Vendor Specific Mechanism(s)	Autonomous Mechanisms
00b	0	No	Yes	Yes
	1	No	No	No
01b	Any	No	Yes	Yes
10b	Any	Yes	Yes	Yes
11b	Reserved			

Functions may indicate that they require re-initialization on exit from this state:

- If the Emergency Power Reduction Initialization Required bit is Clear (see § Section 7.5.3.15):
 - On entry to this state, the Function either operates normally (perhaps with reduced performance), or enters a device specific “power reduction dormant state”. The Upstream Port of the Device remains operating. Outstanding requests initiated by or directed to the Function must complete normally.
 - On exit from this state, the Function operates normally (perhaps resuming normal performance). Functions that entered a “power reduction dormant state” exit that state. In either case, no software intervention is required.
- If the Emergency Power Reduction Initialization Required bit is Set (see § Section 7.5.3.15):
 - On entry to this state, the Function ceases normal operation. The Upstream Port of the associated Device is permitted to enter DL_Down.
 - If the Upstream Port remains in DL_Up, outstanding requests directed to or initiated by the Function must complete normally.
 - If the Upstream Port enters DL_Down, outstanding request behavior is defined in § Section 2.9.1 . This transition may result in a Surprise Down error.
 - Sticky bits must be preserved in this state.
 - On exit from this state, software intervention is required to resume normal operation. The mechanism used to indicate to software when this is required is outside the scope of this specification (e.g., a device specific interrupt). If the Upstream Port entered DL_Down, all Functions of the Device are reset and a full reconfiguration is required (see § Section 2.9.2).

The following rules apply to the Emergency Power Reduction State:

- A Device supports Emergency Power Reduction State if at least one Function in the Upstream Port indicates support (i.e., Emergency Power Reduction Supported is non-zero).
- Emergency Power Reduction State is associated with a Device. All Functions in a Device that support it enter and exit this state at the same time.
- For SR-IOV Devices, if the Emergency Power Reduction Supported field in a VF is non-zero, that VF enters and exits the Emergency Power Reduction State at the same time as its associated PF. For such VFs, the Emergency Power Reduction Detected bit must be hardwired to Zero, but software can use the associated PF's bit to emulate the bit in its VFs.
- Functions where the Emergency Power Reduction Supported field is 00b are not affected by the Emergency Power Reduction State of the Device as long as the Upstream Port remains in DL_Up. The Emergency Power Reduction Detected bit is RsvdZ.
- Functions where the Emergency Power Reduction Supported field is 01b or 10b:
 - Set the Emergency Power Reduction Detected bit when the Device enters Emergency Power Reduction State.
 - Clear the Emergency Power Reduction Detected bit when requested if the Device has exited the Emergency Power Reduction State.
- For Switches, Downstream Switch Ports enter and exit Emergency Power Reduction State at the same time as the associated Upstream Switch Port. The corresponding fields in Configuration Space are reserved for Downstream Switch Ports.
- For SR-IOV Devices, VFs enter and exit Emergency Power Reduction State at the same time as their PF. The corresponding fields in Configuration Space are reserved for VFs.
- Encoding 10b shall not be used unless the associated form factor specification defines a mechanism for requesting Emergency Power Reduction.

- It is strongly recommended that the Emergency Power Reduction Supported field be initialized by hardware or firmware within the Function prior to initial device enumeration. This initialization is permitted to be deferred to device driver load when this is not practical (e.g., when there is no firmware ROM).

IMPLEMENTATION NOTE: DIAGNOSTIC CHECKING OF EMERGENCY POWER REDUCTION DETECTED §

The Emergency Power Reduction Detected bit permits system software to detect that Emergency Power Reduction State was entered, even momentarily. The Emergency Power Reduction Request bit can be used by software to request entry. Normally, software would use a system specific method to enter the Emergency Power Reduction State using external mechanisms.

IMPLEMENTATION NOTE: EMERGENCY POWER REDUCTION STATE: EXAMPLE ADD-IN CARD §

§ Figure 6-27 shows an example multi-Device add-in card supporting Emergency Power Reduction. Note that Device C does not support the Emergency Power Reduction State. Device C might be a Switch that fans out to Devices A and B.

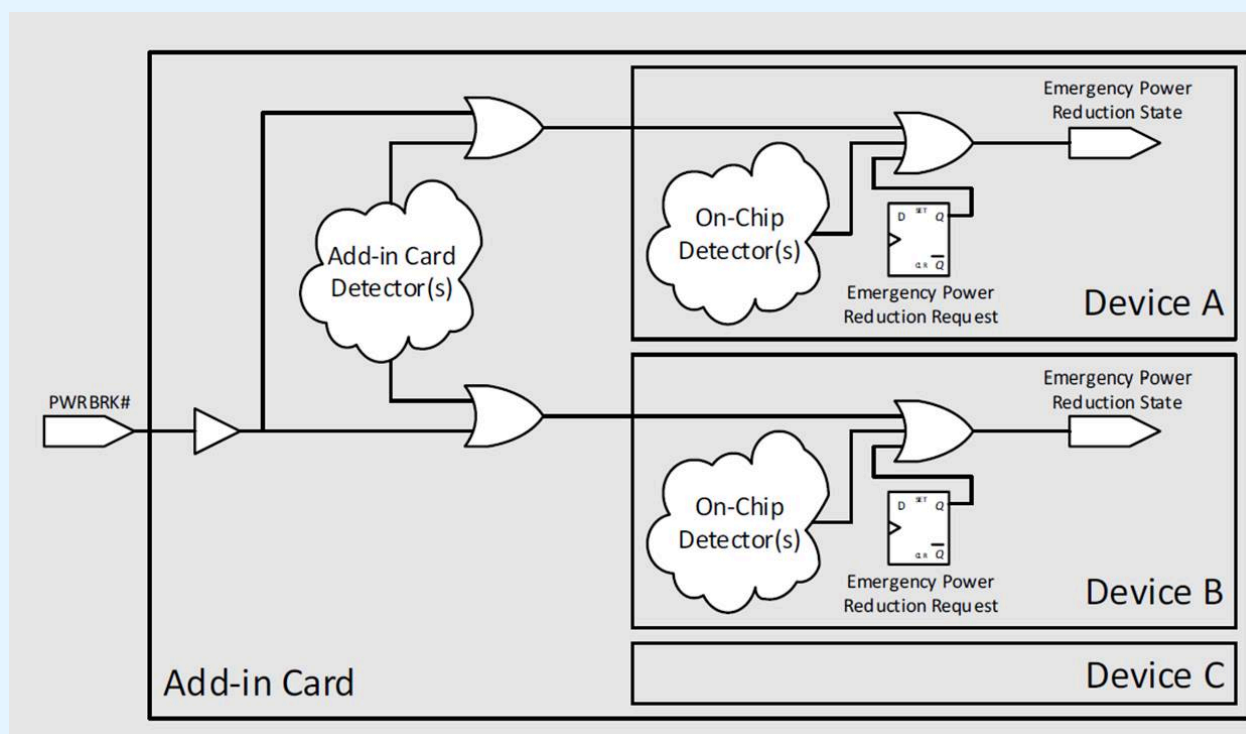


Figure 6-27 Emergency Power Reduction State: Example Add-in Card §

6.25 Hierarchy ID Message §

When software initializes a PCI Hierarchy, it assigns unique Bus and Device numbers to each component so that every Function in the Hierarchy has a unique Routing ID within that Hierarchy. To ensure that Routing IDs are unique in large systems that contain more than one Hierarchy and in clustered systems that contain multiple Hierarchies, additional information is required to augment the Routing ID to produce a unique number. Functions can be uniquely identified by the combination of:

- Unique Identifier for the System (or Root Complex)
- Unique Identifier for the Hierarchy within that Root Complex
- Routing ID within that Hierarchy

The Hierarchy ID Message (see § Section 2.2.8.6.4) is used to provide the additional information needed for a Function to uniquely identify itself in a multi-hierarchy platform.

Hierarchy ID Messages are generated by a Downstream Port upon software request. Received messages at an Upstream Port are reported in the Hierarchy ID Extended Capability (see § Section 7.9.17).

Hierarchy ID Messages are a PCI-SIG-Defined Type 1 VDM. Hierarchy ID Messages can safely be sent at any time and components that do not comprehend them will silently ignore them.

Hierarchy ID Messages typically are sent from a Downstream Port at the top of the Hierarchy (e.g., a Root Port). In systems where the Root Port does not support Hierarchy ID Messages, Hierarchy ID Messages can be sent from Switch Downstream Ports.

The Hierarchy ID Message is intended for use by software, firmware, and/or hardware. When using the Hierarchy ID Message, all bits of the Hierarchy ID, System GUID, System GUID Authority ID fields must be compared, without regard to any internal structure. How this information is used is outside the scope of this specification.

Layout of the Hierarchy ID Message is shown in § Figure 2-61. Fields in the Hierarchy ID Message are as follows:

Hierarchy ID contains the Segment Group Number associated with this Hierarchy (as defined by the *PCI Firmware Specification*). This field can be used in conjunction with the Routing ID to uniquely identify a Function within a System. The value 0000h indicates the default (or only) Hierarchy of the Root Complex. Non-zero values indicate additional Hierarchies.

System GUID[143:0], in conjunction with System GUID Authority ID, provides a globally unique identification for a System.

System GUID[143:136] is byte 14 in the Hierarchy ID Message.
 System GUID[135:128] is byte 15 in the Hierarchy ID Message.
 System GUID[127:120] is byte 16 in the Hierarchy ID Message.
 System GUID[119:112] is byte 17 in the Hierarchy ID Message.
 System GUID[111:104] is byte 18 in the Hierarchy ID Message.
 System GUID[103:96] is byte 19 in the Hierarchy ID Message.
 System GUID[95:88] is byte 20 in the Hierarchy ID Message.
 System GUID[87:80] is byte 21 in the Hierarchy ID Message.
 System GUID[79:72] is byte 22 in the Hierarchy ID Message.
 System GUID[71:64] is byte 23 in the Hierarchy ID Message.
 System GUID[63:56] is byte 24 in the Hierarchy ID Message.
 System GUID[55:48] is byte 25 in the Hierarchy ID Message.
 System GUID[47:40] is byte 26 in the Hierarchy ID Message.
 System GUID[39:32] is byte 27 in the Hierarchy ID Message.
 System GUID[31:24] is byte 28 in the Hierarchy ID Message.
 System GUID[23:16] is byte 29 in the Hierarchy ID Message.
 System GUID[15:8] is byte 30 in the Hierarchy ID Message.
 System GUID[7:0] is byte 31 in the Hierarchy ID Message.

System GUID Authority ID identifies the mechanism used to ensure that the System GUID is globally unique. The mechanism for choosing which Authority ID to use for a given system is implementation specific. The defined values are shown in § Table 6-17.

Table 6-17 System GUID Authority ID Encoding §

Authority ID	Description
00h	None - System GUID[143:0] is not meaningful.

Authority ID	Description
	System GUID[143:0] must be 0.
01h	Timestamp - System GUID[63:0] contains a timestamp associated with the particular system. Encoding is a Unix 64 bit time (number of seconds since midnight UTC January 1, 1970). The mechanism of choosing the timestamp to represent a system is implementation specific. System GUID[143:64] must be 0.
02h	IEEE EUI-48 - System GUID[47:0] contains a 48 bit Extended Unique Identifier (EUI-48) associated with the particular system. Encoding is defined by the IEEE. See [EUI-48] for details. EUI-48 values are frequently used as network interface MAC addresses. The mechanism of choosing the EUI-48 value to represent a system is implementation specific. System GUID[143:48] must be 0.
03h	IEEE EUI-64 - System GUID[63:0] contains a 64 bit Extended Unique Identifier (EUI-64) associated with the particular system. Encoding is defined by the IEEE. See [EUI-64] for details. The mechanism of choosing the EUI-64 value to represent a system is implementation specific. System GUID[143:64] must be 0.
04h	RFC-4122 UUID - System GUID[127:0] contain a UUID as defined by the IETF in [RFC-4122]. This definition is technically equivalent to [ITU-T-Rec-X-667] or [ISO-IEC-9834-8]. The mechanism of choosing the UUID value to represent a system is implementation specific. System GUID[143:128] must be 0
05h	IPv6 Address - System GUID[127:0] contains the unique IPv6 address of one of the network interfaces of the system. The mechanism of choosing the IPv6 value to represent a system is implementation specific. System GUID[143:128] must be 0.
06h to 7Fh	Reserved - System GUID[143:0] contains a unique value. The mechanism used to ensure uniqueness is outside the scope of this specification.
80h to FFh	PCI-SIG Vendor Specific - System GUID Authority ID values 80h to FFh are reserved for PCI-SIG vendor-specific usage. System GUID[143:128] contains a PCI-SIG assigned Vendor ID. System GUID[127:0] contain a unique number assigned by that vendor. The mechanism used for assigning numbers is implementation specific. One possible mechanism would be to use the serial number assigned to the system. The mechanism used to choose between these System GUID Authority IDs is implementation specific. One usage would be to allow a vendor to define up to 128 distinct 128-bit System GUID schemes.

IMPLEMENTATION NOTE:

SYSTEM GUID CONSISTENCY AND STABILITY §

To support the purpose of System GUID, software should ensure that a single system uses identical System GUID and System GUID Authority ID values everywhere.

Implementers should carefully consider their stability requirements for the System GUID value. For example, some use cases may require that the value not change when the system is rebooted. In those cases, a mechanism that picks the EUI-48 value associated with the first Ethernet MAC address discovered might be problematic if the result changes due to hardware failure, system reconfiguration, or variations/parallelism in the discovery algorithm.

IMPLEMENTATION NOTE:

HIERARCHY ID VS. DEVICE SERIAL NUMBER §

The Device Serial Number mechanism can also be used to uniquely identify a component (see § Section 7.9.3). Device Serial Number may be a more expensive solution to this problem if it involves a ROM associated with each component.

IMPLEMENTATION NOTE:

VIRTUAL FUNCTIONS AND HIERARCHY ID §

The Hierarchy ID capability can be emulated by the Virtualization Intermediary (VI). Doing so provides VF software access to this Hierarchy ID information.

When VF hardware needs access to this information, the VF should implement the Hierarchy ID capability. This provides access to both VF software and hardware.

In some situations, the VF should get the same information as the PF. In other situations, particularly those involving migration of Virtual Machines, it may be appropriate to present the VF with Hierarchy ID information that differs from the associated PF and from other VFs associated with that PF.

The following mechanisms are supported:

	VF Hierarchy ID Capability	Hierarchy ID VF Configurable	Hierarchy ID Writeable	VF Software has access	VF Hardware has access	VF Hierarchy Data / GUID
1	Not Present	n/a	n/a	No	No	Not Emulated
2				Yes	No	Emulated
3	Present	0b	0b	Yes	Yes	Same as PF
4		1b	0b	Yes	Yes	Same as PF
5		1b	1b	Yes	Yes	Configured by VI

In mechanism 1, the Virtualization Intermediary does not emulate the capability. VF software and hardware have no access.

In mechanism 2, the Virtualization Intermediary emulates the capability and returns whatever Hierarchy ID information is desired. VF software has access. VF hardware does not have access.

In mechanisms 3 and 4, VF information is the same as the PF and is automatically filled in from received Hierarchy ID messages. Both VF hardware and software have access.

In mechanism 5, VF information is configured by software (probably the VI). Both VF hardware and software have access.

6.26 Flattening Portal Bridge (FPB) §

6.26.1 Introduction §

The Flattening Portal Bridge (FPB) is an optional mechanism which can be used to improve the scalability and runtime reallocation of Routing IDs and Memory Space resources.

For non-ARI Functions associated with an Upstream Port, the Routing ID consists of a 3-bit Function Number portion, which is determined by the construction of the Upstream Port hardware, and a 13-bit Bus Number and Device number portion, determined by the Downstream Port above the Upstream port.

For ARI Functions associated with an Upstream Port, the Routing ID consists of an 8-bit Function Number portion, and only the 8-bit Bus Number portion is determined by the Downstream Port above the Upstream port.

A bridge that implements the FPB Capability can itself also be referred to as an FPB. The FPB Capability can be applied to any logical bridge, as illustrated in § Figure 6-28.

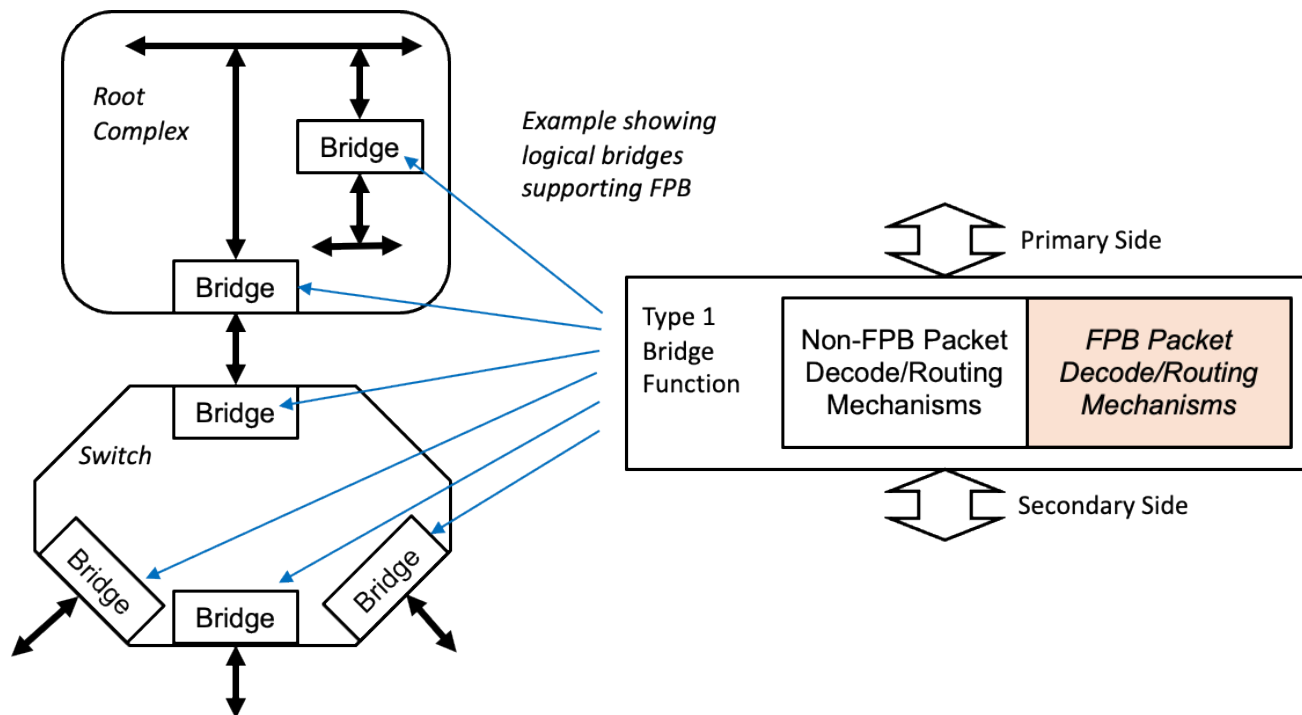


Figure 6-28 FPB High Level Diagram and Example Topology §

FPB changes the way Bus Numbers are consumed by Switches to reduce waste, by “flattening” the way Bus Numbers are used inside of Switches and by Downstream Ports (see § Figure 6-29).

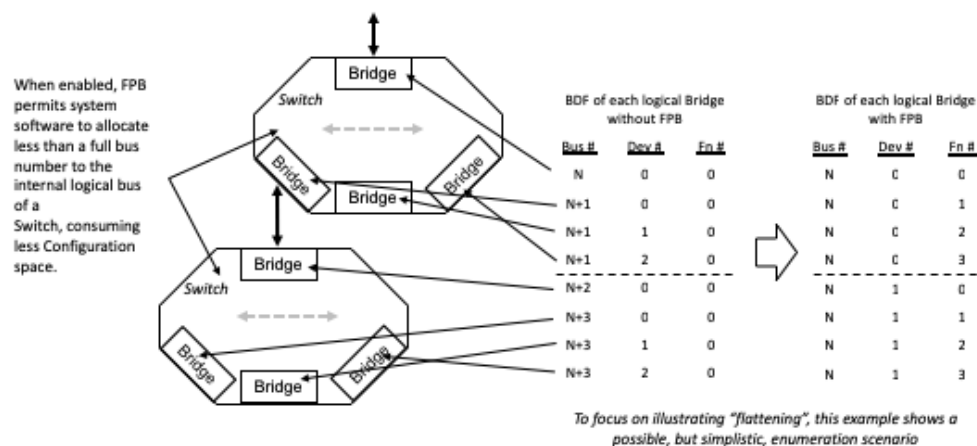


Figure 6-29 Example Illustrating "Flattening" of a Switch §

FPB defines mechanisms for system software to allocate Routing IDs and Memory Space resources in non-contiguous ranges, enabling system software to assign pools of these resources from which it can allocate "bins" to Functions below the FPB. This is done using a bit vector where each bit when Set assigns a corresponding range of resources to the Secondary Side of the bridge (see § Figure 6-30).

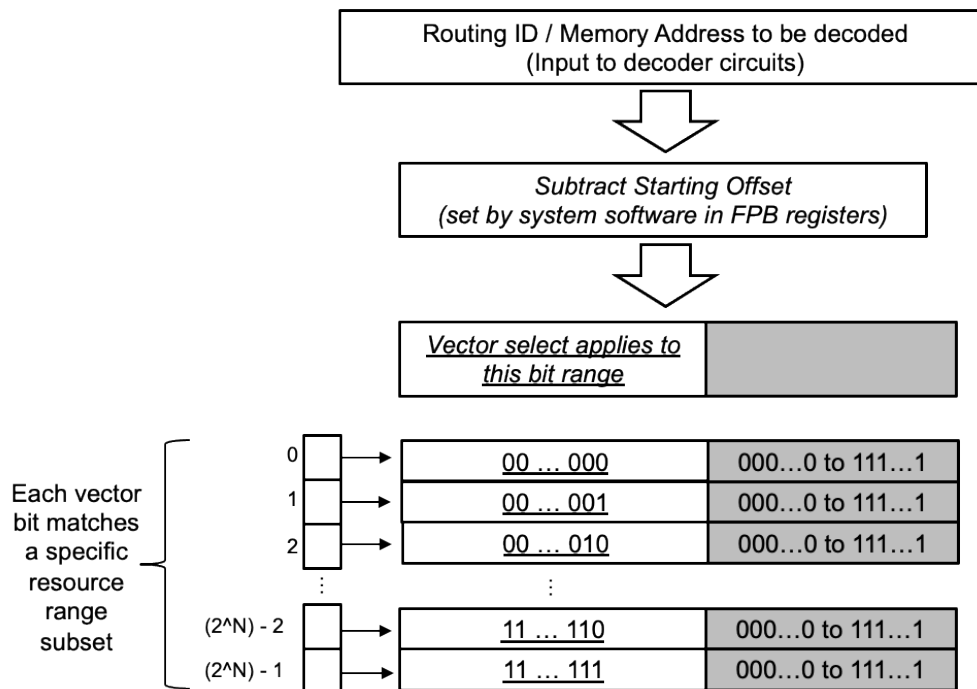


Figure 6-30 Vector Mechanism for Address Range Decoding §

This allows system software to assign Routing IDs and/or Memory Space resources required by a device hot-add without having to rebalance other, already assigned resource ranges, and to return to the pool resources freed, for example by a hot remove event.

FPB is defined to allow both the non-FPB and FPB mechanisms to operate simultaneously, such that, for example, it is possible for system firmware/software to implement a policy where the non-FPB mechanisms continue to be used in parts of the system where the FPB mechanisms are not required (see § Figure 6-31). In this figure, the decode logic is assumed to provide a 1b output when a given TLP is decoded as being associated with the bridge's Secondary Side. The non-FPB decode mechanisms apply as without FPB, so for example only the Bus Number portion (bits 15:8) of a Routing ID is tested by the non-FPB decode logic when evaluating an ID routed TLP.

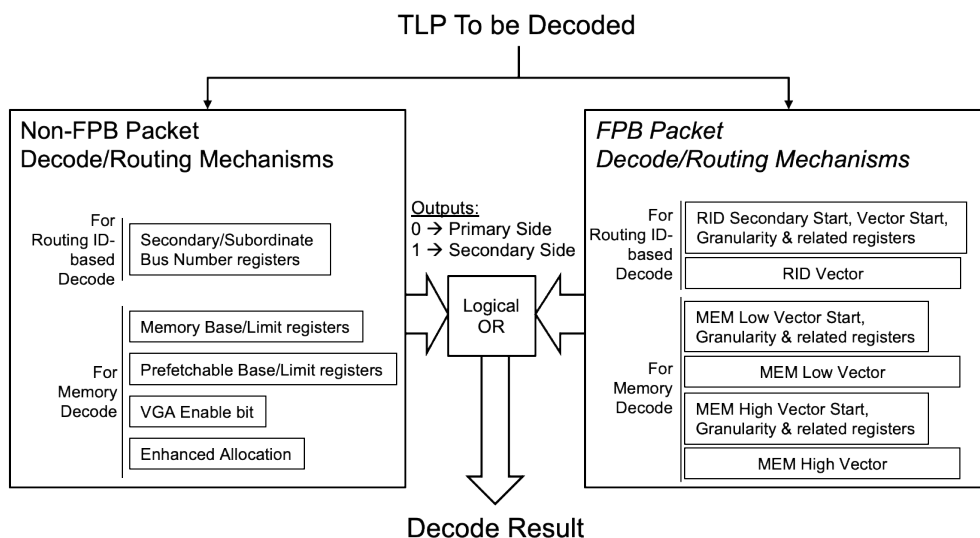


Figure 6-31 Relationship between FPB and non-FPB Decode Mechanisms §

It is important to recognize that, although FPB adds additional ways for a specific bridge to decode a given TLP, FPB does not change anything about the fundamental ways that bridges operate within the Switch and Root Complex architectural structures. FPB uses the same architectural concepts to provide management mechanisms for three different resource types:

1. Routing IDs
2. Memory below 4 GB (“MEM Low”)
3. Memory above 4 GB (“MEM High”)

A hardware implementation of FPB is permitted to support any combination of these three mechanisms. For each mechanism, FPB uses a bit-vector to indicate, for a specific subset range of the selected resource type, if resources within that range are associated with the Primary or Secondary side of the FPB. Hardware implementations are permitted to implement a small range of sizes for these vectors, and system firmware/software is enabled to make the most effective use of the available vector by selecting an initial offset at which the vector is applied, and a granularity for the individual bits within the vector to indicate the size of the resource range to which the bits in a given vector apply.

6.26.2 Hardware and Software Requirements §

The following rules apply when any of the FPB mechanisms are used:

- If system software violates any of the rules concerning FPB, the hardware behavior is undefined.
- It is permitted to implement FPB in any PCI bridge (Type 1) Function, and every Function that implements FPB must implement the FPB Capability (see § Section 7.8.11).
- If a Switch implements FPB then the Upstream Port and all Downstream Ports of the Switch must implement FPB.
- Software is permitted to enable FPB at some Switch Ports and not others.
- A Root Complex is permitted to implement FPB on some Root Ports but not on others.

- A Type 1 Function is permitted to implement the FPB mechanisms applying to any one, two or three of these elemental mechanisms:
 - Routing IDs (RID)
 - Memory below 4 GB (“MEM Low”)
 - Memory above 4 GB (“MEM High”)
- System software is permitted to enable any combination (including all or none) of the elemental mechanisms supported by a specific FPB.
- The error handling and reporting mechanisms, except where explicitly modified in this section, are unaffected by FPB.
- Following any reset of the FPB Function, the FPB hardware must Clear all bits in all implemented vectors.
- Once enabled (through the FPB RID Decode Mechanism Enable, FPB MEM Low Decode Mechanism Enable, and/or FPB MEM High Decode Mechanism Enable bits), if system software subsequently disables an FPB mechanism, the values of the entries in the associated vector are undefined, and if system software subsequently re-enables that FPB mechanism the FPB hardware must Clear all bits in the associated vector.
- If an FPB is implemented with the No_Soft_Reset bit Clear, when that FPB is cycled through $\underline{D0} \rightarrow \underline{D3_{Hot}} \rightarrow \underline{D0}$, then all FPB mechanisms must be disabled, and the FPB must Clear all bits in all implemented vectors.
- If an FPB is implemented with the No_Soft_Reset bit Set, when that FPB is cycled through $\underline{D0} \rightarrow \underline{D3_{Hot}} \rightarrow \underline{D0}$, then all FPB configuration state must not change, and the entries in the FPB vectors must be retained by hardware.
- Hardware is not required to perform any type of bounds checking on FPB calculations, and system software must ensure that the FPB parameters are correctly programmed
 - It is explicitly permitted for system software to program Vector Start values that cause the higher order bits of the corresponding vector to surpass the resource range associated with a given FPB, but in these cases system software must ensure that those higher order bits of the vector are Clear.
 - Examples of errors that system software must avoid include duplication of resource allocation, combinations of start offsets with set vector bits that could create “wrap-around” or bounds errors

The following rules apply to the FPB Routing ID (RID) mechanism:

- FPB hardware must consider a specific range of RIDs to be associated with the Secondary side of the FPB if the Bus Number portion falls within the Bus Number range indicated by the values programmed in the Secondary and Subordinate Bus Number registers logically OR'd with the value programmed into the corresponding entry in the FPB RID Vector.
- System software must configure the Configuration Request Type 1 to Type 0 conversion mechanisms in a Bridge Function before attempting to pass Configuration Requests through that Bridge.
- System software must either program the legacy and FPB mechanisms for Configuration Request Type 1 to Type 0 conversion in a Bridge Function such that they give identical results, or such that one of the two mechanisms is disabled.
 - If it is intended to use only the FPB RID mechanism for BDF decoding, then system software must ensure that both the Secondary and Subordinate Bus Number registers are 0.
 - If it is intended to enable the FPB RID Decode Mechanism, but to use only the legacy mechanism for Configuration Request Type 1 to Type 0 conversion, then system software must write bits 7:3 of the RID Secondary Start field to 0 0000b.
- System software must ensure that the FPB routing mechanisms are configured such that Configuration Requests targeting Functions Secondary side of the FPB will be routed by the FPB from the Primary to Secondary side of the FPB.

When ARI is not enabled, the FPB RID mechanism can be applied with different granularities, programmable by system software through the FPB RID Vector Granularity field in the FPB RID Vector Control 1 Register. § Figure 6-32 illustrates the relationships between the layout of RIDs and the supported granularities. The reader may find it helpful to refer to this figure when considering the requirements defined below and in the definition of the Flattening Portal Bridge (FPB) Capability (see § Section 7.8.6).

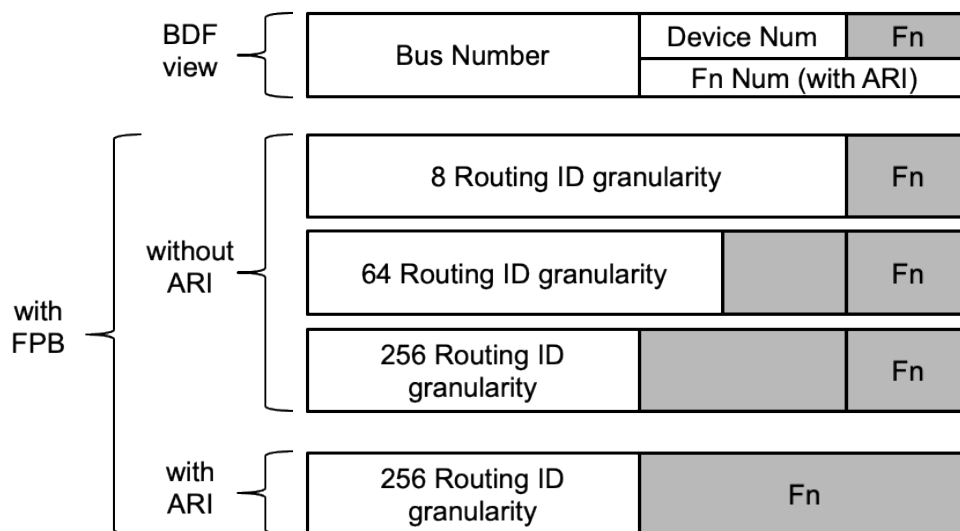


Figure 6-32 Routing IDs (RIDs) and Supported Granularities §

- System software must program the FPB RID Vector Granularity and FPB RID Vector Start fields in the FPB RID Vector Control 1 Register per the constraints described in the descriptions of those fields.
- For all FPBs other than those associated with Upstream Ports of Switches:
 - When ARI Forwarding is not supported, or when the ARI Forwarding Enable bit in the Device Control 2 Register is Clear, FPB hardware must convert a Type 1 Configuration Request received on the Primary side of the FPB to a Type 0 Configuration Request on the Secondary side of the FPB when bits 15:3 of the Routing ID of the Type 1 Configuration Request matches the value in the RID Secondary Start field in the FPB RID Vector Control 2 Register, and system software must configure the FPB accordingly.
 - When the ARI Forwarding Enable bit in the Device Control 2 Register is Set, FPB hardware must convert a Type 1 Configuration Request received on the Primary side of the FPB to a Type 0 Configuration Request on the Secondary side of the FPB when the Bus Number portion of the Routing ID of the Type 1 Configuration Request matches the value in the Bus Number address (bits 15:8 only) of the RID Secondary Start field in the FPB RID Vector Control 2 Register, and system software must configure the FPB accordingly.
- For FPBs associated with Upstream Ports of Switches only, when the FPB RID Decode Mechanism Enable bit is Set, FPB hardware must use the FPB Num Sec Dev field of the FPB Capabilities register to indicate the quantity of Device Numbers associated with the Secondary Side of the Upstream Port bridge, which must be used by the FPB in addition to the RID Secondary Start field in the FPB RID Vector Control 2 Register to determine when a Configuration Request received on the Primary side of the FPB targets one of the Downstream Ports of the Switch, determining in effect when such a Request must be converted from a Type 1 Configuration Request to a Type 0 Configuration Request, and system software must configure the FPB appropriately.
 - System software configuring FPB must comprehend that the logical internal structure of a Switch will change depending on the value of the FPB RID Decode Mechanism Enable bit in the Upstream Port of a Switch.

- Downstream Ports must use their corresponding RID values, and their Requester IDs and Completer IDs, as determined by the Upstream Port's FPB Num Sec Dev and RID Secondary Start values
 - All implemented Functions in the range determined by the Switch Upstream Port Function's RID Secondary Start and FPB Num Sec Dev must be Switch Downstream Ports associated with that Switch Upstream Port; System Software is required to scan all Functions in this range to determine which are implemented.
 - It is strongly recommended that System Software assign the RID Secondary Start such that the Bus and Device Numbers are not the same as for the Switch Upstream Port; otherwise, the resulting hardware behavior is undefined.
- For FPBs associated with Upstream Ports of Switches only, hardware must comprehend that Configuration Requests targeting the Upstream Port itself and any Downstream Ports of the Switch flattened into the range of Function Numbers with the same Bus and Device Numbers as the Upstream Port itself will be converted from Type 1 to Type 0 by the Downstream Port above the Switch, but any other Downstream Ports of the Switch flattened into successive Device Numbers will not be converted from Type1 to Type0 by the Downstream Port above the Switch and so must effectively be converted from Type 1 to Type 0 by the Switch Upstream Port itself.
This is a special case, but the concept is not unique to FPB, and is a reflection of the definition of the relationship between Bus/Device Numbers and Function Numbers – Function Numbers are always determined by the hardware of the Upstream Port, whereas the Bus and Device Numbers for an Upstream Port are always determined by the Downstream Port immediately above the Upstream Port.
- FPBs must implement bridge mapping for INTx virtual wires (see § Section 2.2.8.1)
 - Hardware and software must apply this algorithm (or the logical equivalent) to determine which entry in the FPB RID Vector applies to a given Routing ID (RID) address:
 - IF the RID is below the value of FPB RID Vector Start, then the RID is out of range (below the start) and so cannot be associated with the Secondary side of the bridge, ELSE
 - calculate the offset within the vector by first subtracting the value of FPB RID Vector Start, then dividing this according to the value of FPB RID Vector Granularity to determine the bit index within the vector.
 - IF the bit index value is greater than the length indicated by FPB RID Vector Size Supported, then the RID is out of range (beyond the top of the range covered by the vector) and so cannot be associated with the Secondary side of the bridge, ELSE
 - if the bit value within the vector at the calculated bit index location is 1b, THEN the RID address is associated with the Secondary side of the bridge, ELSE the RID address is associated with the Primary side of the bridge.

The following rules apply to the FPB MEM Low mechanism:

The FPB MEM Low mechanism can be applied with different granularities, programmable by system software through the FPB MEM Low Vector Granularity field in the FPB MEM Low Vector Control Register. § Figure 6-33 illustrates the relationships between the layout of addresses in the memory address space below 4 GB to which the FPB MEM Low mechanism applies. The reader may find it helpful to refer to this figure when considering the requirements defined below and in the definition of the Flattening Portal Bridge (FPB) Capability (see § Section 7.8.11).

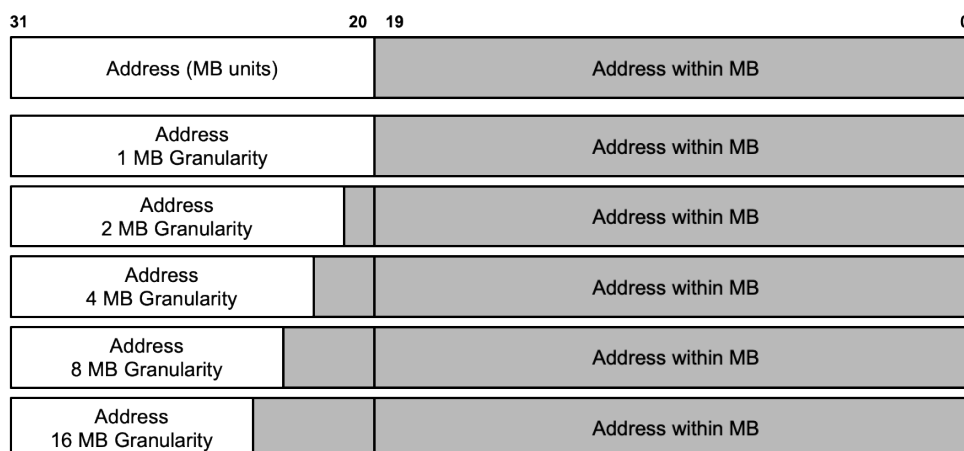


Figure 6-33 Addresses in Memory Below 4 GB and Effect of Granularity §

- System software must program the FPB MEM Low Vector Granularity and FPB MEM Low Vector Start fields in the FPB MEM Low Vector Control Register per the constraints described in the descriptions of those fields.
- FPB hardware must consider a specific Memory address to be associated with the Secondary side of the FPB if that Memory address falls within any of the ranges indicated by the values programmed in other bridge Memory decode registers (enumerated below) logically OR'd with the value programmed into the corresponding entry in the FPB MEM Low Vector. Other bridge Memory decode registers include:
 - Memory Base/Limit registers
 - 64-bit Memory Base/Limit registers
 - VGA Enable bit in the Bridge Control Register
 - Enhanced Allocation (EA) Capability (if supported)
 - FPB MEM High mechanism (if supported and enabled)
- Hardware and software must apply this algorithm (or the logical equivalent) to determine which entry in the FPB MEM Low Vector applies to a given Memory address:
 - If the Memory address is below the value of FPB MEM Low Vector Start, then the Memory address is out of range (below) and so is not associated with the Secondary side of the bridge by means of this mechanism, else
 - calculate the offset within the vector by first subtracting the value of FPB MEM Low Vector Start, then dividing this according to the value of FPB MEM Low Vector Granularity to determine the bit index within the vector.
 - If the bit index value is greater than the length indicated by FPB MEM Low Vector Size Supported, then the Memory address is out of range (above) and so is not associated with the Secondary side of the bridge by means of this mechanism, else
 - if the bit value within the vector at the calculated bit index location is 1b, then the Memory address is associated with the Secondary side of the bridge, else the Memory address is associated with the Primary side of the bridge.

The following rules apply to the FPB MEM High mechanism:

- System software must program the FPB MEM High Vector Granularity and FPB MEM High Vector Start Lower fields in the FPB MEM High Vector Control 1 Register per the constraints described in the descriptions of those fields.

- FPB hardware must consider a specific Memory address to be associated with the Secondary side of the FPB if that Memory address falls within any of the ranges indicated by the values programmed in other bridge Memory decode registers (enumerated below) logically OR'd with the value programmed into the corresponding entry in the FPB MEM High Vector. Other bridge Memory decode registers include:
 - Memory Base/Limit registers
 - 64-bit Memory Base/Limit registers
 - VGA Enable bit in the Bridge Control Register
 - Enhanced Allocation (EA) Capability (if supported)
 - FPB MEM Low mechanism (if supported and enabled)
- Hardware and software must apply this algorithm to determine which entry in the FPB MEM High Vector applies to a given Memory address:
 - If the Memory address is below the value of FPB MEM High Vector Start Upper/FPB MEM High Vector Start Lower, then the Memory address is out of range (below) and so is not associated with the Secondary side of the bridge by means of this mechanism, else
 - calculate the offset within the vector by first subtracting the value of FPB MEM High Vector Start Upper/FPB MEM High Vector Start Lower, then dividing this according to the value of FPB MEM High Vector Granularity to determine the bit index within the vector.
 - If the bit index value is greater than the length indicated by FPB MEM High Vector Size Supported, then the Memory address is out of range (above) and so is not associated with the Secondary side of the bridge by means of this mechanism, else
 - if the bit value within the vector at the calculated bit index location is 1b, then the Memory address is associated with the Secondary side of the bridge, else the Memory address is associated with the Primary side of the bridge.

IMPLEMENTATION NOTE:

FPB ADDRESS DECODING §

FPB uses a bit vector mechanism to decode ranges of Routing IDs, and Memory Addresses above and below 4 GB. A bridge supporting FPB contains the following for each resource type/range where it supports the use of FPB:

- A Bit vector
- A Start Address
- A Granularity

These are used by the bridge to determine if a given address is part of the range decoded by FPB as associated with the secondary side of the bridge. An address that is determined not to be associated with the secondary side of the bridge using either or both of the non-FPB decode mechanisms and the FPB decode mechanisms is (by default) associated with the primary side of the bridge. Here, when we use the term “associated” we mean, for example, that the bridge will apply the following handling to TLPs:

- Associated with Primary, Received at Primary → Unsupported Request (UR)
- Associated with Primary, Received at Secondary → Forward upstream
- Associated with Secondary, Received at Primary → Forward downstream
- Associated with Secondary, Received at Secondary → Unsupported Request (UR)

In FPB, every bit in the vector represents a range of resources, where the size of that range is determined by the selected granularity. If a bit in the vector is Set, it indicates that TLPs addressed to an address within the corresponding range are to be associated with the secondary side of the bridge. The specific range of resources each bit represents is dependent on the index of that bit, and the values in the Start Address & Granularity. The Start Address indicates the lowest address described by the bit vector. The Granularity indicates the size of the region that is represented by each bit. Each successive bit in the vector applies to the subsequent range, increasing with each bit according to the Granularity.

For example, consider a bridge using FPB to describe a MEM Low range. FPB MEM Low Vector Start has been set to FC0h, indicating that the range described by the bit vector starts at address FC00 0000. FPB MEM Low Vector Granularity has been set to 0000b, indicating that each bit represents a 1 MB range.

From these values we can determine that bit 0 of the vector represents a 1MB range starting at FC000 0000 (FC00 0000-FC0F FFFF), bit 1 represents FC10 0000-FC1F FFFF, etc.

Bits in the vector that are set to 0 indicate that the range is not included in the range described by FPB. In the above example, If bit 0 is Clear, packets addressed to anywhere between FC00 0000 and FC0F FFFF should not be routed to the secondary bus of the bridge due to FPB.

IMPLEMENTATION NOTE:

HARDWARE AND SOFTWARE CONSIDERATIONS FOR FPB §

FPB is intended to address a class of issues with PCI/PCIe architecture that relate to resource allocation inefficiency. These issues can be categorized as “static” or “dynamic” use case scenarios, where static use cases refer to scenarios where resources are allocated at system boot and then typically not changed again, and dynamic use cases refer to scenarios where run-time resource rebalancing (e.g., allocation of new resources, freeing of resources no longer needed) is required, due to hot add/remove, or by other needs.

In the Static cases there are limits on the size of hierarchies and number of Endpoints due to the use of additional Bus Numbers and the lack of use of Device Numbers caused by the PCI/PCIe architectural definition for Switches and Downstream Ports. FPB addresses this class of problems by “flattening” the use of Routing IDs (RIDs) so that Switches and Downstream Ports are able to make more efficient use of the available RIDs.

For the Dynamic cases, without FPB, the “best known method” to avoid rebalancing has been to reserve large ranges of Bus Numbers and Memory Space in the bridge above the relevant Port or Endpoint such that hopefully any future needs can be satisfied within the pre-allocated ranges. This leads to potentially unused allocations, which makes the Routing ID issues worse, and in a resource constrained platform this approach is difficult to implement, even for relatively simple cases, where, for example, one might have an add-in card implementing a single Endpoint replaced by another add-in card that has a Switch and two Endpoints, so that although an initial allocation of just one Bus would have been sufficient, the initial allocation breaks immediately with the new add-in card.

For Memory Space the pre-allocation approach is problematic when hot-plugged Endpoints may require the allocation of Memory Space below 4 GB, which by its nature is a limited resource, which is quickly used up by pre-allocation of even relatively small amounts, and for which pre-allocation is unattractive because of the multiple system elements placing demands on system address space allocation below 4 GB.

FPB includes mechanisms to enable discontinuous resource range allocation/reallocation for both Requester IDs and Memory Space. The intent is to allow system software the ability to maintain resource “pools” which can be allocated (and freed back to) at run-time, without disrupting other operations in progress as is required with rebalancing.

To support the run time use of FPB by system software, FPB hardware implementations should avoid introducing stalls or other types of disruptions to transactions in flight, including during the times that system software is modifying the state of the FPB hardware. It is not, however, expected that hardware will attempt to identify cases where system software erroneously modifies the FPB configuration in a way that does affect transactions in flight. Just as with the non-FPB mechanisms, it is the responsibility of system software to ensure that system operation is not corrupted due to a reconfiguration operation.

It is not explicitly required that system firmware/software perform the enabling and/or disabling of FPB mechanisms in a particular sequence, however care should be taken to implement resource allocation operations in a hierarchy such that the hardware and software elements of the system are not corrupted or caused to fail.

6.27 Vital Product Data (VPD) §

Vital Product Data (VPD) is the information that uniquely defines items such as the hardware, software, and microcode elements of a system. The VPD provides the system with information on various FRUs (Field Replaceable Unit) including Part Number, Serial Number, and other detailed information. VPD also provides a mechanism for storing information such as performance and failure data on the device being monitored. The objective, from a system point of view, is to collect this information by reading it from the hardware, software, and microcode components.

Support of VPD within add-in cards is optional depending on the manufacturer. Though support of VPD is optional, add-in card manufacturers are encouraged to provide VPD due to its inherent benefits for the add-in card, system manufacturers, and for Plug and Play.

The mechanism for accessing VPD is documented in § Section 7.9.18.

VPD for PCI Express is unchanged from the definition in the [PCI-3.0]. That definition, in turn, was based on earlier versions of the [PCI] as well as the [PLUG-PLAY-ISA-1.0a].

Vital Product Data is made up of Small and Large Resource Data Types.

Table 6-19 Small Resource Data Type Tag Bit Definitions §

Offset	Field Name		
Byte 0	Value = 0xxx xyyyb		
	Bit 7	Small Resource Type	0b
	Bits 6:3	Small Item Name	xxxx
	Bits 2:0	Length in bytes	yy
Bytes 1 to n	Actual information		

Table 6-20 Large Resource Data Type Tag Bit Definitions §

Offset	Field Name		
Byte 0	Value = 1xxx xxxxb		
	Bit 7	Large Resource Type	1b
	Bits 6:0	Large Item Name	xxxxxxx
Byte 1	Length in bytes of data items bits[7:0] (lsb)		
Byte 2	Length in bytes of data items bits[15:8] (msb)		
Bytes 3 to n	Actual data items		

The first VPD tag is the Identifier String (02h) and provides the product name of the device.

One VPD-R (10h) tag is used as a header for the read-only keywords. Attempts to write the read-only data will be executed as a no-op.

One VPD-W (11h) tag is used as a header for the read-write keywords. The storage component containing the read/write data is a non-volatile device that will retain the data when powered off.

The last tag must be the End Tag (0Fh).

A small example of the resource data type tags used in a typical VPD is shown in § Table 6-21.

Table 6-21 Resource Data Type Flags for a Typical VPD §

TAG 02h	Identifier String	Large Resource Data Type
TAG 10h	VPD-R list containing one or more VPD keywords	Large Resource Data Type
TAG 11h	VPD-W list containing one or more VPD keywords	Large Resource Data Type
TAG 0Fh	End Tag	Small Resource Data Type

6.27.1 VPD Format §

Information fields within a VPD resource type consist of a three-byte header followed by some amount of data (see § Figure 6-34). The three-byte header contains a two-byte keyword and a one-byte length. A keyword is a two-character (ASCII) mnemonic that uniquely identifies the information in the field. The last byte of the header is binary and represents the length value (in bytes) of the data that follows.

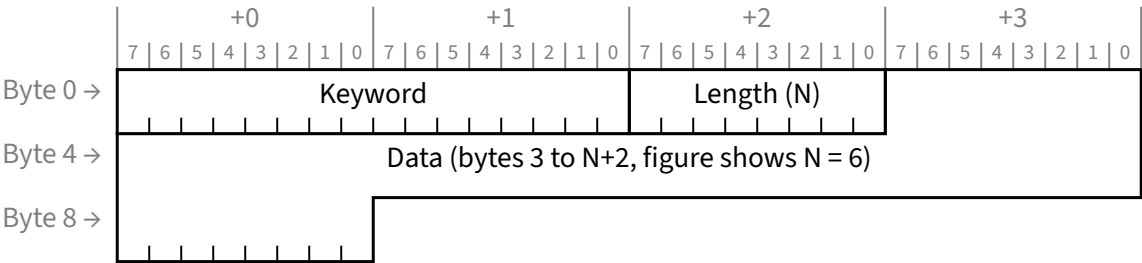


Figure 6-34 VPD Format §

VPD keywords are listed in two categories: read-only fields and read/write fields. Unless otherwise noted, keyword data fields are provided as ASCII characters. Use of ASCII allows keyword data to be moved across different enterprise computer systems without translation difficulty.

An example of the “add-in card serial number” VPD item is as follows:

Table 6-22 Example of Add-in Serial Card Number §

Byte 0	53h “S”	Keyword: SN
Byte 1	4Eh “N”	
Byte 3	08h	Length: 8
Byte 4	30h “0”	Data: “00000194”
Byte 5	30h “0”	
Byte 6	30h “0”	
Byte 7	30h “0”	
Byte 8	30h “0”	
Byte 9	31h “1”	
Byte 10	39h “9”	
Byte 11	34h “4”	

6.27.2 VPD Definitions §

This section describes the current VPD large and small resource data tags plus the VPD keywords. This list may be enhanced at any time. Companies wishing to define a new keyword should contact the PCISIG. All unspecified values are reserved for SIG assignment.

6.27.2.1 VPD Large and Small Resource Data Tags §

VPD is contained in four types of Large and Small Resource Data Tags. The following tags and VPD keyword fields may be provided in PCI devices.

Table 6-23 VPD Large and Small Resource Data Tags §

Tag	Description
Large resource type Identifier String Tag (02h)	This tag is the first item in the VPD storage component. It contains the name of the add-in card in alphanumeric characters.
Large resource type VPD-R Tag (10h)	This tag contains the read only VPD keywords for an add-in card.
Large resource type VPD-W Tag (11h)	This tag contains the read/write VPD keywords for an add-in card.
Small resource type End Tag (0Fh)	This tag identifies the end of VPD in the storage component.

6.27.2.2 Read-Only Fields §

Table 6-24 VPD Read-Only Fields §

Keyword	Name	Description
PN	Add-in Card Part Number	This keyword is provided as an extension to the Device ID (or Subsystem ID) in the Configuration Space header in § Figure 6-34.
EC	Engineering Change Level of the Add-in Card	The characters are alphanumeric and represent the engineering change level for this add-in card.
FG	Fabric Geography	Reserved for legacy use by [PICMG] specifications.
LC	Location	Reserved for legacy use by [PICMG] specifications.
MN	Manufacture ID	This keyword is provided as an extension to the Vendor ID (or Subsystem Vendor ID) in the Configuration Space header in § Figure 6-34. This allows vendors the flexibility to identify an additional level of detail pertaining to the sourcing of this device.
PG	PCI Geography	Reserved for legacy use by [PICMG] specifications.
SN	Serial Number	The characters are alphanumeric and represent the unique add-in card Serial Number.

Keyword	Name	Description
TR	Thermal Reporting	This keyword provides a standard interface for reporting four fields: AFI Level, MaxTherm, DTherm, and MaxAmbient. The data area for this field is four bytes long. This data is encoded as a 4-byte binary value in little endian order (byte 0 contains bits 7:0). This value contains the four fields as follows: AFI Level bits [3:0], MaxTherm bits [7:4], DTherm bits [11:8], and MaxAmbient bits [19:12] are placed in bits 19:0. Bits 31:20 are Reserved and must be set to 000h. Field description is provided within the [CEM]. This keyword is intended to be used only in designs based on that form factor specification. Note that due to the character nature of the VPD encoding mechanism, this binary value is permitted to start on any byte boundary within the VPD.
Vx	Vendor Specific	This is a vendor specific item and the characters are alphanumeric. The second character (x) of the keyword can be 0 through 9 or A through Z.
CP	Extended Capability	This field allows a new capability to be identified in the VPD area. Since dynamic control/status cannot be placed in VPD, the data for this field identifies where, in the device's memory or I/O address space, the control/status registers for the capability can be found. Location of the control/status registers is identified by providing the index (a value between 0 and 5) of the Base Address register that defines the address range that contains the registers, and the offset within that Base Address register range where the control/status registers reside. The data area for this field is four bytes long. The first byte contains the ID of the extended capability. The second byte contains the index (zero based) of the Base Address register used. The next two bytes contain the offset (in little-endian order) within that address range where the control/status registers defined for that capability reside.
RV	Checksum and Reserved	The first byte of this item is a checksum byte. The checksum is correct if the sum of all bytes in VPD (from VPD address 0 up to and including this byte) is zero. The remainder of this item is reserved space (as needed) to identify the last byte of read-only space. The read-write area does not have a checksum. This field is required.
FF	Form Factor	This keyword indicates a string that identifies the form factor and version associated with this add-in-card. Values are a lower case string. The string consists of a list of one or more elements separated by colons (":"). The first element is a sequence of lowercase strings separated by periods that identifies the specification(s) associated with the form factor. The first element string is the reserved domain name associated with the authority defining that form factor (i.e., like those used in [DNS] records), followed by one or more strings that identify a particular form factor, followed by a Version Number for that form factor specification. Any characters in the form factor name other than "a" to "z", "0" to "9", "*", "?", and "-" are dropped (e.g., M.2 becomes m2). The character "*" represents a wild card (arbitrary characters, including "."). The value "-" is used to indicate no form factor. The value "?" or the absence of the FF keyword is used to indicate that the form factor is unknown. Subsequent elements are optional and contain attributes describing variations of that form factor (e.g., size, connector, keying, ...). These elements are defined by the indicated form-factor specification and consist of either an "option" string or a "key=value" string. Valid characters in the "option" or "key" portion are "a" to "z", "0" to "9", "*", "?", and "-". The "value" portion may contain any character other than the colon ":". The order of attributes is not significant. Attributes are not permitted when the first element is "-" or "?" (no form factor or unknown form factor). Examples are: - com.pcisig.cem.4.0 - com.pcisig.cem.* - com.pcisig.m2.1.0:2280 - com.pcisig.m2.1.0:size=2280:key=M - com.pcisig.m2.1.0:bga

Keyword	Name	Description
		• org.snia.sff-ta-1001.1.0.2 A given add-in card is permitted to claim conformance to multiple form factor specifications. This is indicated by using the wild card character “*” or by using multiple FF fields (which in turn could use the wild card character). When the form factor of a slot does not match some FF keyword of the add-in card in that slot, this indicates the presence of one or more “Carrier Cards” to convert power and sideband signals of the slot to those of the add-in card. The mechanism(s) used to identify a particular Carrier Card and to describe how it operates are outside the scope of this specification.

6.27.2.3 Read/Write Fields §

Table 6-25 VPD Read/Write Fields §

Keyword	Name	Description
Vx	Vendor Specific	This is a vendor specific item and the characters are alphanumeric. The second character (x) of the keyword can be 0 through 9 or A through Z.
Yx	System Specific	This is a system specific item and the characters are alphanumeric. The second character (x) of the keyword can be 0 through 9 or B through Z.
YA	Asset Tag Identifier	This is a system specific item and the characters are alphanumeric. This keyword contains the system asset identifier provided by the system owner.
RW	Remaining Read/Write Area	This descriptor is used to identify the unused portion of the read/write space. The product vendor initializes this parameter based on the size of the read/write space or the space remaining following the Vx VPD items. One or more of the Vx, Yx, and RW items are required.

6.27.2.4 VPD Example §

The following is an example of a typical VPD.

Table 6-26 VPD Example §

Offset	Item Value
0	Large Resource Type ID String Tag (02h) 82h “Product Name”
1	Length 0021h
3	Data “ABCD Super-Fast Widget Controller”
36	Large Resource Type VPD-R Tag (10h) 90h
37	Length 0059h
39	VPD Keyword “PN”
41	Length 08h
42	Data “6181682A”
50	VPD Keyword “EC”

Offset	Item Value
52	Length 0Ah
53	Data “4950262536”
63	VPD Keyword “SN”
65	Length 08h
66	Data “00000194”
74	VPD Keyword “MN”
76	Length 04h
77	Data “1037”
81	VPD Keyword “RV”
83	Length 2Ch
84	Data Checksum
85	Data Reserved (00h)
128	Large Resource Type VPD-W Tag (11h) 91h
129	Length 007Ch
131	VPD Keyword “V1”
133	Length 05h
134	Data “65A01”
139	VPD Keyword “Y1”
141	Length 0Dh
142	Data “Error Code 26”
155	VPD Keyword “RW”
157	Length 61h
158	Data Reserved (00h)
255	Small Resource Type End Tag (0Fh) 78h

6.28 Native PCIe Enclosure Management §

NPEM is an optional PCIe Extended Capability that provides mechanisms for enclosure management. This mechanism is designed to provide management for enclosures containing PCIe SSDs that is consistent with the established capabilities in the storage ecosystem.

This section defines the architectural aspects of the mechanism. The NPEM extended capability is defined in [§ Section 7.9.19](#).

An enclosure is any platform, box, rack, or set of boxes that contain one or more PCIe SSDs. The NPEM capability provides storage related enclosure control (e.g., status LED control) for a PCIe SSD. The NPEM capability may reside in a Downstream Port, or an Endpoint (i.e., the PCIe SSD). § Figure 6-35 shows an example configuration with a single Downstream Port containing the NPEM capability and vendor specific logic to control the associated LEDs.

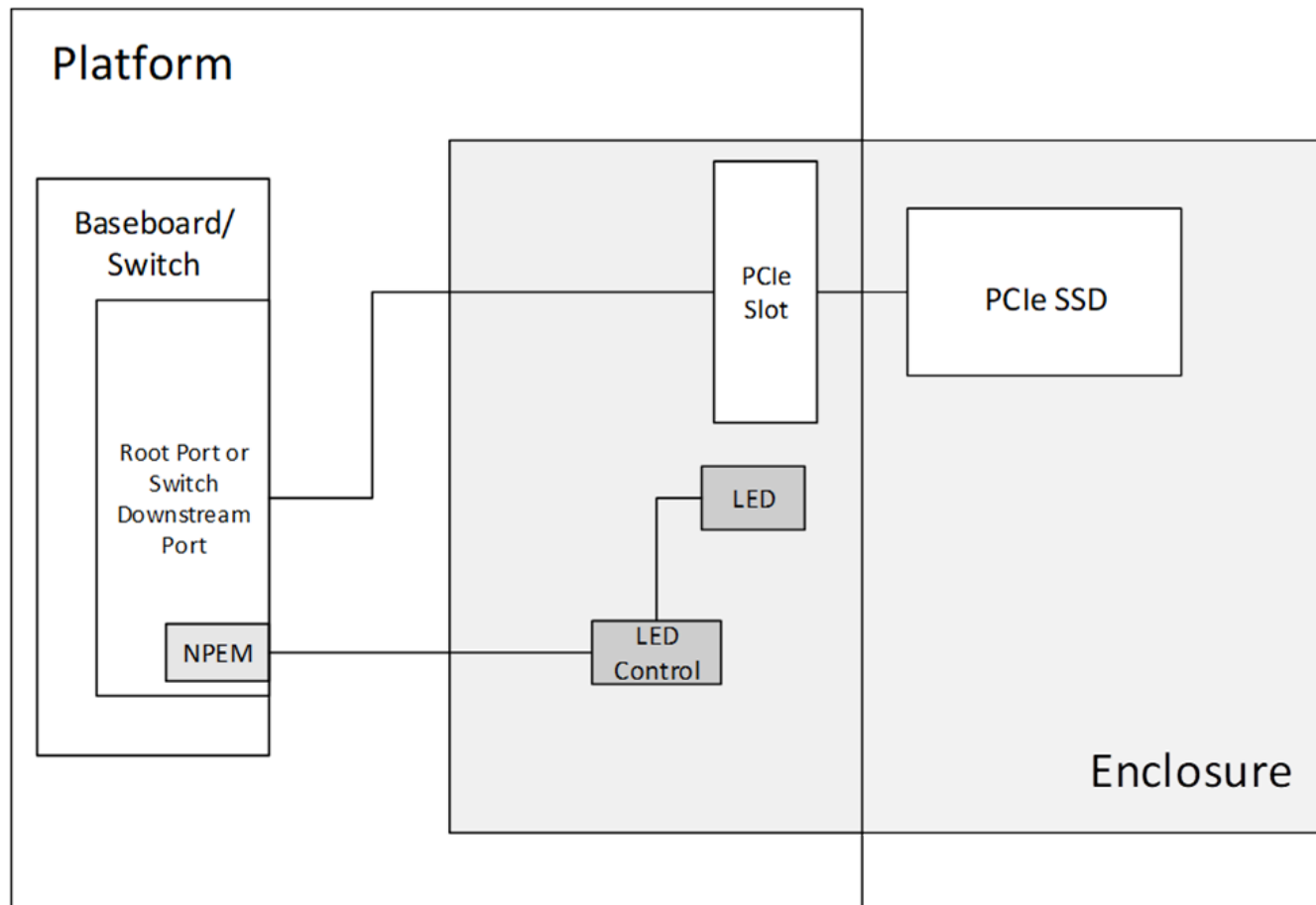


Figure 6-35 Example NPEM Configuration using a Downstream Port §

§ Figure 6-36 shows an example configuration with the NPEM capability located in the Upstream Port (in this case, the SSD function).

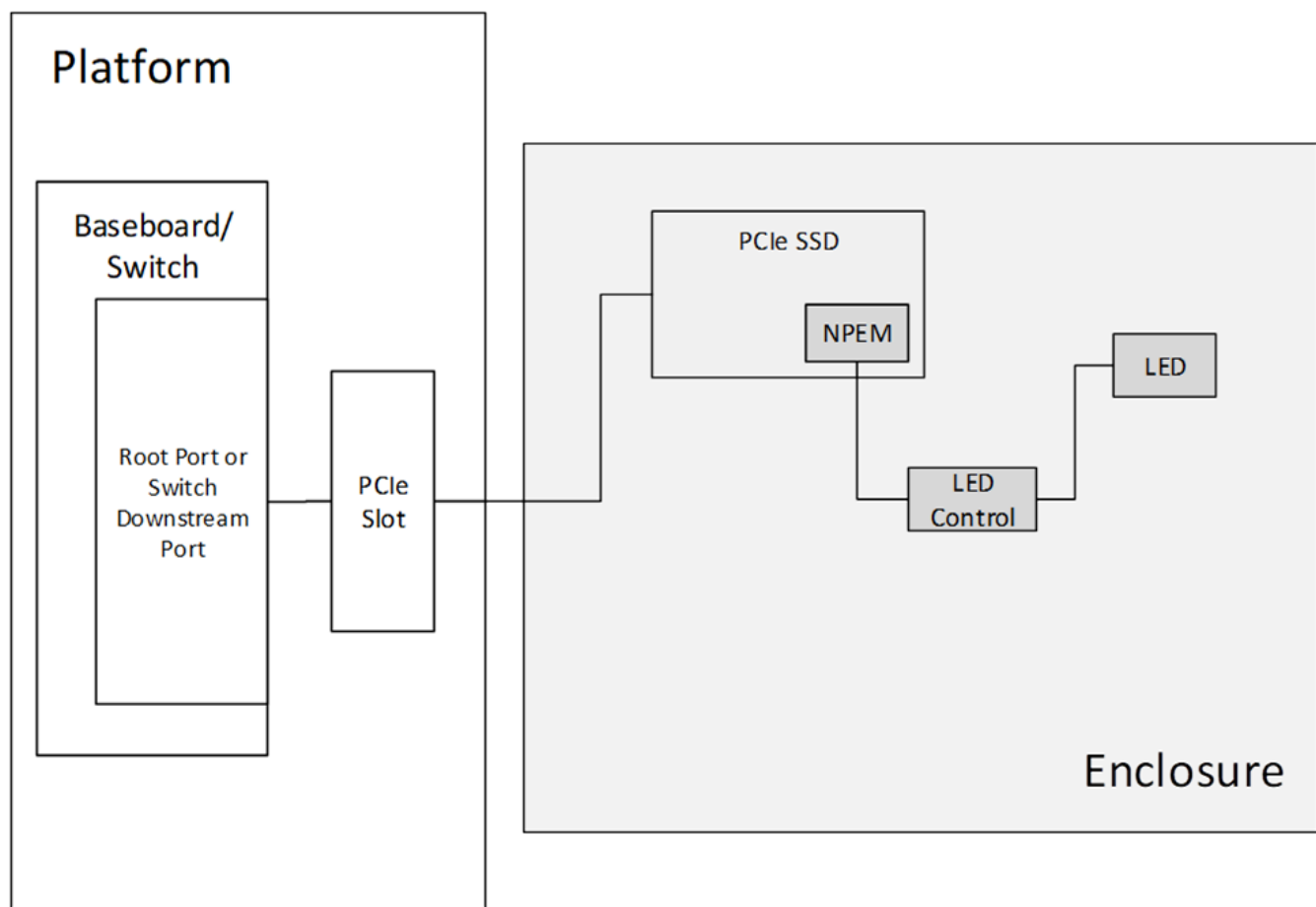


Figure 6-36 Example NPEM Configuration using an Upstream Port §

Software issues an NPEM command by writing to the NPEM Control register to change the indications associated with an SSD. NPEM Command is a single write to the NPEM Control register that changes the state of zero or more bits. NPEM indicates a successful completion to software using the command completed mechanism. § Figure 6-37 shows the overall flow.

This specification defines the software interface provided by the NPEM capability. The Port to enclosure interface, enclosure, enclosure to LED interface, number of LEDs per SSD, and associated LED blink patterns are all outside the scope of this specification.

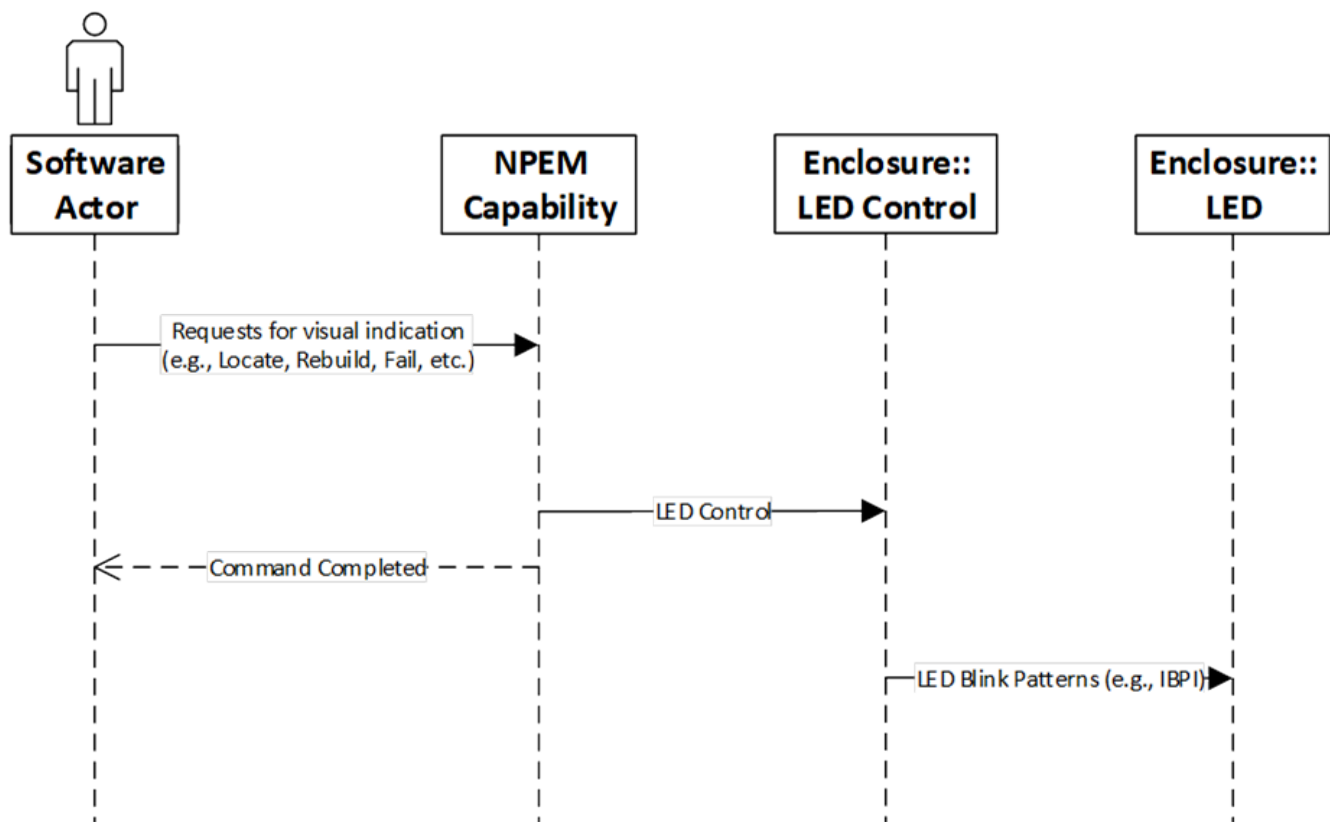


Figure 6-37 NPEM Command Flow §

NPEM provides a mechanism for system software to issue a reset to the LED control element within the enclosure by means of the NPEM Reset mechanism, which is independent of the PCIe Link itself. The NPEM command completed mechanism also applies to NPEM Reset.

Storage system admin or software controls the indications for various device states through the NPEM capability.

IMPLEMENTATION NOTE:

NPEM STATES §

§ Table 6-27 shows an example of NPEM states and a possible meaning that some enclosures may assign to the architected NPEM states.

Table 6-27 NPEM States §

NPEM State	Actor	Definition
OK	System Admin or Storage Software	OK state may mean the drive is functioning normally. This state may implicitly mean that an SSD is present, powered on, and working normally as seen by the software. A more granular indication of drive not physically present or present but not powered up are both outside the scope of this specification.
Locate	System Admin	Locate state may mean the specific drive is being identified by an admin.
Fail	Storage Software	Fail state may mean the drive is not functioning properly
Rebuild	Storage Software	Rebuild state may mean this drive is part of a multi-drive storage volume/array that is rebuilding or reconstructing data from redundancy on to this specific drive.
PFA	Storage Software	PFA stands for Predicted Failure Analysis. This state may mean the drive is still functioning normally but predicted to fail soon.
Hot Spare	Storage Software	Hot Spare state may mean this drive is marked to be automatically used as a replacement for a failed drive and contents of the failed drive may be rebuilt on this drive.
In A Critical Array	Storage Software	In A Critical Array state may mean the drive is part of a multi-drive storage array and that array is degraded.
In A Failed Array	Storage Software	NPEM In A Failed Array state may mean the drive is part of a multi-drive storage array and that array is failed.
Invalid Device Type	Storage Software	Invalid Device Type state may mean the drive is not the right type for the connector (e.g., An enclosure supports SAS and NVMe drives and this drive state indicates that a SAS drive is plugged into an NVMe slot).
Disabled	Storage Software	Disabled state may mean the drive in this slot is disabled. A removal of this drive from the slot may be safe. The power from this slot may be removed.

IMPLEMENTATION NOTE:

SOFTWARE POLLING OF NPEM COMMAND COMPLETED §

Different NPEM implementations may vary widely in how long they take to complete NPEM commands, from instantaneous to tens of ms. To avoid or minimize software polling overheads, it is recommended that software implement one or both of the following optimizations.

Instead of software writing a command and then immediately polling for completion, it is recommended that software reverse this order. When ready to write a new command, software first polls for completion of the previous command, and then writes the new command. This enables overlapped operation, often completely hiding the time it takes hardware to execute an NPEM command. To enable this polling model, software must initialize the hardware following a reset by writing a no-op command in order to have hardware generate the first NPEM command completion.

For the case where an NPEM command has not completed when software polls the bit, it is recommended that software not continuously “spin” on polling the bit, but rather poll under interrupt at a reduced rate; for example at 10 ms intervals.

6.29 Conventional PCI Advanced Features Operation §

For Conventional PCI devices integrated into a Root Complex, the Conventional PCI Advanced Features Capability (AF) provides mechanisms for using advanced features originally developed for PCI Express.

- The Function Level Reset (INITIATE_FLR) mechanism enables software to quiesce and reset hardware with Function-level granularity.

FLR applies on a per Function basis. Only the targeted Function is affected by the FLR operation.

- The Transactions Pending (TP) mechanism is used to indicate that the Function is expecting one or more Completions.

The FLR and TP mechanisms defined here are strictly for Conventional PCI devices integrated into a Root Complex where the implementation permits non-posted transactions for a given Conventional PCI Function to complete even if the value of the Bus Master Enable bit in its Command Register is 0b. Implementations that do not meet this requirement must not implement the FLR and TP mechanisms.

FLR modifies the Function state as follows:

Function registers and Function-specific state machines must be set to their initialization values as specified in this document, except for the following bits, which must not be modified: Fast Back-to-Back Transactions Enable, Cache Line Size, Latency Timer, Interrupt Line, PME_En, PME_Status.

Note that the controls that enable the Function to initiate bus transactions are cleared, including the Bus Master Enable bit in the Command Register, the MSI Enable bit in the MSI Capability Structure, and the like, effectively causing the Function to become quiescent.

After an FLR has been initiated, the Function must complete the FLR within 100 ms. If software initiates an FLR when the Transactions Pending bit is 1b, then software must not initialize the Function until allowing adequate time to achieve reasonable certainty that any outstanding transactions will have completed. The Transactions Pending bit must be clear upon completion of the FLR.

FLR modifies Function state not described by this specification (in addition to state that is described by this specification), and so the following criteria must be applied using Function- specific knowledge to evaluate the Function's behavior in response to an FLR:

- The Function must not give the appearance of an initialized adapter with an active host on any external interfaces controlled by that Function. The steps needed to terminate activity on external interfaces are outside of the scope of this specification.
 - For example, a network adapter must not respond to queries that would require adapter initialization by the host system or interaction with an active host system, but is permitted to perform actions that it is designed to perform without requiring host initialization or interaction. If the network adapter includes multiple Functions that operate on the same external network interface, this rule affects only those aspects associated with the particular Function reset by FLR.
- The Function must not retain within itself software readable state that potentially includes secret information associated with any preceding use of the Function. Main host memory assigned to the Function must not be modified by the Function.
 - For example, a Function with internal memory readable directly or indirectly by host software must clear or randomize that memory.
- The Function must return to a state such that normal configuration of the Function's PCI interface will cause it to be usable by drivers normally associated with the Function.

When an FLR is initiated, the targeted Function must behave as follows:

- The Function must complete normally the configuration write that initiated the FLR operation and then initiate the FLR.
- While an FLR is in progress:
 - The Function must not respond to any request on the bus (i.e., requests targeting the Function will Master Abort).

The Transactions Pending (TP) bit indicates that the Function is expecting one or more Completions. This field may be used by software to determine when a Function has become quiescent.

IMPLEMENTATION NOTE:

AVOIDING ISSUES WITH PENDING TRANSACTIONS §

An FLR causes a Function to lose track of any pending (outstanding Non-Posted and UIO) transactions. Depending upon the specific implementation of the RC-integrated PCI Function, if software issues an FLR while there are pending transactions, there is a possibility for data corruption as described in the “[Avoiding Data Corruption From Stale Completions](#)” Implementation Note.

To avoid potential issues with Root Complex implementations where Stale Completions are possible or a Discard Timer is present, it is recommended that software use an algorithm similar to the following:

1. Software that's performing the FLR synchronizes with other software that might potentially access the Function directly, and ensures that such accesses will not occur during this algorithm.
2. Software clears the entire [Command register](#), disabling the Function from mastering any new transactions.
3. Software polls the Transactions Pending bit in the AF Status Register either until it's clear or until it's been long enough to achieve reasonable certainty that any remaining outstanding Transactions will never complete. On many systems, the Transactions Pending bit will usually clear within a few milliseconds, so software might choose to poll during this initial period using a tight software loop. On rare cases when the Transactions Pending bit doesn't clear by this time, software will need to poll for a longer system-specific period (potentially seconds), so software might choose to conduct this polling using a timer-based interrupt polling mechanism.
4. Software initiates the [FLR](#).
5. Software waits 100 ms.
6. Software reconfigures the Function and enables it for normal operation.

6.30 Data Object Exchange (DOE) §

Data Object Exchange (DOE) is an optional mechanism for system firmware/software to perform data object exchanges with a Function or RCRB. Software discovers DOE support via the Data Object Exchange (DOE) Extended Capability structure. Because DOE depends on Configuration Requests it is not usable for peer-to-peer operations directly between Functions, although system software can provide a mechanism to relay data objects between Functions if such capabilities are desired. When DOE is implemented in an RCRB, it is permitted to block peer-to-peer operation via implementation specific means.

DOE is a prerequisite Extended Capability for a Function to support in-band access by system firmware/software using Configuration Requests to Component Measurement and Authentication (CMA). CMA in turn builds on [\[SPDM\]](#).

It is permitted to implement DOE in any type of Function, and in an RCRB. It is permitted to implement DOE more than once in a single Function or RCRB.

DOE uses the terms “type” and “feature” with specific meanings. A data object type indicates a group of one or more data objects, and can be used to determine how to further construct or parse a specific data object. Every data object includes an indication of its type. Data object types are defined by particular vendors and distinguished by Vendor ID. It is permitted for features/types defined by one vendor to require the use of features/types defined by another vendor.

A feature is a capability of some sort that uses data objects. A feature can be associated with one or more data object types. Each specific feature defines the mechanism(s) used by software to discover support for the feature, for example

by means of a capabilities bit in Configuration Space, or implied by the presence of an associated Extended Capability structure. A feature using data objects must specify the scope of the specific features in relation to the Function(s) implementing that feature via DOE, for example if only the Function itself is associated with the feature, or if Function 0 of a Multi-Function Device represents the Device as a whole, etc.

It is permitted, but not recommended, for a feature using data object exchanges to require that a Function implement a unique instance of DOE for that specific feature, and/or to allow sharing of a DOE instance to only a specific set of features using data object exchange, and/or to allow a Function to implement multiple instances of DOE supporting the specific feature.

It is permitted to use DOE when a Function is in non-D0 states, although it is permitted for a specific data object feature to restrict operation in non-D0 states.

IMPLEMENTATION NOTE: SUPPORTING MULTIPLE HARDWARE/SOFTWARE BINDINGS §

When multiple features are supported by a device, or when a given feature can be used by more than one software entity at a time, it is typically necessary to coordinate the use of the hardware/software interface between entities implementing the multiple features. SPDm Version 1.2.0 introduced the SPDm connection model, addressing some important needs, but not itself addressing the hardware/software interface problems arising when multiple software entities need to maintain different contexts and/or configurations. DOE provides two basic mechanisms for addressing this type of issue.

One option is that specific data object features may require the use of dedicated instances of DOE, particularly to allow system software to assign ownership of a specific DOE instance with the software entity using the specific data object feature(s) associated with that particular instance. For example, data object features A and B may perform different tasks, and so by instantiating dedicated instances of DOE for each, system software avoids the need to implement ownership control and arbitration mechanisms between A and B.

When this is done, if the underlying hardware uses shared resources to implement A and B, then the hardware may require the ability to maintain separate contexts for each data object feature, because system software will allow the two features to be operated at the same time.

Alternatively, for CMA/SPDM and related use cases, the optional Connection ID mechanism can eliminate the need for multiple DOE instances. Typically, the Connection ID mechanism provides much better scaling, and it is recommended that the Connection ID mechanism be used where applicable, instead of requiring the use of dedicated DOE instances. § Figure 6-38 illustrates a “stack diagram” where six applications are all using CMA/SPDM over DOE in three groups.

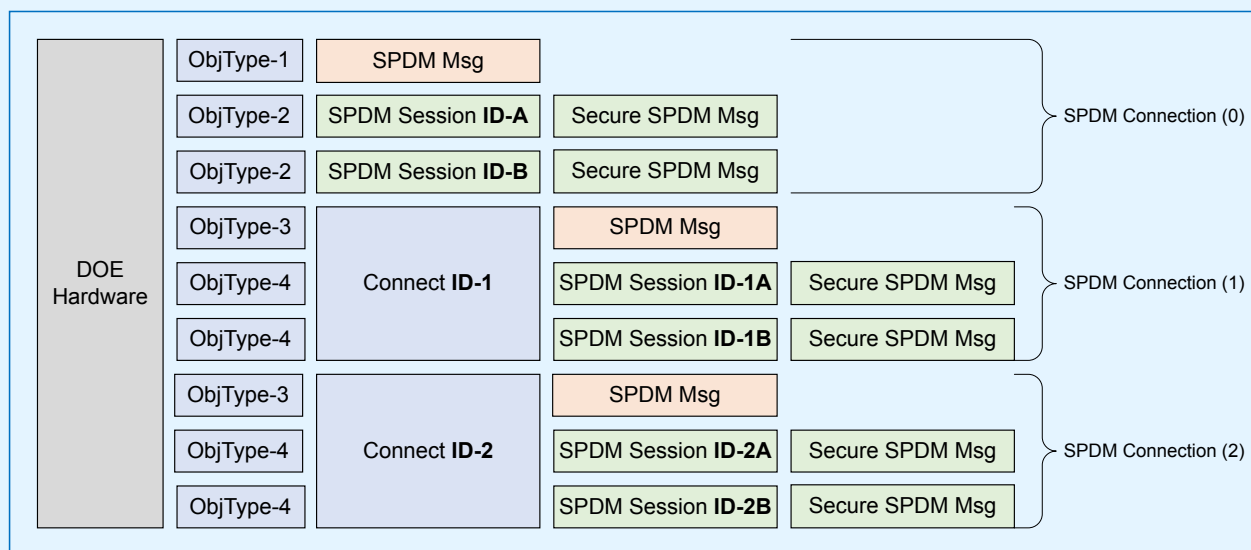


Figure 6-38 Stack Diagram Illustration of Multiple Sessions and Connections §

In each case, two distinct SPDm uses are shown, A and B, and within each group, the SPDm configuration must be the same, but each group may have distinct requirements from the others, e.g., different hashing algorithms. The SPDm Driver and PCI Driver software components must coordinate the establishment of each connection, including the assignment of a Connection ID, and, at run time, must manage the transfer of data objects across

the DOE interface itself to ensure that multiple software entities cannot attempt to access the DOE Mailbox registers during the transfer of a single data object. However, context is maintained in the device for each session. § Figure 6-39 illustrates at a high level how these elements relate to each other.

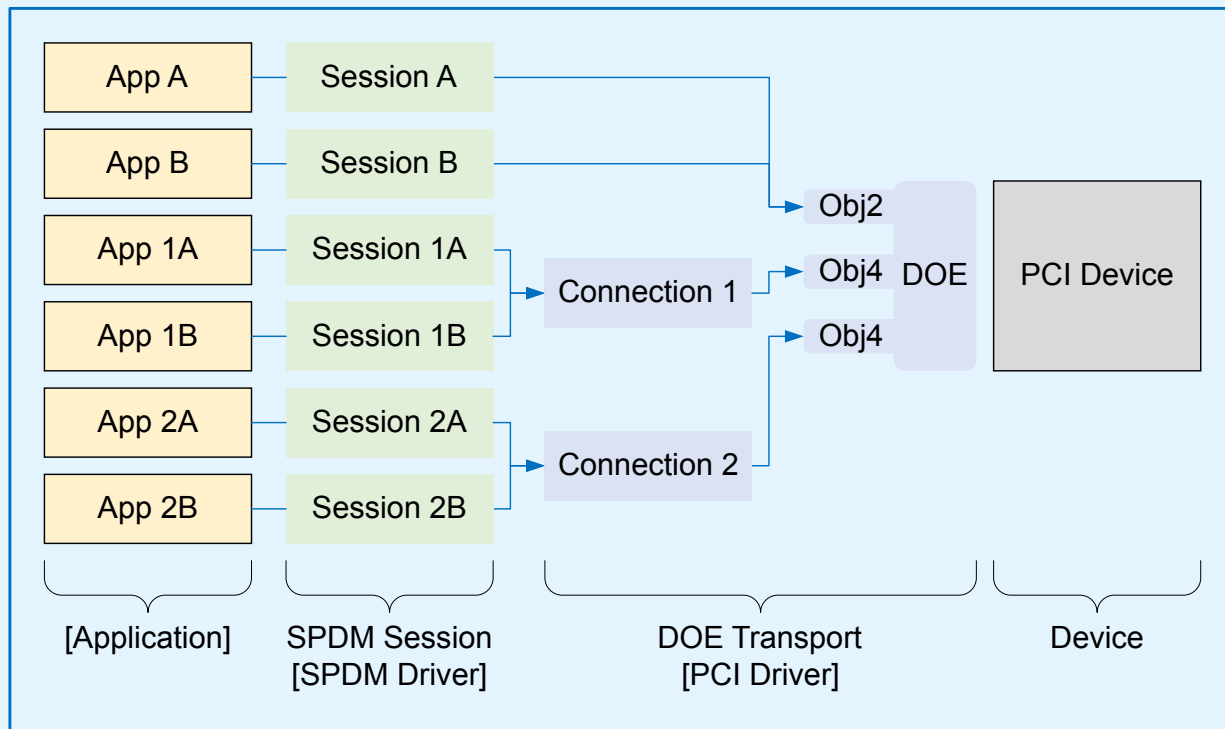


Figure 6-39 Example Showing Relationships of Software and Hardware Elements §

6.30.1 Data Objects and Features §

Data objects must consist of 2 DW to 256K DW, as shown in § Figure 6-40.

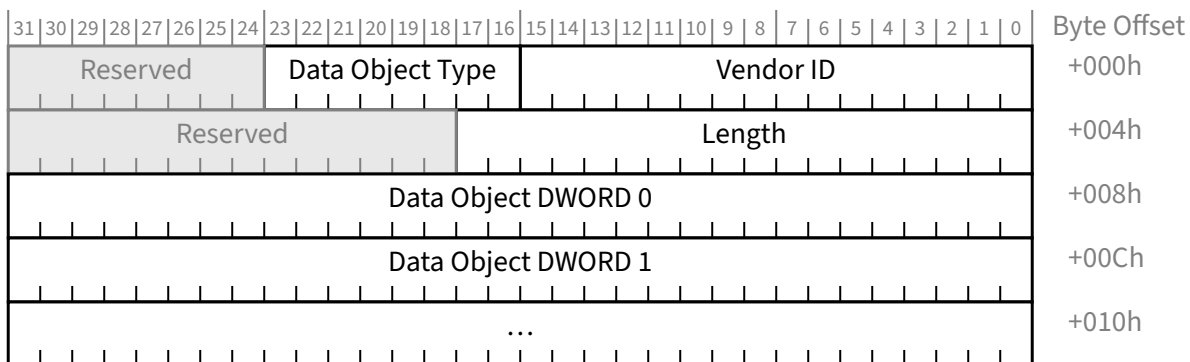


Figure 6-40 DOE Data Object Format §

The first DW of a data object must be formatted as defined in § Table 6-28 and illustrated in § Figure 6-41.

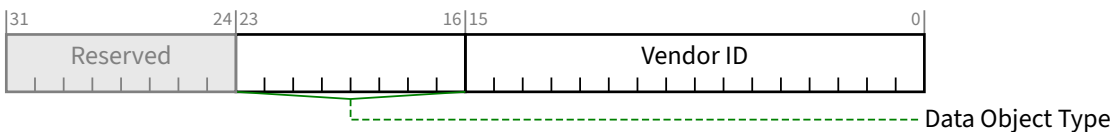


Figure 6-41 DOE Data Object Header 1 §

Table 6-28 DOE Data Object Header 1 §

Bit Location	Description
15:0	Vendor ID - PCI-SIG Vendor ID of the entity that defined the type of data object.
23:16	Data Object Type - The type of data object.

The Second DW of a data object must be formatted as defined in § Table 6-29 and illustrated in § Figure 6-42.

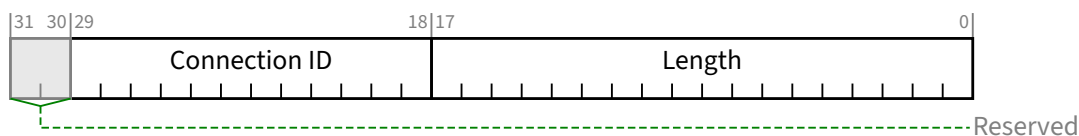


Figure 6-42 DOE Data Object Header 2 §

Table 6-29 DOE Data Object Header 2 §

Bit Location	Description
17:0	Length - Length of the data object being transferred in number of DWs including the 2 DWs of the Data Object Header, encoded such that a value of 0 0002h indicates 2 DWs, and a value of 0 0000h indicates 2 ¹⁸ DWs. A value of 0 0001h is not allowed.
29:18	Connection ID - For data object types using Connection ID, this field contains the Connection ID. For all other data object types, this field is Reserved.

Each data object is uniquely identified by the Vendor ID of the vendor publishing the data object definition and a Data Object Type value assigned by that vendor (see § Table 6-28). See § Table 6-33 for a list of data objects defined by PCI-SIG. Data objects of a specific type are permitted to vary in size and composition.

Unless a data object type definition specifies a different requirement, the following rules apply:

- If the number of DW transferred does not match the Length indicated in DOE Data Object Header 2 for a data object, then the entire data object must be silently discarded.
- If the Length indicated in DOE Data Object Header 2 is shorter than expected for a specific data object, then the data object must be silently discarded.

- If the Length indicated in DOE Data Object Header 2 is greater than expected for a specific data object, then the portion of the data object up to the expected length must be processed normally and the remainder of the data object must be silently discarded.
- If the data object type is not supported, then that data object must be silently discarded.

6.30.1.1 DOE Discovery Feature §

The DOE Discovery feature must be implemented, and provides a means for software to discover the data object types supported by a DOE instance. The DOE Discovery type consists of the request and response data objects, with the 3rd DW of the data object content as defined in § Table 6-30 and § Table 6-31 respectively. Where indicated in § Table 6-33, response data objects include a 4th DW, as defined in § Table 6-32. The DOE Discovery data object feature must be operable in D0, D1, D2 and D3_{hot}.

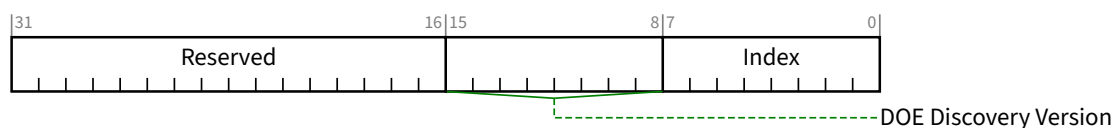


Figure 6-43 DOE Discovery Request Data Object Contents (3rd DW) §

Table 6-30 DOE Discovery Request Data Object Contents (3rd DW) §

Bit Location	Description
7:0	Index – Indicates DOE Discovery entry index queried. Indices must start at 00h and increase monotonically by 1.
15:8	DOE Discovery Version – must be 02h if the Capability Version in the <u>Data Object Exchange Extended Capability</u> is 02h or greater.
31:16	Reserved Reserved - Requesters must place 0000h in this field. Responders must ignore the value in this field.



Figure 6-44 DOE Discovery Response Data Object Contents (3rd DW) §

Table 6-31 DOE Discovery Response Data Object Contents (3rd DW) §

Bit Location	Description
15:0	Vendor ID – PCI-SIG Vendor ID of the entity that defined the type of data object. FFFFh if index is invalid or out of range.
23:16	Data Object Type – Indicates the identity of the data object type associated with the Index value supplied with the DOE Discovery Request.

Bit Location	Description
	The PCI-SIG defined data object type for DOE Discovery must be implemented at index 00h. The index values used for other data object types is implementation specific and has no meaning defined by this specification. Undefined if Vendor ID value is FFFFh.
31:24	Next Index – Indicates the next DOE Discovery Index value. If the responding DOE instance supports entries with indices greater than the index indicated in the received DOE Discovery Request, it must increment the queried index by 1 and return the resulting value in this field. Must be 00h to indicate the final entry. Undefined if Vendor ID value is FFFFh.



Figure 6-45 DOE Discovery Response Data Object Contents (4th DW) §

Table 6-32 DOE Discovery Response Data Object Contents (4th DW) §

Bit Location	Description
17:0	Max DO Length – Maximum length supported for this data object type, expressed as a number of DWs including the 2 DWs of the Data Object Header, encoded such that a value of 0 0002h indicates 2 DWs, and a value of 0 0000h indicates 2^{18} DWs. A value of 0 0001h is not permitted. Undefined if Vendor ID value is FFFFh.
31:18	Additional Info – The contents of this field are Data Object Type specific. Values are defined for each Data Object Type in § Table 6-33. Undefined if Vendor ID value is FFFFh.

PCI-SIG defined data object types are defined in § Table 6-33.

Table 6-33 PCI-SIG Defined Data Object Types (Vendor ID = 0001h) §

Vendor ID (h)	Data Object Type (h)	Description
0001	00	DOE Discovery – Every DOE instance must support this data object feature and associated type, both indicated by this entry. A requester uses DOE Discovery to discover all other Data Object types supported by the DOE instance. Must not include the <u>DOE Discovery Response Data Object Contents (4th DW)</u>.
0001	01	CMA-SPDM – Data object type for SPDM messages. See § Section 6.31 .

Vendor ID (h)	Data Object Type (h)	Description
		Permitted to include the <u>DOE Discovery Response Data Object Contents (4th DW)</u> . The <u>Additional Info</u> field is Reserved.
0001	02	Secured CMA-SPDM – Data object type for SPDM secure sessions. See § Section 6.31.4 If this type is supported, then the CMA/SPDM (01h) type must also be supported. Permitted to include the <u>DOE Discovery Response Data Object Contents (4th DW)</u>. The <u>Additional Info</u> field is Reserved.
0001	03	CMA/SPDM with Connection ID – Data object type for SPDM messages with Connection ID. If this type is supported, then the CMA/SPDM (01h) type must also be supported. Required to include the DOE Discovery Response Data Object Contents 4th DW. Bits 11:0 of the Additional Info field must indicate the largest supported Connection ID value, such that all Connection IDs must be in the range from 0 to the indicated value: 0000 0000 0000b 1 Connection ID ... 1111 1111 1111b 4096 Connection IDs Bits 13:12 of the <u>Additional Info</u> field are Reserved.
0001	04	Secured CMA/SPDM with Connection ID – Data object type for SPDM secure sessions with Connection ID. If this type is supported, then the CMA/SPDM (01h) and CMA/SPDM with Connection ID (03h) types must also be supported. Required to include the DOE Discovery Response Data Object Contents 4th DW. Bits 11:0 of the Additional Info field must indicate the largest supported Connection ID value, such that all Connection IDs must be in the range from 0 to the indicated value: 0000 0000 0000b 1 Connection ID ... 1111 1111 1111b 4096 Connection IDs Bits 13:12 of the <u>Additional Info</u> field are Reserved.
0001	05	Async Message – Data object type for messages generated asynchronously by the device. Required to include the DOE Discovery Response Data Object Contents 4th DW.
0001	06 to FF	Reserved

6.30.1.2 DOE Async Message §

The DOE Async Message data object feature allows a DOE instance to transfer device-initiated messages with system software, enabling the device to act as a requester in cases where otherwise the host would act as requester. The DOE Async Message data object type consists of 1 DW that indicates the contents, defined in § Table 6-34, which, for Async

Message REQUEST (01h) or Async Message RESPONSE (02h), must be followed by an encapsulated DOE data object. The encapsulated data object must be a full data object as illustrated in § Figure 6-40.

Table 6-34 DOE Async Message Data Object Contents (1 DW) §

Bit Location	Description
7:0	Variety – Indicates Async Message variety:
	00h Async Message GET
	01h Async Message REQUEST
	02h Async Message RESPONSE
	03h Async Message DONE
	04-FFh Reserved
31:8	Reserved – Requesters must place 00 0000h in this field. Responders must ignore the value in this field.

Each data object type definition must specify if that type is permitted to use the Async Message mechanism.

The following data object types are permitted in the encapsulated data object:

- CMA/SPDM
- Secure CMA/SPDM
- CMA/SPDM with Connection ID
- Secure CMA/SPDM with Connection ID

An Async Message is not permitted in the encapsulated data object.

A DOE instance advertises support for sending DOE Async Messages by having the DOE Async Message Support bit Set. A DOE instance is only allowed to send DOE Async Messages, when the DOE Async Message Enable bit is Set.

For a DOE instance to send a DOE Async Message to the host, it must Set DOE Async Message Status and wait to receive a DOE Async Message GET from the host.

System software supporting DOE Async Message must check the DOE Async Message Status bit, and, when that bit is Set and the system software is ready to process the message, then, the host must send an Async Message GET. The DOE instance must in response return the Async Message REQUEST. System software then, in turn, must send Async Message RESPONSE. If the DOE instance has more Async Messages to transfer, then the device must return another Async Message REQUEST, to which system software must return a corresponding Async Message RESPONSE. If the DOE instance has no more Async Messages to send, it must return Async Message DONE and Clear DOE Async Message Status.

A DOE instance must only return Async Message REQUEST in response to a received Async Message GET. It is permitted but not required for system software to abort (as defined in § Section 6.30.1) other data object transfers to give priority to Async Message transfers.

6.30.2 Operation §

Other than the DOE Discovery feature, DOE is used to transport data objects as part of a feature defined outside of DOE itself. This section defines requirements of DOE itself, although in some cases these requirements are permitted to be modified per the definition of a specific feature.

For features that define request/response protocols, unless there is a feature-specific requirement, a DOE instance must complete processing a received data object and, if a data object is required in response, must generate the response and Set the Data Object Ready bit in the DOE Status register within 1 second after the DOE Go bit was Set in the DOE Control

Register, otherwise the DOE instance must Set the DOE Error bit in the DOE Status register within the same time limit. At any time, the system firmware/software is permitted to set the DOE Abort bit in the DOE Control Register, and the DOE instance must Clear the Data Object Ready bit, if not already Clear, and Clear the DOE Error bit, if already Set, in the DOE Status Register, within 1 second.

Once the transfer of a specific data object has been started, except in case of error or abort, that transfer must be completed before the transfer of another data object is started.

If a single DOE instance supports multiple data object features, system firmware/software is permitted to interleave data objects associated with different data object features, if and only if the data object features allow such interleaving.

Data object buffering requirements are determined by the data object feature(s) supported, and data object features must ensure that maximum data object sizes are well defined. When required, the Max DO Length must be used to indicate the maximum length supported for a specific data object type. All data object types that are defined to require the use of Capability Version 02h or greater in the Data Object Exchange Extended Capability must require the use of Max DO Length.

It is strongly recommended that implementations ensure that the functionality of the DOE Abort bit is resilient, including that DOE Abort functionality is maintained even in cases where device firmware is malfunctioning.

An FLR to a Function must result in the aborting of any DOE transfer in progress. Data object features must specify the handling of FLR and, as appropriate, other conditions that impact the data object feature. While an FLR is in-progress, all writes to the DOE Write Data Mailbox must be dropped, and all reads to the DOE Read Data Mailbox must be terminated with UR, or return all 0's. If these or other erroneous actions are detected while an FLR is in progress, it is permitted that, upon exit from FLR, the DOE Error bit be Set.

If software permits DOE operations to overlap with potential FLRs, software cannot depend on the RRS mechanism, but rather must use time-based mechanisms or FRS to determine the completion of an FLR.

It is not required that FLR result in any type of reset to the internal processing engine for DOE operations, although such behavior is permitted, and may be required or forbidden by specific data object features.

DOE errors cover errors in the operation of DOE itself, and, except as noted below, do not extend to errors associated with a data object feature. Any of the following events must result in the DOE Error bit being Set:

- A Poisoned Configuration Write to any of the DOE registers
- Overflow of the Write Data Mailbox mechanism
- Optionally, if the associated data object feature does not provide an alternate mechanism for reporting such errors, the transfer of a data object that is shorter than the minimum expected length of that data object

No response must be generated when the DOE Error bit is Set because of a condition associated with the receipt of a data object that would, in a non-error condition, have an associated data object response, no response must be generated.

Hardware behavior is undefined unless software issues writes to the DOE Write Data Mailbox Register and reads from the DOE Read Data Mailbox Register with all Bytes enabled. It is permitted, but not required, for hardware to check for accesses without all Bytes enabled, and if checked, it is strongly recommended that violations result in the DOE Error bit being Set.

The optional DOE Attention Mechanism supports improved power management of DOE implementations, by enabling a DOE instance to temporarily enter an unresponsive state, and providing software a mechanism to direct the instance back to a responsive state. If the DOE Attention Mechanism Support bit is Set, for backwards compatibility, the default is that the DOE instance must remain in a responsive state. System software is recommended to Set the DOE Attention Not Needed bit when it is acceptable for the DOE instance to enter and stay in a state where it is not immediately available for use. When that bit is Clear, the DOE instance must remain in a state where it is ready to respond in a timely way to system software. The DOE At Attention bit, when Set, indicates the DOE interface is presently in a state of readiness. This bit must only be Cleared if the DOE Attention Not Needed bit in the DOE Control Register is Set. It is permitted for this bit to remain Clear for up to 50 ms following the Clearing of DOE Attention Not Needed.

IMPLEMENTATION NOTE:

EXCHANGE OF DATA OBJECTS §

Exchange of Data Objects Data objects are exchanged through the mailboxes provided by the Data Object Exchange (DOE) Extended Capability. The DOE mailbox is defined to flexibly support a variety of data objects, and as a result of the definition of specific data objects and their associated features, it is necessary to provide the information required for requesters and responders to appropriately size their data buffers and robustly implement the associated feature.

At the level of individual DW transfers, the DOE responder can use the Completions for the DOE mailbox Configuration Read and Write operations as a flow control mechanism. It should be understood by hardware designers, however, that delaying Configuration Completions will typically stall the software operating DOE, and in some cases may also cause stalls in other software/hardware operations.

The DOE Busy bit can be used to indicate that the DOE responder is temporarily unable to accept a data object. It is necessary for a DOE requester to ensure that individual data object transfers are completed, and that a request/response contract is completed, for example using a mutex mechanism to block other conflicting traffic for cases where such conflicts are possible. The following example shows how system firmware/software transfers a request from system firmware/software to a DOE instance, and the response back to system firmware/software from the DOE instance:

1. System firmware/software checks that the DOE Busy bit is Clear to ensure that the DOE instance is ready to receive a DOE request.
2. System firmware/software writes the entire data object, starting with the first DWORD, a DWORD at a time via the DOE Write Data Mailbox Register.
3. System firmware/software writes 1b to the DOE Go bit.
4. The DOE instance consumes the DOE request from the DOE mailbox.
5. The DOE instance generates a DOE Response and Sets the Data Object Ready bit and generates a DOE Software notification, if supported and enabled.
6. System firmware/software waits for an interrupt if applicable, checks/polls the Data Object Ready bit and, provided it is Set, reads data from the DOE Read Data Mailbox Register and then writes any value to the DOE Read Data Mailbox Register to indicate a successful read, starting with the first DWORD, a DWORD at a time until the entire DOE Response is read.

The above example does not illustrate error handling or additional software mechanisms necessary to manage cases where more than one software entity could potentially attempt to use DOE.

6.30.3 Interrupt Generation §

A DOE instance is permitted to support the generation of DOE interrupts, as indicated by the DOE Interrupt Support bit in the DOE Capabilities Register. If DOE interrupts are supported, the DOE instance must support MSI and/or MSI-X. INTx interrupt signaling is not permitted with DOE. DOE interrupts are enabled by the DOE Interrupt Enable bit in the DOE Control Register. DOE interrupts are indicated by the DOE Interrupt Status bit in the DOE Status register.

If enabled (see § Section 6.1.4.3), an interrupt message must be triggered when the associated vector is unmasked (see § Section 6.1.4.5 Per-vector Masking and Function Masking), the DOE Interrupt Enable bit is Set, and the value of the DOE Interrupt Status bit transitions from 0b to 1b.

The interrupt message will use the vector indicated by the DOE Interrupt Message Number field in the DOE Capabilities Register. Multiple DOE instances in the same Function or RCRB are permitted to use the same interrupt vector.

6.31 Component Measurement and Authentication (CMA-SPDM) §

Component Measurement and Authentication/SPDM (CMA-SPDM) defines optional security features based on the adaptation of the data objects and underlying protocol defined in [SPDM]. These provide mechanisms to perform security exchanges (where this term is used generically to refer to all defined capabilities of [SPDM]) with a component, or Device/Function. It is intended that CMA-SPDM inherit all new capabilities of [SPDM] as that specification is enhanced, and identifiable by the versioning mechanisms defined for [SPDM]. CMA-SPDM is part of a layered architecture intended to support a consistent and structured approach to security (see § Figure 6-46).

As security requirements evolve, and sometimes older technologies become compromised, for example by new types of attacks that had not been foreseen, so that they are no longer considered secure, CMA-SPDM is intended to balance the need for maintaining sufficient security against the sometimes conflicting goal of maximizing interoperability. Backwards compatibility will be maintained wherever possible, but this may not be possible in all cases.

It must also be understood that many aspects of security, including the establishment of policies, and the evaluation of specific trust decisions, very often cannot be made locally to a specific device or platform, and so are necessarily outside the scope of this specification. CMA-SPDM should be understood as a tool that provides a common framework for some key elements of security.

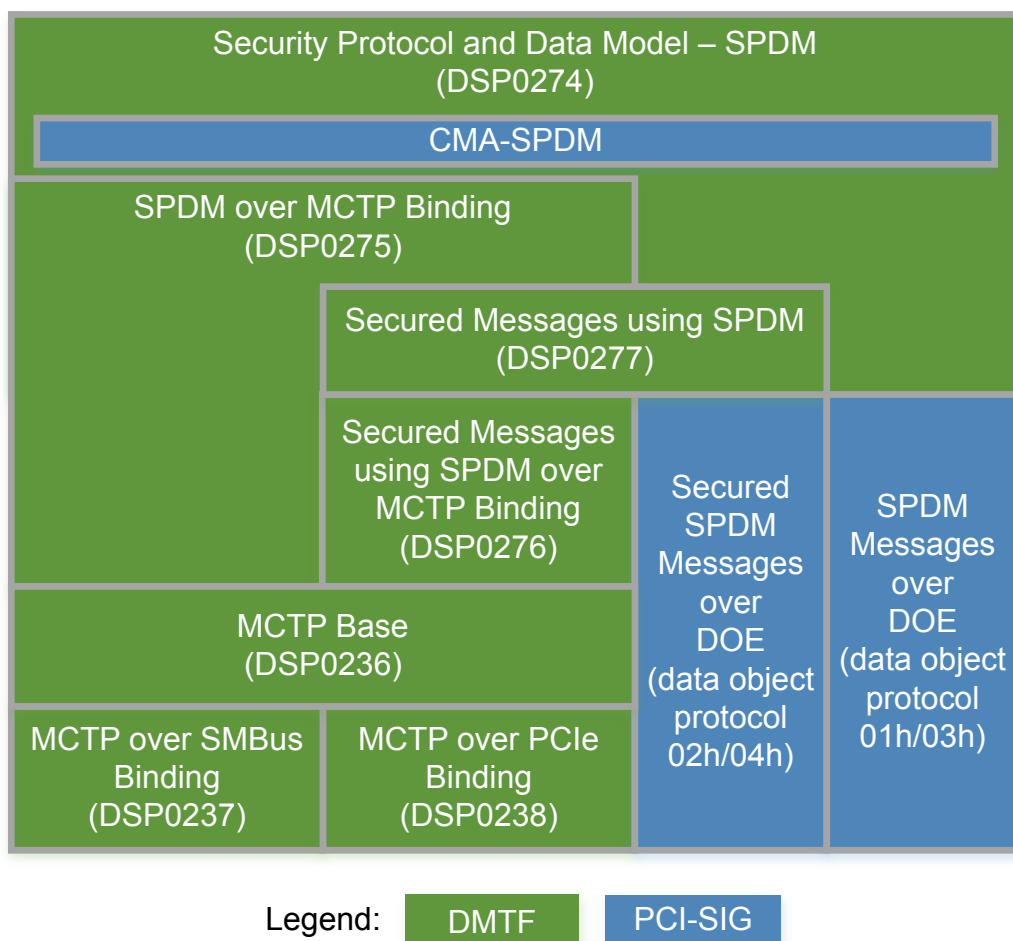


Figure 6-46 CMA-SPDM as Part of a Layered Architecture §

CMA-SPDM can use the Data Object Exchange (DOE) Capability (See § Section 6.30) as a mechanism for security exchanges, and can also be used for out-of-band or other in-band security exchanges, for example using MCTP [SPDM-MCTP] Binding.

It is possible to use CMA-SPDM in many scenarios—for example:

- Remote system administrators can dynamically generate a manifest of cryptographic identities of components of a system, especially at the level of removable units (e.g., add-in-cards/modules), without physical examination of the system, via a BMC or other platform root-of-trust.
- The identity of a Function can be verified by OS drivers before assigning resources to the Function during runtime/hot-plug scenarios without requiring a host reboot.
- Virtual Machine Monitors (VMMs) or TEE Security Managers (TSMs – see § Chapter 11.) can establish the hardware and firmware identities of a Function before assigning it to a Virtual Machine, and provide a service such to enable these to be confirmed by the Virtual Machine guest directly with the assigned Function.

The high-level overview of the CMA-SPDM security features and their associated PCIe-specific requirements are given in the following sections, whereas the foundational architecture, protocol and message definitions for the security

exchanges used by CMA-SPDM are defined in [SPDM]. The messages exchanged between the requester and a responder for the CMA-SPDM security features are denoted as CMA-SPDM Messages.

IMPLEMENTATION NOTE: UNDERSTANDING AND IMPLEMENTING CMA-SPDM §

CMA-SPDM is part of a layered architecture to support device and platform security (see § Figure 6-46). Building on the DMTF Security Protocol & Data Model specification [SPDM], CMA-SPDM provides a “mapping” of that foundation, with the intent that future [SPDM] enhancements can, in most cases, be implemented without requiring modifications to CMA-SPDM. In addition to device authentication & firmware measurement, capabilities such as mutual authentication and cryptographic key exchange are also possible. Following [SPDM], CMA-SPDM uses the term “requester” to refer to agents initiating CMA-SPDM protocol requests, and “responder” to refer to an agent that ultimately services those requests and generates responses.

Depending on the use models supported, it may be desirable to implement support for more than one way of transporting CMA-SPDM requests and responses.

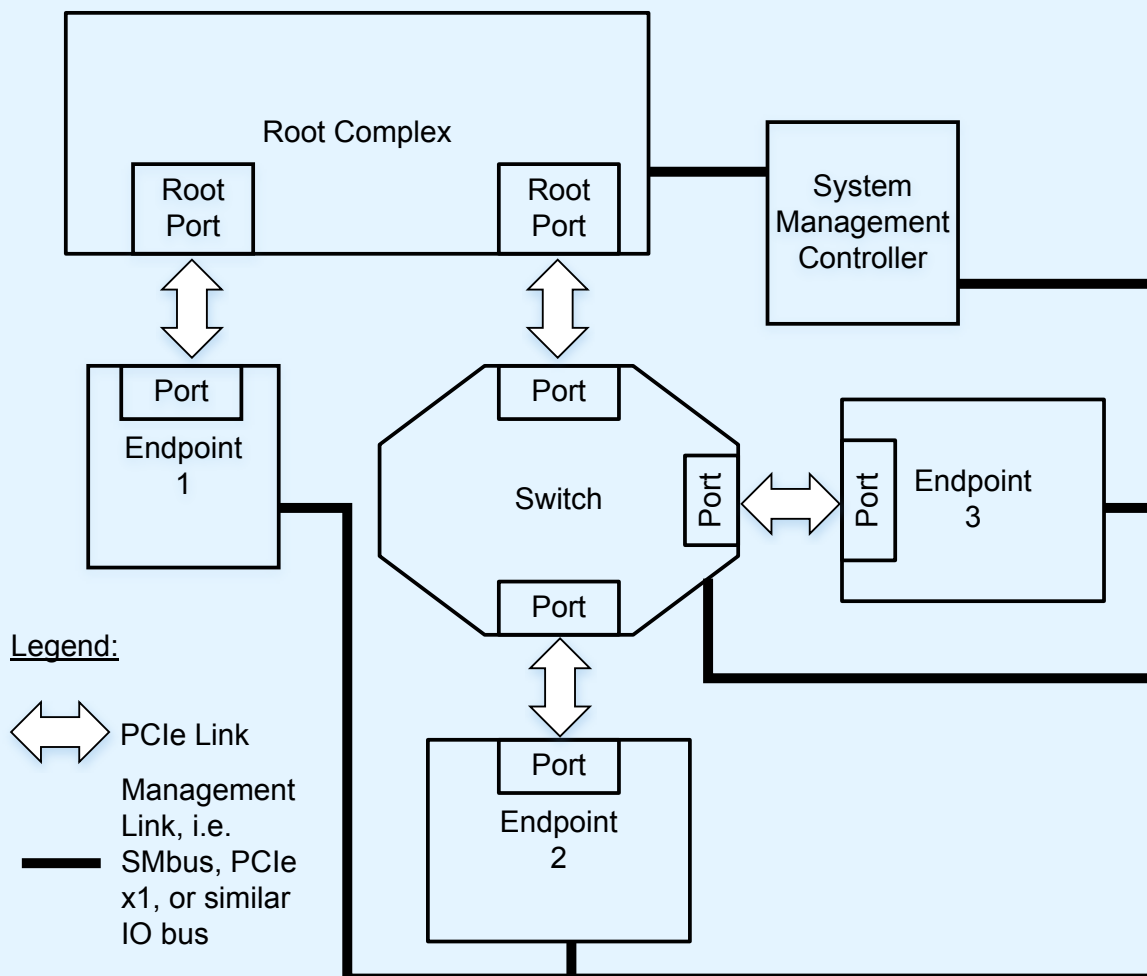


Figure 6-47 Example System Showing Multiple Access Mechanisms §

§ Figure 6-48 shows an example of a device that supports multiple platform use models and multiple access mechanisms. In this example, there is a System Management Controller that has an out-of-band connection to the other elements in the platform, enabling it to use CMA-SPDM even when the PCIe Links are not active and/or Fundamental Reset is active. When the PCIe Links are operating, CMA-SPDM can be used via [SPDM-MCTP]. The Root Complex can use CMA-SPDM over DOE. It is a platform implementation choice of which methods are used.

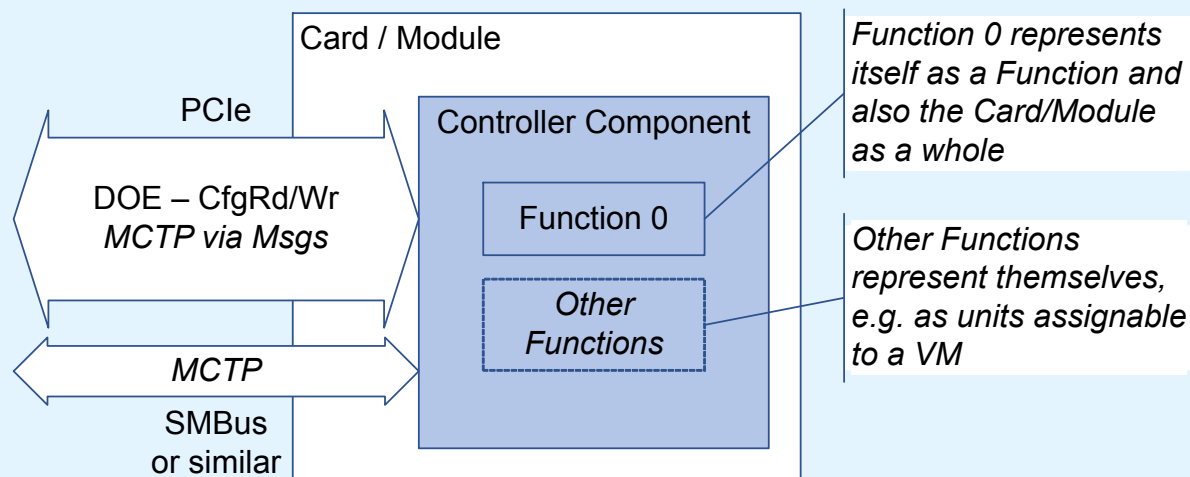


Figure 6-48 Example Add-In-Card Supporting CMA-SPDM §

Correspondingly, a device controller can support CMA-SPDM in multiple ways, depending on the associated form-factor and platform requirements and implementation choices. For maximum flexibility it may be desirable to support all of the following:

- Out-of-band (e.g., over SMBus, I2C, I3C, etc.) using MCTP, according to the specifications for [SPDM] + CMA-SPDM + [SPDM-MCTP]
- In-band using MCTP encapsulated in PCIe Messages, according to the specifications for [SPDM] + CMA-SPDM + [SPDM-MCTP] + [MCTP-VDM]
- In-band using the Configuration mailbox mechanism defined by the DOE Extended Capability, according to the specifications for [SPDM] + CMA-SPDM + DOE

Or, when the requirements for a specific implementation do not require this level of flexibility, it may be preferable to support only one or two of these approaches.

The platform requirements for CMA-SPDM are determined by the use cases supported. Example use cases that apply to an add-in-card as a unit include:

- Before platform boot, a management controller checks the add-in-card out of band.
- During platform boot, system firmware checks the add-in card in-band via DOE.
- During run-time, a management controller or system software/firmware checks the add-in-card via [MCTP-VDM].

To support use cases where the management controller and system firmware/software communicate the results of their checks with each other, it is necessary that the results of those checks be consistent, subject to any

changes applied since earlier checks were made. It is for this use case that Function 0 is required to match identically the results received via these other mechanisms.

For use cases where the add-in-card is being evaluated as a unit, the device controller can, through implementation specific means, attest and/or measure other board elements, to ensure that the identity and integrity of the add-in-card as a whole is correct. How this can be done is outside the scope of CMA-SPDM.

For improved interoperability, CMA-SPDM requires support for certain algorithms, while allowing support for any of the algorithms supported by [SPDM]. Flexibility is allowed for Responder algorithm selection with the intent that device vendors can, if desired, align their choices with common standards such as those defined in the Commercial National Security Algorithm (CNSA) Suite and in NIST Special Publication 800-57.

The applications of CMA/SPDM are numerous and varied. For complex environments, the issues of secure identity provisioning, measurement collection/reporting, and verification of attested state may require evaluating additional standards and industry best practices. Examples include:

- TCG Reference Integrity Manifest (RIM) Information Model: <https://trustedcomputinggroup.org/resource/tcg-reference-integrity-manifest-rim-information-model>
- OCP Attestation of System Components v1.0 Requirements and Recommendations: <https://www.opencompute.org/documents/attestation-v1-0-20201104-pdf>

IMPLEMENTATION NOTE:

OVERVIEW OF THREAT MODEL §

A detailed threat model analysis typically requires consideration of the context in which a system is operating and the composition of the system along with many other factors, and as such cannot be provided here. A high level overview of the types of threats for which CMA-SPDM may be applicable includes:

- Remote and Local software-based attacks, e.g., to install corrupted device firmware or roll back device firmware to an older version
- Threats from attacker in physical possession of the device including:
 - Software-based (similar to above)
 - Presentation (“impersonation”)
 - Hardware attacks of various sorts
- Attacks during manufacturing, provisioning or maintenance, including:
 - Provisioning of improper configuration and/or firmware
 - Improper “repair” of a module

CMA-SPDM requires the leaf certificate to include the information typically used by system software for device driver binding. This requirement is intended to support scenarios where an attacker device attempts to gain access to system resources by appearing to be a valid device type. Responding devices can include the device serial number value in the leaf certificate to simplify system implementation of policies that require specific unit instances to be identified, for example to support scenarios where a modified unit is substituted for a valid, but otherwise identical, unit.

6.31.1 Removed §

6.31.2 Removed §

6.31.3 CMA-SPDM Rules §

CMA-SPDM defines how the responder role as defined in [SPDM] must be implemented for PCIe devices, regardless of the communication path(s) implemented between the requester(s) and the responder.

It is permitted, but not required, to support CMA-SPDM and the responder role as defined in [SPDM] at the level of individual Functions. When this is done, then the Function(s) must implement both CMA-SPDM and DOE. For a Multi-Function Device, each Function implementing the responder role must implement CMA-SPDM and DOE. For Switches that support CMA-SPDM, each Switch Upstream Port Function and Downstream Port Function implementing the responder role must implement CMA-SPDM and DOE.

It is permitted, but not required, for CMA-SPDM to be implemented using one or more access mechanisms other than DOE, in support of various use models (e.g. see § Figure 6-46). When a use model requires CMA-SPDM to be applied at the level of a replaceable unit it may be necessary to support security exchanges operated by means other than DOE. For example, an add-in-card being evaluated by a Baseboard Management Controller (BMC), or similar element, may use MCTP over a sideband bus. For devices that implement such support, the certificate chain in slot 0 (referring to slots as defined in [SPDM]) must match identically the certificate chain in slot 0 returned by Function 0 via DOE, if DOE is also supported.

CMA-SPDM does not apply to Root Ports, and a Root Port must not implement CMA-SPDM.

The value of all measurements must always reflect the firmware in use at the time of the measurement being read, including for components that support runtime update of firmware without a system reset.

When using CMA-SPDM with DOE:

- The instance of DOE used for CMA-SPDM must support:
 - the DOE Discovery data object protocol,
 - the CMA-SPDM data object protocol,
 - if IDE is supported, the IDE_KM data object protocol using Secured CMA-SPDM (See § Section 6.31.4),
 - and no other data object protocol(s).
- A responder must support operation when the associated Function is in the D0 state, and is permitted but not required to support operation in non-D0 states.
- Behavior is undefined if a Function that does not support operation in non-D0 states is transitioned into a non-D0 state during a CMA-SPDM protocol operation.
 - It is strongly recommended that system software avoid placing a Function into a non-D0 state while a CMA-SPDM protocol operation is taking place.
- An FLR to a Function during the processing of a CMA-SPDM request must result in that Function terminating its processing of the request, and that Function not returning a response to the request.
 - An FLR to a Function during the DOE transfer of a CMA-SPDM request or response data object must follow the rules defined in § Section 6.30 .

When using CMA-SPDM with a transport mechanism other than DOE:

- For the CMA-SPDM data object type or SPDM with connection ID data object type, the SPDM message payloads must start from “Data Object DW 0”. The SPDM message payloads must follow [SPDM] specification Generic SPDM message field definitions, starting with SPDM version, Request Response Code, Param 1 and Param 2. The Byte mapping of SPDM Messages is shown in § Figure 6-49. If required, SPDM Message payloads must be padded with 0’s to maintain DW alignment, when using DOE.



Without the connection ID, one DOE instance can only have one SPDM connection. With the connection ID, one DOE instance can have multiple SPDM connections. The maximum valid connection ID can be discovered by using DOE Discovery Data Object Protocol.

Some components provide a debug mode where a debugger is granted access to hardware security properties, allowing the debugger to influence the measurement process itself. It is strongly recommended that components report when a debug mode is active. The reporting mechanism is outside the scope of this specification. It can be SPDMA debug and device mode in measurement record or DICE operation flags in DiceTcbInfo.

- For BaseAsymAlgo, Requesters must support all, and Responders must support one or more of the following:
 - TPM_ALG_RSASSA_3072
 - TPM_ALG_ECDSA_ECC_NIST_P256
 - TPM_ALG_ECDSA_ECC_NIST_P384
- For BaseHashAlgo, Requesters must support all, and Responders must support one or both of the following:
 - TPM_ALG_SHA_256
 - TPM_ALG_SHA_384
- For MeasurementSpecification, Requesters and Responders must support the following:
 - DMTF measurement specification format
- Requesters and responders must, for MeasurementHashAlgo, Requesters must support all, and Responders must support one or both of the following:

- TPM_ALG_SHA_256
- TPM_ALG_SHA_384
- It is permitted for Requesters and Responders to support additional algorithms defined for [SPDM] beyond those required.
- Responders must implement a Cryptographic Timeout (CT), as defined in [SPDM], of not more than 2^{23} μ s.
 - Per [SPDM] CT is in turn indicated through the value of CTExponent.
 - It is strongly recommended that the CT be as short as practical.

6.31.4 Secured CMA-SPDM §

Secured CMA-SPDM provides security for data object protocols based on [SPDM] mechanisms, including the IDE key management (IDE_KM) protocol. Once a secure session has been established per [SPDM] (Revision 1.1 or later), it is permitted, and for some uses required, to use Secured CMA-SPDM to transfer other Data Objects with integrity/encryption, using the algorithm negotiated in the SPDM session establishment, per [Secured SPDM].

It is permitted to continue to perform non-secured CMA-SPDM operations after a session has been established, provided the specific use allows non-secure transport.

Secured CMA-SPDM data objects must be formatted per [Secure-SPDM]. It is permitted for specific data object protocols to constrain the use and content of optional fields, but if no such constraints are applied then the use of such fields is implementation specific.

An FLR to a Function for which there is an established secure session must not change the state of the secure session. However, as with non-secured CMA-SPDM, an FLR to a Function during the processing of a CMA-SPDM request must result in that Function terminating its processing of the request, and that Function not returning a response to the request, and this may impact the usability of the secure session and possibly render the secure session unusable.

For the Secured CMA-SPDM data object type or Secured CMA-SPDM with connection ID data object type, the Secured CMA-SPDM message payloads must start from “Data Object DW 0”. The Secured CMA-SPDM message payloads must follow the Secured CMA-SPDM specification Secure Message fields definition, starting with Session ID. “Sequence Number” field must be absent (S=0). “Random Data” field must be absent (R=0). The “Application Data” field must be the in-session SPDM message. The Byte mapping of Secured CMA-SPDM Messages is shown in § Figure 6-50. If required, Secured CMA-SPDM Message payloads must be padded with 0’s to maintain DW alignment, when using DOE.

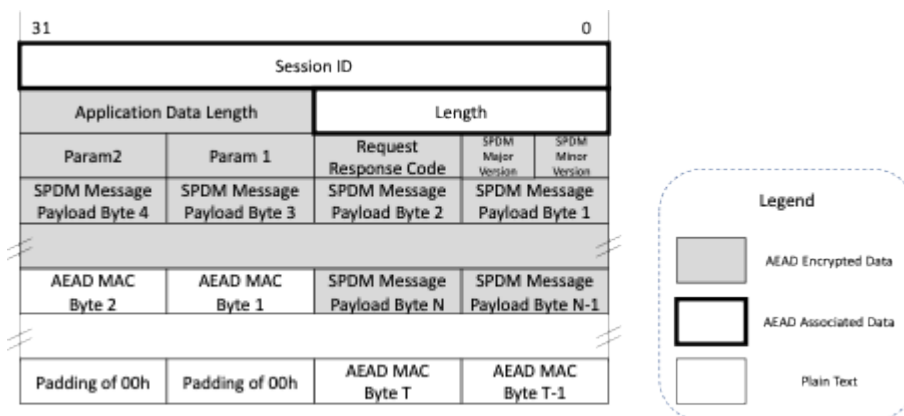


Figure 6-50 Byte Mapping of Secured CMA-SPDM Messages Including Example Payload §

Additional Rules:

- Requesters and Responders must support ENCRYPT_CAP and MAC_CAP.
- For DHE group, Requesters must support all, and Responders must support one or both of the following:
 - secp256r1
 - secp384r1
- For AEAD structure, Requesters and Responders must support the following:
 - AES-256-GCM with 16 Byte MAC
- For KeySchedule structure, Requesters and Responders must support the following:
 - SPDM Key Schedule
- For Secured Message format, Requesters and Responders must support the following:
 - DSP0277 Secured Messages opaque data format as defined in Section 8 of DSP0277, if SPDM 1.1 or below is used.
 - DSP0274 General opaque data format as defined in Section 14 of SPDM 1.2, if SPDM 1.2 or above is used. The OtherParamsSupport field of NEGOTIATE_ALGORITHMS request and ALGORITHMS response must set the bit for OpaqueDataFmt1.
- Requesters and Responders must use DSP0277 specification version as the Secured Message transport binding version (format as defined in Section 6 of DSP0277) required in the OpaqueData fields of KEY_EXCHANGE request / KEY_EXCHANGE_RSP response / PSK_EXCHANGE request / PSK_EXCHANGE_RSP response messages defined in DSP0274, when using DOE.
- Requesters and Responders may support mutual authentication. If mutual authentication is supported, for ReqBaseAsymAlgo, Requesters and Responders must support one or more of the following:
 - TPM_ALG_RSASSA_3072
 - TPM_ALG_ECDSA_ECC_NIST_P256
 - TPM_ALG_ECDSA_ECC_NIST_P384
- Requesters and Responders are permitted to implement additional algorithms defined in [\[SPDM\]](#)

6.32 Deferrable Memory Write §

The Deferrable Memory Write (DMWr) is an Optional Non-Posted Request that enables a scalable high-performance mechanism to implement shared work queues and similar capabilities. With DMWr, devices can have a single shared work queue and accept work items from multiple non-cooperating software agents in a non-blocking way. A DMWr Completer, i.e., a Function whose [DMWr Completer Supported](#) bit is Set, is a Function that supports being the target of DMWr Requests.

The mechanisms for generating DMWr Requests are outside the scope of this document. For cases where a DMWr Requester supports generation of DMWr Requests directly via software mechanisms, it is recommended that the software thread that invoked the Request is informed of the corresponding Completion Status in an implementation specific manner.

As with AtomicOps (See § Section 6.15), the use model for DMWr requires that, from the initial trigger to generate a DMWr Request, the subsequent routing to the Completer, the Completer's resulting actions, and the return of the corresponding Completion, that each of these be "atomic" in the sense that a single trigger must result in a single Request, which must not be subdivided, being acted upon by the Completer in such a way that no observer can see a partial result, and a single Completion being returned to the Requester, without subdivision. Therefore, the following rules apply:

- Switches and Root Complexes that support DMWr routing must route DMWr Requests and Completions without modification.
- DMWr Completers must ensure that operations performed on behalf of a given DMWr Request are performed atomically with respect to each host processor or device access to that target location range.
- The internal implementation of DMWr Requesters and Completers to enforce the required “atomic” behavior is outside the scope of this document.

The target of a DMWr Request that is not a DMWr Completer (i.e., a Function whose DMWr Completer Supported bit is Clear) responds as per the Request handling rule for a Request Type that is not supported (see § Section 2.3.1). In Non-Flit Mode, components designed to earlier revisions of this specification that treat the Request Type value corresponding to DMWr Request as Reserved are permitted to treat a received DMWr Request as a Malformed TLP.

The following requirements apply to DMWr Completers:

- Functions indicate their ability to act as DMWr Completers by Setting DMWr Completer Supported and by indicating the largest DMWr TLP that the Function can receive (see DMWr Lengths Supported).
- Properly formed DMWr Requests must be handled as a Successful Completion (SC), Request Retry Status (RRS), Unsupported Request (UR), or Completer Abort (CA).
 - Properly formed DMWr Requests with types or operand sizes not supported by the Completer, or targeting an address not intended by the device’s programming model to be a target of DMWr Requests, or crossing an address boundary between two different resources, must be handled as Completer Abort (CA), and the value of the target location must remain unchanged.
 - Switches that support DMWr Routing but do not support serving as a DMWr Completer must handle properly formed DMWr Requests that target internal resources of the Switch as Completer Abort (CA), and the value of the target location must remain unchanged.
- When a DMWr Request cannot be completed successfully due to a temporary condition, the Completer is permitted to return a Completion with Request Retry Status (RRS)
 - When this is done, the value of the target location must remain unchanged, and the Completer must not assume that the Requester will repeat the request.
 - This is not an error
- Completers are permitted to use implementation specific mechanisms to determine when to use the RRS Completion Status in order to establish policies for fairness or for other reasons, and these mechanisms may be based on Requester ID, Traffic Class, PASID, payload contents, or other criteria as may be appropriate.
- Completers supporting DMWr are allowed to implement a restricted programming model.
 - If a Request that is not a DMWr targets an address intended by the device’s programming model to be a target of DMWr Requests, it is strongly recommended that the value of the target location remain unchanged, and, if the Request is a Non-Posted Request or a UIO Request, that the Completion returned does not include any sensitive information.
 - See Implementation Note: Optimizations Based on a Restricted Programming Model in § Section 2.3.1 for additional general guidance.
- Completers supporting DMWr that return Successful Completion (SC) must guarantee that the observed update granularity will not be smaller than 64 bytes, or the size of the Request, whichever is smaller.
 - This requirement applies to DMWr targeting “plain” memory, a shared work queue, or other implementation specific structures, when such operations are supported by the programming model of the Completer.
 - See also § Section 2.4.3 and § Section 2.4.4.

- If any Function in a Multi-Function Device associated with an Upstream Port supports DMWr Completer or DMWr routing capability, all Functions with Memory Space BARs in that device must decode properly formed DMWr Requests and handle any they don't support as an Unsupported Request (UR).
 - In such devices, Functions lacking DMWr Completer capability are forbidden from handling properly formed DMWr Requests as Malformed TLPs.
- Unless there is a higher precedence error, a DMWr-aware Completer must handle a Poisoned DMWr Request as a Poisoned TLP Received error (see § Section 2.7.2.1).
 - The Completer must return a Completion with a Completion Status of either Unsupported Request (UR) or Request Retry Status (RRS).
 - The value of the target location must remain unchanged.
- If the Completer of a DMWr Request encounters an uncorrectable error accessing the target location, the Completer must handle it as a Completer Abort (CA).
 - The subsequent state of the target location is implementation specific.
- Completers are permitted to support DMWr Requests on a subset of their target Memory Space as needed by their programming model (see § Section 2.3.1).
 - Memory Space structures defined or inherited by PCI Express (e.g., the MSI-X Table structure) are not required to be supported as DMWr targets unless explicitly stated in the description of the structure.
- If an RC has any Root Ports that support DMWr routing capability, all RCiEPs in the RC reachable by forwarded DMWr Requests must decode properly formed DMWr Requests and handle any they do not support as an Unsupported Request (UR).

The following requirements apply to Root Complexes and Switches that support DMWr routing:

- Root Ports and Switch Ports indicate their support of DMWr routing by Setting DMWr Request Routing Supported and by indicating the largest DMWr TLP that the associated Function supports (see DMWr Lengths Supported).
- Switches and Root Ports supporting the DMWr routing capability or DMWr Completer capability (or both) that receive a properly formed DMWr Requests must either forward it to another Port or handle it as a Successful Completion (SC), Request Retry Status (RRS), Unsupported Request (UR), Completer Abort (CA), or DMWr Egress Blocked error.
- If a Switch supports DMWr routing for any of its Ports, it must do so for all of them.
- For Switches and Root Ports supporting the DMWr routing capability, if a DMWr Request is received that crosses a decoding boundary between two different destinations, the Ingress Port must not propagate the Request, and must return a Completion with a Completion Status of UR.
- For a Switch or an RC, when DMWr Egress Blocking is enabled in an Egress Port and a DMWr Request targets going out that Egress Port, then the Egress Port must handle the Request as a DMWr Egress Blocked error and must also return a Completion with a Completion Status of UR.
 - If the severity of the DMWr Egress Blocked error is non-fatal, then this case must be handled as an Advisory Non-Fatal Error as described in § Section 6.2.3.2.4.1.
 - This is a reported error associated with the Egress Port (see § Section 6.2).
- For an RC, support for peer-to-peer routing of DMWr Requests and Completions between Root Ports is optional and implementation specific.
 - When supported, the associated Ports must Set the DMWr Request Routing Supported in the Device Capabilities 3 Register.
 - When supported, DMWr TLPs must be routed without modifying the size of the data payload.
 - Even when DMWr Request Routing Supported is Set in two Root Ports, it is implementation specific whether forwarding is supported between those Ports.

- If one Root Port in a Root Complex supports DMWr Completer or DMWr Routing, a DMWr Request received by that Port that is routed to another Root Port that does not support DMWr must be handled as an Unsupported Request (UR).
 - This is a reported error associated with the Ingress Port (See § [Section 6.2](#)).

The following requirements apply to DMWr Requesters:

- A Function is only permitted to generate DMWr Requests when the DMWr Requester Enable bit in the Device Control 3 register is Set.
- When a DMWr Request is completed with Request Retry Status (RRS), the Requester is permitted, but not required, to re-issue the Request.
 - The Requester is permitted to use any implementation specific criteria to determine if/when to re-issue the Request.
 - Subsequent Requests are permitted to be the same or modified.

IMPLEMENTATION NOTE: CONSIDERATIONS FOR THE USE OF DEFERRABLE MEMORY WRITE (DMWR) §

The intended use model for the Deferrable Memory Write (DMWr) is to implement efficient hardware/software interface control mechanisms, using specialized hardware in the Completer to process the DMWr Request and generate the appropriate Completion Status. For example, a device could implement “enqueue registers” that enable commands to the device to be issued via a single DMWr Request, and based on the Completion Status the device can indicate to the Requester of the command was accepted.

Users of DMWr must understand the functional implications of transaction ordering. A DMWr Request is a Non-Posted Request with Data, which means that Posted Requests are permitted to pass DMWr Requests. Additionally, there is no guaranteed ordering among all types of Non-Posted Requests (see Table 2-4, entries B3, B4, C3 & C4).

As with all types of “control” mechanisms, it is necessary for all participants to comprehend the specific requirements placed by the particular mechanism, and these will vary between different systems and different device types. In many cases it will be necessary to distinguish Requests issued from different software environments (e.g., from multiple Virtual Machine guests where the guests use different drivers) all sharing the same work queue. PASID is one mechanism that can be used for this purpose, although there are many alternatives (e.g., different ranges of addresses could be assigned to each environment that would be mapped to the same resources in the Completer). In some systems, system and application level software is capable of generating DMWr Requests according to a specific template (§ Figure 6-51), where bits 31:0 are defined by the system architecture, the P bit at bit 31 indicates if user (0b) or supervisor (1b) code triggered the Request, and bits 19:0 of the payload include the PASID to indicate the context in which the Request was generated.



Figure 6-51 Example DMWr Data Payload Template §

For performance reasons it is not recommended that DMWr be used for sending bulk data.

Being a Non-Posted Request, DMWr TLPs require a Completion. In addition, PCIe ordering rules dictate that Non-Posted TLPs cannot pass Posted TLPs, making Posted transactions preferable for improved performance.

Because DMWr TLPs and Memory Read Request TLPs can pass each other, and DMWr TLPs can be deferred by the Completer, care must be taken by Device and Device Driver manufacturers when attempting to read a memory location that is also the target of an outstanding DMWr Transaction, if this is even supported by the programming model of the Completer. Because of these properties, use of DMWr TLPs when transferring large amounts of data is not recommended.

When DMWr Transactions are used to enable a shared work queue, care must be taken to ensure that no Requesters are denied access indefinitely to the queue due to competition with other Requesters. Software entities that submit work to such a queue may choose to implement a flow control mechanism or rely on a particular programming model to ensure that all entities are able to make forward progress, for example to

include a feedback mechanism or an indication from the Function to software on the state of the queue, or a timer that delays DMWr Requests after a Completion with Completion Status RRS. The DMWr mechanism does not itself provide protection against software entities issuing Requests (either maliciously or unintentionally) at a rate high enough to cause problems with other software entities accessing the single shared work queue. The details of such mechanisms and programming models are outside of the scope of this specification.

6.33 Integrity & Data Encryption (IDE) §

Integrity & Data Encryption (IDE) provides confidentiality, integrity, and replay protection for TLPs Transmitted and Received between two Ports. It flexibly supports a variety of use models, while providing broad interoperability. The cryptographic mechanisms are aligned to industry best practices and can be extended as security requirements evolve. The security model considers threats from physical attacks on Links, including cases where an adversary uses lab equipment, purpose-built interposers, malicious Extension Devices, etc. to examine data intended to be confidential, modify TLP contents, reorder and/or delete TLPs. TLPs can be secured as they transit Switches, extending the security model to address attacks mounted by reprogramming Switch routing mechanisms, or using “malicious” Switches. IDE can be used to secure traffic within trusted execution environments, also known as trust domains, composed of multiple components – the frameworks for such composition are outside the scope of IDE.

IDE establishes an IDE Stream between two Ports (see § Figure 6-52). There are two types of IDE Streams: a Selective IDE Stream applies to selected TLPs as determined by association rules defined in this section, and a Link IDE Stream applies to all TLPs Transmitted using a particular TC except those that are associated with a Selective IDE Stream. When there are no Switches between the Ports, then it is possible to secure all, or only selected, TLP traffic on the Link, using Link IDE or Selective IDE, respectively. There is no required relationship, or restriction, between Link IDE and Selective IDE. It is possible to use both Link IDE and Selective IDE between two directly connected Ports, as shown in § Figure 6-52 between Ports A and B, in which case TLPs associated with the Selective IDE Stream are secured using that Stream’s key set, and all other TLPs are secured using the key set for the Link IDE Stream. Such a configuration may be desirable if, for example, different security policies are applied to the Selective IDE TLPs than to other Link traffic. It is possible to use Selective IDE in cases where the IDE Terminus is a Switch Port, as shown between Ports C and D. IDE does not establish security beyond the boundary of the two terminal Ports, and mechanisms for securing and/or isolating secure traffic within a Component are outside the scope of this document. Again referring to the example shown in § Figure 6-52, the Selective IDE Streams between Ports C and G, and between Ports G and H, are secured as they pass through the Switch. All other Link IDE and Selective IDE streams illustrated are secured by IDE from Port to Port, but must be secured by implementation specific means within the Component past the terminal Port. By implication, when Link IDE is used with TLPs flowing “hop-by-hop” through one or more Switches, it is necessary to ensure acceptable security is maintained within the Switch(es), but how this is done is outside the scope of this document.

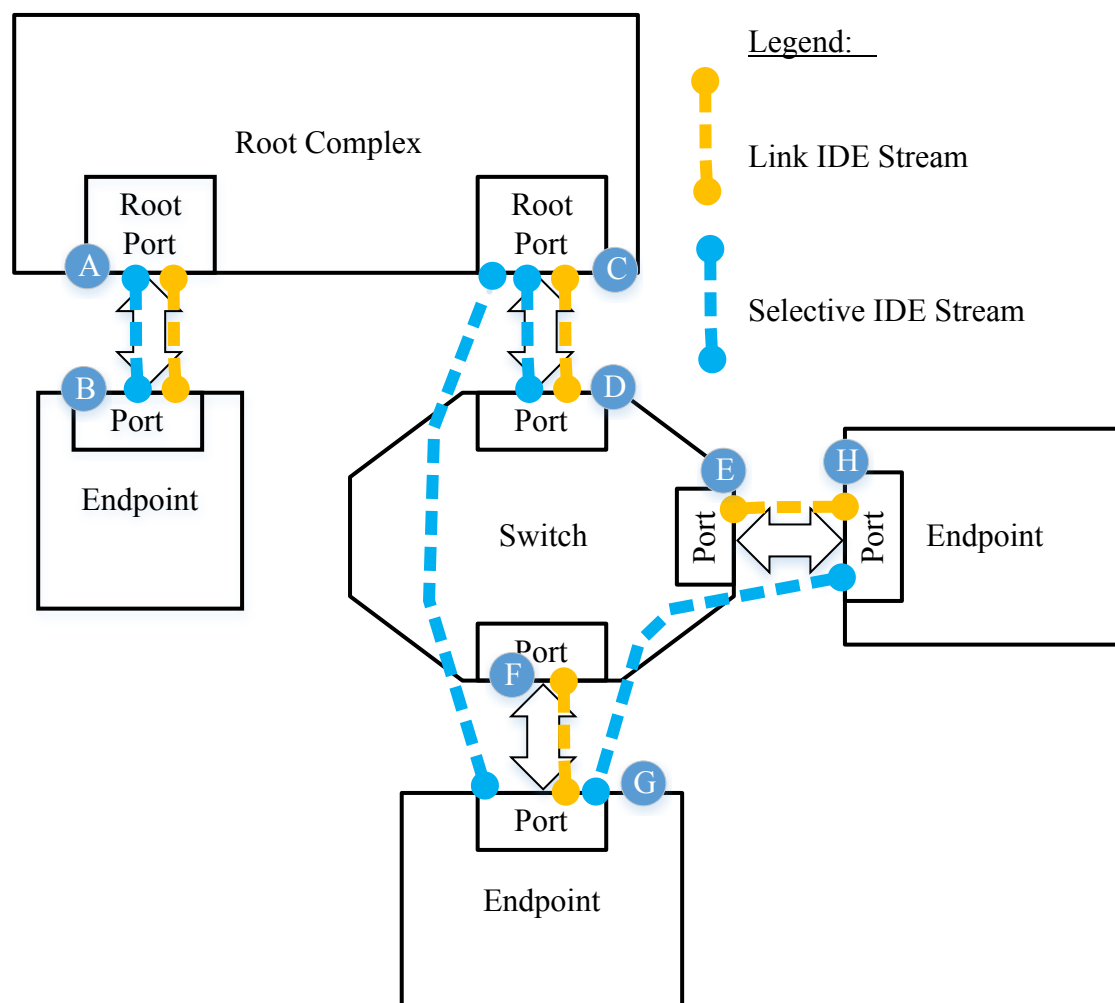


Figure 6-52 IDE Secures TLPs Between Ports §

In addition to the in-line securing of TLPs, as a “data plane” capability, IDE defines interoperable mechanisms for establishing Streams and programming keys, as a “control plane” capability, based on industry specifications. For example, for an Endpoint connected directly to the Root Complex (A to B above), one way to establish IDE is to use IDE Key Management (IDE_KM – see § Section 6.33.3) via DOE to allow host Firmware/Software to configure the Ports, including securely programming the IDE keys into both Ports. In another example, for two Endpoints communicating peer to peer (G to H above), the two Endpoints can implement communication directly via [MCTP-VDM] and [Secured-MCTP], where one will take the Requester role and the other the Responder role, and then applying the IDE_KM flow for secure key establishment. In an alternate example, it is also possible for some kind of management controller to apply IDE_KM over a sideband management connection, to program IDE keys in Ports throughout a system. The mechanisms for a management controller to program keys into a Root Complex are outside the scope of this document.

Policies for establishing trust between elements in a platform are outside the scope of IDE. It is strongly recommended that a platform-appropriate policy be implemented via platform-specific means. It is not necessary that this policy be applied prior to establishing IDE Streams, and in some cases it may be preferable to establish IDE first, and subsequently

apply security policy mechanisms, or to apply some policy mechanisms prior to establishing IDE and some additional mechanisms after.

IMPLEMENTATION NOTE:

THREAT MODEL AND RELATED CONSIDERATIONS FOR IDE §

This implementation note provides a very general treatment of the threat model assumed for IDE. A detailed treatment will necessarily require knowledge of the platform environment and other elements that are outside the scope of this document.

This threat model covers:

Attacks using a logic analyzer or interposer type device, including e.g. “rogue” Retimers, where the attack devices attempt to add or delete TLPs, observe TLP payload data, and/or reorder TLPs. Example attacks include delaying a flag write to bypass a data write causing stale data to be accepted, or delaying a read to bypass a write to same location causing a stale value to be read. Reordering is discussed in more detail in the Implementation Note “Detection of Improper Reordering”.

IDE secures host systems against device substitution attacks because, once authenticated key exchange is completed, a subsequent attack by a different unit trying to masquerade as the authenticated unit will fail because the masquerading unit cannot generate IDE TLPs using the correct key.

Provided an implementation includes appropriate self-protection measures, IDE also supports the detection of attacks involving the removal of a unit, for example by moving the unit to a different system and attempting to operate the unit masquerading as the authenticated host.

It is assumed that in an attack, the attacker will thwart error reporting attempts, e.g. by blocking Messages from the Port that detected the error, and such reporting messages are only intended to be used in debugging improperly configured systems. If a specific use model requires timely detection of security failures, some type of “heartbeat” mechanism should be used, rather than assuming that the failure will be reported directly.

This threat model does not cover:

Security exposures caused by inadequate Device implementation. For example, implementations are necessarily required to secure local keys, interconnects, and memory, including, for example, local memory implemented on an add-in-card using discrete memory components. IDE does not protect against on-die traffic redirection, for example between Functions of a Multi-Function Device.

Debug mechanisms should be given careful review as they can easily cause information exposure when improperly implemented. It is strongly recommended that debug state be reported using measurement mechanisms, and that any change in debug configuration that could expose data intended to be secured result in a transition to Insecure (see § Section 6.33.1).

There are many considerations regarding secure key generation, programming, and storage, and it is strongly encouraged that non-experts consult with experts to evaluate all levels of implementation to ensure that good practices are followed. In all cases, it is essential to avoid exposure of plain text keys by any means including debug features such as tracers, configuration registers etc.

If partial header encryption (see § Section 6.33.4) is not used, “side channel” attacks may be possible in some cases based on attacker analysis of the information included in the headers. For example, see <https://www.ieee-security.org/TC/SP2015/papers-archived/6949a640.pdf>.

Considerations:

IDE secures TLP traffic from one Port to another Port. TLP content is not secured on-die by IDE past the terminal Partner Ports, and so it is necessary to provide appropriate implementation specific protective measures based on use model requirements to ensure that TLP traffic is secured prior to transmission, and following reception.

IDE assumes the implementation of appropriate isolation mechanisms to ensure that information remains secured beyond the Port to Port connection secured via IDE. In some cases, entire components can be considered “secure” and there is no need to distinguish traffic on-die, in other cases the establishment of one or more Trusted Execution Environments (TEEs) may be needed to isolate secure traffic from non-secure traffic, and different secure environments from each other. Although it is permitted to establish more than one IDE Stream between the same two Ports, this is not generally needed or useful, because it is assumed that once on-die, all secure traffic is “equally” secure, and using separate IDE Streams provides no additional protection. The details of how such TEEs are implemented and managed are outside the scope of IDE. The XT bit and the T bit are used for TEE management mechanisms (see § Chapter 11.). IDE mechanisms ensure that the XT and T bits (like other TLP content) are secured during transit.

Good practices for implementing TEEs include, but are not limited to:

- Securing secrets through the use of local encryption, access control, and/or other mechanisms
- Ensuring that secure data cannot “leak” due to errors, power management, or other operations
- Detecting inappropriate attempts to reconfigure IDE, e.g. writes to any of the IDE control registers, and/or other internal conditions that could compromise secure data and taking appropriate measures, including potentially forcing the Port into Insecure
- Ensuring that secret keys are never exposed or stored in non-secure buffers
- Ensuring that the establishment & management of TEEs is itself secure

The implementation of TEEs can be very complex, and it is strongly recommended that persons with appropriate security expertise are intimately involved in the development and validation of components and systems.

Although Link IDE applies to all kinds of TLPs, Selective IDE can only be applied to certain types of TLPs (see § Table 6-35). Memory operations are required to be supported for virtually all use models, and are supported by Selective IDE. I/O operations are not commonly used, and are not supported by Selective IDE to simplify design and validation. Selective IDE can be applied to Messages, and, optionally, to Configuration Requests & Completions. Selective IDE can be applied to TLPs with Prefixes, but Local TLP Prefixes are not protected when the TLP is associated with a Selective IDE Stream. In NFM, End-End TLP Prefixes are protected along with the associated TLP, and in FM, OHC content is protected along with other Header content.

6.33.1 IDE Stream and TEE State Machines §

Conceptually, the initialization of a Link or Selective IDE Stream involves multiple steps, although some of these steps can be merged or performed in a different order. The first step is to establish the authenticity and identity of the components containing the two Partner Ports to be the IDE Terminuses of the IDE Stream. This may be done using CMA-SPDM, by implementation specific means, or in some cases implicitly. The second step is to establish the IDE Stream keys – the IDE Key Management (IDE_KM – § Section 6.33.3) provides a way to do this. Third, the Secure Connection must be configured, and, finally, the establishment of the IDE Stream is triggered.

Conceptually, each IDE Stream is associated at each Partner Port with a state machine illustrated in § Figure 6-53.

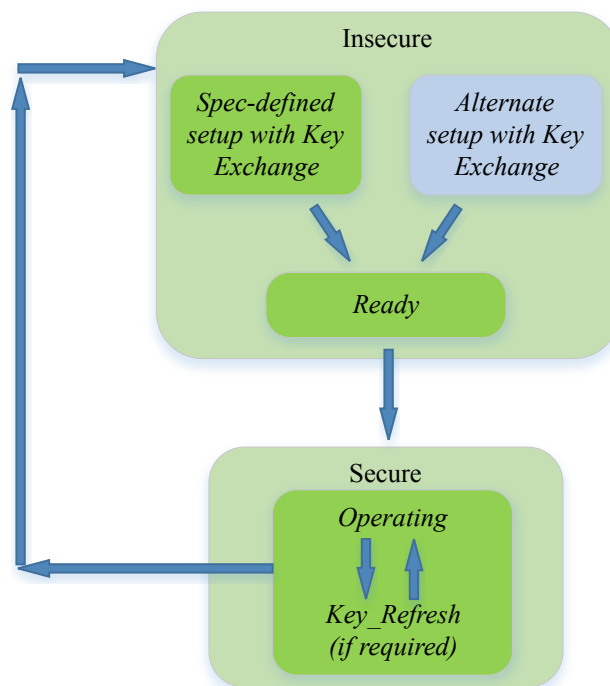


Figure 6-53 IDE Stream State Machine §

- The Insecure state indicates that the necessary steps to operate the IDE Stream have not been completed, or that some event has ended the operation of a previously operating IDE Stream.
 - Typically the Insecure will include various conceptual substates that are not directly observable by the hardware, and only the system firmware/software configuring the IDE Stream will have the ability to comprehend when all necessary steps have been completed.
 - The Ready conceptual sub-state of the Insecure state is entered when all necessary configuration has been performed; this condition must be tracked by system firmware/software.
 - In many cases it will not be possible for hardware to distinguish when all necessary configuration has been performed, and there is no requirement for hardware to track the transition into the Ready sub-state.
- The IDE Stream State Machine for a specific Stream of a Port must transition from Secure to Insecure when the corresponding Link/Selective IDE Stream Enable bit is Cleared.
 - As further discussed below, it is essential that the Port internally block all transmissions that are intended to be secure if the corresponding IDE Stream State Machine is not in the Secure state.
- If at any time a condition compromising the security of the IDE Stream is detected at the Port, the Port must transition to Insecure.
 - It is permitted to transition to Insecure for implementation specific reasons.

A trusted execution environment (TEE) using IDE must prevent the transmission of TLPs intended to be secure using non-IDE TLPs, and must reject non-IDE TLPs received if the TEE requires those TLPs to be secure. A specific architecture for devices that support TEEs is defined by TDISP (see § Chapter 11.). In order to precisely define the normative requirements for IDE in relation to TEEs, we will assume that TEEs have internal states that correspond to those shown in § Figure 6-54.

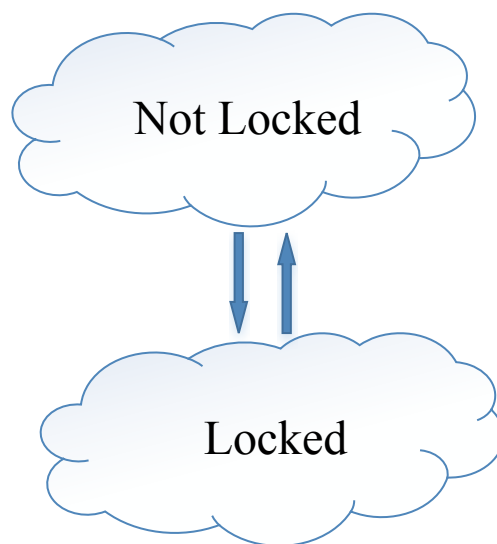


Figure 6-54 IDE Stream State Machine §

TDISP (see § Chapter 11.) defines specific requirements that extend the following rules, that apply broadly to all TEEs using IDE:

- A TEE must distinguish between at least two operating conditions, that may be called by any names but here will be called Not Locked and Locked.
 - A TEE must transition to the Locked state if and only if all IDE Streams required for the secure operation of the TEE are in the Secure state.
 - A TEE must transition to the Not Locked state if any IDE Stream required for the secure operation of the TEE is in the Insecure state.
- A TEE using an IDE Stream must precisely define the essential configuration information that could affect the security of the IDE Stream, and, once that IDE Stream is established, that essential configuration information must be confirmed and maintained by secure means so as to detect “adversary-in-the-middle” (AITM) attacks attempted during or after IDE Stream establishment, and changes to that information blocked and/or detected, as to detect/prevent attacks during operation.
 - The specific configuration information to which this requirement applies is implementation specific and dependent on the hardware elements involved, the security attributes required, and potentially on assumptions of use, all of which are outside the scope of this specification.
 - How the information is confirmed is implementation specific, but would typically include securely transferring a data structure that contains a local snapshot of the information to a secure partner to be compared against the expected values.

6.33.2 IDE Stream Establishment §

To establish IDE Streams interoperably based on this specification, system firmware/software acts as a central authority to create and program keys into the two Partner Ports. The following rules apply:

- For Endpoints, including Functions of a Multi-Function Device, associated with an Upstream Port, only Function 0 must implement the IDE Extended Capability.
- For Switches, including cases where one or more Functions of a Multi-Function Device represent the Upstream Port of a Switch, the IDE Extended Capability must only be implemented in Function 0, and implemented such that Function 0 represents the Multi-Function Device as a whole.
- For a Downstream Port, the Bridge Function associated with the Port must implement the IDE Extended Capability.
- All Ports other than Root Ports must implement support for key management by means of the IDE key management (IDE_KM) protocol defined in § Section 6.33.3 .
 - For Switch and Root Ports it is permitted for one Port to provide the DOE and CMA-SPDM responder function on behalf of other Ports as defined in § Section 6.33.3 .
 - Root Ports are permitted to implement support for the IDE_KM protocol.

It is also permitted for systems to implement the IDE_KM protocol via MCTP (see § Section 6.33.3).

It is also permitted for system firmware/software to enable pass-through communications between the two Partner Ports, where one of the two takes the Requester Role and the other takes the Responder Role, implementing the IDE_KM protocol defined below directly between the two Partner Ports (as an example see the Selective IDE Stream between Ports G and H in § Figure 6-52). How this is discovered and enabled is outside the scope of this specification.

6.33.3 IDE Key Management (IDE_KM) §

IDE Key Management (IDE_KM) builds upon [SPDM] and [Secured-SPDM], and can be used over multiple transports (see § Figure 6-55).

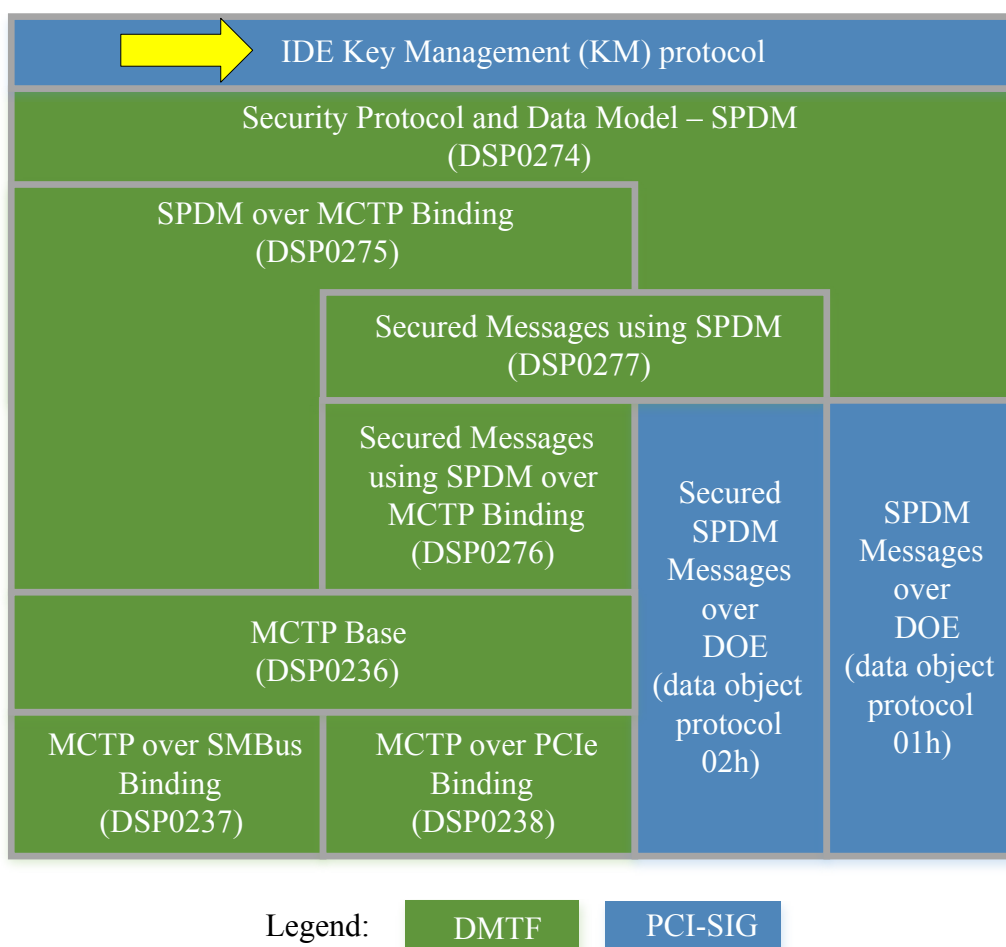


Figure 6-55 IDE Key Management (IDE_KM) and Related Specifications & Capabilities §

The following rules define the IDE key management (IDE_KM) protocol, and must be followed for Ports that support the use of IDE_KM:

- The IDE_KM protocol uses the data objects defined below, where:
 - The Requester must use the [SPDM] VENDOR_DEFINED_REQUEST format, the Responder must use the [SPDM] VENDOR_DEFINED_RESPONSE format.
 - The StandardID field of the VENDOR_DEFINED_REQUEST/ VENDOR_DEFINED_RESPONSE must contain the value assigned in [SPDM] to identify PCI-SIG.
 - The VendorID field of the VENDOR_DEFINED_REQUEST/ VENDOR_DEFINED_RESPONSE must contain the value 0001h, which is assigned to the PCI-SIG.
 - The VendorDefinedReqPayload/VendorDefinedRespPayload field of the VENDOR_DEFINED_REQUEST/VENDOR_DEFINED_RESPONSE must be the data object content as defined below.
 - The VENDOR_DEFINED_REQUEST/VENDOR_DEFINED_RESPONSE must in turn form the Application Data field of a Secured Message per [Secured-SPDM].

- It is strongly recommended that the cryptographic strength used in the secure session be at least as strong as selected for IDE itself.
- If any IDE_KM data object is received that has not been transferred securely per [Secured-SPDM], the received data object must not be used for key management, and, if it is a request, must not result in a response.
- The size of the VendorDefinedReqPayload/VendorDefinedRespPayload must match the size of the data object defined below.
 - If the size does not match, the received data object must not be used for key management, and, if it is a request, must not result in a response.
- For Endpoint Functions, including Functions of a Multi-Function Device, associated with an Upstream Port, Function 0 must implement DOE and CMA-SPDM for Authentication and key exchange, including the secure session establishment mechanism (see [SPDM]) and the IDE key management (IDE_KM) protocol as a Responder, as defined below.
 - IDE operates at a per-Port level, and Function 0 of an Upstream Port must be used for the purposes of establishing the authenticity and identity of the associated Component, performing key exchange, and the configuration and management of IDE Stream(s) for that Port.
- Each Upstream Port Function, regardless of Function Number, representing a Switch for which Link IDE Stream Supported and/or Selective IDE Streams Supported are Set, including in Multi-Function Devices, must implement DOE and CMA-SPDM supporting the IDE key management (IDE_KM) protocol as a Responder, as defined below.
- It is permitted for a Root Complex to:
 - support the IDE_KM protocol using a DOE instance for some or all Root Ports per-Port, or using some Root Ports to represent other Root Ports
 - implement a DOE instance in an RCRB supporting IDE_KM for Root Ports,
 - use implementation specific key management.
- For Switches and Root Complexes, it is permitted for one Port to implement the IDE_KM interface as a responder for itself and for other Ports of the Switch/Root Complex.
- Ports are permitted to support the IDE_KM protocol transported via MCTP.
- Within each data object, the Protocol ID field in bits [7:0] of the first DW must be 00h to indicate IDE.
- The Object ID field in bits [15:8] of the first DW indicates the IDE_KM data object type and is encoded as:
 - 00h: Query (QUERY)
 - 01h: Query Response (QUERY_RESP)
 - 02h: Key Programming (KEY_PROG)
 - 03h: Key Programming Acknowledgement (KP_ACK)
 - 04h: Key Set Go (K_SET_GO)
 - 05h: Key Set Stop (K_SET_STOP)
 - 06h: Key Set Go/Stop Acknowledgement (K_GOSTOP_ACK)
 - all other encodings Reserved.
- A Requester is permitted to use QUERY to determine the capabilities and configuration of a Port (see § Figure 6-56)
 - The PortIndex field must be used to indicate the Port addressed by the QUERY.
- A Responder must respond to a QUERY indicating a PortIndex value of 00h.

- IDE_KM assigns unique Port numbers (PortIndex) for each Port of an Endpoint, Switch or Root Complex, implementing IDE_KM.
- For a Switch that supports the IDE_KM responder role:
 - The Switch Upstream Port must implement the responder role for itself and the Upstream Port must respond to a PortIndex of 00h.
 - Downstream Ports of the Switch that are represented by the Upstream Port must respond to PortIndex ranging from 01h to FFh, where the order is established by the Device/Function numbers assigned by the Switch construction to the Downstream Ports from lowest to highest.
 - It is permitted for a Switch to implement a responder capability in a Downstream Port, for example by implementing a DOE instance in the Downstream Port, in which case that Downstream Port must respond to a PortIndex of 00h.
 - It is permitted for that Port to represent other Downstream Ports, in which case the represented Downstream Ports must respond to PortIndex ranging from 01h to FFh, where the order is established by the Device/Function numbers assigned by the Switch construction to the Downstream Ports from lowest to highest.
- For a Root Port that supports the IDE_KM Responder role:
 - The Root Port must implement the responder role for itself and must respond to a PortIndex of 00h.
 - It is permitted for that Port to represent other Root Ports, in which case the represented Root Ports must respond to PortIndex ranging from 01h to FFh, where the order is established by the Device/Function numbers assigned by the Root Complex construction to the Root Ports from lowest to highest.
- For an Endpoint Upstream Port that supports the IDE_KM responder role, the Port must respond to a PortIndex of 00h.
- Ports/RCRBs implementing the [SPDM] responder role must respond to a QUERY with QUERY_RESP (see § Figure 6-57).
 - The PortIndex field must contain the PortIndex field value from the corresponding QUERY.
 - The MaxPortIndex field value must indicate the maximum PortIndex value for the Ports represented by this Port/RCRB.
 - If only one Port is represented, including for all Endpoint Upstream Ports, the MaxPortIndex field must be 00h.
 - The Bus Number field must contain the Bus Number of the Function corresponding to the PortIndex field value.
 - The Segment field must:
 - for Ports that are not Root Ports, be zero
 - for Root Ports, contain the Segment Number value for the Root Port, or zero if the Root Complex implements only one Segment.
 - For Non-ARI Functions the Device/Function Number field must contain the Device and Function number of the Function corresponding to the PortIndex field value.
 - For ARI Functions the Function Number field must contain the Function number of the Function corresponding to the PortIndex field value.
 - The remainder of QUERY_RESPONSE must consist of the contents of the IDE Extended Capability Structure, other than the IDE Extended Capability Header itself, for the addressed Port.
 - The Supported Algorithms and Selected Algorithm field values returned in QUERY_RESP must be compared against the values read from the corresponding fields in the Responder Port's IDE Extended Capability structure and if non-matching values are detected then the

IDE_KM protocol for this secure session must be aborted, or other appropriate corrective action taken to avoid a possible “downgrade” attack.

- Requesters must not issue other requests after issuing a QUERY command until receipt of the corresponding QUERY_RESP.
- KEY_PROG, KP_ACK, K_SET_GO, K_SET_STOP and K_GOSTOP_ACK all apply to a single Sub-Stream, direction (Tx or Rx) and Key Set.
 - The Key Sub-Stream field indicates the Key Sub-Stream, using the same encodings as defined for the Sub-Stream identifier (see § Section 6.33.5)
 - The direction is indicated by the RxTxB bit encoded:
 - 0b – Receive
 - 1b – Transmit
 - The Key Set field indicates the Key Set, corresponding to the K bit value in the IDE TLP Prefix (NFM)/OHC-C (FM).
- For Ports implementing the Responder role, key programming and the ability to select the initial value of the IV must be supported using the KEY_PROG command (see § Figure 6-58).
 - The length of the key must correspond to the length indicated in the Selected Algorithm field for the Stream.
 - The Requester must not send another KEY_PROG command to the same Port until it has received a KP_ACK from the Port.
 - If the Requester does not receive a KP_ACK from the Responder within 1 second plus a sufficient time to account for transport delay the Requester is permitted to consider that the Responder is not operating correctly.
 - Fields specific to the KEY_PROG command are:
 - PortIndex, indicating the Port to which the key is to be programmed, corresponding to the order established in the QUERY_RESP
 - Stream_ID
 - Key, which must be of the size required for the Selected Algorithm for the Stream
 - IFV, indicating the initial value for the invocation field of the IV, which must be 64 bits in size, and must initially set to the value 0000_0001h upon establishment of the Stream and when performing a key refresh.
- A Port implementing the Responder role must acknowledge receipt of a KEY_PROG command by returning KP_ACK, defined in § Figure 6-59.
 - The Status field must indicate the result of the KEY_PROG command, encoded as:
 - 00h: Successful
 - 01h: Failed to parse command – Incorrect Length
 - 02h: Failed to parse command – Unsupported value in PortIndex
 - 03h: Failed to parse command – Unsupported value in other field(s)
 - 04h: Unspecified Failure
 - 05-FFh: Reserved – Must not be used in generating KP_ACK, but if received must be treated as Unspecified Failure
 - The Responder must return KP_ACK within 1 second of the receipt of the KEY_PROG command.
 - It is strongly recommended to return KP_ACK as quickly as possible.
 - Return of KP_ACK, regardless of Status, indicates the Port is able to receive and process another KEY_PROG command .

- Mechanisms for generating keys are outside the scope of this document.
- It is strongly recommended to complete key programming for a Stream before Setting the Enable bit in the IDE Extended Capability entry for that Stream.
 - It is permitted, but strongly not recommended, to Set the Enable bit in the IDE Extended Capability entry for a Stream prior to the completion of key programming for that Stream.
- If the Enable bit is Set in the IDE Extended Capability entry for a Stream, but that IDE Stream is not already in Secure, the receipt of a K_SET_GO for must trigger the Port to Transmit/Receive IDE TLPs for the indicated Stream, Sub-Stream, direction and Key Set.
 - The agent implementing the Requester role for IDE_KM must send K_SET_GO commands to enable the Receivers at both IDE Partner Ports, and then send K_SET_GO commands to enable the Transmitters at both IDE Partner Ports.
 - The Port must use the indicated Key Set for IDE TLP transmissions associated with the IDE Stream, Sub-Stream, and direction starting not more than 10 ms after the receipt of the K_SET_GO command to enable the Transmitter.
 - The Port must be capable of processing received IDE TLPs using the indicated Key Set within 10 ms after the receipt of the K_SET_GO command enabling the Receiver.
 - The Port must respond by returning an K_GOSTOP_ACK once it is capable of receiving another IDE_KM Request.
- If the Enable bit is Set in the IDE Extended Capability entry for a Stream, and that IDE Stream is already in Secure (a key refresh operation), the receipt of a K_SET_GO for must trigger the Port to Transmit/Receive IDE TLPs for the indicated Stream, Sub-Stream, direction and Key Set.
 - The agent implementing the Requester role for IDE_KM must send K_SET_GO commands to enable the Receivers at both IDE Partner Ports, and then send K_SET_GO commands to enable the Transmitters at both IDE Partner Ports.
 - The Port must use the indicated Key Set for IDE TLP transmissions associated with the IDE Stream, Sub-Stream, and direction starting not more than 10 ms after the receipt of the K_SET_GO command to enable the Transmitter.
 - The Port must be capable of processing received IDE TLPs using the indicated Key Set within 10 ms after the receipt of the K_SET_GO command enabling the Receiver.
 - For each Sub-Stream of the Stream, until the Port receives an IDE TLP using the new key set, as indicated by the K bit value toggling, the Port must continue to accept IDE TLPs using the established key set.
 - Once the Port receives an IDE TLP using the new key set on a Sub-Stream it must invalidate and render unreadable the old key set, and discard subsequently received IDE TLPs using the old key set on that Sub-Stream.
 - The Port must respond by returning an K_GOSTOP_ACK once it is capable of receiving another IDE_KM Request.
- If the Enable bit in the IDE Extended Capability for a Stream is Clear, the receipt of K_SET_GO for both receive and transmit must cause the Port to become ready to Transmit/Receive IDE TLPs for the indicated Stream and Sub-Stream within 10 ms following the receipt of the last K_SET_GO, after which system software is permitted to Set the Enable bit for the Stream. When the Enable bit is Set:
 - The Port must be capable of processing received IDE TLPs using the Key Set armed by the received K_SET_GO Request.
 - System software must ensure that the Partner Ports initiate IDE TLPs sequenced appropriately so that a Port will not receive an IDE TLP before the Enable bit has been set.
 - The Port must respond by returning an K_GOSTOP_ACK once it is capable of receiving another IDE_KM Request.

It is strongly recommended that all Sub-Streams in both directions be fully programmed before Setting the Enable bit for the Stream.

- Is permitted for the IDE_KM Requester to transmit the K_SET_STOP command, defined in § Figure 6-61, to indicate that a Key Set must stop being used at a Port for the indicated Stream, Sub-Stream, and direction.
 - The Port implementing the responder role must invalidate and render unreadable the indicated Key Set not more than 10 ms after the receipt of the K_SET_STOP command.
 - Upon receipt of the KEY_STOP command, for the indicated Key Set and direction, all keys must be invalidated and rendered unreadable.
 - It should be observed that this action does not directly transition the Stream to Insecure, but any subsequent attempt to use the indicated Key Set will result in the Stream transitioning to Insecure.
 - The Port must respond by returning an K_GOSTOP_ACK once it is capable of receiving another IDE_KM Request.
- When an error is detected in a received K_SET_GO or K_SET_STOP (such as an invalid Stream_ID), it is recommended that no K_GOSTOP_ACK be returned.
- When using DOE for IDE_KM, or when IDE is enabled/disabled using the Enable bit for an IDE Stream, the following rules apply:
 - For a Configuration Request that triggers the start IDE, the Port must first return the Configuration Completion as a non-IDE TLP, and then trigger the start of IDE.
 - For a Configuration Request that stops IDE, the Port must first return the Configuration Completion as an IDE TLP, and then stop IDE.

An IDE error condition will occur if system software fails to ensure the correct sequencing of IDE enablement at the two Partner Ports.

- For a given IDE Stream, once a secure [SPDM] session has been used to respond to one QUERY or KEY_PROG request:
 - While the secure [SPDM] session that was used for initial key programming remains open, all QUERY and/or KEY_PROG requests that are received through a different secure [SPDM] session must be discarded by the Responder, and must not result in a response.
 - If the secure [SPDM] session that was used for initial key programming is closed, any subsequent QUERY and/or KEY_PROG requests received through a different secure [SPDM] session must first cause the responder to invalidate and render unreadable all keys must for the IDE Stream, then transition that IDE Stream to the Insecure state, and only then respond to the QUERY/KEY_PROG request, unless it can be ensured through implementation specific means that the new session has been established with the same requester as performed the initial key programming.

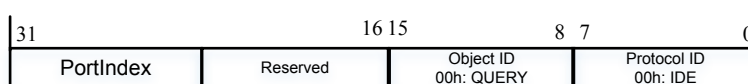


Figure 6-56 Query (QUERY) Data Object §

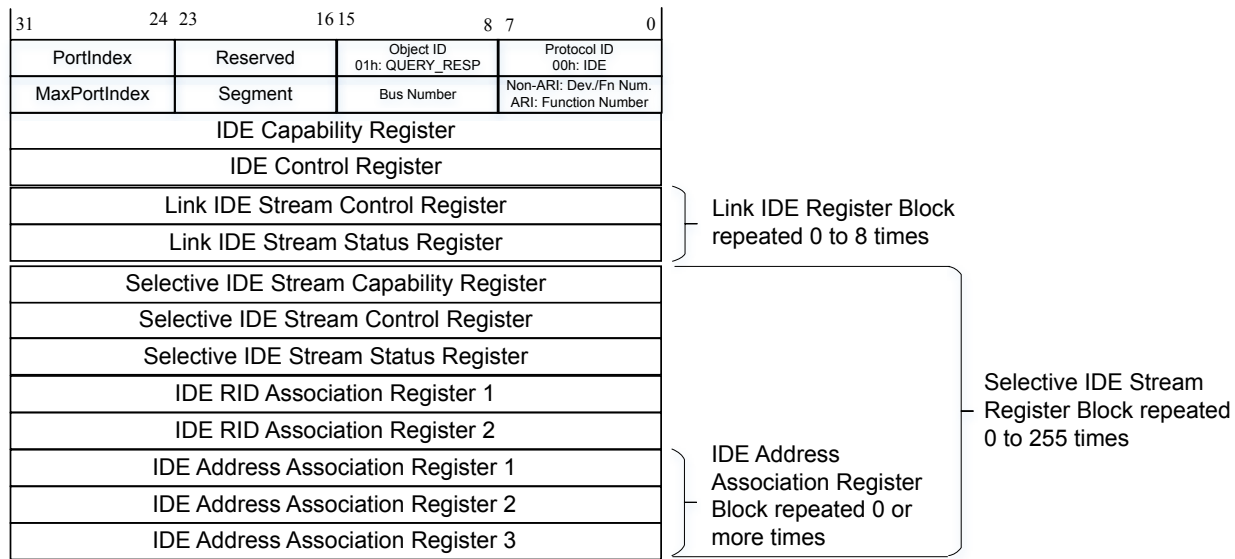


Figure 6-57 Query Response (QUERY_RESP) Data Object §

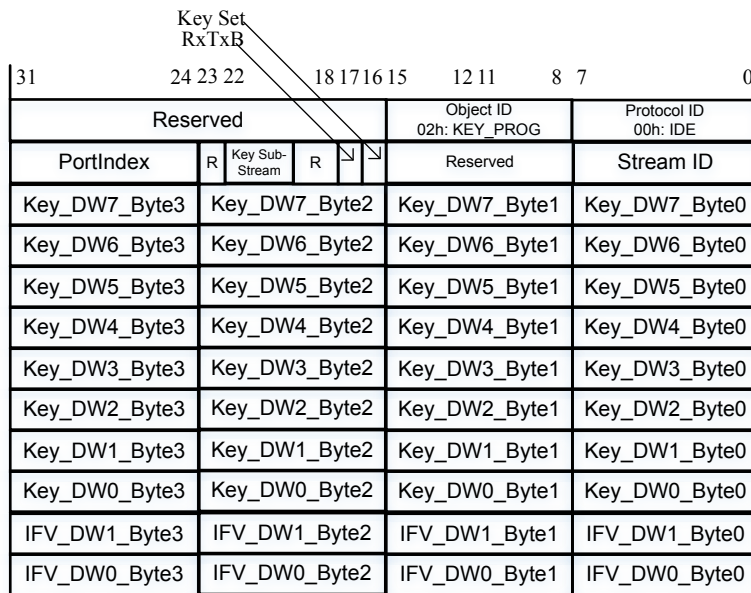


Figure 6-58 Key Programming (KEY_PROG) Data Object with Example 256b Key §



For implementations not requiring interoperable authentication and key exchange, it is permitted to use other mechanisms for key management.

Rules related to keys:

- Key size must be 256 bits.
- Following key exchange, implementation specific means must be used to ensure key security - the specific requirements for maintaining key security are platform and use case specific, and are out of scope for this document.
- Separately generated keys must be used for the Transmitter and the Receiver, and for each Sub-Stream.
 - It is strongly recommended that all keys for all Streams be separately generated.
- To support key updates without requiring an active IDE Stream to be put into a quiescent state, two key sets are defined, and the appropriate key set indicated in the IDE TLP Prefix (NFM)/OHC-C (FM) by the Transmitter via the K bit.
 - The initial value of the K bit is permitted to be 0b or 1b, although it is recommended the initial value be 0b. Software must ensure that the selected key set has been provided with keys in both Partner Ports.
 - Once the Transmitter has indicated a change of key set via the K bit, the Receiver must mark the other key set/bank invalid until it is reprogrammed.
 - The specific requirements for the frequency of key updates are determined by platform security requirements that are outside the scope of this document.
 - It is generally recommended that hardware provide enough key storage and management resources to support changing the keys for at least one active Stream without disruption to IDE operation.
 - It is expected that key update operations for multiple streams may need to be performed by firmware/software in a serialized fashion based on hardware key storage and management resource limitations.

IMPLEMENTATION NOTE: UNDERSTANDING THE IDE KEY MANAGEMENT FLOW §

The following diagram illustrates a detailed example key programming flow using the IDE_KM protocol defined above, although it should be understood that there are many possible variations on this flow.

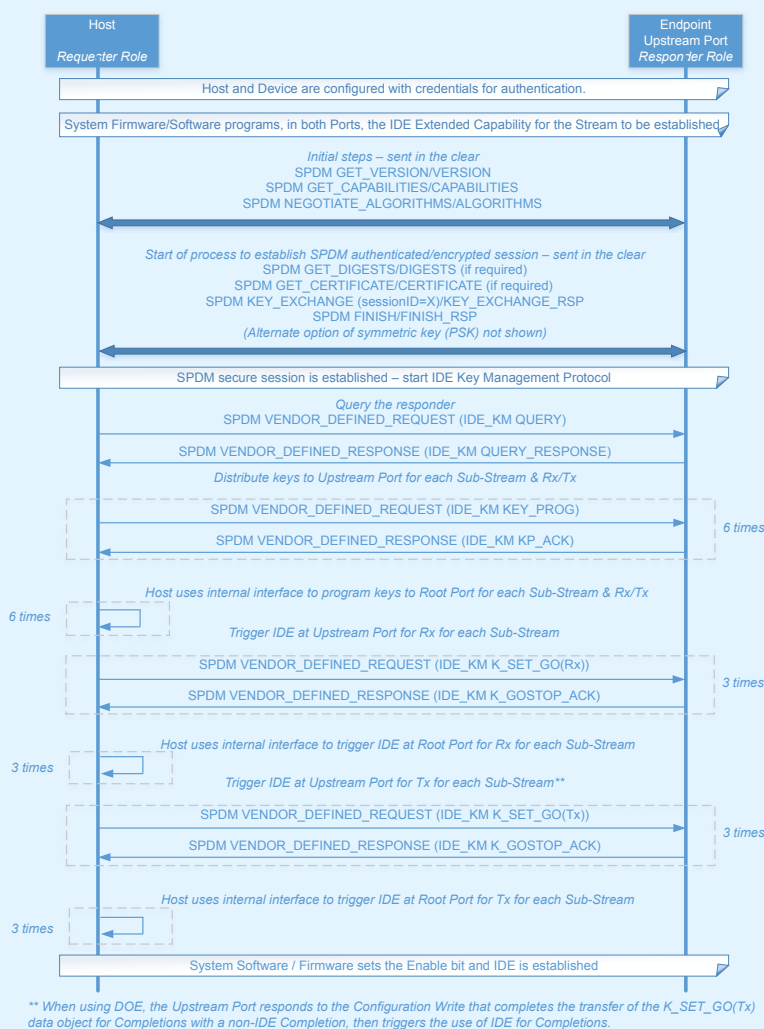


Figure 6-63 IDE_KM Example §

6.33.4 IDE TLPs §

TLPs secured by IDE are called IDE TLPs. In Non-Flit Mode, all IDE TLPs must use the IDE prefix (see § Figure 6-64), and this prefix must precede all other End-End TLP prefixes. In Flit Mode, all IDE TLPs must include OHC-C.

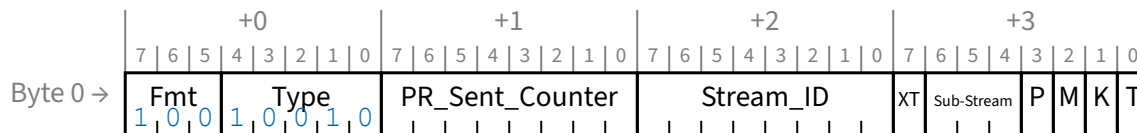


Figure 6-64 IDE TLP Prefix (NFM) §

The IDE Prefix (NFM) includes:

- M bit – When Set, indicates this TLP includes a MAC
 - When aggregation is not used, the M bit must be Set for all IDE TLPs.
 - Rules for the use of aggregation are given below.
- K bit – Indicates the key set used for this TLP
 - After Transmitting a TLP with the K bit toggled for any Sub-Stream, subsequent TLP Transmissions for different Sub-Streams of the same Stream must also use the new value for the K bit.
 - After receiving a TLP with the K bit toggled, the Receiver must transition to the new key and IV set for that TLP and all subsequent TLPs associated with the Sub-Stream, and must mark the old key and IV set invalid until reprogrammed.
 - Such transitions must not affect other Sub-Streams of the IDE Stream at the Receiver.
- XT and T bits – The XT (eXtended TEE) bit and the T (TEE) bit indicate the trusted execution environment (TEE) attributes associated with the TLP (see § Section 11.1).
 - If the TEE-IO Supported bit in the Device Capabilities Register is Clear:
 - It is permitted for IDE TLPs to originate from both trusted and non-trusted execution environments, and the values of the XT and T bits do not modify the handling of TLPs within IDE per se; the rules for trusted execution environments are not defined in this document.
 - The XT and T bits must be Clear unless the use of the XT and T bits have been explicitly defined by TEE management mechanisms outside the scope of this document.
 - If the TEE-IO Supported bit in the Device Capabilities Register is Set, the XT and T bits must be used for TEE management mechanisms as defined in § Chapter 11. .
- P bit – When Set, indicates the TLP includes PCRC.
 - Must only be Set when the M bit is also Set.
- Sub-Stream[2:0] – Indicates the Sub-Stream identifier value¹⁵⁵
- Stream_ID[7:0] – Indicates the associated Stream_ID value
- PR_Sent_Counter[7:0] – For non-UIO Non-Posted Requests and Completions the value must be determined according to the rules below. This field must be Reserved for Posted Requests and for UIO Requests/ Completions.

In Flit Mode:

- The presence of MAC and/or PCRC are indicated using the TS field.
- The K bit, XT bit, T bit, Sub-Stream, Stream_ID, and PR_Sent_Counter are included in OHC-C, and have the same meaning as in the IDE Prefix.

155. In earlier versions of this specification, the Sub-Stream field was 4 bits. See § Section 6.33.5 .

IDE uses Galois/Counter Mode (GCM) as defined in [AES-GCM], referred to as AES-GCM. For IDE TLPs, TLP data payload content forms the “Plaintext”, also known as *P*, as defined in [AES-GCM], and the TLP Header and certain other elements (defined below) form the “Additional Authenticated Data”, also known as *A*, as defined in [AES-GCM].

The Message Authentication Code (MAC)¹⁵⁶ size, also known as *t*, as defined in [AES-GCM], must be 96b (see § Figure 6-65).

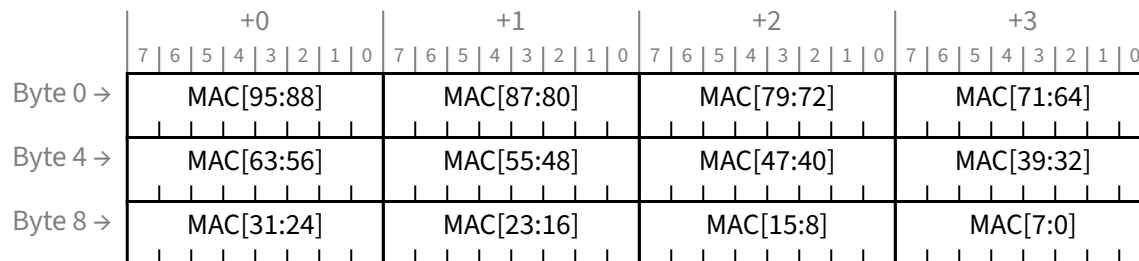


Figure 6-65 MAC Layout §

Flow Control credit accounting for IDE TLPs must handle the MAC as covered by the Header Credit.

For IDE TLPs, AES-GCM can be applied to each IDE TLP, or aggregation can be used to apply AES-GCM to multiple IDE TLPs, reducing the per-TLP overhead for the IDE TLP MAC. For a Link IDE Stream, local prefixes must be covered by the MAC (see § Figure 6-66, § Figure 6-67, § Figure 6-70, and § Figure 6-71). For Selective IDE Streams, local prefixes must not be covered by the MAC (see § Figure 6-68, § Figure 6-69, § Figure 6-72, and § Figure 6-73).

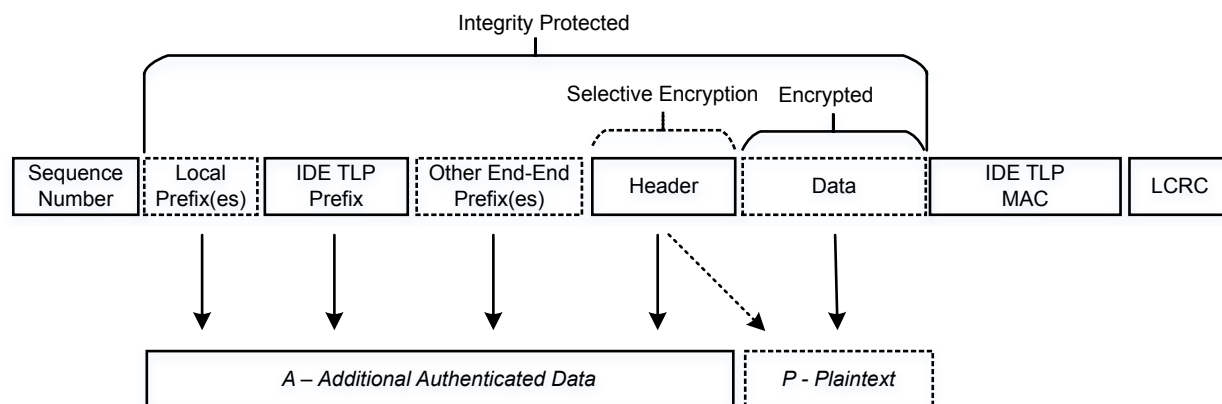


Figure 6-66 Example of IDE TLP for a Link IDE Stream without Aggregation (Non-Flit Mode) §

156. Referred to as “authentication tag” or *T* in [AES-GCM] but renamed here to avoid confusion with other uses of “tag”

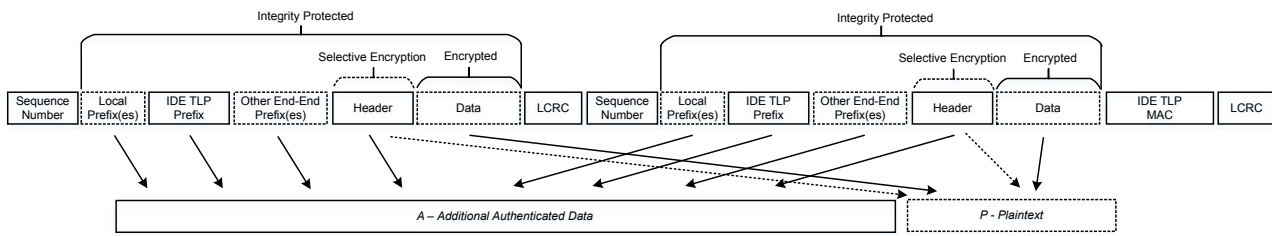


Figure 6-67 IDE TLP – Example Showing Aggregation of Two TLPs for a Link IDE Stream (Non-Flit Mode) §

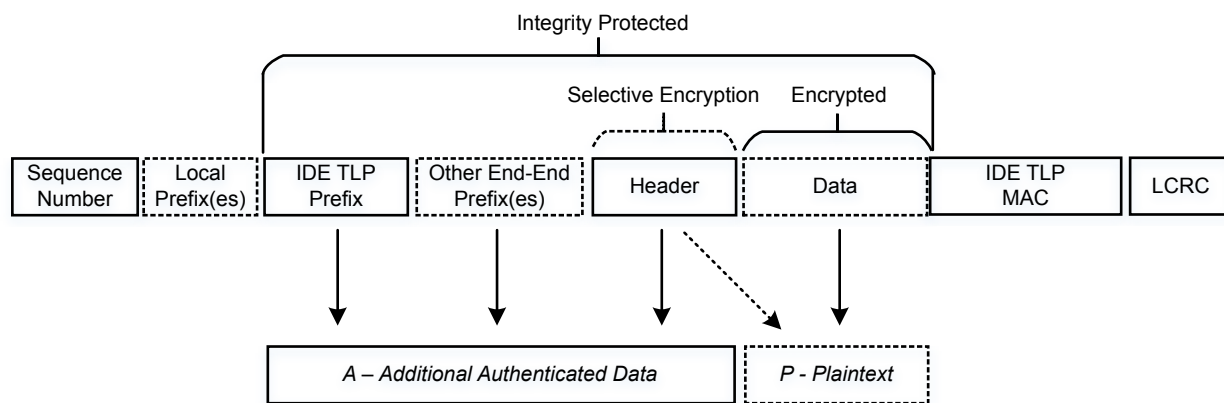


Figure 6-68 IDE TLP – Example of IDE TLP for a Selective IDE Stream without Aggregation (Non-Flit Mode) §

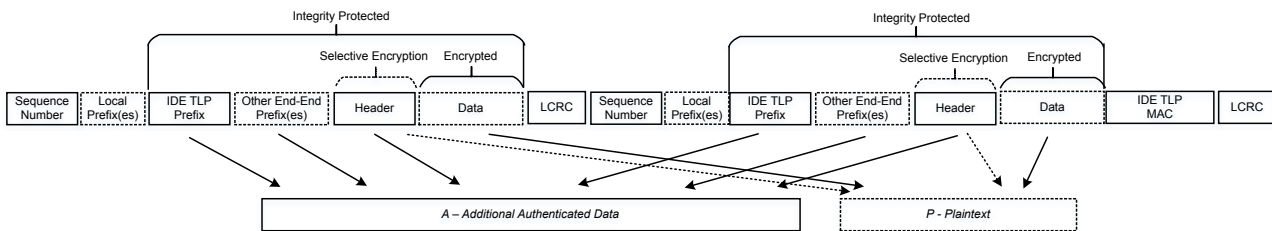


Figure 6-69 IDE TLP – Example Showing Aggregation of Two TLPs for a Selective IDE Stream (Non-Flit Mode) §

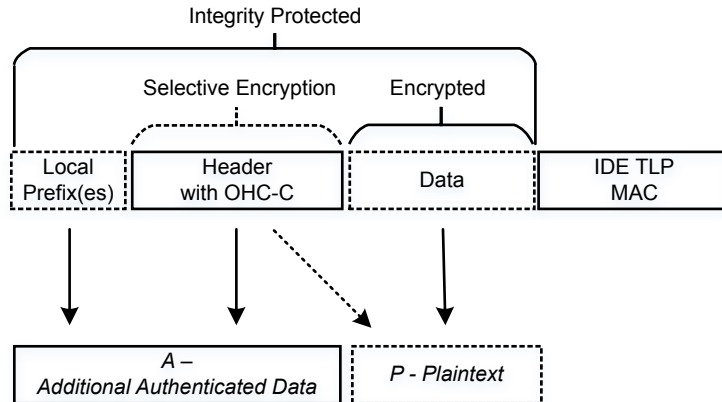


Figure 6-70 Example of IDE TLP for a Link IDE Stream without Aggregation (Flit Mode) §

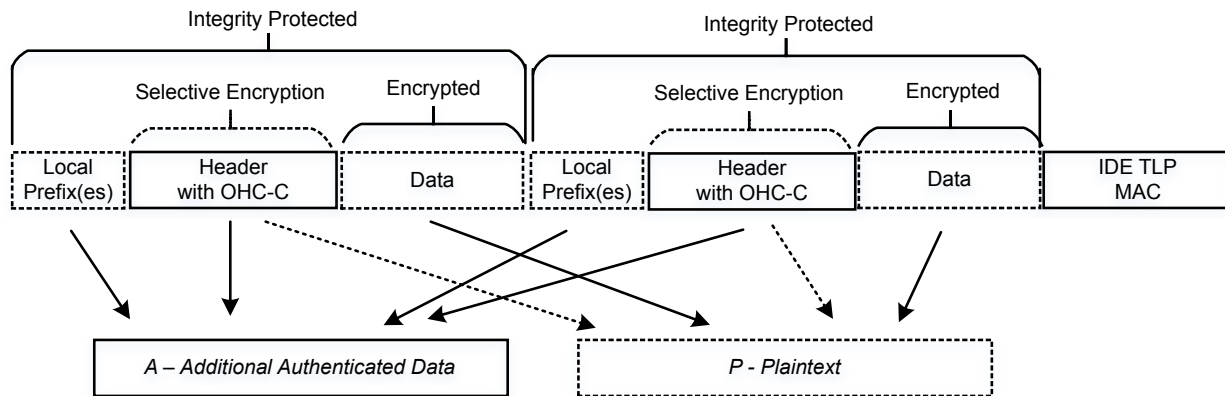


Figure 6-71 IDE TLP – Example Showing Aggregation of Two TLPs for a Link IDE Stream (Flit Mode) §

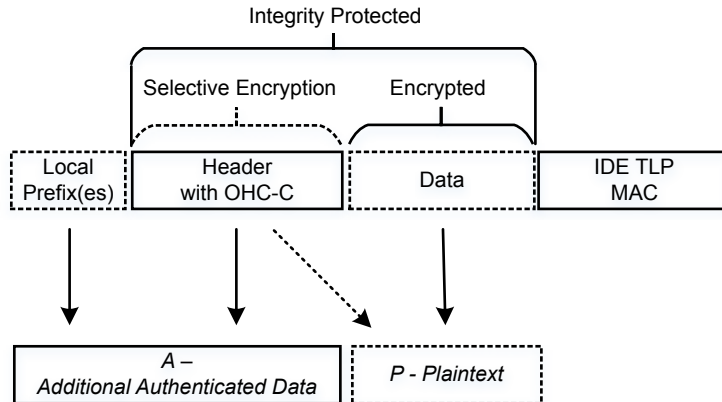


Figure 6-72 IDE TLP – Example of IDE TLP for a Selective IDE Stream without Aggregation (Flit Mode) §

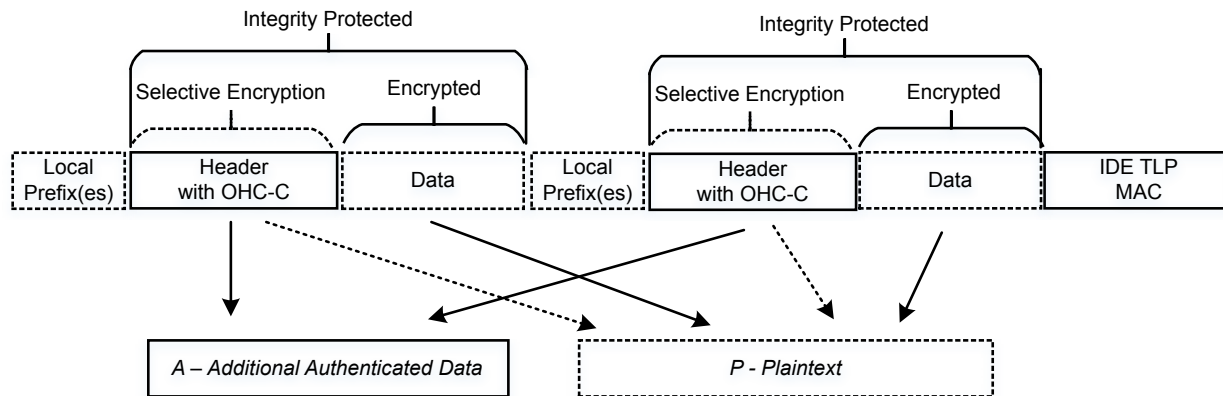


Figure 6-73 IDE TLP – Example Showing Aggregation of Two TLPs for a Selective IDE Stream (Flit Mode) §

The inputs *A* and *P* must be formed by concatenating the included TLP content in Byte order as defined in § Section 2.1.2. Although the *A* and *P* content is conceptually concatenated as illustrated in these figures, the content placement in the IDE TLPs is the same as in non-IDE TLPs. Once the *A* and *P* content is constructed, [AES-GCM] defines how *A* and *P* must be padded – this padding is not illustrated here, and the padding is used in the [AES-GCM] calculations but is not included in the TLPs transmitted/received. When aggregation is used, the *A* and *P* content for aggregated TLPs is conceptually concatenated, for each type of content, prior to padding.

Partial header encryption provides the ability to reduce potential exposure to side-channel attacks by encryption some portions of the Header of an IDE Memory Request while maintaining information that is required for TLP routing and low-level TLP processing in the clear. § Figure 6-74 illustrates, at a high level, the application of partial header encryption.

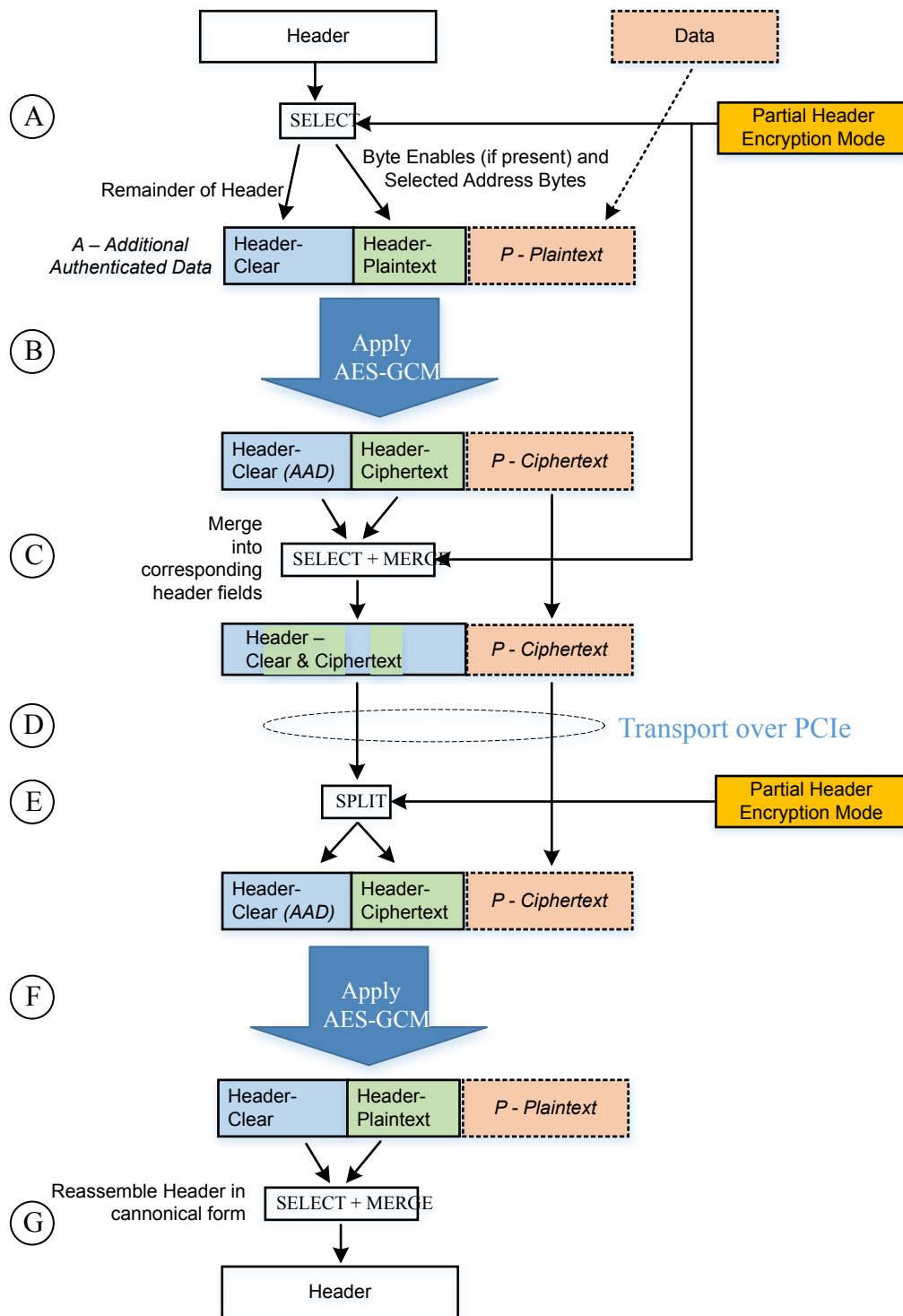


Figure 6-74 High Level Flow For Partial Header Encryption §

In partial header encryption, the encrypted portions of the Header are determined by the setting in the Partial Header Encryption Mode field of the Selective IDE Stream Control Register or Link IDE Control Register.

Rules for partial header encryption:

- When encrypted, the First DW BE and Last DW BE fields must be in the first byte of *P*
 - In NFM, the First DW BE and Last DW BE fields must be encrypted in all Memory Requests, except for:
 - AtomicOp Requests
 - Translation Requests
 - Memory Read/DMWr Requests with the TH bit Set
 - In FM, for Memory Requests, if OHC-A1 is present, then the First DW BE and Last DW BE fields must be encrypted.
- Address bits selected for encryption must follow the First DW BE and Last DW BE fields, if included, in *P*, and are formed as:
 - Address[17:2]:
 - Byte +0: Address[17:10]
 - Byte +1: Address[9:2]
 - Address[25:2]:
 - Byte +0: Address[25:18]
 - Byte +1: Address[17:10]
 - Byte +2: Address[9:2]
 - Address[33:2]:
 - Byte +0: Address[33:26]
 - Byte +1: Address[25:18]
 - Byte +2: Address[17:10]
 - Byte +3: Address[9:2]
 - Address[41:2]:
 - Byte +0: Address[41:34]
 - Byte +1: Address[33:26]
 - Byte +2: Address[25:18]
 - Byte +3: Address[17:10]
 - Byte +4: Address[9:2]
- In the case of Translation Requests, header Byte 14 and header Byte 15 (see § Section 10.2.2) are handled in the same way as for Memory Reads, including the Reserved bits and the CXL Src bit.
- At the Transmitter, the Header content selected for encryption is concatenated at the front of *P* and removed from *A*, increasing the size of *P* and decreasing the size of *A* accordingly.
- If PCRC is enabled when using partial header encryption:
 - Relative ordering of bits for PCRC input is maintained, with the same Header content used as for *P*.
 - For the PCRC calculation only, the selected portions of the header must be padded to 64bits with 0's in the most significant bits.
- At the Receiver, the operation is reversed in order to apply AES-GCM to the *A* and *C* content and then finally the complete header is reconstructed.

- The PCRC is calculated at the Receiver using the decrypted *P* content and the Header portion padded to 64bits with 0's in the most significant bits.
- When an error causes an IDE TLP with partial header encryption to be logged in the Header Log Register, the header fields selected for partial header encryption are permitted to contain all 0's or the encrypted values, but must not contain the decrypted values unless through implementation specific means it is assured that doing so will not violate any security requirements.

§ Figure 6-75 through § Figure 6-78 illustrate the application of partial header encryption.

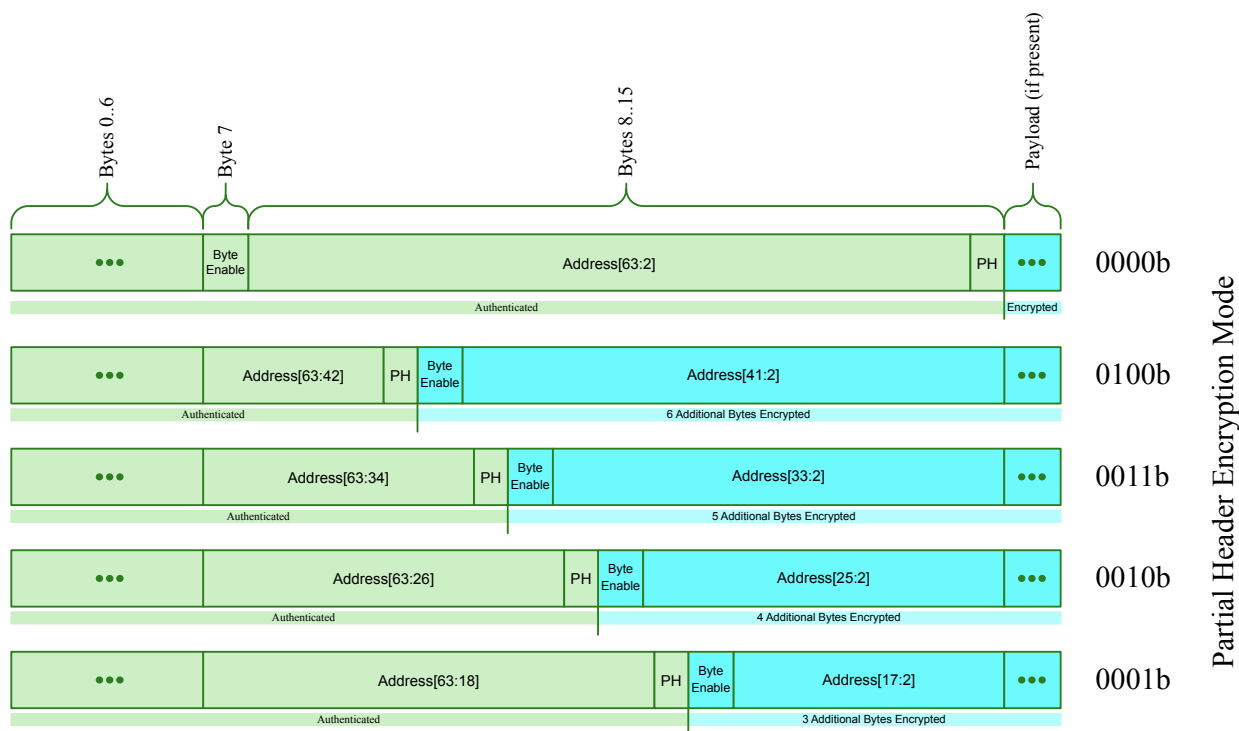


Figure 6-75 Partial Header Encryption in NFM with Byte Enables §

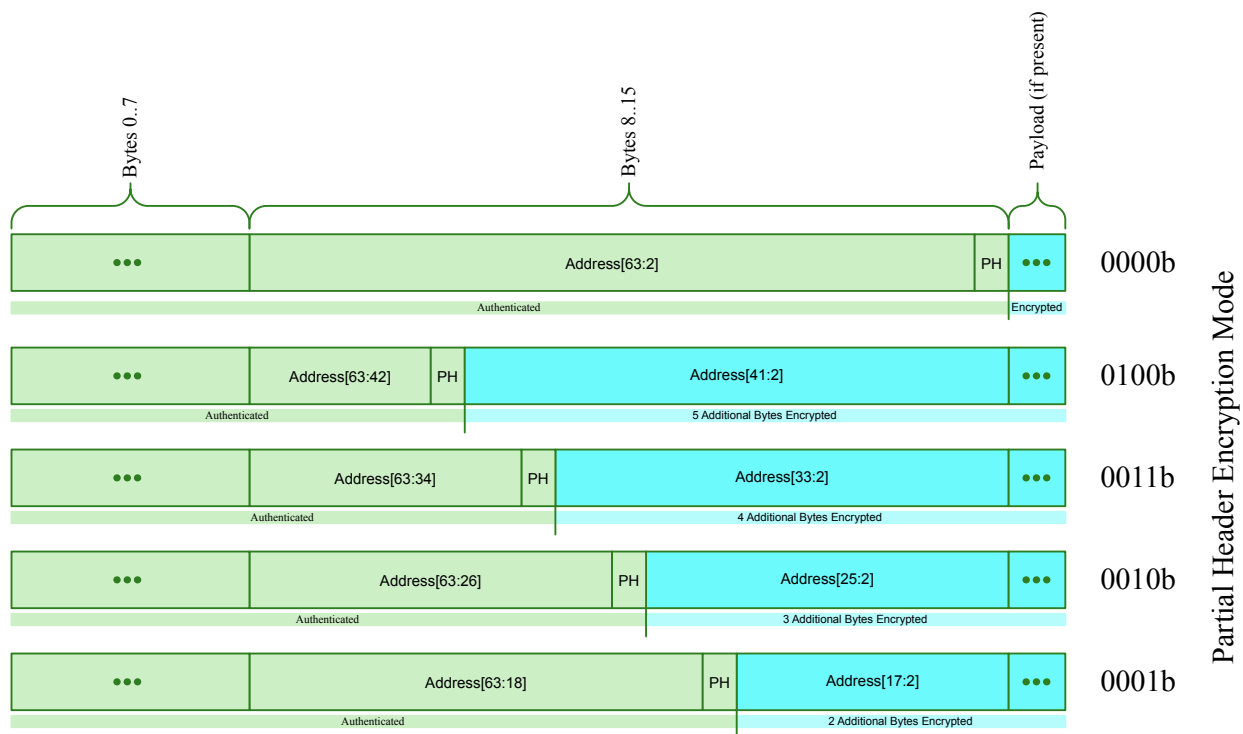


Figure 6-76 Partial Header Encryption in NFM without Byte Enables §

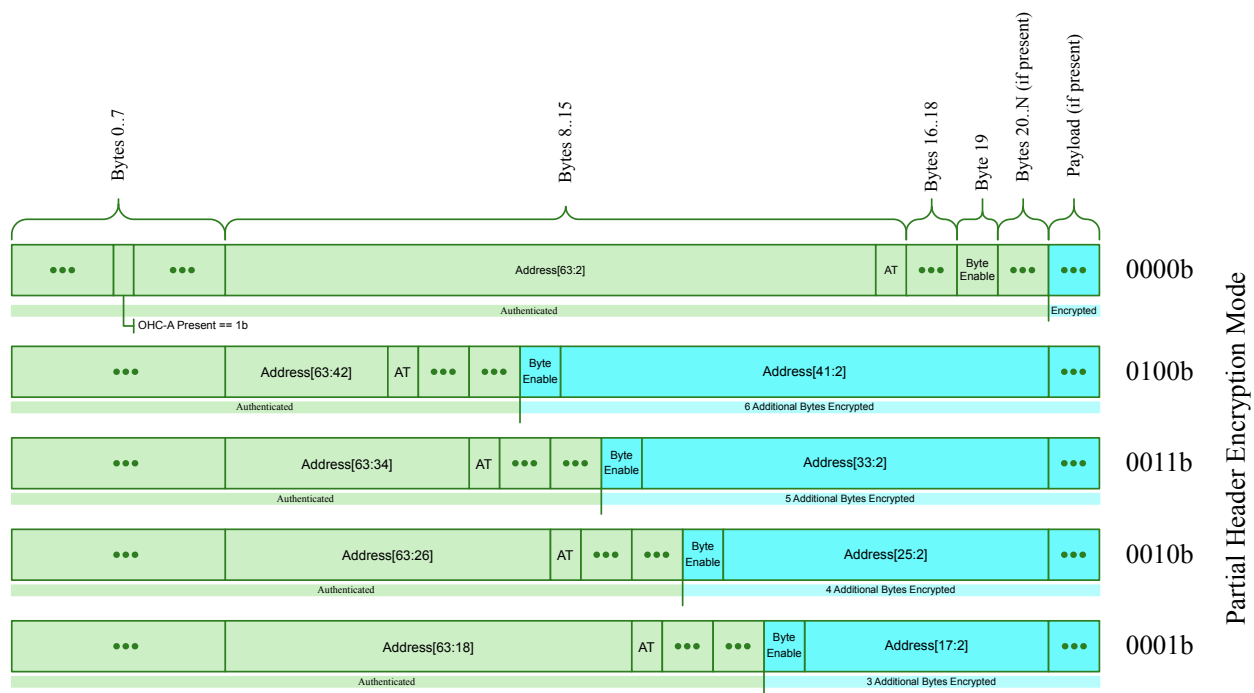


Figure 6-77 Partial Header Encryption in FM with OHC-A1 §

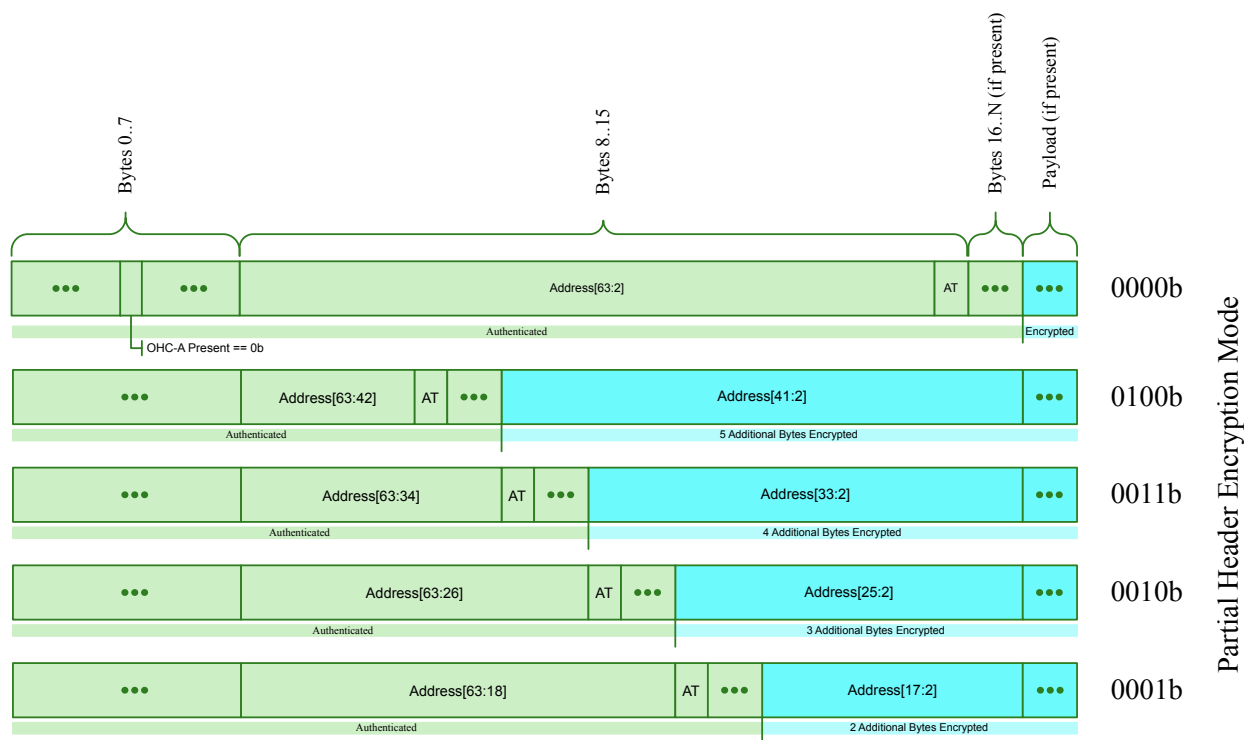


Figure 6-78 Partial Header Encryption in FM without OHC-A1 §

The use of Local TLP Prefixes with IDE TLPs is permitted. Local TLP Prefixes, if present, must precede the IDE Prefix (NFM). For Link IDE Streams, Local TLP Prefixes must be included in A. For Selective IDE Streams, Local TLP Prefixes must not be included in A or P.

The IDE Prefix (NFM) must be included in A. All OHC content (FM) must be included in A.

In NFM, the use of other End-End TLP Prefixes, besides the IDE Prefix, with IDE TLPs is permitted. End-End TLP Prefixes, if present, must follow the IDE Prefix, and must be included in A.

When aggregation is used, all TLPs associated with a single MAC are considered to be part of an “aggregated unit.”

As defined in [AES-GCM], a single invocation is performed for each TLP when aggregation is not used, and for each aggregated unit when aggregation is used.

As with all TLPs, IDE TLPs are covered by Data Link Layer mechanisms, such that physical Link errors are detected and corrected before received TLPs are presented to the Receiver’s cryptographic processing mechanisms.

The use of ECRC with IDE TLPs is not permitted. If ECRC is enabled for non-IDE TLPs, then IDE TLPs must be formed as if ECRC were not enabled, and the TD bit in the TLP Header must be Clear.

To enable the detection of faults in the encryption/decryption logic, which occur outside of the path protected by the MAC, IDE implementations are permitted optionally to support the Plaintext CRC (PCRC) mechanism, for which the following rules apply:

- Software must only enable PCRC when both Partner Ports support the PCRC mechanism.
 - It is permitted to enable the PCRC mechanism on a per-IDE Stream basis.

- When PCRC is enabled for an IDE Stream, all Transmitted TLPs associated with that Stream that include a MAC must also include PCRC, if and only if there is *P* content in the TLP or aggregated unit of TLPs.
 - In NFM, presence of a MAC is indicated by the M bit in the IDE Prefix being Set and presence of the PCRC is indicated by the P bit in the IDE Prefix being Set
 - In FM, the TS field is used to indicate presence of MAC / PCRC (see § Section 2.2.1.2 .
 - When aggregation is used, the TLPs of an aggregated unit that do not include a MAC also must not include PCRC (see § Figure 6-79).

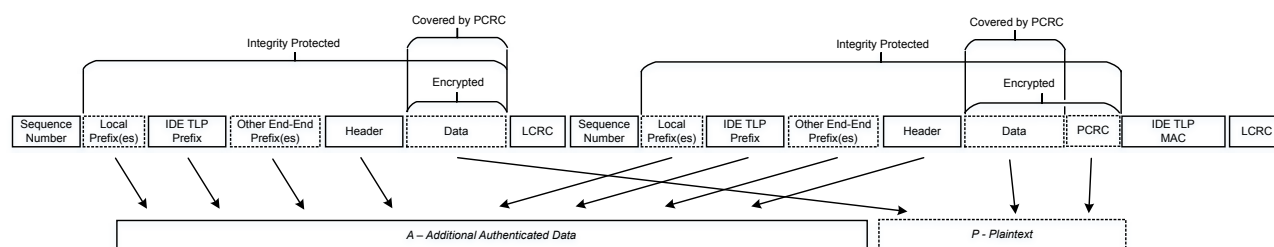


Figure 6-79 Example Illustrating PCRC Application to Two Aggregated IDE TLPs for a Link IDE Stream (NFM) §

- In NFM, when PCRC is enabled for an IDE Stream, the ultimate Receiver must check that all received TLPs or aggregated units associated with that Stream that include *P* content and a MAC (as indicated by the M bit in the IDE Prefix) also have the P bit in the IDE Prefix Set.
 - If, for a TLP, the P bit in the IDE Prefix is Clear and the M bit is Set, the Receiver must report this as a PCRC Check Failed error.
- PCRC must be calculated across all *P* content¹⁵⁷ for a given invocation of AES-GCM, and per the following:
 - The polynomial used has coefficients expressed as 04C1 1DB7h
 - The seed value must be FFFF FFFFh
 - All *P* content must be included in the PCRC calculation
 - PCRC calculation starts with bit 0 of byte 0 and proceeds from bit 0 to bit 7 of each byte of *P*
 - The result of the PCRC calculation must be complemented, mapped as shown in § Table 2-55 (following the same mapping as for ECRC), and appended to the *P* content, and encrypted/decrypted along with the other *P* content, with the encrypted PCRC value appended to the other *P* content and preceding the MAC (see § Figure 6-79).
- The PCRC must only be checked by the ultimate Receiver of the IDE TLP including PCRC.
 - A failure of the PCRC check indicates that one or more bits of the data payload have been corrupted — the Receiver's use of the data payload is outside the scope of this specification, but it is strongly recommended that the corrupted data not be used as if it were uncorrupted.
 - PCRC Check Failed is a reported error.
- Flow Control credit accounting for IDE TLPs that include PCRC must handle the PCRC as covered by the Header Credit.

157. As with ECRC, the PCRC is calculated using the TLP as-transmitted, including, for any data payload, all bytes as-transmitted regardless of the byte enable values.

IMPLEMENTATION NOTE: PCRC REUSE OF ECRC LOGIC §

In most respects, PCRC is calculated in the same way as ECRC, and the CRC polynomial is the same.

In some implementations it may be possible to reuse the same logic for PCRC calculation as is used for ECRC, when both PCRC and ECRC are supported.

ECRC is not allowed on IDE TLPs, and so no TLP could have both PCRC and ECRC.

These rules apply to Selective IDE Stream TLPs:

- § Table 6-35 defines which TLP types are permitted for a Selective IDE Stream.
 - TLP types that are not permitted for a Selective IDE Stream are still permitted to be used, and can be secured using Link IDE, if enabled.
- Receipt of an IDE TLP associated with a Selective IDE Stream that is not a permitted TLP Type is an IDE Check Failed error.
 - The Receiver must transition to Insecure for the associated IDE Stream.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 6-35 TLP Types for Selective IDE Streams §

TLP Type	Description	Permitted for Selective IDE Streams
MRd	Memory Read Request	Y
MRdLk	Memory Read Request-Locked	Y
MWrr	Memory Write Request	Y
IORd	I/O Read Request	N
IOWr	I/O Write Request	N
CfgRd0	Type 0 Configuration Read Request	N
CfgWr0	Type 0 Configuration Write Request	N
CfgRd1	Type 1 Configuration Read Request	Y
CfgWr1	Type 1 Configuration Write Request	Y
DMWr	Deferrable Memory Write	Y
Msg	Message Request	Y when Routed by ID, Routed by Address, or Routed to Root complex ¹⁵⁸ N for other routing mechanisms
MsgD	Message Request with data	Y when Routed by ID, Routed by Address, or Routed to Root complex ¹⁵⁹ N for other routing mechanisms

158. Routed to Root Complex is permitted only when the Partner Port is a Root Port, as indicated by the Default Stream bit being Set.

159. Routed to Root Complex is permitted only when the Partner Port is a Root Port, as indicated by the Default Stream bit being Set.

TLP Type	Description	Permitted for Selective IDE Streams
Cpl	Completion without Data	Y
CplD	Completion with Data.	Y
CplLk	Completion for Locked Memory Read without Data	Y
CplDLk	Completion for Locked Memory Read – otherwise like CplD.	Y
FetchAdd	Fetch and Add AtomicOp Request	Y
Swap	Unconditional Swap AtomicOp Request	Y
CAS	Compare and Swap AtomicOp Request	Y
LPrfx	Local TLP Prefix	Allowed to be present, but not secured if TLP is associated with a Selective IDE Stream
EPrfx (applicable in NFM only)	End-End TLP Prefix	Y if the TLP is part of a Selective IDE Stream

- All IDE TLPs must be associated with an IDE Stream, identified via an IDE Stream_ID.
 - Software must assign IDE Stream IDs such that two Partner Ports use the same value for a given IDE Stream.
 - Software must assign IDE Stream IDs such that every enabled IDE Stream associated with a given terminal Port is assigned a unique Stream_ID value at that Port
 - It is permitted for a platform to further restrict the assignment of Stream IDs.
- When only a Link IDE Stream is enabled for a particular TC, all TLPs using that TC must be secured using the corresponding Link IDE Stream.
- For a Transmitter to associate a specific TLP with a specific Selective IDE Stream, the following criteria must be satisfied:
 - The Selective IDE Stream Enable bit in the Selective IDE Stream Control Register must be Set.
 - The TLP type must be permitted for Selective IDE Streams (see § Table 6-35).
 - The TC of the TLP must match the TC value in the Selective IDE Stream Control Register.
 - For a Configuration Request, the Selective IDE for Configuration Requests Enable bit must be Set (applicable to Root Ports only).
 - For a Completion, the Stream_ID must match those in the corresponding Request.
 - If an ACS mechanism in the Port redirects the TLP towards the Root Complex, the Default Stream bit must be Set, indicating that the Selective IDE Stream targets the Root Complex. See § Section 6.12.3 .
 - For a Routed-to-Root-Complex Message, the Default Stream bit must be Set, indicating that the Selective IDE Stream targets the Root Complex.
 - For an ID-Routed Message not associated with a Default Stream, unless there is an exception based on implementation-specific criteria:
 - The destination RID must be greater than or equal to the RID Base and less than or equal to the RID Limit in the Selective IDE RID Association Register block, and
 - In Flit Mode, the destination Segment value must also match the value of Segment Base in the Selective IDE RID Association Register block.

- Unless there is an exception made via implementation specific means, when ATS is supported and enabled, all Translation Requests and Untranslated Memory Requests are associated with the default Stream, otherwise for a Memory Request not already associated with a Default Stream, the destination address must be greater than or equal to the Memory Base value and less than or equal to Memory Limit value in a Selective IDE Address Association Register block (as applies when targeting a specific Function's BAR or the Base/Limit range of addresses assigned to a Device) ¹⁶⁰.
- For a TLP not already associated with any other Stream, the Default Stream bit must be Set.
- The TLP is selected or precluded through implementation specific means. For example, the Transmitter could associate all Memory Requests initiated by one or more internal Functions with a specific Selective IDE Stream, particularly when it is known that the Partner Port is a Root Port, and that all Requests initiated by that internal Function target system memory.
- If the XT Enable bit in the Selective IDE Stream Control Register is Clear, and the TEE-Limited Stream bit in the Selective IDE Stream Control Register is Set, Requests with the T bit Set are associated with this Selective IDE Stream.
 - Requests that otherwise would be associated with this Selective IDE Stream and have the T bit Clear must be Transmitted as non-IDE TLPs if Link IDE is not enabled, or as Link IDE TLPs if Link IDE is enabled.
- If both the XT Enable bit and the TEE-Limited Stream bit are Set in the Selective IDE Stream Control Register, Requests with the XT bit and/or the T bit Set are associated with this Selective IDE Stream.
 - Requests that otherwise would be associated with this Selective IDE Stream and have the XT bit Clear and the T bit Clear must be Transmitted as non-IDE TLPs if Link IDE is not enabled, or as Link IDE TLPs if Link IDE is enabled.
- TLPs not determined by the Transmitter to be associated with a specific Selective IDE Stream or that are precluded through implementation specific means, must not be associated with any Selective IDE Stream.
- If supported, the TEE-Limited Stream bit must be configured in both Partner Ports before enabling a Selective IDE Stream.
 - It is permitted for the Partner Ports to have different values for the TEE-Limited Stream bit.
- Separate TCs must use separate Selective IDE Streams.
- Software must ensure that Selective IDE Streams are assigned to RID and address ranges that are not overlapping.
 - If software violates this rule by programming overlapping ranges, hardware must associate a given TLP with one Selective IDE Stream matching the RID/address range, but it is implementation specific which Selective IDE Stream from the overlapping set is associated.

These rules apply to IDE Fail Messages:

- Receivers must accept IDE Fail Messages received as IDE TLPs and also as non-IDE TLPs.
- When an IDE Fail Message is to be transmitted on an enabled Stream that is in the Insecure state, the Message must be transmitted as a non-IDE TLP. This includes cases where the transition of that Stream to Insecure was the trigger for the IDE Fail Message.
- When an IDE Fail Message is to be transmitted on an enabled Stream that is in the Secure state, the Message must be transmitted as an IDE TLP. This includes cases where the transition to Insecure on a Selective IDE Stream is the trigger for the IDE Fail Message, and the transmission Stream is either a different Selective IDE Stream or a Link IDE Stream.

These rules apply to TLPs other than IDE Fail Messages:

160. As with the Base/Limit address routing mechanism for Type 1 (Bridge) Functions, the IDE Address Association mechanism does not consider the value of the AT field to determine TLP routing.

- When ready to schedule a TLP to be Transmitted that is associated with an enabled Stream that is in the Insecure state, the Transmitter must instead discard the TLP.
-
- When both Link IDE Stream and one or more Selective IDE Stream(s) are enabled at the same Port, for Transmitted TLPs the Selective IDE Stream(s) take precedence: selected TLPs must be associated with the Selective IDE Stream(s); TLPs not associated with any Selective IDE Stream must use Link IDE.
 - If both Link IDE and Selective IDE are enabled, and a TLP to be Transmitted is associated with an enabled Selective IDE Stream that is in the Insecure state, the Transmitter must not use Link IDE for the TLP and must instead discard the TLP.
- At the ultimate Receiver, IDE TLPs must be associated with the Stream_ID indicated in the IDE Prefix (NFM)/OHC-C (FM).
- Once Link IDE has been enabled (see § Section 6.33.3) for a particular TC, for each Sub-Stream of the Stream, until the Port receives an IDE TLP on that TC and that Sub-Stream, the Port must continue to accept non-IDE TLPs on that TC of the FC type corresponding to that Sub-Stream.
 - Once the Port receives an IDE TLP on a Sub-Stream it must silently discard subsequent non-IDE TLPs on that Sub-Stream for as long as the associated Stream is enabled. This silent discard operation takes precedence over reporting errors associated with these non-IDE TLPs (e.g., ECRC, Poison)¹⁶¹.
- For Root Ports, when Selective IDE for Configuration Requests Enable for a particular Selective IDE Stream is Set, all Configuration Requests that match that Selective IDE Stream must be Transmitted as IDE TLPs associated with that Selective IDE Stream.
 - After enabling Selective IDE for Configuration Requests it is recommended that system software perform a Configuration Request using that Selective IDE Stream, so that the Partner Port will be triggered to reject subsequent Configuration Requests not associated with the Selective IDE Stream.
 - How the Root Complex ensures that only authorized system software generates Configuration Requests is outside the scope of this document.
- Once Selective IDE for Configuration Requests Enable for a particular Selective IDE Stream is Set by system software, it is strongly recommended that system software not Clear the bit for as long as the Selective IDE Stream itself is enabled.
- Specific Selective IDE Streams on Root Ports for which Selective IDE for Configuration Requests Enable is Set must transmit Configuration Requests that are associated with those Selective IDE Streams as Type 1 Configuration Requests.
- Configuration Requests associated with a Selective IDE Stream will be received at an Upstream Port as Type 1 Configuration Requests.
- Configuration Requests received as IDE TLPs that are directed to pass through the Switch must pass without modification through a Switch when Flow-Through IDE Stream Enabled is Set in both the Upstream Port and the Egress Downstream Port.
- Configuration Requests that are not directed to pass through a Switch must be accepted by the Receiver provided the Target RID is an implemented Function either associated with that Upstream Port or a Downstream Port of a Switch in that Upstream Port. If the Target RID is not an implemented Function, the Configuration Request must be handled as an Unsupported Request.
- For Upstream Ports that are the IDE Terminus of a Selective IDE Stream with Selective IDE for Configuration Requests Supported Set, until the Port receives a Configuration Request as an IDE TLP on that Selective IDE Stream, the Port must continue to accept Configuration Requests as non-IDE TLPs. Once a Configuration Request has been received as an IDE TLP on that Selective IDE Stream, then the Port must accept only

161. Earlier versions of this specification required discarding non-IDE TLPs but did not specify whether error reports were suppressed. Such behavior is still permitted but it is recommended to lower the impact of non-IDE TLPs by avoiding consequences (e.g., reporting ECRC errors).

Configuration Requests associated with that Selective IDE Stream, and must discard all Configuration Requests received on other IDE Streams or as non-IDE TLPs, for as long as that Selective IDE Stream is enabled.

- For Selective IDE, for TLP types other than Configuration Requests, the requirements for rejecting TLPs are determined by the TEE associated with the Selective IDE Stream (see § Section 6.33.1).
- The use of TLP poisoning, as indicated by the EP bit in the TLP header being Set, is permitted for IDE TLPs when applied by the originating Transmitter.
- For IDE TLPs, it is not permitted to modify any part of a TLP, including the EP bit, at any intermediate point between the two Partner Ports.
- Software must configure Selective IDE such that all Links on the entire path between the Partner Ports are operating either in FM, or in NFM.
- If TEE-IO Supported is Set, for an Endpoint Upstream Port:
 - In Flit Mode, if Segment Captured is Set ¹⁶², on a Selective IDE Stream:
 - A Requester must include OHC-C with the Requester Segment Valid (RSV) bit Set.
 - A Completer must include OHC-A5.
 - A Completer receiving a Request on a Selective IDE Stream other than the Default Stream must handle it as an Unsupported Request if the Requester ID field of the Request is less than the RID Base or greater than the RID Limit in the Selective IDE RID Association Register block of the Stream.
 - In Flit Mode, the Requester Segment value (if included in the Request) must also match the value of Segment Base in the Selective IDE RID Association Register block.
 - A Requester receiving a Completion on a Selective IDE Stream other than the Default Stream must handle it as an Unexpected Completion if the Completer ID field of the Completion is less than the RID Base or greater than the RID Limit in the Selective IDE RID Association Register block of the Stream.
 - In Flit Mode, the Completer Segment value (if included in the Completion) must also match the value of Segment Base in the Selective IDE RID Association Register block.
- If TEE-IO Supported is Set, for a Root Port:
 - A Completer receiving a Request on a Selective IDE Stream for which the Root Port is an IDE Terminus must handle it as an Unsupported Request if the Requester ID field of the Request is less than the RID Base or greater than the RID Limit in the Selective IDE RID Association Register block of the Stream.
 - In Flit Mode, if the Requester Segment Valid (RSV) bit is Set, the Requester Segment value must also match the value of Segment Base in the Selective IDE RID Association Register block.
 - A Requester receiving a Completion on a Selective IDE Stream for which the Root Port is an IDE Terminus must handle it as an Unexpected Completion if the Completer ID field of the Completion is less than the RID Base or greater than the RID Limit in the Selective IDE RID Association Register block of the Stream.
 - In Flit Mode, if the Completion includes OHC-A5, the Completer Segment value must also match the value of Segment Base in the Selective IDE RID Association Register block.
- If the XT Supported bit in the IDE Capability Register is Set:
 - If the XT Enable bit for the associated IDE Stream is Clear:
 - The Transmitter must Clear the XT bit in the Transmitted IDE TLPs associated with the Stream.
 - The Receiver should process the TLP as if the XT bit was Clear.
 - If the XT Enable bit for the associated IDE Stream is Set:
 - The Transmitter must include the XT bit in the Transmitted IDE TLPs as defined by the TEE management mechanisms in § Chapter 11. .

162. The value of Segment Captured indicates that Segment values are being applied – the captured Segment value is not used for the tests defined here.

- The Receiver must process the XT bit in the Received IDE TLPs as defined by the TEE management mechanisms in § Chapter 11. .
- If the XT Supported bit in the IDE Capability Register is Clear:
 - The Transmitter must Clear the XT bit in the Transmitted IDE TLPs associated with the Stream.
 - The Receiver should process the TLP as if the XT bit was Clear.

The use of Multicast (see § Section 6.14) with Selective IDE Streams is outside the scope of this specification.

6.33.5 IDE TLP Sub-Streams §

With [AES-GCM] it is desirable to maintain TLPs in-order so that the Transmitter and Receiver can independently maintain IV in sync with each other without the overhead required to transmit the IV for each TLP or aggregated unit. However, certain TLP bypassing is required for deadlock avoidance, and this is reflected in the different types of Flow Control Credit Types – Posted Request header/data payload, Non-Posted Request header/data payload, and Completion header/data payload (see § Section 2.6.1). To provide in-order TLP processing where possible, and to simplify implementations that structure their internal buffering according to these Flow Control Credit types, IDE introduces the concept of a Sub-Stream within which TLP traffic is maintained fully in-order between the IDE Partner Ports. It is ensured within IDE Sub-Streams that TLPs travel in-order between the IDE Partner Ports both to reduce the per-TLP overhead as noted, and also to ensure that certain attack scenarios, such as the reordering or replaying by an Adversary-in-the-Middle, of Posted Requests, will be detected by the Receiver without additional TLP tracking logic.

With [AES-GCM] it is essential to maintain synchronization between the Partner Ports such that all TLPs associated with an IDE Stream are always routed from one Partner Port to the other. If any IDE TLPs are misrouted the result will typically be an unrecoverable error for the associated IDE Stream (see detailed requirements below).

Each IDE Stream includes Sub-Streams distinguished by TLP credit type and direction, for which the following rules apply:

- For each Sub-Stream there is a Sub-Stream identifier:
 - 000b – Posted Requests and UIO Writes
 - 001b – Non-Posted Requests and UIO Reads
 - 010b - Completions
 - Values 011b-110b are Reserved
 - 111b - In NFM, Reserved; In FM, indicates a TLP that includes OHC-C but is not an IDE TLP.
 - In earlier versions of this specification, the Sub-Stream field was 4 bits in Symbol 3, bits 7:4. If the TEE-IO Supported bit in the Device Capabilities Register is Set, components must not treat bit 7 as part of the Sub-Stream field. If the TEE-IO Supported bit in the Device Capabilities Register is Clear, components are permitted – though strongly discouraged – to treat bit 7 as part of the Sub-Stream field.
- For each Sub-Stream, per [AES-GCM], there must be a 96b initialization vector IV of deterministic construction, consisting of:
 - a fixed field in bits 95:64 of the IV, where bits 95:64 are all 0's
 - an invocation field in bits 63:0 of the IV, containing the value of a counter, initially set to the value 0000_0001h for each Sub-Stream upon establishment of the Stream and each time the Sub-Stream key is refreshed, and incremented every time an IV is consumed.
- Each Sub-Stream must support the use of its own unique key value and invocation field initial counter value.

§ Section 6.33.7 defines additional requirements for Switches and Root Complexes that support peer-to-peer routing of Selective IDE Streams.

The ordering rules defined in § Section 2.4 define constraints on TLP/TLP ordering, but do not provide mechanisms to detect improper reordering. With IDE, counters are used to enable the ultimate Receiver to detect improper reordering of non-UIO TLPs while allowing permitted reordering. These counters and associated mechanisms, including error checks, do not apply to UIO TLPs. Separate sets of these counters must be operated for each IDE Stream, and operated according to the following rules:

- For the Transmitter, two 8 bit counters must be maintained: a count of Posted Requests Transmitted since the last Non-Posted Request or IDE Sync Message was Transmitted, called PR_Sent_Counter-NPR, and a count of Posted Requests Transmitted since the last Completion or IDE Sync Message was Transmitted, called PR_Sent_Counter-CPL
 - Upon entry to the Secure state, both counters must be initialized to 0.
 - Both counters must be incremented for each Posted Request IDE TLP Transmitted associated with the IDE Stream.
 - When a Non-Posted Request associated with the IDE Stream is Transmitted, the PR_Sent_Counter-NPR value must be used in the PR_Sent_Counter field of the IDE Prefix (NFM)/OHC-C (FM) for the Non-Posted Request, and then PR_Sent_Counter-NPR must be reset to zero.
 - When a Completion associated with the IDE Stream is Transmitted, the PR_Sent_Counter-CPL value must be used in the PR_Sent_Counter field of the IDE Prefix (NFM)/OHC-C (FM) for the Completion, and then PR_Sent_Counter-CPL must be reset to zero.
 - When either the PR_Sent_Counter-NPR or the PR_Sent_Counter-CPL reaches 245, an IDE Sync Message (see § Section 2.2.8.11) must be transmitted to the Partner Port as an IDE TLP.
 - It is permitted to Transmit an IDE Sync Message at other times.
 - Every time an IDE Sync Message is Transmitted to the Partner Port, both the PR_Sent_Counter-NPR and PR_Sent_Counter-CPL must be incremented prior to forming the IDE Sync Message, and then both the PR_Sent_Counter-NPR and PR_Sent_Counter-CPL must be reset to zero.
- For the Receiver, two 64 bit counters must be maintained: a count of Posted Requests received since the last Non-Posted Request was received, called PR_Received_Counter-NPR and a count of Posted Requests received since the last Completion was received, called PR_Received_Counter-CPL
 - Upon entry to the Secure state, both counters must be initialized to zero.
 - Both counters must be incremented for each Posted Request IDE TLP received associated with the IDE Stream.
 - When a Non-Posted Request associated with the IDE Stream is received then the PR_Sent_Counter value carried in the IDE prefix (NFM)/OHC-C (FM) must be subtracted from the PR_Received_Counter-NPR, and the PR_Received_Counter-NPR updated with the result. If this subtraction underflows, this is an error – see rules related to error handling below.
 - When a Completion associated with the IDE Stream is received then the PR_Sent_Counter value carried in the IDE prefix (NFM)/OHC-C (FM) must be subtracted from the PR_Received_Counter-CPL, and the PR_Received_Counter-CPL updated with the result. If this subtraction underflows, this is an error – see rules related to error handling below.
 - When an IDE Sync Message associated with the IDE Stream is received then:
 - the PR_Sent_Counter-NPR value carried in the IDE Stream Sync Message must be subtracted from the PR_Received_Counter-NPR, and the PR_Received_Counter-NPR updated with the result,
 - the PR_Sent_Counter-CPL value carried in the IDE Stream Sync Message must be subtracted from the PR_Received_Counter-CPL, and the PR_Received_Counter-CPL updated with the result.

If either subtraction underflows, this is an error – see rules related to error handling in § Section 6.33.7.

IMPLEMENTATION NOTE: DETECTION OF IMPROPER REORDERING §

As with other TLPs, IDE TLPs need to be reordered to satisfy requirements for deadlock avoidance, but some other forms of reordering are forbidden as IDE TLPs pass over PCIe between Ports. These ordering requirements are defined in § Table 6-36, and are stated in terms of Posted Requests (PR), Non-Posted Requests (NPR), and Completions (Cpl). The following examples illustrate selected reordering cases.

An attack based on TLP reordering (or a delay that has the effect of reordering) can be implemented using a variety of mechanisms that all result in the same observed behavior, and will be detected using the mechanisms defined by IDE.

§ Figure 6-80 illustrates the case where a Posted Request is allowed to bypass a Non-Posted Request, as is required for deadlock avoidance. IDE supports having Posted Requests bypass Non-Posted Requests through the use of Sub-Streams within a given Stream. There is a similar requirement for Posted Request to be able to bypass Completions (not illustrated).

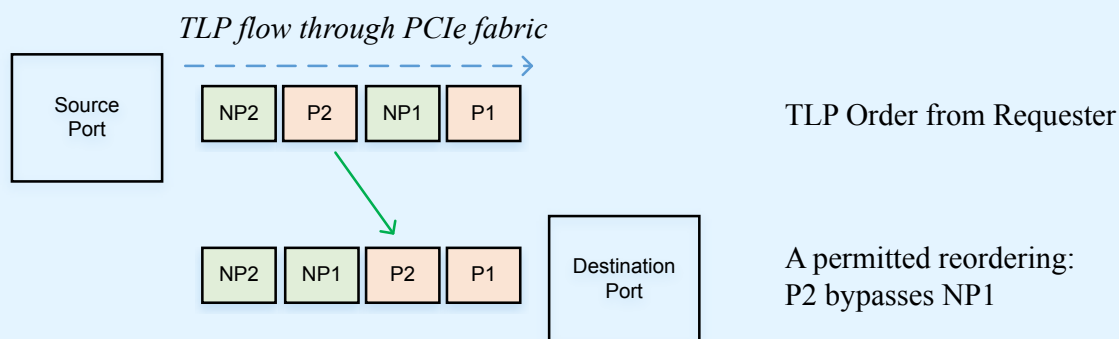


Figure 6-80 Example – Posted Requests Allowed to Bypass Non-Posted Requests §

§ Figure 6-81 illustrates a case where an attacker attempts to bypass a Posted Request with a Non-Posted Request, which could, for example, cause the consumption of stale data. This case will be detected through the use of the PR Sent Counter mechanism, through which the Transmitter at source Port associated with the Stream indicates to the Receiver at the Destination Port how many Posted Requests were transmitted between successive Non-Posted Requests. This indication is carried in the IDE TLP Prefix (NFM)/OHC-C (FM), which is integrity protected and so cannot be modified without detection at the Receiver. In this example, NP1 will carry the indication that a Posted Request (P1) was transmitted, and this will not match the Receiver's count of Posted Requests received, enabling this illegal reordering to be detected.

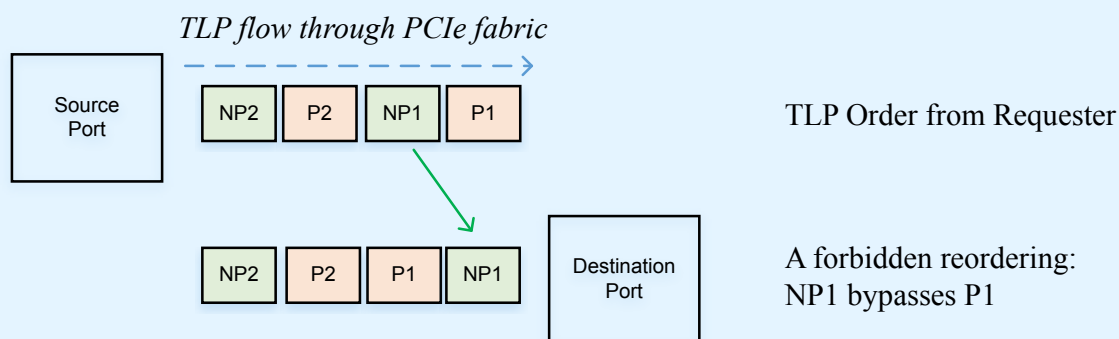


Figure 6-81 Example – Non-Posted Requests Never Allowed to Bypass Posted Requests §

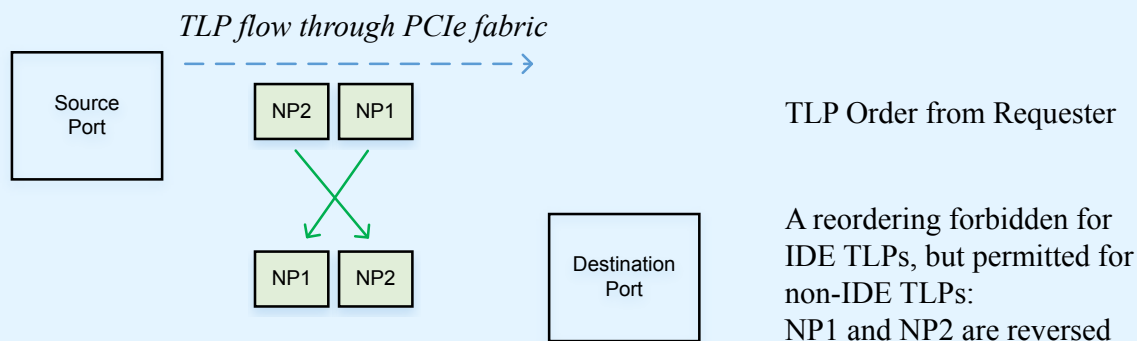


Figure 6-82 Example – Secure Non-Posted Request Reordering Not Allowed Over PCIe Fabric §

Without IDE, Non-Posted Requests are allowed to bypass each other, but within an IDE Stream, reordering of TLPs of the same type is disallowed in order to simplify the operation of the integrity/encryption mechanisms. § Figure 6-82 illustrates that this will cause the reordering of Non-Posted Requests to be detected as an integrity check failure, even though there isn't a security exposure per se.

Note that reordering attacks are possible through Retimers, Switches, and any other device or equipment that can alter the flow of TLPs at any point between the originating Port and the Destination Port.

The counters used to implement the reordering checks are of different sizes in the Transmitter and Receiver. The Transmitter counters only need to be 8 bits wide in order to accommodate the arbitrarily chosen limit of 245 that triggers the required transmission of an IDE Sync Message. Because routing elements can modify the relative order of TLPs in different Sub-Streams, Receivers cannot assume any particular limit on the amount of reordering that can occur, and so the Receiver counters were chosen to be 64 bits wide so as to ensure that under normal operating conditions it is not possible for them to overflow.

6.33.6 IDE TLP Aggregation §

The following rules relate to aggregation (see § Figure 6-67 and § Figure 6-69):

- When aggregation is supported, as indicated at the Port level by the Aggregation Supported bit in the IDE Capability Register, then a Receiver must be capable of supporting:
 - the aggregation within any Sub-Stream of up to the lesser of 8 TLPs, or as many TLPs as can be received based on the outstanding Flow Control credits applicable to that Sub-Stream
 - the receipt of other TLPs that are not part of the Sub-Stream between the TLPs of the aggregated unit.
- When aggregation is supported, as indicated at the Port level by the Aggregation Supported bit in the IDE Capability Register, then for each Sub-Stream where aggregation is to be permitted software must enable aggregation at the Transmitting Port.
- If aggregation is to be used, Software must enable aggregation prior to enabling the IDE Stream.
- When aggregation is enabled, a Transmitter:
 - must apply aggregation only among TLPs within the same Stream and Sub-Stream,
 - must aggregate at most the selected number of TLPs in the corresponding Aggregation Mode field,
 - must limit the number of TLPs aggregated such that the sum of the data payloads of all TLPs of an aggregated unit does not exceed 256 DW,
 - is permitted to Transmit other TLPs that are not part of the Sub-Stream between the TLPs of the aggregated unit,
 - must, when transmitting an IDE Sync Message, treat the IDE Sync Message as the last TLP of an aggregated unit,
 - is permitted to aggregate fewer TLPs than the number permitted.
- When aggregating TLPs, the Transmitter must include a MAC only for the last TLP of the aggregated unit, for which the included MAC must cover the A and P content for all the TLPs of the unit.
 - The Transmitter must treat a TLP as the last TLP of an aggregated unit unless the Transmitter can guarantee that it will transmit another TLP within the aggregated unit within 1 μ s.
- The same key and IV set must be used for all TLPs in an aggregated unit.
- If the K bit is to be toggled, it must only be toggled for the first TLP of an aggregated unit.
 - Receivers must check for violations of this rule – a violation is an IDE Check Failed error.
 - The Receiver must transition to Insecure for the associated IDE Stream.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).
- It is permitted for the TLPs of an aggregated unit to be interleaved with other TLPs, including IDE TLPs associated with other Streams, or non-IDE TLPs.
- All aggregated TLPs of a unit must be processed before it is possible for the Receiver to complete authenticated decryption of the unit.
- If an IDE TLP without a MAC is received at a Receiver where Aggregation is not supported, or if the Receiver detects a violation of any of the rules in this section, this is an IDE Check Failed error.
 - The Receiver must transition to Insecure for the associated IDE Stream.

IMPLEMENTATION NOTE:

USE OF AGGREGATION §

To reduce the bandwidth overheads associated with the MAC, the use of aggregation is encouraged. Although aggregation may increase the latency for a Receiver to make use of the received TLPs, typically this increase will not be significant, and in many cases the overall performance should be improved through the use of aggregation.

Although the specific optimizations will depend on traffic types and use model requirements, typically aggregating about 4 TLPs will provide a good balance between the possible bandwidth improvement and the added latency. Typically a Transmitter policy should assume that the Receiver will buffer all the TLPs of an aggregated unit until the Receiver checks are complete before releasing any of the TLPs for further processing. Transmitters can reduce the effective latency impacts of aggregation when there is knowledge of the underlying traffic, for example by ensuring that a doorbell or other “trigger” TLP is either not aggregated, or that an aggregated unit is ended with the transmission of such TLPs.

Receivers should be designed to ensure that aggregated units of TLPs can be buffered with minimal stalling. Typically this implies that the amount of aggregation a Receiver supports should be significantly less than the amount of Receiver Flow Controller buffering advertised.

The IDE TLPs of an aggregated unit may be interleaved with other TLPs, and in some cases this may be required. For example, if we consider a case where a Switch receives a number of aggregated Memory Reads followed by a Memory Write, all of which target the same egress Port. If, at the egress Port, there are insufficient Flow Control credits to transmit all of the Memory Reads, then the Switch may be required to allow the Memory Write to bypass the blocked Memory Reads. Receivers are required to be tolerant to such “interruptions” of an aggregated unit.

6.33.7 Flow-Through Selective IDE Streams §

A Switch or Root Complex is permitted to support Flow-Through Selective IDE Streams without supporting operation as an IDE Terminus (i.e., it is permitted for a Switch or RC to have Flow-Through IDE Stream Supported Set, Link IDE Stream Supported Clear, and Selective IDE Streams Supported Clear). It is also permitted for a Switch/Root Port that is enabled for flow-through IDE to be an IDE Terminus, and in such cases, flow-through IDE TLPs must be routed without modification, and only TLPs for which the Switch/Root Port is the source or ultimate destination must be handled by the Port as an IDE Terminus.

A Switch that supports Flow-Through Selective IDE Streams must implement the IDE Extended Capability at every Port of the Switch.

Switches and RCs that support Flow-Through IDE Streams must, when enabled, implement modified ordering rules defined in § Table 6-36, and § Table 6-37 applied per-Stream, for IDE TLPs that pass through the Switch/RC from a given Ingress Port to a given Egress Port. The entries A2, B3, B4, C3, C4, and D5 are all No to ensure that there is no reordering within any Sub-Stream. In all other cases, the rules defined in § Section 2.4 must be followed; for example: between different Stream IDs, different Ingress Ports, different Egress Ports, or between IDE TLPs and non-IDE TLPs.

Hardware is not required to follow this modified ordering model beyond the TLP flow between the two Partner Ports for an IDE Stream, e.g. within an Endpoint or RC, and so at the system level it must not be assumed that the ordering behavior observed will match the modified IDE Ordering model.

Although Switches/RCs must not reorder IDE TLPs within a Flow-Through IDE Stream based on Relaxed Ordering or IDO, including when ACS mechanisms are being used, it is permitted for those TLPs to have the RO and/or IDO bits Set.

The IDE Sync Message, like all other Messages, is a Posted Request, and the IDE protocol depends on the IDE Sync Message being ordered with all other Posted Requests for a given IDE Stream.

Table 6-36 IDE Revised Ordering Rules for Flow-Through non-UIO IDE Streams - Per Stream §

Row Pass Column?		Posted Request (Col 2)	Non-Posted Request		Completion (Col 5)
			Read Request (Col 3)	NPR with Data (Col 4)	
Posted Request (Row A)		No	<u>Yes</u>	<u>Yes</u>	a) <u>Y/N</u> b) <u>Yes</u>
Non-Posted Request	Read Request (Row B)	No	No	No	<u>Y/N</u>
	NPR with Data (Row C)	No	No	No	<u>Y/N</u>
Completion (Row D)		No	<u>Yes</u>	<u>Yes</u>	No

Table 6-37 IDE Revised Ordering Rules for Flow-Through UIO IDE Streams - Per Stream §

Row Pass Column?	UIO PR-FC TLP (Col U1)	UIO NPR-FC TLP (Col U2)	UIO Completion (Col U3)
UIO PR-FC TLP (Row UA)	No	Yes/No	Yes/No
UIO NPR-FC TLP (Row UB)	Yes/No	No	Yes/No
UIO Completion (Row UC)	Yes	Yes	No

Switches/RCs must not modify Flow-Through IDE TLPs between the ingress and egress Ports.

Switches/RCs must only route IDE TLPs through Ports with the Flow-Through IDE Stream Enabled bit Set. If an IDE TLP is received by an Ingress Port or routed to an Egress Port, and the Port's Flow-Through IDE Stream Enabled bit is Clear, the Port must handle the TLP as a Misrouted IDE TLP error unless there is a higher precedence error. Further:

- For an Ingress Port, the Port must not forward the IDE TLP.
- For an Egress Port, the Port must not Transmit the IDE TLP.
- If the IDE TLP is a Non-Posted Request or a UIO Request, the Port must not return a Completion.

The use of ACS in combination with Flow-Through IDE Streams is permitted, but requires care to ensure that the modified ordering defined above will be satisfied. It should also be understood that, because Relaxed Ordering does not apply within the modified ordering rules defined above, certain use cases such as those involving ACS P2P Completion Redirect are likely to have reduced performance.

Because hardware used with IDE will typically be required to satisfy platform-level trust requirements, in many cases ACS is not required to achieve and maintain secure platform behaviors when IDE is in use.

6.33.8 Other IDE Rules §

Rule for Non-Posted IDE Requests and UIO Requests:

- If the TEE-IO Supported bit in the Device Capabilities Register is Clear, Requesters of Non-Posted Requests and UIO Requests must check that the corresponding received Completion(s) be returned using the same Stream_ID and the same T bit value as was associated with the Non-Posted Request – a violation is an IDE Check Failed error.
 - The Requester must transition to Insecure for the associated IDE Stream.
- If the TEE-IO Supported bit in the Device Capabilities Register is Set, Requesters of Non-Posted Requests and UIO Requests must check that the corresponding received Completion(s) be returned using the same Stream_ID as was associated with the Non-Posted Request – a violation is an IDE Check Failed error.
 - The Requester must transition to Insecure for the associated IDE Stream

The following rules relate to resets:

- Any Conventional Reset to an Upstream Port or to the Bridge Function of a Downstream Port, or any FLR to a Function containing an IDE Extended Capability, must result in all IDE Streams associated with that Function transitioning to the Insecure state, and all keys must be invalidated and rendered unreadable.
 - Additional implementation specific mechanism are required in many cases to ensure security of all associated data is maintained.
- An FLR to a Function that does not contain an IDE Extended Capability must not affect IDE operation
 - In some cases IDE_KM can be affected by an FLR to a Function that does not contain an IDE Extended Capability, but does implement the IDE_KM responder role via DOE – see § Section 6.33.3 .

Use of mechanisms that result in the blocking or termination of TLPs, such as exist for AtomicOps, DMWr, and the End-End TLP Prefix Blocking mechanism, must be carefully coordinated with the use of Selective IDE Streams, to avoid the dropping of Selective IDE TLPs in such a way that would result in a IDE Check Failed error.

The following rules relate to the use of Access Control Services (ACS - see § Section 6.12) with IDE.

- When Link IDE is used, ACS mechanisms are permitted to be used as architected without restrictions.
- If Selective IDE is used with ACS to enable Direct I/O as documented in the Implementation Note: ACS Redirect and Guest Physical Addresses (GPAs) (see § Section 6.12.4), ACS mechanisms are permitted to be used as architected without restrictions.
- If Selective IDE is used for P2P communication without any ACS redirect mechanisms enabled, the remaining ACS services are permitted to be used as architected without restrictions.
- Use of Selective IDE for P2P communication with ACS redirect mechanisms enabled has a number of major issues with ordering and portions of Sub-Streams taking different paths. Such use is out of scope of this specification.
- Use of Selective IDE under any use case where ACS services (or any other mechanism) blocks or otherwise terminates IDE TLPs will result in the associated Selective IDE Stream going to Insecure due to the failure of the intended ultimate destination Port to receive the blocked/terminated IDE TLP.

The following rules relate to error handling

- Receipt of a Link IDE TLP or Selective IDE TLP for which there is not an associated IDE Stream is a Misrouted IDE TLP error; this is a reported error associated with the Receiving Port.
- Receipt of a Link IDE TLP by a Switch that targets an Egress Port for which there is not a Link IDE Stream associated with the same TC and in the Secure state is a Misrouted IDE TLP error; this is a reported error associated with the Ingress Port.
- The Transaction Layer must return flow control credit and handle as a Misrouted IDE TLP, but take no other action in response to a received Misrouted IDE TLP.

- The detection of any of the following conditions is IDE Check Failed error, a reported error associated with the Receiving Port (see § Section 6.2).
 - a MAC check failure occurs when the Receiver's check of the MAC of a received TLP, or aggregated unit of TLPs, fails,
 - underflow of the PR-Received-Counter-NPR or PR_Received_Counter-CPL (indicating an improper reordering has been detected),
 - either or both of the PR-Received-Counter-NPR/PR_Received_Counter-CPL 64-bit counters overflow (indicating a failure to receive the Transmitted NPR/CPL TLPs).
 - The Sub-Stream identifier field contains a Reserved/unsupported value.

Upon detection of one or more of these conditions, the IDE Stream State Machine for this IDE Stream must enter Insecure. This is a reported error associated with the Receiving Port (see § Section 6.2). The TLP that triggered the error and all subsequent IDE TLPs received associated with the same IDE Stream must be discarded, following update of Flow Control credits, for as long as the associated Stream is enabled. There must be no additional error(s) logged for subsequent TLPs received associated with an IDE Stream already in the Insecure state. These rules also apply to TLPs received while in the Insecure state if that state was reached for reasons other than those listed above.

- Receiving an IDE TLP that is a Completion with UR or UC status is not a security error and must not by itself trigger a transition to Insecure.
- Upon detection of an error associated with an aggregated unit of TLPs, when Advanced Error Reporting is supported, only the last TLP of the unit must be logged in the Header Log Register and TLP Prefix Log Register.
- All IDE Streams associated with a Link must transition to Insecure when DL_Active transitions from asserted to deasserted for that Link.
- Upon transition from Secure to Insecure for any reason, other than that the corresponding Link/Selective IDE Stream Enable bit is Cleared or the associated Traffic Class designator maps to a non-UIO VC, for a given Stream, the Port must transmit an IDE Fail Message indicating the Stream_ID to the Partner Port
- Upon receipt of an IDE Fail Message, for the indicated Stream the Port must transition to Insecure
 - When an IDE Stream enters Insecure due to the receipt of a IDE Fail Message then that transition into Insecure must not result in the Transmission of an IDE fail message.
- Upon entry into Insecure, all active key sets and IVs for the associated IDE Stream must be marked as invalid.
- For an IDE Stream to exit Insecure and return to Secure, the IDE Stream must be re-established using a new key and IV set.
- In the Insecure state, private data associated with the affected IDE Stream(s) must be secured.
 - How this is done is implementation specific.
- To prepare to exit the Insecure state for a Stream and return to the Secure state, software must write a 0b to the corresponding Selective IDE Stream Enable or Link IDE Stream Enable bit, even if it is already 0b.
 - Hardware must not return a Stream to the Secure state until the Enable bit has been written to 0b and subsequently Set.
 - Additional actions not defined here are typically necessary based on specific use model requirements.

To return a Link IDE Stream for TC0/VC0 to the Secure state, it is necessary to reset the Device, e.g. using a Hot Reset, to make it possible for Configuration Requests/Completions to pass across the Link, unless the Device provides an alternate, implementation-specific, mechanism.

- When processing received IDE TLPs, all error checks must be completed, or an equivalent delay must be inserted, prior to signaling an error, such that it is not possible for an external observer to determine at which stage in error checking the error(s) was(were) detected.

The following rules relate to Power Management:

- The No_Soft_Reset bit must be Set
- All state related to keys and counters must be maintained in D0, D1, D2 and D3_{hot}.
 - The IDE Extended Capability is subject to the same rules as other register structures defined by this specification, and because it is itself essential for IDE operation, any condition where the IDE Extended Capability programming is lost also necessarily results in a loss of the ability to maintain IDE operation; Such conditions must result in all IDE Streams transitioning to the Insecure state, and all keys must be invalidated and rendered unreadable.
- It is permitted to support retention of state related to keys and counters in D3_{cold}, however such mechanisms are beyond the scope of this document.

The following rules relate to maintaining a secure local environment:

- Attempts to modify IDE registers, BARs, and other structures that could affect the security of the device or an IDE Stream must be detected
 - The resulting actions are implementation specific
- It is permitted to enter Insecure when a condition is detected that could affect the security of the device or an IDE Stream
 - In some cases, as determined by implementation specific criteria, it may be desirable to implement some other implementation specific action.
- IDE must be enabled in a coordinated way between the two Partner Ports such that both are enabled for IDE prior to either one Transmitting an IDE TLP to the other.
- For Link IDE, software must enable the Upstream Port and then the Downstream Port, while ensuring that no TLPs are transmitted on the Link between these two events.
- For Selective IDE, the Stream must not be used until it has been enabled in both Partner Ports. For cases where one of the Partner Ports is a Root Port and Selective IDE for Configuration Requests is enabled, the other Partner Port must be enabled prior to the Root Port. For other scenarios, the mechanisms to satisfy this requirement are implementation-specific.

6.34 Unordered I/O (UIO) §

Unordered I/O (UIO) is an optional capability, intended to address the limitations of the PCI/PCIe fabric-enforced ordering rules. UIO enables fabrics with multiple paths between a source and destination to be supported, and more closely matches the semantics of common IO fabrics (including on-die fabrics). UIO is suitable for all combinations of Requesters and Completers including Host-to-Device, Device-to-Host, and Device-to-Device (P2P).

UIO provides the ability for hardware to maintain full backwards compatibility with the PCI/PCIe producer-consumer model, shifting the responsibility for enforcing the observed ordering from the fabric to the Requester. All UIO Requests have corresponding UIO Completions, and the UIO Completions provide the Requester with the ability (and responsibility) to enforce ordering requirements. UIO can be used only when the entire path from Requester to Completer uses Flit Mode, supports UIO, and has UIO enabled.

6.34.1 UIO Rules §

For the path between a UIO Requester and Completer, UIO must be enabled end-to-end through all intermediate routing elements. For a given operation, the Requester is permitted to select, using implementation specific means, between using UIO Requests or non-UIO Requests.

Non-tree topologies and multi-path support are permitted for UIO.

- The mechanism used to enable and manage such topologies is outside the scope of this specification.
- The mechanisms required to avoid deadlocks and loops for such topologies are outside the scope of this specification.
- A tree topology must exist as a subset of any non-tree topologies for enumeration and configuration operations and other non-UIO traffic.
 - Until enabled by implementation specific software, any Links outside of this tree topology must not carry any TLPs other than Messages with Local (r[2:0] = 100b) Message routing.

When a VC is configured for UIO, Posted credits are used for UIO Memory Write Requests and Non-Posted credits are used for all other UIO Memory Requests.

A Requester, Completer, or intermediate routing element advertises UIO support through the SVC VC Protocols Supported field in the SVC Resource Capability Register. For UIO functionality that has specific capability and control configuration (see § Section 7.7.9.2 and § Section 7.7.9.3), all UIO-supporting VCs implemented by a Port must have the same capabilities. It is not required for software to enable all implemented UIO-supporting VCs identically.

UIO, as with other PCIe traffic, does not provide protection against address hazarding, that is, the fabric ordering does not consider Request addresses. If such protection is required, it must be implemented outside the scope of this specification.

6.35 MMIO Register Blocks §

MMIO Register Blocks are optional mechanisms for exchanging various types of data structures between system software and a Function. Because all runtime operations are performed using Memory Space, the performance is generally much better than mechanisms such as DOE that use Configuration Space, and the use of Memory Space is better suited to direct assignment of the hardware resources to the appropriate software element.

MMIO Register blocks are discovered using the MMIO Register Block Locator (MRBL) Extended Capability (§ Section 7.9.30). The registers are mapped in Memory Space allocated via a BAR. Each MRBL Locator Register (§ Section 7.9.30.3) entry in the MRBL Extended Capability structure defines the type of MMIO Register Block and the BAR number and the offset within the BAR where these registers are mapped. It is permitted that the BAR contains other items besides the MMIO Register Blocks described in this section.

It is strongly recommended that all accesses to offsets within the MMB Memory Space be naturally aligned. If an access is not naturally aligned, then the request is permitted to be handled as a restricted programming model violation as defined in § Section 2.3.1.

§ Figure 6-83 portrays the relationship between the MRBL Extended Capability and the MMIO Register Blocks described in this section.

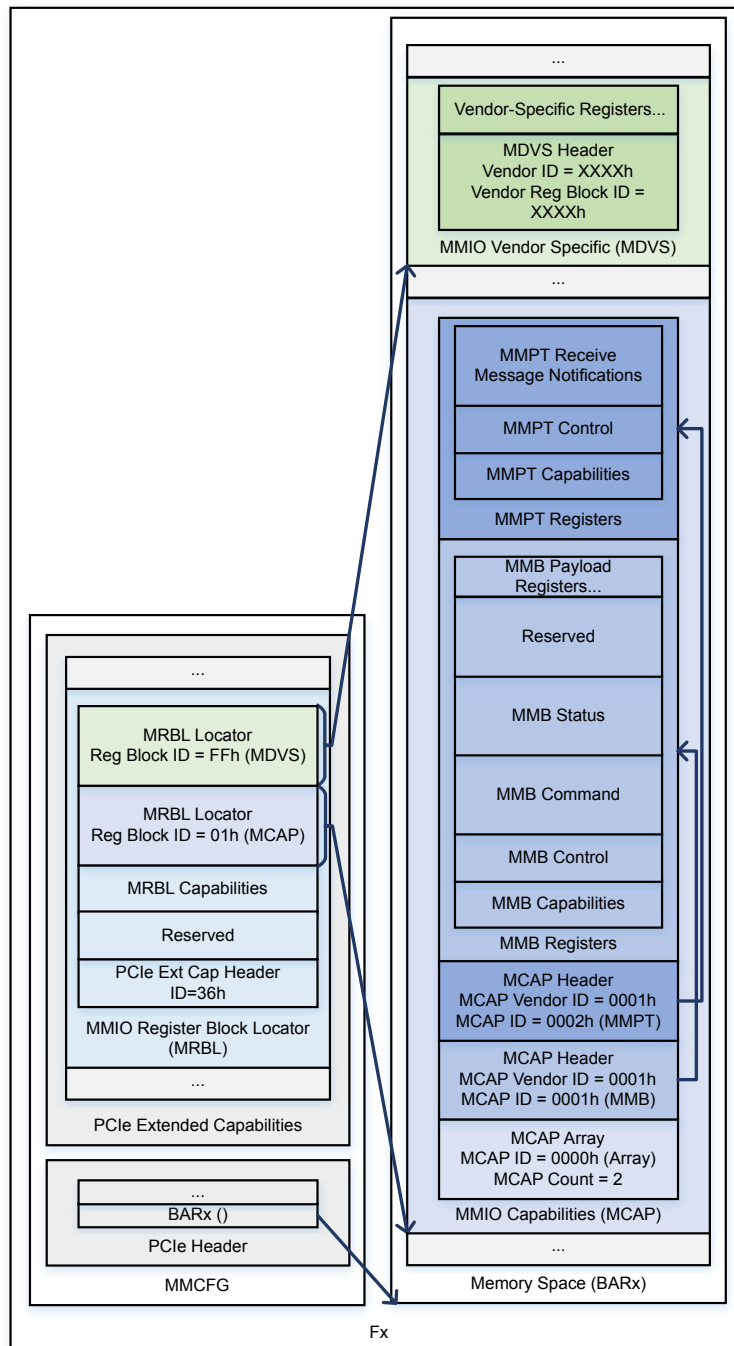


Figure 6-83 MMIO Register Blocks §

6.35.1 MMIO Capabilities Register Block (MCAP) §

The MMIO Capabilities Register Block is an array of capability structures. Each capability in the MCAP array is described by an MCAP Header Register that identifies the specific capability and points to the capability register structure in Memory Space. At the beginning of the MCAP Register Block is the MCAP Array Register (see § Section 6.35.1.1) that defines the size of the array followed by a list of MCAP Header Registers (See § Section 6.35.1.2).

The MCAP Register Block is discovered using the MMIO Register Block Locator Extended Capability (MRBL) (see § Section 7.9.30).

§ Figure 6-84 illustrates the MCAP Register Block.

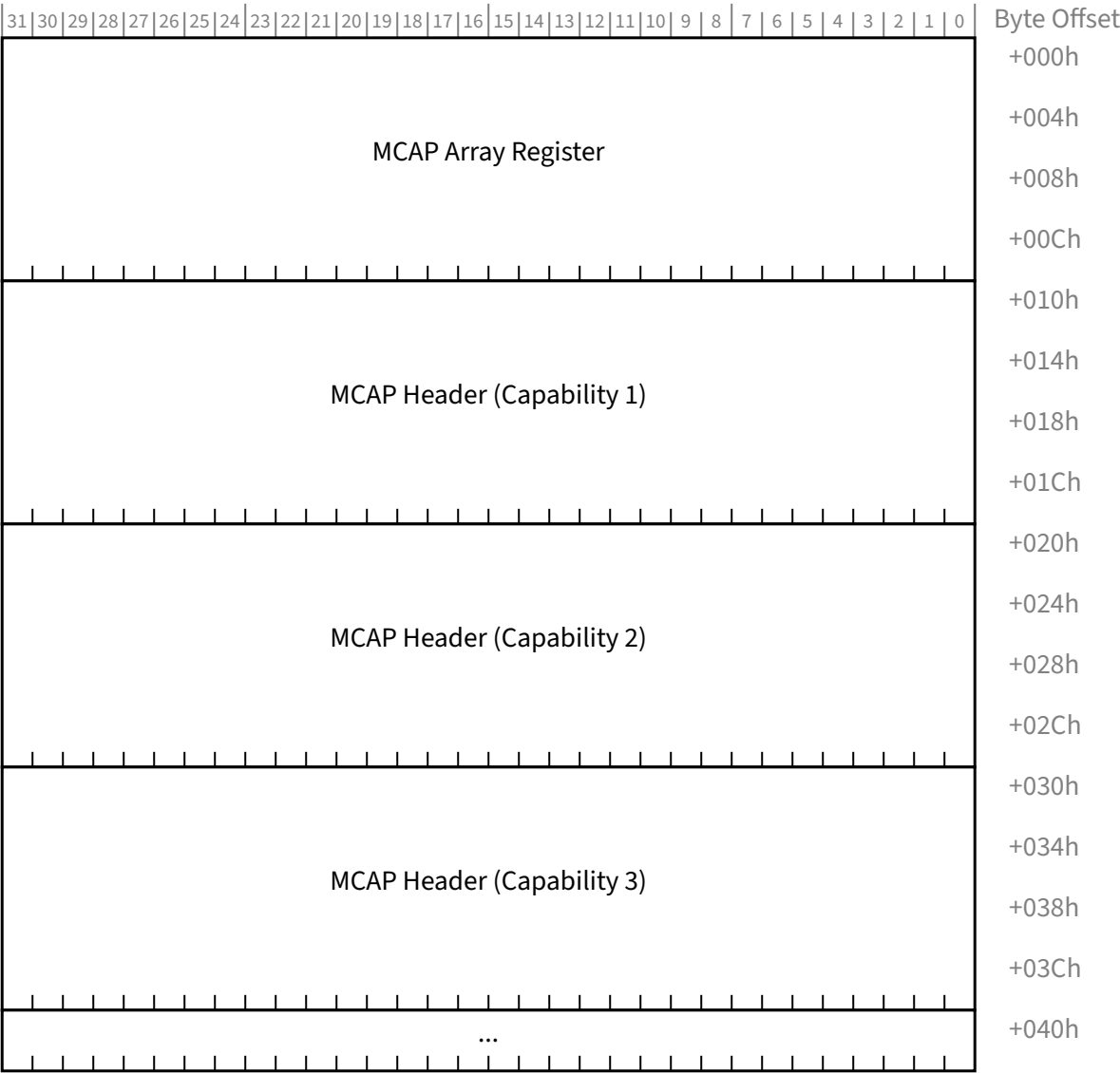


Figure 6-84 MCAP Register Block §

6.35.1.1 MCAP Array Register (Offset 00h) §

§ Figure 6-85 detail allocation of register fields in the MCAP Array Register Block.

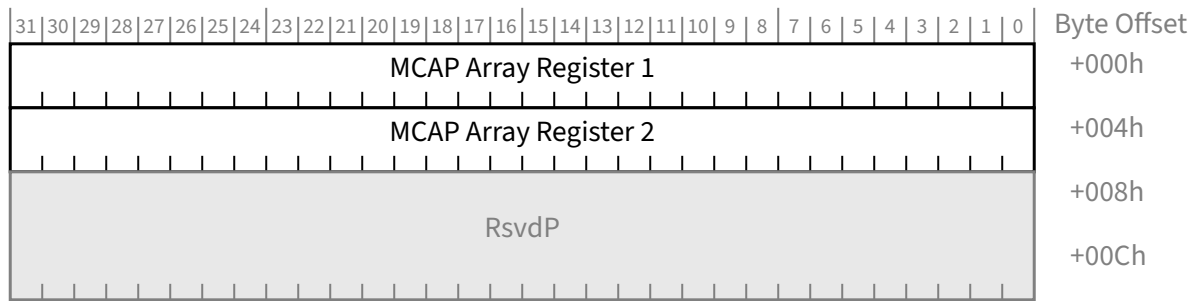


Figure 6-85 MCAP Array Register Block §

§ Figure 6-86 and § Figure 6-87 detail allocation of register fields registers that form the MCAP Array Register Block; § Table 6-38 and § Table 6-39 provide the respective bit definitions.

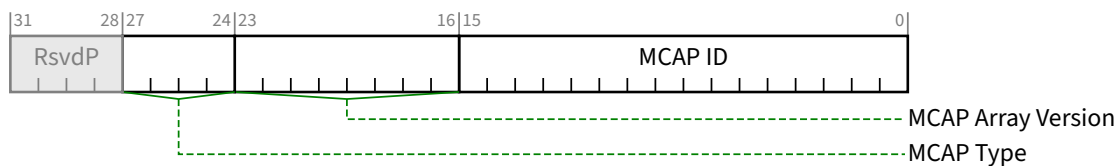


Figure 6-86 MCAP Array Register 1 §

Table 6-38 MCAP Array Register 1 §

Bit Location	Description	Attributes
15:0	MCAP ID – The MCAP array identifier. For the <u>MCAP Array Register</u> , this field shall be set to 0000h.	<u>RO</u>
23:16	MCAP Array Version – Defines the format of the MCAP Header Registers (see § Section 6.35.1.2 and § Table 6-40 through § Table 6-43) in the <u>MCAP Register Block</u> . Encodings are: 01h The MCAP Header format is defined in § Figure 6-88. Others All other encodings are reserved.	<u>RO</u>
27:24	MCAP Type – Defines the type associated with any type-specific capabilities (MCAP IDs in the range 4000h-7FFFh) in the <u>MCAP Register Block</u> . Encodings are: 0h The type is inferred from the 24-bit value corresponding to the Class Code of the Function. If the Class Code is not associated with any type-specific capabilities, then no type-specific capabilities shall be present. 1h-7h 1h-7h Reserved – Allocated for [CXL]. Others All other encodings are reserved.	<u>RO</u>

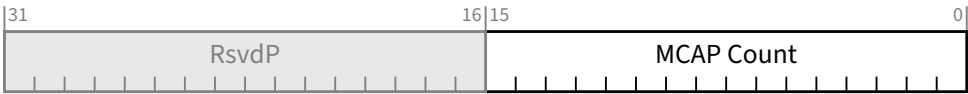


Figure 6-87 MCAP Array Register 2 §

Table 6-39 MCAP Array Register 2 §

Bit Location	Description	Attributes
15:0	MCAP Count – The number of capabilities in the MCAP Register Block, not including the MCAP Array Register. Each MCAP Header Register (see § Section 6.35.1.2) is contiguous to previous MCAP Header Register.	RO

6.35.1.2 MCAP Header Register Block (Offset Varies) §

§ Figure 6-88 details allocation of register fields in the MCAP Header Register Block.

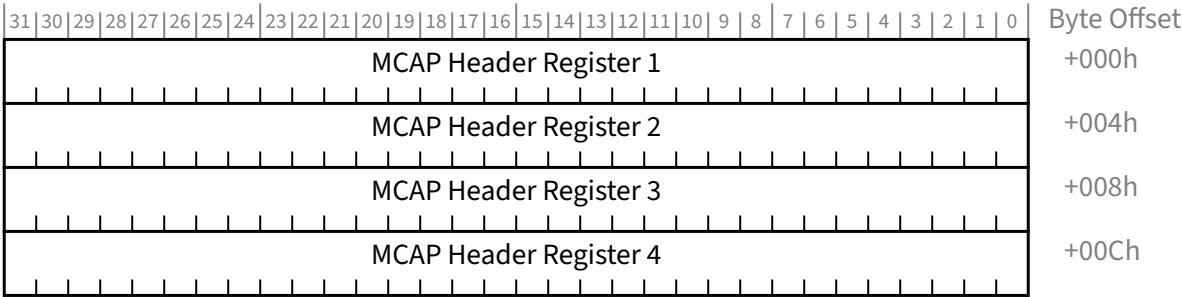


Figure 6-88 MCAP Header Register Block §

§ Figure 6-89 through § Figure 6-92 detail allocation of register fields in the MCAP Header Register Block; § Table 6-40 through § Table 6-43 provides the respective bit definitions.

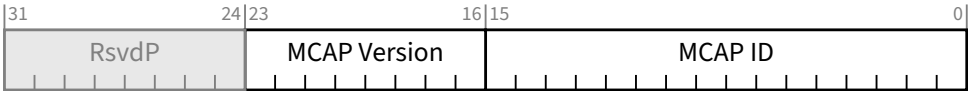


Figure 6-89 MCAP Header Register 1 §

Table 6-40 MCAP Header Register 1 §

Bit Location	Register Description	Attributes
15:0	MCAP ID – The capability identifier.	RO
23:16	MCAP Version – Defines the version of the capability register structure. The MCAP Version is incremented whenever the capability register structure is extended to add more functionality. Backward compatibility shall be maintained during this process. For all values of n, version n+1 may extend version n by replacing fields that are marked as reserved in version n but must not redefine the meaning of existing fields. Software that was written for a lower version may continue to operate on capability structures with a higher version but will not be able to take advantage of new functionality. If backwards compatibility cannot be maintained, a new MCAP ID shall be created. Each field in a capability register structure is assumed to be introduced in version 1 of that structure unless specified otherwise in the field's definition.	RO

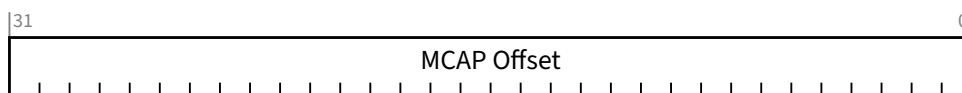


Figure 6-90 MCAP Header Register 2 §

Table 6-41 MCAP Header Register 2 §

Bit Location	Register Description	Attributes
31:0	MCAP Offset – Offset of the capability register structure from the start of the MCAP Register Block in bytes. The offset of performance sensitive MCAPs and security sensitive MCAPs shall be 4 KB aligned within Memory Space.	RO

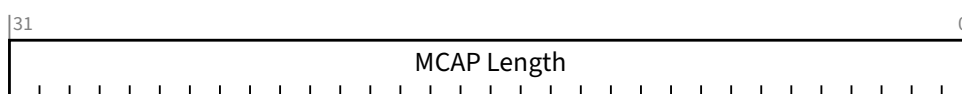


Figure 6-91 MCAP Header Register 3 §

Table 6-42 MCAP Header Register 3 §

Bit Location	Register Description	Attributes
31:0	MCAP Length – Size of the capability register structure in bytes.	RO

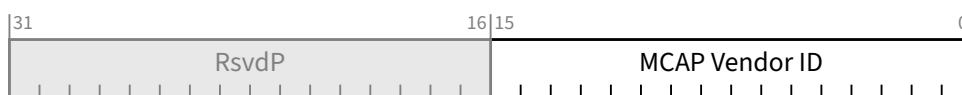


Figure 6-92 MCAP Header Register 4 §

Table 6-43 MCAP Header Register 4 §

Bit Location	Register Description	Attributes
15:0	MCAP Vendor ID – The PCI-SIG assigned Vendor ID for the entity that defined the capability register structure. A value of 0000h in this field is reserved for legacy compatibility with existing [CXL] defined register structures only.	RO

Table 6-44 PCI-SIG Defined MCAP Identifiers (MCAP Vendor ID = 0001h) §

MCAP Vendor ID	MCAP ID	Description
N/A	0000h	MCAP Array Register – Describes the MCAP Array Register. See § Section 6.35.1.1 .
0001h	0001h	MMIO Mailbox (MMB) – Describes the MMIO mailbox capability registers (MMB). At most one instance of this register structure can exist per Function. See § Section 6.35.1.3 .
0001h	0002h	Management Message Passthrough (MMPT) – Describes the registers required for a Function that supports the Management Message Passthrough (MMPT) command set (§ Section 6.36.1). At most one instance of this register structure can exist per Function (See § Section 6.35.1.4.1).
0001h	0003h-3FFFh	Reserved
N/A	4000h-7FFFh	Type-Specific – Identifies type-specific capabilities associated with the MCAP Type specified in the MCAP Array Register (see § Section 6.35.1.1).
0001h	8000h-FFFFh	Reserved

6.35.1.3 MMIO Mailbox Capability (MMB) (Offset: Varies) §

The MMIO Mailbox Capability (MMB) provides the ability to issue a command to a Function.

The MMB interface shall only be used in a single-threaded manner. It is software’s responsibility to avoid simultaneous, uncoordinated access to the MMB Registers using techniques such as locking.

The MMB command timeout is 2 seconds. This is the maximum permitted time after the Doorbell is Set in the MMB Control Register (see § Section 6.35.1.3.2.2) for the Function to complete the command, Clear the Doorbell, and optionally signal the Command Ready Interrupt, if configured.

MMB commands do not continue to execute across Conventional Resets. It is not required that FLR results in any type of reset to the internal processing engine for MMB operations, although such behavior is permitted, and may be required or forbidden by specific commands.

The optional MMB Attention Mechanism supports improved power management of MMB implementations, by enabling an MMB instance to temporarily enter an unresponsive state, and providing software, a mechanism to direct the instance

back to a responsive state. If MMB Attention Mechanism Support is Set, for backwards compatibility, the default is that the MMB instance must remain in a responsive state. System software is recommended to Set MMB Attention Not Needed when it is acceptable for the MMB instance to enter and stay in a state where it is not immediately available for use. When Clear, the MMB instance must remain in a state where it is ready to respond in a timely way to system software. MMB At Attention, when Set, indicates the MMB interface is presently in a state of readiness. This bit must only be Cleared if MMB Attention Not Needed is Set. It is permitted for this bit to remain Clear for up to 50 ms following the Clearing of MMB Attention Not Needed.

Functions may support sending MSI/MSI-X interrupts to indicate MMB command status. Support for MMB interrupts is enumerated in the MMB Capabilities Register (see § Section 6.35.1.3.2.1) and enabled in the MMB Control Register (see § Section 6.35.1.3.2.2).

Unless specified otherwise in the field definitions for the MMB Registers, each field is present in version 1 and later of these structures. The Function must report the version of these structures in the MCAP Version field of the MCAP Header Register (see § Section 6.35.1.2).

6.35.1.3.1 MMB Operation §

The flow for executing a command is described below. The term “caller” represents the entity submitting the command.

- The caller ensures the Function is ready to accept a new command on the MMB.
 - After a Conventional Reset, the caller ensures the MMB is initialized.
 - The caller polls for MMB Ready to be Set in the MMB Status Register (see § Section 6.35.1.3.2.4).
 - The caller ensures the MMB is in a state of readiness.
 - If MMB Attention Mechanism Support in the MMB Capabilities Register (see § Section 6.35.1.3.2.1) is Set, the caller reads MMB At Attention in the MMB Status Register (see § Section 6.35.1.3.2.4). If Clear:
 - The caller Clears MMB Attention Not Needed in the MMB Control Register (see § Section 6.35.1.3.2.2).
 - The caller either polls for MMB At Attention to be Set in the MMB Status Register (see § Section 6.35.1.3.2.4) or waits for the Command Ready Interrupt if configured.
 - The caller ensures the Function is ready to accept a new command.
 - The caller polls for the Doorbell to be Cleared in the MMB Control Register (see § Section 6.35.1.3.2.2).
- The caller issues a new command.
 - The caller writes the MMB Command Register (see § Section 6.35.1.3.2.3).
 - The caller writes the MMB Payload Registers (see § Section 6.35.1.3.2.5) if the input payload is non-empty.
 - The caller Sets the Doorbell in the MMB Control Register (see § Section 6.35.1.3.2.2).
- The caller waits for the command to complete.
 - The caller either polls for the Doorbell to be Cleared in the MMB Control Register (see § Section 6.35.1.3.2.2) or waits for the Command Ready Interrupt if configured.
 - In case of a command timeout, the caller may attempt to recover the Function by issuing a Conventional reset to the Function.
- The caller retrieves the result of the command.
 - The caller reads the Return Code in the MMB Status Register (see § Section 6.35.1.3.2.4).

- The caller reads the MMB Payload Length in the MMB Command Register (see § Section 6.35.1.3.2.3). If non-zero, the caller reads the MMB Payload Registers (see § Section 6.35.1.3.2.5) to retrieve the output payload.

6.35.1.3.2 MMB Registers §

§ Figure 6-93 illustrates the MMB Registers structure.

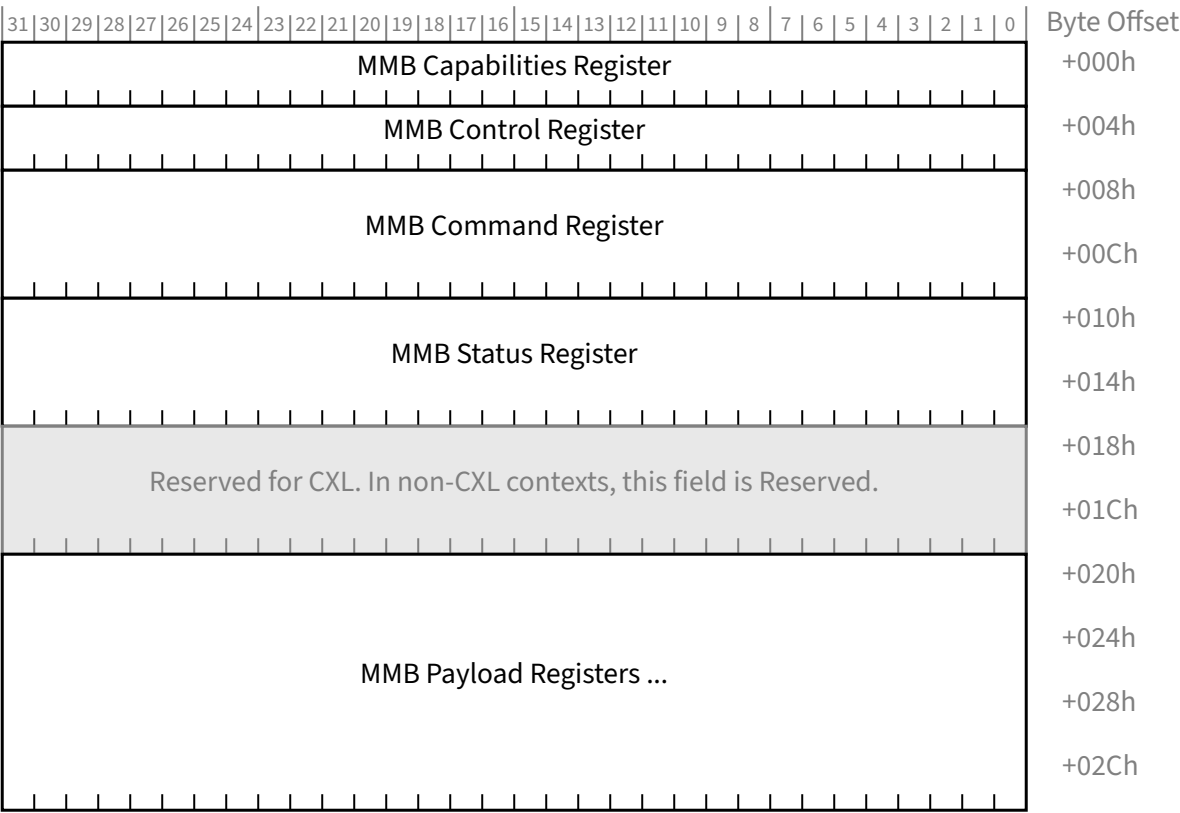


Figure 6-93 MMB Registers §

6.35.1.3.2.1 MMB Capabilities Register (Offset 00h) §

§ Figure 6-94 details allocation of register fields in the MMB Capabilities Register; § Table 6-45 provides the respective bit definitions.

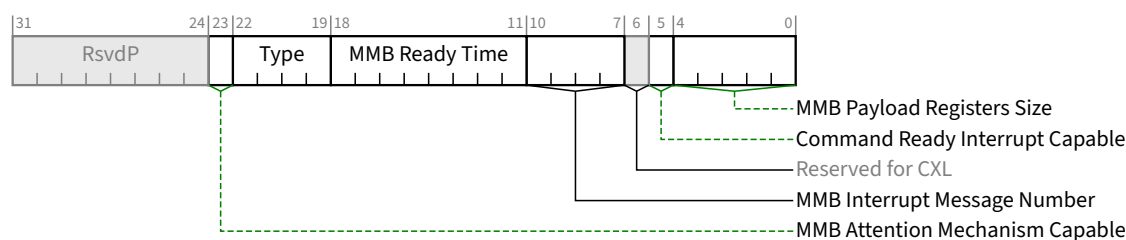


Figure 6-94 MMB Capabilities Register §

Table 6-45 MMB Capabilities Register §

Bit Location	Register Description	Attributes
4:0	MMB Payload Registers Size – Size of the MMB Payload Registers (see § Section 6.35.1.3.2.5) in bytes, expressed as 2^n . The minimum size is 256 bytes ($n=8$) and the maximum size is 1 MB ($n=20$).	HWInit
5	Command Ready Interrupt Capable – This field indicates if the MMB supports signaling an MSI/MSI-X interrupt when the Doorbell in the MMB Control Register (see § Section 6.35.1.3.2.2) transitions from Set to Clear or MMB At Attention in the MMB Status Register (see § Section 6.35.1.3.2.4) transitions from Clear to Set. 0 Not supported 1 Supported	HWInit
6	Reserved for CXL – This field is assigned for use by [CXL]. In non-CXL contexts, this field is Reserved.	RsvdP
10:7	MMB Interrupt Message Number – When MSI/MSI-X is implemented, this field indicates which MSI/MSI-X vector is used for the interrupt message generated in association with this MMB instance. For MSI, the value in this field indicates the offset between the base Message Data and the interrupt message that is generated. Hardware is required to update this field so that it is correct if the number of MSI Messages assigned to the Function changes when software writes to the Multiple Message Enable field in the Message Control register for MSI. For MSI-X, the value in this field indicates which MSI-X Table entry is used to generate the interrupt message. The entry shall be one of the first 16 entries even if the Function implements more than 16 entries. The value in this field shall be within the range configured by system software to the Function. For a given MSI-X implementation, the entry shall remain constant. If both MSI and MSI-X are implemented, they are permitted to use different vectors, though software is permitted to enable only one mechanism at a time. If MSI-X is enabled, the value in this field shall indicate the vector for MSI-X. If MSI is enabled or neither is enabled, the value in this field indicates the vector for MSI. If software enables both MSI and MSI-X at the same time, the value in this field is undefined.	RO/RsvdP
18:11	MMB Ready Time – This field indicates the maximum amount of time in seconds after a Conventional Reset for MMB Ready in the MMB Status Register (see § Section 6.35.1.3.2.4) to become Set. A value of 0 indicates 2 seconds.	HWInit
22:19	Type – This field identifies the type-specific commands supported by the MMB. 0h The type is inferred from the 24-bit value corresponding to the Class Code of the Function. If the Class Code is not associated with any type-specific commands, then no type-specific commands are present. 1h-7h Reserved – Allocated for [CXL]. Others All other encodings are reserved.	HWInit

Bit Location	Register Description	Attributes
23	MMB Attention Mechanism Capable – This field indicates if the MMB supports the optional MMB Attention Mechanism. 0 Not supported 1 Supported	HWInit

6.35.1.3.2.2 MMB Control Register (Offset 04h) §

§ Figure 6-95 details allocation of register fields in the MMB Control Register; § Table 6-46 provides the respective bit definitions.

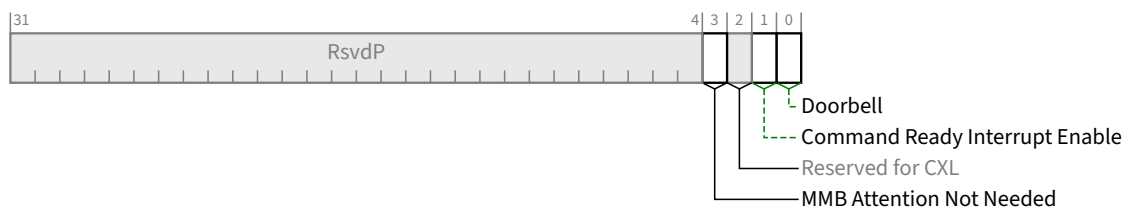


Figure 6-95 MMB Control Register §

Table 6-46 MMB Control Register §

Bit Location	Register Description	Attributes
0	Doorbell – When MMB Ready is Set, this bit, when Clear, indicates that the Function is ready to accept a new command. Set by the caller to notify the Function that the command inputs are ready. Read-only when Set. Cleared by the Function when the command completes. 0 Ready to accept a command 1 Busy executing a command or initializing Default value of this field is 1b. Cleared by the Function when <u>MMB Ready</u> is Set in the <u>MMB Status Register</u> .	RW
1	Command Ready Interrupt Enable – When Command Ready Interrupt Capable is Set, this bit, when Set, enables the MMB to signal an MSI/MSI-X interrupt when the Doorbell transitions from Set to Cleared or MMB At Attention in the MMB Status Register (see § Section 6.35.1.3.2.4) transitions from Clear to Set. Read-only when the Doorbell is Set. When Command Ready Interrupt Capable is Clear, this bit is permitted to be RsvdP. 0 Disabled 1 Enabled Default value of this field is 0b.	RW/RsvdP
2	Reserved for CXL – This field is assigned for use by [CXL]. In non-CXL contexts, this field is Reserved.	RsvdP
3	MMB Attention Not Needed – If MMB Attention Mechanism Capable is Set, this bit, when Set, enables the MMB to enter and stay in a state where it is not immediately available for use. When Clear the MMB must remain in a responsive state. When MMB Attention Mechanism Support is Clear, this bit is permitted to be RsvdP.	RW/RsvdP

Bit Location	Register Description	Attributes
	0 Attention needed 1 Attention not needed Default value of this bit is 0b.	

6.35.1.3.2.3 MMB Command Register (Offset 08h) §

The MMB Command Register shall only be used by the caller when the Doorbell in the MMB Control Register (see § Section 6.35.1.3.2.2) is Cleared.

§ Figure 6-96 details allocation of register fields in the MMB Command Register; § Table 6-47 provides the respective bit definitions.

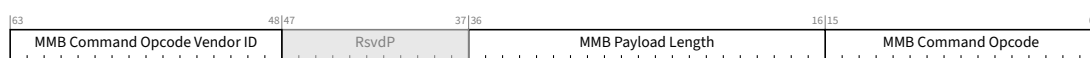


Figure 6-96 MMB Command Register §

Table 6-47 MMB Command Register §

Bit Location	Register Description	Attributes
15:0	MMB Command Opcode – The command identifier. Default value of this field is 0000h.	RW
36:16	MMB Payload Length – The size of the data in the MMB Payload Registers (see § Section 6.35.1.3.2.5) in bytes. Valid values for this field are less than or equal to the MMB Payload Registers Size specified in the MMB Capabilities Register (see § Section 6.35.1.3.2.1). Written by the caller to provide the command input payload size to the Function prior to setting the Doorbell in the MMB Control Register (see § Section 6.35.1.3.2.2). Values specified by the caller that are greater than the MMB Payload Registers Size result in an error of Invalid Payload Length being returned in the MMB Return Code field. Written by the Function to provide the command output payload size to the caller when the Doorbell is Cleared in the MMB Control Register (see § Section 6.35.1.3.2.2). Default value of this field is 00000h.	RW
63:48	MMB Command Opcode Vendor ID – The PCI-SIG assigned Vendor ID for the entity that defined the MMB Command Opcode. A value of 0000h in this field is reserved for legacy compatibility with existing [CXL] defined commands only. Default value of this field is 0000h.	RW

IMPLEMENTATION NOTE: MMB COMMAND OPCODE VENDOR ID LEGACY COMPATIBILITY §

For legacy compatibility with OS software, platform firmware should clear the MMB Command Opcode Vendor ID to 0000h prior to OS hand off.

6.35.1.3.2.4 MMB Status Register (Offset 10h) §

Reports information about the state of the MMB and the last command executed since Conventional Reset.

§ Figure 6-97 details allocation of register fields in the MMB Status Register; § Table 6-48 provides the respective bit definitions.

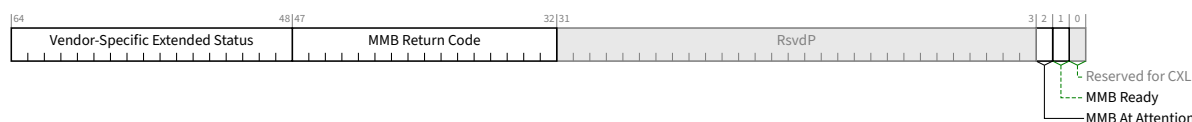


Figure 6-97 MMB Status Register §

Table 6-48 MMB Status Register §

Bit Location	Register Description	Attributes
0	Reserved for CXL – This field is assigned for use by [CXL]. In non-CXL contexts, this field is Reserved.	RsvdP
1	MMB Ready – This bit, when Set, indicates that the Function is ready to accept commands through the MMB interface. Functions that report a non-zero MMB Ready Time shall Set this bit after a Conventional Reset within the time reported in the MMB Ready Time field and it shall remain Set until the next Conventional Reset, or the Function encounters an error that prevents any MMB communication. 0 Not ready 1 Ready	RO
2	MMB At Attention – When MMB Attention Mechanism Capable is Set, this bit, when Set, indicates the MMB interface is presently in a state of readiness. The transition of this bit from Clear to Set triggers a Command Ready interrupt if Command Ready Interrupt Enable is Set. When MMB Attention Mechanism Support is Clear, this bit is RsvdP. This bit is Set by the Function when MMB Ready is Set. 0 Not at Attention 1 At Attention	RO/RsvdP
47:32	MMB Return Code – The result of the command. Only valid after the Doorbell is Cleared in the MMB Control Register (see § Section 6.35.1.3.2.2) after a command completes.	RO
64:48	Vendor-Specific Extended Status – The Vendor-Specific extended status information. Only valid after the Doorbell is Cleared in the MMB Control Register (see § Section 6.35.1.3.2.2) after a command completes.	RO

6.35.1.3.2.4.1 MMB Command Return Codes §

Command Return Codes are associated with the entity that defined the command as indicated by the MMB Command Opcode Vendor ID in the MMB Command Register.

The MMB Command Return Code is Set by the Function to indicate to the Caller the result of a command. If more than one MMB Command Return Code applies, the Function chooses which one to return.

In general, retries are not recommended for commands that return an error except when indicated in the return code definition.

PCI-SIG defined command return codes are only valid for PCI-SIG defined commands.

Table 6-49 MMB PCI-SIG Defined Command Return Codes (Vendor ID = 0001h) §

Value	Description
0000h	Success: The command successfully completed.
0001h	Reserved
0002h	Invalid Input: A command input was invalid.
0003h	Unsupported: The MMB Command Opcode is not supported.
0004h	Internal Error: The command was not completed because of an internal error.
0005h	Retry Required: The command was not completed because of a temporary error. An optional single retry may resolve the issue.
0006h-7FFFh	Reserved: Allocated for [CXL].
8000h	Invalid Payload Length: The input payload length specified for the command is not valid.
8001h	Unsupported Management Message: The MMPT Message Type is not supported.
8002h	Transfer in Progress: Only one transfer can occur at a time. Complete or abort the current data transfer before starting a new one.
8003h	Transfer Out of Order: The transfer was aborted because the data was transferred out of order.
8004h	Transfer Aborted: The data transfer was aborted and must be restarted from the beginning.
8005h-FFFFh	Reserved

6.35.1.3.2.5 MMB Payload Registers (Offset 20h) §

The MMB Payload Registers must consist of 256 Bytes – 1 MB, as shown in § Figure 6-98. The size of the MMB Payload Registers is reported in the MMB Capabilities Register (see § Section 6.35.1.3.2.1). Any data beyond the size specified in the MMB Capabilities Register shall be ignored by the caller and the Function.

The MMB Payload Registers are written by the caller to provide the command input payload to the Function prior to Setting the Doorbell in the MMB Control Register (see § Section 6.35.1.3.2.2). They are written by the Function to provide the command output payload back to the caller prior to Clearing the Doorbell.

These registers must only be used by the caller when the Doorbell in the MMB Control Register (see § Section 6.35.1.3.2.2) is Clear.

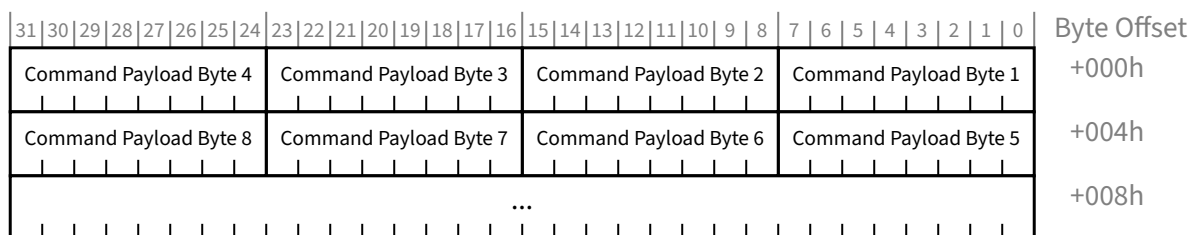


Figure 6-98 MMB Payload Registers §

6.35.1.4 Management Message Passthrough (MMPT) Capability (Offset: Varies) §

The Management Message Passthrough (MMPT) Capability is required for Functions that support the MMPT command set. Refer to the MMPT command interface for usage of this capability (§ Section 6.36.1).

Unless specified otherwise in the field definitions, each field is present in version 1 and later of this structure. The Function shall report the version of this structure in the MCAP Version field of the MCAP Header Register.

6.35.1.4.1 MMPT Registers §

§ Figure 6-99 illustrates the MMPT Register structure.

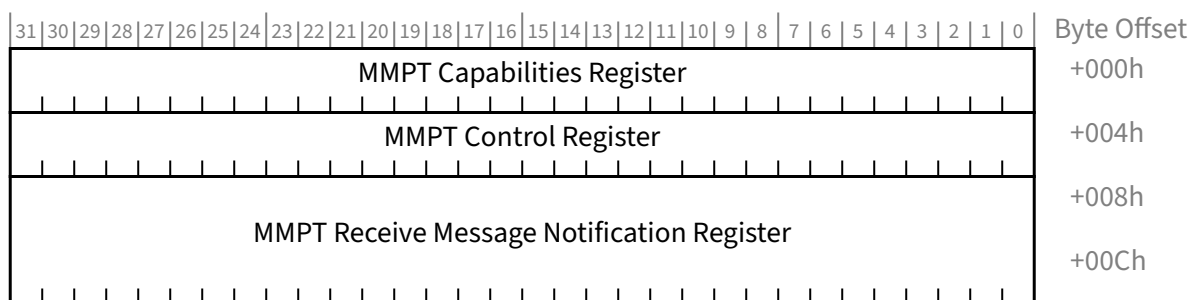


Figure 6-99 MMPT Registers §

6.35.1.4.1.1 MMPT Capabilities Register (Offset 00h) §

§ Figure 6-100 details allocation of register fields in the MMPT Capabilities Register; § Table 6-50 provides the respective bit definitions.

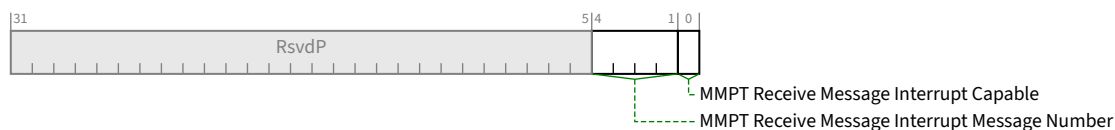


Figure 6-100 MMPT Capabilities Register §

Table 6-50 MMPT Capabilities Register §

Bit Location	Register Description	Attributes
0	MMPT Receive Message Interrupt Capable – This field indicates if the Function supports signaling an MSI/MSI-X interrupt when MMPT Receive Message Ready transitions from Clear to Set. 0 Not supported 1 Supported	RO
4:1	MMPT Receive Message Interrupt Message Number – When MSI/MSI-X is implemented, this field indicates which MSI/MSI-X vector is used for the interrupt message generated when MMPT Receive Message Ready transitions from Clear to Set. For MSI, the value in this field indicates the offset between the base Message Data and the interrupt message that is generated. Hardware is required to update this field so that it is correct if the number of MSI Messages assigned to the Function changes when software writes to the Multiple Message Enable field in the Message Control register for MSI. For MSI-X, the value in this field indicates which MSI-X Table entry is used to generate the interrupt message. The entry shall be one of the first 16 entries even if the Function implements more than 16 entries. The value in this field shall be within the range configured by system software to the device. For a given MSI-X implementation, the entry shall remain constant. If both MSI and MSI-X are implemented, they are permitted to use different vectors, though software is permitted to enable only one mechanism at a time. If MSI-X is enabled, the value in this field shall indicate the vector for MSI-X. If MSI is enabled or neither is enabled, the value in this field indicates the vector for MSI. If software enables both MSI and MSI-X at the same time, the value in this field is undefined.	RO/RsvdP

6.35.1.4.1.2 MMPT Control Register (Offset 04h) §

§ Figure 6-101 details allocation of register fields in the MMTP Control Register; § Table 6-51 provides the respective bit definitions.

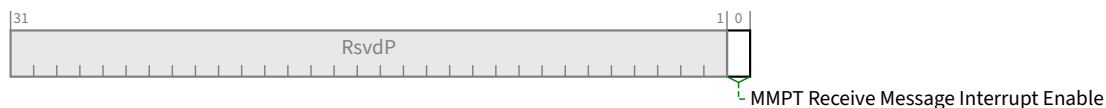


Figure 6-101 MMPT Control Register §

Table 6-51 MMPT Control Register §

Bit Location	Register Description	Attributes
0	MMPT Receive Message Interrupt Enable – When MMPT Receive Message Interrupt Capable is Set, this bit, when Set, enables the Function to signal an MSI/MSI-X interrupt when MMPT Receive Message Ready transitions from Clear to Set. When MMPT Receive Message Interrupt Capable is Clear, this bit is permitted to be RsvdP. 0 Disabled 1 Enabled Default value of this field is 0b.	RW/RsvdP

6.35.1.4.1.3 MMPT Receive Message Notification Register (Offset 08h) §

§ Figure 6-102 details allocation of register fields in the MMPT Receive Message Notification Register; § Table 6-52 provides the respective bit definitions.

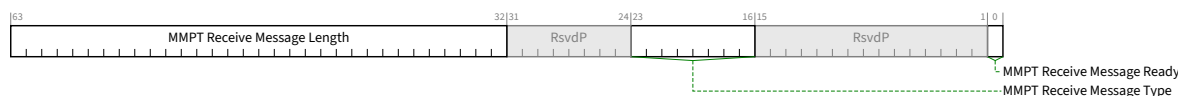


Figure 6-102 MMPT Receive Message Notification Register §

Table 6-52 MMPT Receive Message Notification Register §

Bit Location	Register Description	Attributes
0	MMPT Receive Message Ready – When Set, a new management message is ready to be transferred from the Function to the host using the MMPT Receive Message command (see § Section 6.36.1.2).	RO
23:16	MMPT Receive Message Type – This field indicates the nature and format of the management message ready to be transferred from the Function to the host using the MMPT Receive Message command (see § Section 6.36.1.2). Encodings for this field are provided in the [PCI-Code-and-ID]. All unspecified encodings are Reserved.	RO
63:32	MMPT Receive Message Length – This field indicates the length of the management message in bytes ready to be transferred from the Function to the host using the MMPT Receive Message command (see § Section 6.36.1.2).	RO

6.35.2 MMIO Designated Vendor-Specific Register Block (MDVS) §

The MMIO Designated Vendor-Specific Register Block (MDVS) allows a Vendor-Specific Memory Space register block to be discovered by utilizing the MRBL Extended Capability (§ Section 7.9.30). The format of the MDVS Register Block starts with the MDVS Register Block Header Register (see § Section 6.35.2.1 and § Section 6.35.3) as illustrated in § Figure 6-103. The remainder of the MDVS Register BlockMDVS Register Block is vendor defined. A single Function is permitted to implement more than one in a MDVS Register Block.

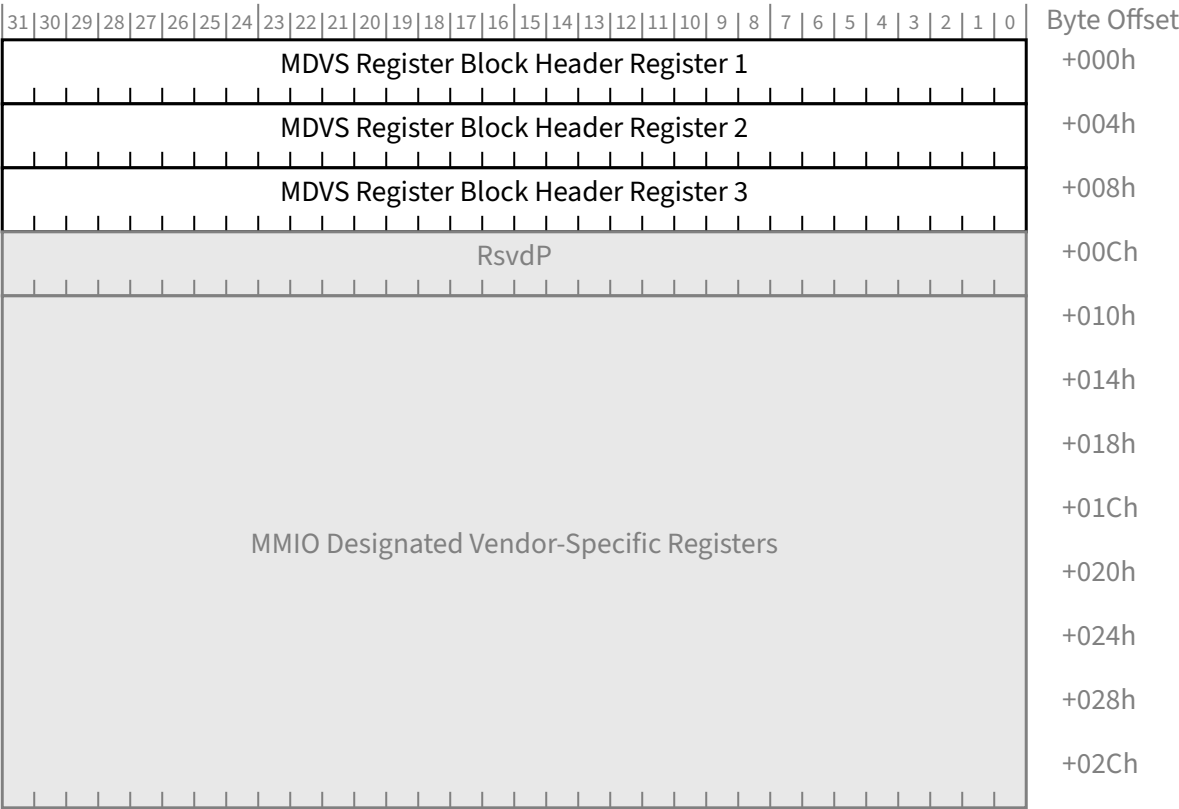


Figure 6-103 MDVS Register Block §

6.35.2.1 MDVS Register Block Header Register 1 (Offset 00h) §

§ Figure 6-104 details allocation of register fields in the MDVS Register Block Header Register 1; § Table 6-53 provides the respective bit definitions.

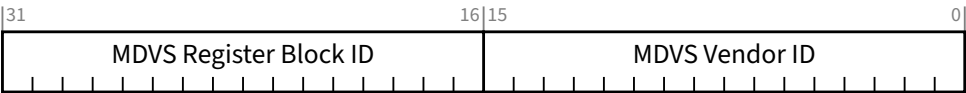


Figure 6-104 MDVS Register Block Header Register 1 §

Table 6-53 MDVS Register Block Header Register 1 §

Bit Location	Register Description	Attributes
15:0	MDVS Vendor ID – The PCI-SIG assigned Vendor ID for the organization that defined the layout and controls the specification for this register block.	<u>RO</u>

Bit Location	Register Description	Attributes
31:16	MDVS Register Block ID – Value defined by the Vendor ID in bits 15:0 that indicates the nature and format of the Vendor-Specific registers.	RO

6.35.3 MDVS Register Block Header Register 2 (Offset 04h) §

§ Figure 6-105 details allocation of register fields in the MDVS Register Block Header Register 2; § Table 6-54 provides the respective bit definitions.

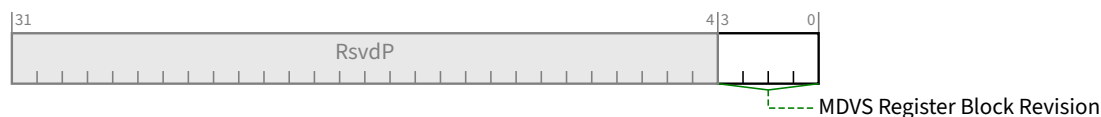


Figure 6-105 MDVS Register Block Header Register 2 §

Table 6-54 MDVS Register Block Header Register 2 §

Bit Location	Register Description	Attributes
3:0	MDVS Register Block Revision – Version number defined by the Vendor ID in bits 15:0 that indicates the version of the register block.	RO

6.35.4 MDVS Register Block Header Register 3 (Offset 08h) §

§ Figure 6-106 details allocation of register fields in the MDVS Register Block Header Register 3; § Table 6-55 provides the respective bit definitions.

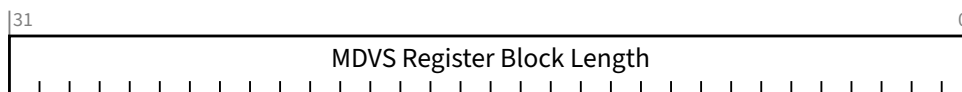


Figure 6-106 MDVS Register Block Header Register 3 §

Table 6-55 MDVS Register Block Header Register 3 §

Bit Location	Register Description	Attributes
31:0	MDVS Register Block Length – The number of bytes in the register block, including the MDVS Register Block Header and the Vendor-Specific registers.	RO

6.36 MMB Command Interface §

MMB commands are identified by a MMB Command Opcode. MMB Command Opcodes also provide an implicit version number, which means a command's definition shall not change in an incompatible way in future revisions of this specification. Instead, if an incompatible change is required, the specification defining the change shall define a new MMB Command Opcode for the changed command. Commands may evolve by defining new fields in areas of the payload definitions that were originally defined as Reserved, but only in a way where software written using the earlier definition will continue to work correctly, and software written to the new definition can use the 0 value or the payload size to detect Function components that do not support the new field. This implicit versioning allows software to be written with the understanding that a MMB Command Opcode shall only evolve by adding backward compatible changes.

Table 6-56 PCI-SIG Defined MMB Command Opcodes (Vendor ID = 0001h) §

MMB Command Opcode					Input Payload Size (Bytes) (Note 1)	Output Payload Size (Bytes) (Note 1)
Command Set Bits[15:8]		Command Bits[7:0]		Combined Opcode		
00h	Reserved	00h-FFh	Reserved	0000h-00FFh	N/A	N/A
01h	Management Message Passthrough (MMPT)	00h	Send Message (see § Section 6.36.1.1)	0100h	10h+	00h+
		01h	Receive Message (see § Section 6.36.1.2)	0101h	10h	00h+
		02h-FFh	Reserved	0102h-01FFh	N/A	N/A
02h-FFh	Reserved	00h-FFh	Reserved	0200h-FFFFh	N/A	N/A

Notes:

1. Indicates the minimum payload size for a successful completion. Commands with variable payloads sizes are marked with '+'. Actual valid bytes in the output payload are indicated by the MMB Payload Length field in the MMB Command Register (see § Section 6.35.1.3.2.3).

6.36.1 Management Message Passthrough (MMPT) §

The optional Management Message Passthrough (MMPT) command set provides an interface to tunnel management messages defined in other industry standard specifications between the host and the Function using the MMB Command Interface. This enables OS standard class drivers to utilize the same management messages used by the BMC, thus making in-band and out-of-band management more consistent.

Management messages may be initiated from the host to the Function using the Send Message command (see § Section 6.36.1.1) or asynchronously from the Function to the host using the Receive Message command (see § Section 6.36.1.2). The Function notifies the host that a new management message is ready to be retrieved using the Receive Message command (see § Section 6.36.1.2) by setting the MMPT Receive Message Ready bit in the MMPT Receive Message Notification Register (see § Section 6.35.1.4.1.3). If more than one management message is ready to be transferred from the Function to the host, it is the Function's responsibility to queue up the management messages until the host retrieves them.

Management messages may be larger than the MMB Payload Registers Size requiring the management message to be transferred in parts. If a management message is transferred in its entirety, the caller makes one call to Send or Receive Message with Action = Full Data Transfer. The Offset field is not used and shall be ignored. If a management message is transferred in parts, the caller makes one call to Send or Receive Message with Action = Initiate Data Transfer, zero or more calls with Action = Continue Data Transfer, and one call with Action = Finish Data Transfer or Abort Data Transfer. The management message parts shall be transferred in ascending order based on the Offset value, and the Function shall return the Transfer Out of Order return code if the management message parts are not transferred in ascending order. Back-to-back retransmission of any management message part is permitted during a transfer. A Send or Receive Message command with Action = Abort Data Transfer shall be supported for management messages that can be transferred in parts. An attempt to call Send or Receive Message with Action = Abort Data Transfer for a message whose data has been fully transferred shall fail with Invalid Input.

If the management message transfer is interrupted by a Conventional Reset, error condition, or MMB command-specific reason, the management message transfer shall be aborted and the Send or Receive Message command shall return Transfer Aborted. If a management message transfer is aborted prior to the entire management message data being transferred, the Function shall require the management message transfer to be started from the beginning of the message data.

6.36.1.1 MMPT Send Message (Opcode 0100h) §

Transfer all or part of a management message from the host to the Function. If a management message is transferred in parts, the entire management data shall be transferred or aborted prior to transferring a new management message. Otherwise, the function shall return Invalid Input.

Possible MMB Command Return Code values (see § Section 6.35.1.3.2.4.1):

- Success
- Invalid Input
- Internal Error
- Retry Required
- Invalid Payload Length
- Unsupported
- Unsupported Management Message
- Transfer Out of Order
- Transfer in Progress
- Transfer Aborted

Table 6-57 *MMPT Send Message Input Payload* §

Byte Offset	Length in Bytes	Description
00h	01h	MMPT Message Type – This field indicates the nature and format of the Message Data. Encodings for this field are provided in the <u>[PCI-Code-and-ID]</u> . All unspecified encodings are Reserved.
01h	03h	Reserved
04h	04h	Flags DW Bits[2:0] Action Specifics – Specifies the stage of the Message Data transfer.

Byte Offset	Length in Bytes	Description
		000b Full Data Transfer 001b Initiate Data Transfer 010b Continue Data Transfer 011b Finish Data Transfer 100b Abort Data Transfer Others All other values are reserved Bits[31:3] Reserved
08h	04h	Offset – The Byte Offset in the Message Data.
0Ch	04h	Reserved
10h	variable	Message Data – The management message data, formatted according to the MMPT Message Type specification.

Table 6-58 MMPT Send Message Output Payload §

Byte Offset	Length in Bytes	Description
00h	Varies	Message Data – The management message response, formatted according to the MMPT Message Type specification.

6.36.1.1.1 MMPT Send Message Operation §

The flow for transferring a management message from the host to the Function is described below. Total input payload size refers to the MMPT Send Message Input Payload structure (see § Table 6-57) including the full Message Data.

- The host transfers the management message to the Function using the Send Message command.
 - If the total input payload size is less than or equal to the MMB Payload Registers Size, the host issues the Send Message Command with Action = Full Data Transfer.
 - If the total input payload size is greater than the MMB Payload Registers Size, the host transfers the message to the Function in ordered parts where each part fits in the MMB Payload Registers (§ Section 6.35.1.3.2.5). If any part fails to transfer, the host can retransmit that part before sending the next part or abort the transfer and start over.
 - The host issues the Send Message Command with Action = Initiate Data Transfer.
 - Depending on the size of the Message Data, the host issues zero or more Send Message Commands with Action = Continue Data Transfer.
 - The host issues the Send Message Command with Action = Finish Data Transfer.
- The Function may optionally send a response message back to the host.
 - If the response message size is less than or equal to the MMB Payload Registers Size and the response message data is available when the Send Message command completes, the Function returns the response message in the output payload.
 - Otherwise, the Function sends the response message using the asynchronous Receive Message flow.

6.36.1.2 MMPT Receive Message (Opcode 0101h) §

Transfer all or part of a management message from the Function to the host.

Possible MMB Command Return Code values (§ Section 6.35.1.3.2.4.1):

- Success
- Invalid Input
- Internal Error
- Retry Required
- Invalid Payload Length
- Unsupported
- Unsupported Management Message
- Transfer Out of Order
- Transfer in Progress
- Transfer Aborted

Table 6-59 MMPT Receive Message Input Payload §

Byte Offset	Length in Bytes	Description
00h	01h	MMPT Message Type – This field indicates the nature and format of the Message Data. Encodings for this field are provided in the <u>[PCI-Code-and-ID]</u> . All unspecified encodings are Reserved.
01h	03h	Reserved
04h	04h	Flags DW:
		Bits[2:0]
		Action Specifics – Specifies the stage of the Message Data transfer.
		000b Full Data Transfer
		001b Initiate Data Transfer
		010b Continue Data Transfer
		011b Finish Data Transfer
		100b Abort Data Transfer
Others All other values are reserved		
Bits[31:3]	Reserved	
08h	04h	Offset – The Byte Offset in the message data to return in the output payload.
0Ch	04h	Message Length – The Length in Bytes of the message data to return in the output payload.

Table 6-60 MMPT Receive Message Output Payload §

Byte Offset	Length in Bytes	Description
00h	Varies	Message Data – The management message data, formatted according to the MMPT Message Type specification.

6.36.1.2.1 MMPT Receive Message Operation §

The flow for transferring a management message asynchronously from the Function to the Host is described below.

- The Function indicates to the host that a new management message is ready to be retrieved.
 - The Function writes the MMPT Receive Message Notification Register (§ Section 6.35.1.4.1.3) and Sets MMPT Receive Message Ready.
 - The host either polls for MMPT Receive Message Ready to be Set or waits for the MMPT Receive Message Interrupt if configured.
- The host transfers the management message from the Function using the Receive Message command.
 - If the MMPT Receive Message Length is less than or equal to the MMB Payload Registers Size, the host issues the Receive Message Command with Action = Full Data Transfer.
 - If the MMPT Receive Message Length is greater than the MMB Payload Registers Size, the host transfers the management message from the Function in ordered parts where each part fits in the MMB Payload Registers (see § Section 6.35.1.3.2.5). If any part fails to transfer, the host can retransmit that part before receiving the next part or abort the transfer and start over.
 - The host issues the Receive Message Command with Action = Initiate Data Transfer.
 - Depending on MMPT Receive Message Length, the host issues zero or more Receive Message Commands with Action = Continue Data Transfer.
 - The host issues the Receive Message Command with Action = Finish Data Transfer.
- The Function Clears the MMPT Receive Message Notification Register (see § Section 6.35.1.4.1.3).

6.37 Debug Over Link §

As complexity of interconnects and SoCs/chiplets increases, the complexity of diagnosing and resolving issues relating to these increases as well. The ability to transmit real-time debug information is very useful for identifying and addressing problems that may arise during operation. Debug mechanisms, such as the ones described in this section, are tools that provide the visibility needed to ease the failure debug process.

6.37.1 NOP Flit §

The NOP Flit debug mechanism is a method for transmitting debug information over the PCI Express link. It utilizes the NOP.Debug and NOP.Vendor Flit types (see § Section 4.2.3.4.3.2 and § Section 4.2.3.4.3.3), which are inherently non-intrusive and conform to existing PCI Express flit transmission rules, to deliver the debug information across the link. This mechanism can be particularly useful for capturing real-time link information, which is vital for debugging transient issues that are not easily accessible or visible after their occurrence.

Both NOP.Debug and NOP.Vendor Flits can provide visibility of internal state information to an observer. Internal states could be related to the PCI Express Link used for sending these NOP Flit types or it could be something unrelated to the PCI Express Link.

Possible Observers include implementation specific entities in the receiving Port and external Protocol/Logic Analyzers, allowing for real-time analysis of link behavior. This immediate access to link information can be important for diagnosing issues that may only be present for a brief period, allowing that critical debug data is not missed or stale.

NOP Flits are transmitted by the Physical Layer, functioning independently from the upper protocol layers with respect to their transmission. While the content encapsulated within these NOP Flits may be derived from or informed by the

upper layers, such as the Data Link or Transaction Layers, the actual process of sending these Flits is managed at the Physical Layer level. This separation ensures that the debug information can be transmitted even in scenarios where the upper layers may not be fully operational or are in a state of initialization.

NOP.Debug Flits can be transmitted periodically during normal operation as a proactive measure serving as checkpoints, as an early indicator preceding potential error conditions, in direct response to error conditions that have been detected, or manually triggered. Note: NOP.Vendor Flits can be used in a similar manner.

Possible usages of NOP.Debug Flits:

1. Periodic Transmission:
 - By periodically transmitting the current state of the link when no error conditions are present, the NOP.Debug Flits can serve as valuable checkpoints when debugging a failure and trying to identify the failure point with respect to regular link traffic.
2. Precursor to Error Conditions:
 - The transmission of NOP.Debug Flits may be triggered as a preemptive signal when the system identifies patterns that typically precede error conditions, alerting an observer to the state of the link and providing context for debugging.
3. Response to Error Conditions:
 - In the event of an error, NOP.Debug Flits can be transmitted immediately to capture the state of the link at the time of the error, providing valuable context for debugging.
4. Manual Trigger:
 - Software-initiated transmission of NOP.Debug Flits allows for on-demand generation of debug information, giving the ability to manually trigger NOP.Debug Flits and control the number of Flits and debug content for targeted diagnostic purposes.

The transmission of NOP.Debug Flits operates on a best-effort basis, with no guarantee of delivery. The receiver has no requirements on how to handle the received Flits, but receiving NOP.Debug Flits must not affect the link state. This approach is designed to minimize the impact on the primary function of the PCI Express Link while still providing a channel for essential debug information. NOP.Debug and NOP.Vendor Flits are not replayed and are thus subject to loss due to bit errors.

In instances where direct physical access to a PCI Express Link is not possible (e.g., on-chip Links or Links where the traces are not wired to a connector might not be visible by a Protocol Analyzer), Switches and Root Complexes are permitted to forward NOP.Debug and/or NOP.Vendor Flits between Ports. This feature allows NOP.Debug and/or NOP.Vendor Flits to be routed to a different, accessible Link, ensuring that debug information remains observable regardless of the physical constraints of the system. Similarly, a single component may have multiple independent entities generating NOP.Debug or NOP.Vendor Flits that are then forwarded to a transmitting Port. In both situations, this forwarding occurs on a best effort basis and may be subject to Flit dropping on buffer overflows. The routing mechanism between Links is implementation specific, but each NOP.Debug and NOP.Vendor Flit includes a NOP Stream ID within the TLP bytes to identify its source. The forwarded NOP.Debug and NOP.Vendor Flits must preserve all TLP bytes of the original NOP.Debug Flit with the exception that the forwarding agent is permitted to overwrite the NOP Flit Counter to FFFh.

Forwarding between Ports of a Switch or Root Complex is not required. When implemented, a subset of Ports may be supported. For example, it is unlikely to make sense to support forwarding between Ports on different sockets of a multi-socket Root Complex. Similarly, to avoid implementing a parallel switching fabric, forwarding might only be supported between “adjacent” Ports of a Switch or Root Complex. The Debug Flit Maximum Rate Hint field can be used to help avoid buffer overflow due to aggregation at forwarding entities.

NOP.Debug and NOP.Vendor Flits carry a unique NOP Flit Counter for each NOP Stream ID. This counter allows observers to detect most missing or dropped Flits, providing an observer a reasonably reliable method for tracking the continuity of the debug information stream. Note that the NOP Flit Counter is a 12-bit field and cannot detect certain loss patterns.

Forwarding agents should change the NOP Flit Counter to FFFh to provide a hint to observers that a large number of Flits were dropped.

NOP.Debug Flits carry a Vendor ID, enabling the transmission of custom debug information tailored to specific vendor requirements. Additionally, PCI-SIG defined Debug Opcodes are available (see § Table 4-25), offering standardized options for debug operations and enhancing the interoperability of the mechanism across different vendor implementations.

Transmission of NOP.Debug and NOP.Vendor Flits must not interfere with entry and exit conditions for power management substates. Upon transition to a new state, transmission of NOP.Debug and NOP.Vendor Flits must be appropriately suspended or modified to align with the activity level permitted by the target state.